



Guidewire InsuranceSuite™ for Guidewire Cloud

PolicyCenter Configuration

Release: 2026.03.0



© 2026 Guidewire Software, Inc.

For information about Guidewire trademarks, visit <https://www.guidewire.com/legal-notices>.

Guidewire Proprietary & Confidential — DO NOT DISTRIBUTE

Product Name: Guidewire InsuranceSuite for Guidewire Cloud

Product Release: 2026.03.0

Document Name: PolicyCenter Configuration

Document Revision: 23-April-2026

Contents

Support.....	13
---------------------	-----------

Part 1

Guidewire PolicyCenter configuration.....	15
--	-----------

1 Validation in PolicyCenter.....	29
Choosing between validation and underwriting authority.....	29
Field-level validation.....	29
Class-based data object validation.....	30
Rules-based data object validation.....	30
Highlighting and linking to validation errors in PCF files.....	31
Highlighting coverage term field with validation error.....	31
Highlighting dynamic step with validation error.....	32
Highlighting clause term in popup with validation error.....	33
2 Class-based validation.....	35
Validation classes in the base configuration.....	35
Configuring class-based validation.....	36
Policy object validation levels.....	37
Adding new validation levels.....	37
Validation package.....	38
Validation chaining.....	41
PolicyPeriodValidation: validateImpl method.....	45
PolicyPeriodValidation validation checks.....	45
Invariant validation checks.....	46
Static validation checks.....	47
Invoking class-based validation.....	47
Example: Invoking validation in a job wizard step.....	48
Invoke validation in a job wizard step.....	48
Trace the validation path.....	48
3 Configuring contingencies.....	51
Contingency object model.....	51
Contingency Gosu class.....	53
Contingency batch processing.....	53
4 Configuring the Spotlight integration.....	55
Accessing risk assessment interactively.....	55
Accessing risk assessment non-interactively.....	58
Risk assessment object model.....	59
Location risk assessment object.....	60
Risk assessment temporary objects.....	60
Configuring risk assessment in the Location Information screen.....	61
Risk assessment information.....	61
Risk assessment thumbnail map.....	62
Deleting temporary risk assessment objects.....	62
Risk assessment configurable classes.....	62
Spotlight service authentication for risk assessment.....	63

	Risk assessment error handling.....	64
	Request and location level errors.....	64
	Enabling Spotlight services.....	64
	Risk assessment configuration parameters.....	64
	Spotlight session time-out.....	65
5	Configuring underwriting authority.....	67
	Configuring authority grants.....	67
	Underwriting authority profile object model.....	68
	Displaying authority grants in the user interface.....	68
	Configuring underwriting rules.....	69
	Configuring underwriting rules through business rules.....	69
	Underwriting rules object model.....	69
	Adding a new checking set.....	71
	Adding a new value comparator.....	73
	Adding a new value formatter.....	75
	Checking sets and evaluators.....	76
	Orphaned underwriting issues.....	77
	Blocking points.....	77
	Configuring underwriting referral reasons.....	78
	Job interactions with underwriting issues.....	79
	Issue keys.....	82
	Configuring underwriting issue history.....	82
	Comparing issue values.....	83
	Configuring default values for assignment type and value offset amount.....	84
	Underwriter issue type verifier class.....	84
	Localizing underwriting issue details.....	84
	Configuring approvals.....	84
	Approval expiration.....	85
	Special approval permission.....	85
	Passing approval requests to underwriters.....	85
	Handling underwriting issues in policy revisions.....	86
	Handling underwriting issues in out-of-sequence policy changes.....	86
	Handling underwriting issues in preempted jobs.....	87
	Creating externally managed underwriting rules in Gosu code.....	87
	UWIssueType fields in underwriting rule.....	89
	Creating underwriting issues with policy evaluation plugin.....	89
	Raise underwriting issues in Gosu code.....	90
6	Configuring business rules.....	91
	Business rule subtypes.....	91
	Modifying the business rules functionality.....	91
	Working with the business rules plugin class.....	92
	Configuring business rule editor functionality.....	92
	Business rule actions.....	93
	Triggering point mapping.....	96
	Custom rule utility functions.....	96
	Rule contexts and rule context definitions.....	96
	Configuring applicability criteria.....	97
	Add a new PolicyCenter applicability criterion.....	98
7	Configuring policy data spreadsheet import/export.....	101
	Register the export data web factory class.....	101
	Create an XML file describing columns to import and export.....	102

Define column data resolvers.....	105
Update the ColumnDataResolverFactory class.....	106
Create an import strategy for each new data column resolver.....	106
Register the import strategy class.....	107
8 Changing the spreadsheet protection password.....	109
Change the worksheet protection password.....	109
9 Configuring locations.....	111
Adding a revisioned field to a location.....	111
Define revisioned field on account location.....	112
Define revisioned field on policy location.....	112
Define revisioned field as syncable on account location.....	113
Define revisioned field as syncable on policy location.....	114
Add get and set methods to policy location.....	114
Add revisioned field to PolicyCenter user interface.....	115
Configuring geocoding.....	115
Implementing your own unused location removal.....	115
10 Configuring the Team tab.....	117
Setting the window size for team statistics.....	117
How PolicyCenter calculates reporting categories.....	118
Setting the maximum number of rows on the Team screens.....	119
11 Configuring earned premium.....	121
How PolicyCenter calculates earned premium.....	121
Methods that calculate earned premium.....	122
PCF files that display earned premium.....	122
12 Configuring quote cloning for business intelligence.....	123
Quote cloning object model.....	124
Enable quote cloning.....	124
Quote clone plugin.....	124
Determining whether to clone a quote.....	124
Cleaning up after cloning policy period.....	125
Processing the cloned policy period.....	125
Adding to the Purge Quote Clones query.....	125
Purge Quote Clones batch processing.....	125
Configuring asynchronous quoting	15
Configure and enable asynchronous quoting.....	16
Asynchronous quoting in PCF files.....	17
Policy period status and asynchronous quoting.....	18
Asynchronous quoting activity.....	19
Asynchronous quoting work queues.....	19
13 Configuring quoting and rating performance.....	127
Configuring parallel product model synchronization.....	127
Configuring two-step quoting.....	128
Enabling two-step quoting.....	128
Quoting process steps.....	128
14 Configuring quote purging.....	131
Quote purging configuration overview.....	131
Quote purging object model.....	131
Objects that get purged or pruned.....	132
Quote purging batch processes.....	134
Enable quote purging batch processes.....	135
Batch processes to run prior to running the purge batch process.....	135

Purge batch process.....	135
Purge Orphaned Policy Periods batch process.....	138
Using events to notify external system of purged or pruned entities.....	139
Purging test tool.....	139
Flushing other work queues with the purging test tool.....	139
Purge or prune a job with the purging test tool.....	139
Prune policy periods with the purging test tool.....	140
Run the Purge batch process from the purging test tool.....	141
15 High Volume Quoting.....	143
Overview of a High Volume Quoting implementation.....	145
Process flow for high volume quotes.....	146
Quote-only instances of PolicyCenter.....	147
Configuring time to live for quotes in Quote Storage Service.....	147
Configure QSS time to live in the storeQuote function.....	148
Configure QSS time to live in config.properties.....	148
Configure validation levels, rating style, and blocking issues.....	149
Observing High Volume Quoting metrics.....	151
High volume quote requests API classes and XML.....	151
Quoting factory.....	151
XML structures and classes for quotes.....	152
Quote data manager.....	153
Quote converter.....	155
Quote store.....	155
Quote search criteria.....	156
16 Product Model Synchronization (V2.0).....	157
17 Configuring Rating Management.....	161
Rating Management data model.....	161
Rate table data model.....	161
Rate routine data model.....	162
Rating Management performance.....	163
Improving startup and first access to Rating Management components.....	163
Improving rate table performance.....	169
Improving rate table cache performance.....	171
Parameters and properties for Rating Management.....	175
Minimum rating level parameter.....	175
Rate table normalization configuration parameters.....	175
Logging properties for Rating Management.....	176
Third party libraries for Rating Management.....	176
POI library.....	176
User authority and permissions for Rating Management.....	176
Rating Management permissions.....	176
Rating Management roles.....	176
Assignment of permission to roles.....	177
Custom physical tables for Rating Management.....	177
Configuring a new custom physical table.....	177
Configuring value providers.....	178
Typelist value provider.....	178
PolicyCenter product model value providers.....	178
Reference factor value provider.....	179
Creating a new value provider.....	180
Configuring matching rule operations.....	180

Configure a new match operation.....	180
Extending the match operation factory class.....	181
Match operation implementation.....	181
Match operation validator.....	182
Configuring rate book matching.....	183
Configuring rate routines.....	183
Configuring the rating engine to execute the rate routine.....	183
Adding a new rate routine function.....	184
Configuring rounding operators.....	185
Configure prefixes that identify types in rate routine steps.....	186
Configuring variant identifiers for a rate routine.....	186
Configuring the rate routine plugin.....	189
Using Gosu to get the rate factor from the rate table.....	189
Configuring new parameters in parameter sets.....	191
Creating a new parameter for inclusion in parameter sets.....	192
Adding a new parameter to the rating engine.....	193
Configuring new wrappers for parameter sets.....	194
Configuring rating worksheets.....	195
Configuring extract and purge rating worksheets.....	195
Logging rate routine functions in a rating worksheet.....	197
Configuring impact testing.....	197
Impact testing warnings and recommendations.....	198
Enabling impact testing for a line of business.....	198
Rate renewals as renewal jobs in impact testing.....	198
Modifying the functionality of impact testing.....	199
Work queues for impact testing.....	199
Configuring the impact testing plugin.....	199
Configuring rate book export to spreadsheet and XML.....	200
Rating Management plugins and interfaces.....	200
Rate routine plugin.....	200
Impact testing plugin.....	204
Rate query lookup plugin.....	207
Adding line-specific cost methods using cost adapters.....	207
18 Configuring Reinsurance Management.....	209
Reinsurance Management object model.....	209
Reinsurance program object model.....	209
Reinsurance agreement object model.....	209
Reinsurance coverage group object model.....	212
Policy period reinsurance object model.....	213
Reinsurance Management permissions.....	214
Configuring reinsurance coverage groups.....	215
Configure reinsurance coverage group on individual coverages.....	215
Disable Reinsurance Management for a line of business.....	215
Reinsurance net retention underwriting issue type.....	215
Gosu methods for calculating the total insured value.....	215
19 Configuring monetary data.....	217
Monetary data in PolicyCenter.....	217
Configuring monetary data storage in the database.....	219
Money data types.....	219
Precision and scale.....	219
Specifying the supported currencies.....	220

Add a currency.....	220
Remove a currency.....	221
Retire a currency.....	221
The default currency.....	221
Specify the default currency.....	222
Display properties for monetary data.....	222
The ICU Library.....	222
Currency formats in a single currency implementation.....	222
Currency formats in a multicurrency implementation.....	224
Configuring currencies by territorial data.....	225
Categorizing territorial typelists by currency.....	225
Mapping currencies to jurisdictions.....	225
Mapping currencies to addresses.....	226
Monetary amounts in Gosu.....	226
20 Configuring multicurrency.....	227
Configuring PolicyCenter for a single currency.....	227
Enabling multicurrency.....	227
Enable multicurrency.....	228
Multicurrency object model.....	228
Multicurrency properties.....	229
Multicurrency in a policy line.....	229
Creating multicurrency policy lines.....	230
Adding coverage currencies to a policy line.....	231
Changing the settlement currency.....	232
PolicyCenter and product model configured for multicurrency.....	232
Configuring the coverage currency.....	233
Coverage currency in accounts.....	233
Coverage currency in policy periods.....	233
Coverage currency in coverages and clauses.....	233
Coverage currency in coverables.....	234
Configuring the settlement currency.....	234
Settlement currency in contacts.....	234
Settlement currency in accounts.....	235
Settlement currency in policy periods.....	235
Configuring multicurrency and reinsurance.....	235
Reinsurance objects and multicurrency.....	236
Configuring multicurrency and rating.....	236
Costs and multicurrency.....	236
Transactions and multicurrency.....	238
Configuring underwriting authority and multicurrency.....	238
Configuring underwriting issues and multicurrency.....	238
Configuring authority profiles and multicurrency.....	239
Implementing an exchange rate service.....	239
The exchange rate service plugin interface.....	239
Configuring the BillingCenter multicurrency integration.....	240
Plans and authority limits for multicurrency in BillingCenter.....	240
21 Configuring personal data destruction.....	243
Data destruction configuration parameters.....	243
Data model configuration for data destruction.....	244
DestructionRootPinnable delegate.....	244
Do Not Process flag.....	245

Display keys for data destruction messages.....	245
Obfuscatable delegate.....	245
Obfuscator interface.....	246
Marking entity fields for obfuscation.....	246
ContactInfo support for personal data destruction in ClaimCenter.....	248
Entity domain graphs.....	248
Identifying archived objects that affect destruction of data.....	250
Data Protection Officer.....	251
Data Protection Officer permissions.....	251
Notifying the data protection officer.....	252
Data destruction purge configuration.....	252
Purging and the ClaimCenter entity domain graph.....	253
Pinnable hierarchy in PolicyCenter.....	253
PersonalDataPurgeTree class in PolicyCenter.....	254
Traversing the pinnable hierarchy in PolicyCenter for personal data purge.....	255
Defining the destroyer.....	256
PersonalDataPurge event in ClaimCenter and PolicyCenter.....	258
Specifying which objects in the graph can be destroyed.....	258
Data obfuscation in InsuranceSuite.....	262
Implementing the Obfuscatable delegate in an entity.....	263
Implementing the Obfuscator interface in an entity.....	263
Personal data obfuscator classes.....	265
Testing data destruction in PolicyCenter during development.....	269
22 Configuring automation.....	271
Changes to PolicyCenter to enable automation.....	272
23 Configuring seasonal coverages in a line of business.....	275
Configuring policy purge.....	20
Configuring preupdate processing.....	21
Implement prioritized preupdate functionality.....	23
Create custom executors.....	23
Configure the IPreupdateHandler registry.....	24
Logging executors.....	25
Migrating from the SurePath prioritized preupdate handler.....	25
The prioritized preupdate screen.....	27
24 Defining activity patterns.....	279
Pattern types and categories.....	279
Activity pattern types.....	280
Categorizing activity patterns.....	280
Using activity patterns in Gosu.....	280
Calculating activity due dates.....	281
Target due dates (deadlines).....	281
Escalation dates.....	281
Defining the business calendar in ClaimCenter.....	281
Base configuration activity patterns.....	282
Activity pattern objects.....	282
Use activity patterns with documents and emails.....	284
Specifying document and email templates in CSV files.....	284
Localizing activity patterns.....	284
25 Guidewire InsuranceSuite and email.....	285
Configure the email plugin class and parameters.....	286
The email object model.....	286

Email utility methods.....	287
Email transmission.....	288
About email templates.....	288
Create an email template file.....	289
Create an email template descriptor file.....	290
Localize an email template.....	290
Sending emails from Gosu code.....	290
Saving an email message as a document.....	291
Creating an HTML email within code.....	291

Part 2

Guidewire PolicyCenter job configuration.....299

26 Configuring jobs.....	301
Local job process.....	301
Gosu classes for local job processes.....	301
Cloud job process.....	302
Configure cloud job processes.....	302
Wizards for jobs.....	304
Job workflows.....	304
Job rule sets.....	304
System configuration parameters for jobs.....	304
Changing jobs to expired status.....	305
Configuring the job expire batch process.....	305
Configuring job history events.....	305
History events in the default configuration.....	306
Configuring history search criteria.....	307
Multiple revision jobs and the job selected branch property.....	307
Selecting the underwriting company through segmentation.....	308
Segmentation classes.....	308
PolicyPeriodBaseEnhancement.....	309
27 Configuring submissions.....	311
Submission process Gosu class.....	311
Submission enhancements.....	312
Submission integration.....	313
Submission web services.....	313
Submission plugins.....	313
Expiring submissions.....	314
Configuring the copy submission feature.....	314
28 Configuring issuance.....	315
Modifying the issuance process.....	315
Issuance integration.....	315
Issuance web services and plugins.....	315
29 Configuring renewals.....	317
Renewal process Gosu class.....	317
Renewal enhancement.....	318
Renewal workflows.....	318
Renewal rule sets	319
Renewal integration.....	319
Renewal web services.....	319
Renewal plugins.....	320

Renewal events.....	321
Configuring batch process renewals.....	321
Renewal plugin calculation.....	322
Renewal lead time in notification config system table.....	323
Example lead time by line of business.....	323
Example lead time by time of year.....	323
Delay for a conflicting job.....	323
Configuring explanations in pre-renewal directions.....	324
30 Configuring cancellations.....	325
Cancellation process class.....	325
Cancellation enhancements.....	326
Complete cancellation workflow.....	326
Cancellation integration.....	326
Cancellation web services.....	326
Cancellation plugins.....	327
Cancellation events.....	327
Calculating the cancellation effective date.....	328
Calculating the cancellation effective date if the source is insured.....	328
Calculating the cancellation effective date if the source is insurer.....	328
Configuring the premium calculation method.....	329
31 Configuring policy change.....	331
Policy change process class.....	331
Modifying the policy change process class.....	332
Policy change integration.....	333
Policy change web services.....	333
Policy change plugins.....	333
Policy change events.....	333
32 Configuring reinstatement.....	335
Reinstatement process Gosu class.....	335
Reinstatement integration.....	336
Reinstatement web services.....	336
Reinstatement plugins.....	336
Reinstatement events.....	337
33 Configuring rewrite.....	339
Rewrite process Gosu class.....	339
Rewrite events.....	340
34 Configuring rewrite new account.....	341
Rewrite new account job.....	341
Copying contacts and locations in rewrite new account job.....	341
Rewrite new account object model.....	342
Rewrite new account history events.....	342
35 Configuring premium audit.....	343
Audit Gosu classes.....	344
Modifying the audit process Gosu class.....	344
Modifying the audit scheduler Gosu class.....	344
Premium audit integration.....	345
Integration with BillingCenter.....	345
Audit web services.....	345
Audit plugins.....	345
Audit events.....	345
Defining audit schedules.....	345

- Final audit schedules..... 345
- Premium report schedules..... 346
- Create audit schedule..... 347
- Configuring audit types..... 347
 - Add new audit type..... 348
 - Extend audit schedule type typelist..... 348
 - Audit schedule properties file..... 349
 - Audit schedule pattern..... 351
- 36 Configuring side-by-side quoting..... 353**
 - Changing the maximum number of versions in side-by-side quoting..... 353
 - Side-by-side quoting object model..... 353
 - Configuring the side-by-side quoting screen..... 354
 - Posting changes to the server..... 354
 - Disable post on enter..... 355
 - Configuring initial behavior of the side-by-side quoting screen..... 355
 - Copying base data for side-by-side quoting..... 355
 - High-level view of base data copy..... 356
 - Gosu methods for base data copy..... 356
 - Excluding side-by-side data from base data..... 358
 - Side-by-side data that base data copy must exclude..... 359
 - Side-by-side data that base data copy can exclude..... 359
 - Side-by-side data in personal auto excluded from base data copy..... 361
 - Exclude entities reachable through two paths..... 362
 - Configuring data excluded from base data..... 365
 - Configuring Quote All to ignore validation warnings..... 365

Support

For assistance, visit the Guidewire Community.

Guidewire customers

<https://community.guidewire.com>

Guidewire partners

<https://partner.guidewire.com>

Guidewire PolicyCenter configuration

It is possible to configure PolicyCenter features such as Guidewire Rating Management and personal data destruction to meet your business needs. You configure these features by modifying configuration files, batch processes, and Gosu classes, among others.

Configuring asynchronous quoting

In the base configuration, asynchronous quoting is not enabled and it is not configured for each line of business. To configure asynchronous quoting, you update the `shouldQuoteAsynchronously` with the criteria which triggers asynchronous quoting. You can use the asynchronous quoting threshold value to define the condition that triggers asynchronous quoting. The default threshold value is 30. For information on how to enable asynchronous quoting and define the asynchronous quoting criteria for one or more lines of business, see “Configure and enable asynchronous quoting” on page 16.

PolicyCenter uses the following components to implement asynchronous quoting:

PCF files

PCF files define the user interface behavior. This applies to products that use `localjobprocess` or `cloudjobprocess`. For more information, see “Asynchronous quoting in PCF files” on page 17.

Policy period status

The policy period status indicates the job status of the asynchronous quote. This applies to products that use `localjobprocess` or `cloudjobprocess`. For more information, “Policy period status and asynchronous quoting” on page 18

Activity pattern code

PolicyCenter creates an asynchronous quoting activity item when the asynchronous quote completes. This applies to products that use `localjobprocess` or `cloudjobprocess`. For more information, see “Asynchronous quoting activity” on page 19.

Work queue

Asynchronous quoting work queues keep track of the asynchronous quotes that are in progress. This applies to products that use `localjobprocess` or `cloudjobprocess`. For more information, see “Asynchronous quoting work queues” on page 19.

Batch jobs

For products that use `cloudjobprocess` and also enable Quoting Services, PolicyCenter uses a set of batch jobs to determine the status and get the results of asynchronous quotes. For more information, see *Application Guide* and “Configure and enable asynchronous quoting” on page 16.

Configure and enable asynchronous quoting

You can enable asynchronous quoting and you can configure the criteria which triggers asynchronous quoting for each line of business.

After reading this topic, you'll be able to:

- Enable asynchronous quoting for PolicyCenter
- Configure asynchronous quoting for a line of business
- Configure the asynchronous quoting threshold

Procedures

Step 1: Enable asynchronous quoting

About this task

Enables asynchronous quoting for all lines that implement asynchronous quoting.

Procedure

Open `config.xml` in Studio, and set the `AsynchronousQuotingEnabled` parameter to `true`.

Step 2: Configure asynchronous quoting for the line of business

About this task

The `PolicyLineMethods` interface of each line of business includes a `shouldQuoteAsynchronously` methods that defines when PolicyCenter quotes that line of business quote asynchronously.

Procedure

In the `PolicyLineMethods` interface of each line of business, override `shouldQuoteAsynchronously` method to specify if and when to quote asynchronously. You can call `AychnQuotingConfiguration.getThreshold` to include the threshold value in the asynchronous quoting criteria that you define.

Example

Example 1: To implement asynchronous quoting for the SBO line using the threshold value, you can override `shouldQuoteAsynchronously` with the following code:

```
class SBOPolicyLineMethods extends SBOPolicyLineMethodsBase {
    var _line : entity.SBOLine
    construct(line : entity.SBOLine) {
        super(line)
        _line = line
    }
    ...
    override function shouldQuoteAsynchronously() : boolean {
        return _line.Branch.SBOLine.SBOLocations.toArray("SB0Buildings").toArray(new SB0Building[0]).Count >
            AychnQuotingConfiguration.getThreshold(_line.PatternCode)
    }
}
```


Example 2: To bypass the threshold and always quote the SBO LOB asynchronously, you can override `shouldQuoteAsynchronously` with the following code:

```
class SBOPolicyLineMethods extends SBOPolicyLineMethodsBase {
  var _line : entity.SBOLine
  construct(line : entity.SBOLine) {
    super(line)
    _line = line
  }
  ...
  override function shouldQuoteAsynchronously() : boolean {
    return true
  }
}
```

Optional: Configure the asynchronous quoting threshold

About this task

The default threshold is 30. However, you can update this value for each line of business.

Procedure

1. To update the threshold value for a line of business, open the `modules/configuration/config/config.properties` file in Studio and add following property:

```
asyncQuoting.<LineCodePattern>.threshold = <value>
```

For example, during development and testing, you can change the threshold for the BOP line to a lower value, such as 3, with the following property setting: `asyncQuoting.BOPLine.threshold = 3`.

2. Repeat step 1 for each line of business for which you want to add a threshold value.

Next steps

To apply the changes, restart PolicyCenter.

Related information

- If you use Quoting Services, the asynchronous batch jobs run every 15 minutes. However, you can increase the frequency to as low as 30 seconds. For more information, see *InsuranceSuite Server Administration* and *InsuranceSuite Server Administration*

Asynchronous quoting in PCF files

Each `JobWizardToolBarButtonSet.Job.pcf` files has two **Quote** buttons, only one of which is visible at a time. One **Quote** button triggers asynchronous quoting. The code in the `visible` property determines whether to display the asynchronous **Quote** button. The code checks the line of business and determines whether the number of coverables on the policy period exceeds the threshold. Both **Quote** buttons use the same display key.

If both asynchronous quoting and two-step quoting are enabled in a policy line, the PCF file displays an asynchronous **Rate** button for the first step. The second step displays a **Finish Quote** button that quotes synchronously.

Cancellation

The `StartCancellation.pcf` file has two **Start Cancellation** buttons, one of which is for asynchronous quoting.

In synchronous quoting mode, the **Start Cancellation** button starts the cancellation job, which quotes, then displays the **Confirmation** screen. In asynchronous quoting mode, **Start Cancellation** starts the cancellation job, which takes you to the **Entry** screen instead of the **Confirmation** screen. You must then click **Quote** to start asynchronous quoting.

Policy period status and asynchronous quoting

On a policy period, the Status property indicates that an asynchronous quote has been requested. If a policy line has asynchronous quoting enabled, the Status property on PolicyPeriod is set to QuoteRequested to indicate that an asynchronous quote has been requested.

However, if a policy line has both asynchronous quoting and two-step quoting enabled, the first step, rating, is done asynchronously. The second step, finish quoting, is synchronous. The Status property is set to RateRequested to indicate that an asynchronous rate has been requested.

Quote requested status

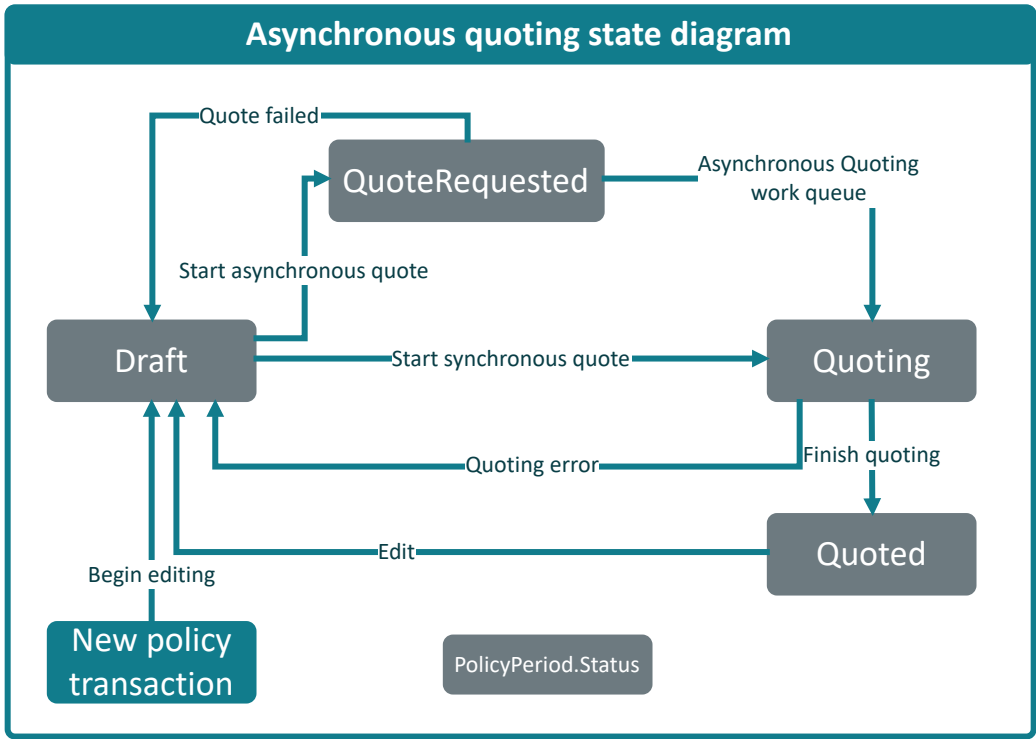
If a policy line has asynchronous quoting enabled, PolicyPeriod.Status is set to QuoteRequested after the user clicks the asynchronous **Quote** button. The status changes to Quoting when quote processing starts. While the policy period is in this status, the user cannot make changes to the job because the job wizard steps are not editable.

PolicyPeriod.Status in a job with asynchronous quoting progresses as follows:

Job outcome	PolicyPeriod.Status
Success	Draft to QuoteRequested to Quoting to Quoted
Quote failure	Draft to QuoteRequested to Quoting to Draft Draft to QuoteRequested to Draft

If quoting fails, the policy period status is set back to Draft. Quoting fails when the number of retries exceeds the WorkItemRetryLimit, specified in config.xml.

The following illustration shows how PolicyPeriod.Status changes as it moves through an asynchronous quote.



Rate requested status

If a policy line has both asynchronous quoting and two-step quoting enabled, PolicyPeriod.Status is set to RateRequested after the user clicks the asynchronous **Rate** button. The status changes to Quoting when rate

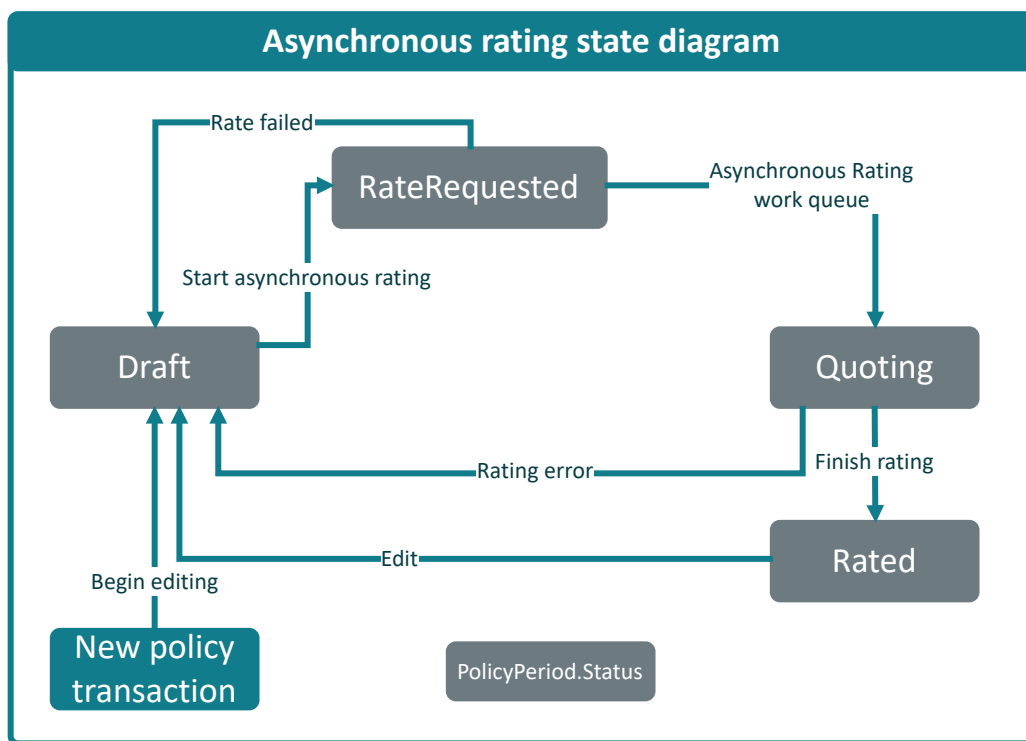
processing starts. While the policy period is in this status, the user cannot make changes to the job because the job wizard steps are not editable.

`PolicyPeriod.Status` in a job with both asynchronous quoting and two-step quoting progresses through rating as follows:

Job outcome	PolicyPeriod.Status
Success	Draft to RateRequested to Quoting to Rated
Quote failure	Draft to RateRequested to Quoting to Draft Draft to RateRequested to Draft

If rating fails, the policy period status is set back to Draft. Rating fails when the number of retries exceeds the `WorkItemRetryLimit`, specified in `config.xml`.

The following illustration shows how `PolicyPeriod.Status` changes as it moves through asynchronous rating.



Asynchronous quoting activity

When an asynchronous quote completes, PolicyCenter generates an activity with activity pattern code (`Activity.ActivityPattern.Code`) set to `async_quote_completed`. The activity is assigned to the user who submitted the quote. PolicyCenter generates an activity with this pattern code for both successful and failed quotes.

If both asynchronous quoting and two-step quoting are enabled in a policy line, PolicyCenter generates this activity when an asynchronous rate completes.

Asynchronous quoting work queues

Two work queues are associated with asynchronous quoting. You can configure these work queues to meet your needs.

Asynchronous Quoting work queue

If a policy line has asynchronous quoting enabled, the Asynchronous Quoting work queue rates and quotes the current policy period asynchronously. PolicyCenter creates an asynchronous quoting work item (AsyncQuotingWorkItem) when a user requests an asynchronous quote from PolicyCenter.

Asynchronous Rating work queue

If a policy line has both asynchronous quoting and two-step quoting enabled, the Asynchronous Rating work queue rates the current policy asynchronously. The second step, finish quoting, is synchronous. PolicyCenter creates an asynchronous rating work item (AsyncRatingWorkItem) when a user requests an asynchronous rate from PolicyCenter.

See also

- *InsuranceSuite Server Administration*

Configuring policy purge

PolicyCenter provides a **Purge Policy Terms** batch process that can be configured to automatically delete policy data that is older than a pre-defined retention threshold (in years). You can use this batch process to reduce the size of your PolicyCenter database. You can define the threshold in the `policy_term_retention_threshold_configs` system table (see the example below).

```
<?xml version="1.0"?>
<import>
  <TermRetentionThresholdConfig public-id="pc:zsjhco2f93r646dg97icfptf9ja">
    <EffectiveDate>2000-01-01 00:00:00.000</EffectiveDate>
    <LineOfBusiness>PersonalAutoLine</LineOfBusiness>
    <RetentionThreshold>10</RetentionThreshold>
  </TermRetentionThresholdConfig>
  <TermRetentionThresholdConfig public-id="pc:zsjhco2f93r646dg97icfptf9jb">
    <EffectiveDate>2000-01-01 00:00:00.000</EffectiveDate>
    <Jurisdiction>AZ</Jurisdiction>
    <LineOfBusiness>PersonalAutoLine</LineOfBusiness>
    <RetentionThreshold>4</RetentionThreshold>
  </TermRetentionThresholdConfig>
  <TermRetentionThresholdConfig public-id="pc:zsjhco2f93r646dg97icfptf9jc">
    <EffectiveDate>2000-01-01 00:00:00.000</EffectiveDate>
    <Jurisdiction>TX</Jurisdiction>
    <LineOfBusiness>PersonalAutoLine</LineOfBusiness>
    <RetentionThreshold>7</RetentionThreshold>
  </TermRetentionThresholdConfig>
</import>
```

The batch process automatically deletes bound policies and policy terms that are older than the threshold. You can also purge policy data based on pre-defined criteria (such as the jurisdiction or the particular line of business), or you can define your own criteria. Note that the purge process only takes into consideration policy terms that came into force after the `EffectiveDate`. The batch process does not require archived policy periods to be retrieved before they can be purged. Archived policy periods are purged in the same way as non-archived policy periods.

PolicyCenter calls a plugin method (`getHistoricPolicyTermsRetentionThreshold()` in `PCPurgePlugin.gs`) to determine the retention threshold. In the base configuration, the plugin method returns the threshold that is defined in the `policy_term_retention_threshold_configs` system table, however you can override the logic and return a different threshold if you wish.

PolicyCenter also contains a number of internal checks that you cannot override. For example, if there are open claims on a policy term then that policy data will not be purged. You can also add your own checks to the plugin methods.

The internal checks are contained in the plugin method `canPurgePolicyTermForTruncation()`, and are carried out at the policy, policy term, and policy period levels. These internal checks cannot be disabled. A policy term will not be deleted if the following conditions apply:

- The DoNotDestroy field is true on any:
 - PolicyTerm
 - Policy associated with the PolicyTerm (PolicyTerm.Policy).
 - Jobs associated with the PolicyTerm (PolicyTerm.job).
- There is no data retention threshold set for the policy term, or the policy term has not exceeded the threshold limit. Note that the data retention threshold is configurable and is returned by the plugin method `getHistoricPolicyTermsRetentionThreshold(policyTerm)`.
- There are any open workflows on PolicyPeriod entities associated with the PolicyTerm.
- There are any open:
 - Jobs with (`CloseDate != null`).
 - Activities on a job associated with the PolicyTerm.
 - Archived jobs associated with the PolicyTerm.

To enable the **Purge Policy Terms** batch process, you must set `TruncatePoliciesEnabled` to true in `config.xml`. `Config.xml` also contains a number of other configuration parameters that you can use to reduce the impact on the database during the purge. See "Policy purge parameters" in *Configuration Parameters*.

Configuring preupdate processing

Prioritized preupdate handler functionality allows you to easily develop and manage preupdate business logic using custom classes.

This documentation describes prioritized preupdate functionality and how you can use it.

Understanding the prioritized preupdate handler

Between the time that a bundle is created and the bundle is committed to the database, the application can perform custom processing on the beans that are being inserted, removed, or modified. This is called *preupdate processing*.

The prioritized preupdate handler is an implementation of the `IPreupdateHandler` plugin, which provides tools for configuring preupdate processing. These tools are in the `gw.plugin.preupdate.prioritized` package, which contains classes and interfaces that allow you to work with executors, priority annotations, and contexts. These terms are defined as follows:

Executors

Gosu classes that contain custom preupdate processing logic for a specific entity. Use executors to manage processing of beans of a given type before beans are committed to the database.

Priority annotations

Class annotations you can add to executors to control the order in which they are executed.

Contexts

objects that contain lists of beans of a specific type that are to be committed to the database as part of a bundle.

For more information about the available classes, interfaces, and other utilities in the prioritized preupdate handler API, see the Javadoc for the `gw.plugin.preupdate.prioritized` package.

Working with executors

Create executors as classes in a Gosu package. Executors are run automatically run by the application when a bundle containing instances of a given entity is committed to the database.

To run an executor, the InsuranceSuite application must first recognize and register the appropriate Gosu classes as executors. The application does this by scanning for executors on application startup. For the application to recognize executors during scanning, executors must meet the following requirements:

- Executors must implement the `PrioritizedPreUpdateExecutor` interface or extend the `StateAwarePrioritizedPreUpdateExecutor` class.
- Executors must be located in the package or packages specified as parameters in the `IPreUpdateHandler` plugin registry. See “Modifying the `IPreUpdateHandler` registry” on page 23.

Executors process beans for specific entity types, and also for subtypes of their target entities. For example, an executor that processes Contact entities also processes Person entities (a subtype of Contact) and Doctor entities (a subtype of Person).

Note: Executors do not process delegates of their target entity types. For example, it is possible for a delegate to inherit from `KeyableBean`, but the delegate and its implementations will not be processed by executors. To ensure proper behavior, executors must target concrete entity types or well-defined abstract entities, rather than delegates.

Executors must override the method `executePreUpdate`. The method signature looks different depending on whether the executor implements the `PrioritizedPreUpdateExecutor` interface or extends the `StateAwarePrioritizedPreUpdateExecutor` class.

- Executors that implement the `PrioritizedPreUpdateExecutor` interface must override the public `void executePreUpdate(PrioritizedPreUpdateContext<T> context)` method. The `PrioritizedPreUpdateContext` object includes properties that contain lists of added, modified, and deleted beans of the given type in the current bundle. You can retrieve the list properties by calling the object's methods `getInsertedBeans`, `getUpdatedBeans`, and `getRemovedBeans`.
- Executors that extend the `StateAwarePrioritizedPreUpdateExecutor` class must override the protected abstract `void executePreUpdate(List<T> beans)` method. The method receives a list of beans that are eligible for preupdate processing, based on which `StateAware` interface is implemented. For more, see [Migrating from the SurePath prioritized preupdate handler](#).

Adding priority annotations

You can add a priority annotation to an executor using the `@ExecutionPriority` annotation. Priority annotations control the order with which the prioritized preupdate handler runs executors. If there are multiple executors that are triggered when a bundle is being committed, the application uses the priority annotation of the executors to determine the order in which executors are run. If there are multiple executors with the same priority, they are executed alphabetically by the name of the executor class.

The `@ExecutionPriority` annotation takes a `PreUpdatePriority` enum constant as an argument. The available constants are:

- HIGHEST
- HIGH
- STANDARD
- LOW
- LOWEST

Higher-priority executors are executed before lower-priority executors. If no execution priority is set, the executor is automatically assigned a priority of `LOWEST`.

For example, a person executor with high priority in the custom package `customer.preupdate` has the following structure:

```
package customer.preupdate

@ExecutionPriority(PreUpdatePriority.HIGH)
class PersonPreUpdateExecutor implements PrioritizedPreUpdateExecutor<Person> {
    override function executePreUpdate(context : PrioritizedPreUpdateContext<Person>) {
        // preupdate logic goes here
    }
}
```

```
}  
}
```

For a more detailed example, see “Implement prioritized preupdate functionality” on page 23.

Modifying the IPreUpdateHandler registry

A plugin registry item `IPreupdateHandler.gwp` is provided for the preupdate handler plugin. To make use of prioritized preupdate functionality, configure the plugin registry. This consists of the following steps:

1. Verifying that the plugin has the correct implementation class.
2. Adding a parameter or parameters that point to the package or packages containing custom executors. The parameters specifies which packages to scan when searching for executors on application startup.

For the procedure to configure the registry, see “Configure the IPreupdateHandler registry” on page 24.

Verifying executors in the user interface

The application displays registered executors and scanned packages on an administrative user interface screen. For information on accessing this screen, see “The prioritized preupdate screen” on page 27.

Order of execution for preupdate logic

You can implement custom business logic in the following ways:

- Writing preupdate rules
- Configuring the prioritized preupdate handler

The prioritized preupdate handler is fully configurable, and contains no built-in preupdate business logic.

However, ClaimCenter and PolicyCenter offer a base configuration implementation of the preupdate handler plugin that contains app-specific business logic. The base configuration implementation specifies preupdate processing related to specific entities and business requirements in the application. This processing is executed before any custom prioritized preupdate processing. For more on this base configuration preupdate processing, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

Therefore, the execution of all preupdate logic occurs in the following order:

1. The base configuration implementation of the preupdate handler.
2. Custom executors built using the prioritized preupdate handler.

Implement prioritized preupdate functionality

You can implement prioritized preupdate functionality to define preupdate processing logic. The main tasks involved are:

- Creating custom executors
- Configuring the `IPreupdateHandler.gwp` file

You can perform these tasks in any order. However, to configure the `IPreupdateHandler.gwp` file, you must know the package or packages where your executors are located.

Create custom executors

About this task

Create custom executors to hold custom preupdate logic for a specific entity.

Procedure

1. Define the Gosu package or packages that contain the executors. You can create new packages or choose existing packages. For example, `customername.plugin.demo`. Any subdirectories within the package will also be scanned for executors.
2. Create a new Gosu class in the package.
 - a) Implement the `PrioritizedPreUpdateExecutor` interface. Set the type parameter to the entity that the executor will process. For example, `PrioritizedPreUpdateExecutor<Account>`.
 - b) Implement the `executePreUpdate` method.
 - c) Write custom business logic using the context provided in the `PrioritizedPreUpdateContext` object.

You can also create executors by extending the `StateAwarePrioritizedPreUpdateExecutor` class. This is described in “Migrating from the SurePath prioritized preupdate handler” on page 25.
3. *Optional:* Add a priority annotation to the custom executors.
 - a) Add the `@ExecutionPriority` annotation above the class declaration, and provide a constant in the `PreUpdatePriority` enum as an argument.

Example

The following is a simple example of a custom executor:

```
package gw.plugin.demo

uses gw.plugin.preupdate.prioritized.ExecutionPriority
uses gw.plugin.preupdate.prioritized.PreUpdatePriority
uses gw.plugin.preupdate.prioritized.PrioritizedPreUpdateContext
uses gw.plugin.preupdate.prioritized.PrioritizedPreUpdateExecutor

@ExecutionPriority(PreUpdatePriority.STANDARD) // set priority annotation
class PersonPreUpdateExecutor implements PrioritizedPreUpdateExecutor<Person> { // implement interface

    // override executePreUpdate method to execute custom business logic; in this case, add a note
    // to persons in the bundle that are inserted, updated, or removed
    override function executePreUpdate(context : PrioritizedPreUpdateContext<Person>) {
        updateContact(context.getInsertedBeans())
        updateContact(context.getUpdatedBeans())
        updateContact(context.getRemovedBeans())
    }

    private function updateContact(contacts : List<Person>) {
        for (contact in contacts) {
            contact.Notes = "Person updated"
        }
    }
}
```

Configure the IPreupdateHandler registry

About this task

For the application to recognize custom executors, you must configure the `IPreupdateHandler` plugin registry item to point to the correct Gosu interface and Gosu package or packages.

Procedure

1. In Studio, go to **configuration > config > plugins > registry > IPreUpdateHandler.gwp**. Open the file.
2. Ensure that the Gosu Class is set to the fully qualified class name of the class that implements the plugin. In the base configuration, this class is `gw.plugin.preupdate.impl.XXPreUpdateHandlerPluginImpl`, where `XX` is `CC` for ClaimCenter, `PC` for PolicyCenter, or `BC` for BillingCenter.
3. Add a parameter for each package containing custom executors.
 - a) In the **Parameters** list, select **Add Parameter**.

- b) Set the **Name** of the parameter to `customerPackage` or some string beginning with `customerPackage`. For example, if there are three packages which contain executors, there could be three parameters, named `customerPackage_feature1`, `customerPackage_feature2`, and `customerPackage_feature3`.
- c) For each parameter, set the **Value** to the fully qualified name of the package that contains the custom executors. For example, `customername.plugin.customexecutors`.

Results

For an implementation in ClaimCenter with two packages containing custom executors, the registry file `IPreUpdateHandler.gwp` might look similar to the following:

```
<?xml version="1.0"?>
<plugin
  interface="IPreUpdateHandler"
  name="IPreUpdateHandler">
  <plugin-gosu
    gosuclass="gw.plugin.preupdate.impl.CCPreUpdateHandlerPluginImpl">
    <param
      name="customerPackage_featureABC"
      value="customername.plugin.customexecutors.featureabc"/>
    <param
      name="customerPackage_featureDEF"
      value="customername.plugin.customexecutors.featuredef"/>
    </plugin-gosu>
  </plugin>
```

After configuring the plugin registry item, preupdate executors are recognized by the application and are triggered when the target entities are modified. You can verify that executors are working as expected by implementing a specialized logger, as explained in the next section.

Logging executors

Guidewire provides a logger for the prioritized preupdate handler. The logging category is `Plugin.PreUpdate.Prioritized`.

When logging is enabled, the following is logged:

- The executor classes that are registered when the application scans for them. This information is logged when the server is started.
- Information about the executors that are run whenever they are triggered by a database transaction.

When the application is started, each executor that has been registered logs a startup message similar to the following:

```
1. PersonPreUpdateExecutor (gw.plugin.demo.PersonPreUpdateExecutor)
  - Priority: HIGH
  - Types:
    1. entity.Person
    2. entity.PersonVendor
    3. entity.UserContact
```

This indicates that this executor will be run for Person entities, as well as PersonVendor and UserContact, which extend Person.

Migrating from the SurePath prioritized preupdate handler

IMPORTANT: New and updating implementations may elect to continue using the SurePath Content preupdate handler implementation, or a custom preupdate handler implementation, instead of adopting the new prioritized preupdate handler functionality. However, if you enable prioritized preupdate functionality as described in this documentation, you must ensure that all preupdate logic is moved from the SurePath component to the prioritized preupdate functionality in your application configuration. The application has a validation to ensure that the Prioritized Preupdate Handler and the SurePath component do not conflict. The application throws a startup error if Prioritized Preupdate functionality is enabled and the

`gw.surepath.suite.integration.preupdate.PrioritizedPreUpdateHandler` package is found in the code base.

Before being added to the InsuranceSuite base configuration, prioritized preupdate handler functionality existed as a SurePath component. If you have implemented the SurePath component, you can migrate your logic to the prioritized preupdate handler by refactoring it. As a convenience, Guidewire provides utilities in the `gw.plugin.preupdate.prioritized` package that are useful for this refactoring.

In this refactoring, you must move all logic from SurePath classes to new executor classes in packages in the customer layer. The package or packages must be referenced as parameters in the `IPreUpdateHandler.gwp` file, as explained in “Configure the `IPreUpdateHandler` registry” on page 24.

The new executor classes can extend the abstract class `StateAwarePrioritizedPreUpdateExecutor` instead of implementing the `PrioritizedPreUpdateExecutor` interface. If executors extend the abstract class, they must implement at least one of the following marker interfaces. These interfaces control which beans in a bundle are affected by the preupdate logic:

- `StateAwareInsert`: Inserted beans are affected by preupdate code (corresponds to the `InsertHandler` interface in the SurePath content).
- `StateAwareRemove`: Removed beans are affected by preupdate code (corresponds to the `RemoveHandler` interface in the SurePath content).
- `StateAwareUpdate`: Updated beans are affected by preupdate code (corresponds to the `UpdateHandler` interface in the SurePath content).

These interfaces mirror the interfaces provided by the SurePath component. This allows you to refactor the code to utilize the prioritized preupdate functionality without significant rewriting.

For example, the following code provides an example of a custom executor using the SurePath migration utilities. In this example, the `StateAwareInsert` and `StateAwareUpdate` interfaces are implemented, so only beans that are inserted or updated in the bundle are affected by the `executePreUpdate` method.

```
@ExecutionPriority(PreUpdatePriority.STANDARD)
class WCStateBenefitEdition_SPPreUpdate
    extends StateAwarePrioritizedPreUpdateExecutor<WCStateBenefitEdition_SP>
    implements StateAwareInsert, StateAwareUpdate {

    override function executePreUpdate(beans : List<WCStateBenefitEdition_SP>) {
        for (b in beans) {
            ReserveWorksheetUtil_SP.updateInUseIndicatorWCBenefitTypeEdition_SP(b.WCBenefitTypeEdition_SP)
        }
    }
}
```

Setting executor priority

In the original SurePath component, executor priority was set using a method. If the method returned `null`, the executor was assigned the lowest priority.

The prioritized preupdate handler uses the `@ExecutionPriority` annotation to control executor priority. The `@ExecutionPriority` annotation is optional. If it is not set, the executor is treated as if it has `LOWEST` priority.

When refactoring a SurePath executor to use the prioritized preupdate functionality, if the original SurePath method returned `null`, you can omit the `@ExecutionPriority` annotation or set it to `LOWEST`.

Best practices

During development of prioritized preupdate business logic, Guidewire recommends that you write tests and maintain coverage to ensure that business logic is correctly registered and executed.

Additionally, Guidewire recommends using loggers to monitor registration and execution of executors. Loggers can be enabled as explained in “Logging executors” on page 25.

The prioritized preupdate screen

The application displays registered executors and scanned packages on an administrative user interface screen. This screen is populated with data when the application scans for executors at startup.

The screen can be accessed as follows:

1. Log in to the application as an administrator, such as username **su** and password **gw**.
2. Press **Alt+Shift+T**.
3. Navigate to **Info Pages > Prioritized Preupdate** in the sidebar.

The screen contains the following information:

Parameters

The parameters used by the `IPreUpdateHandler` plugin as specified in the `IPreUpdateHandler.gwp` file. The following information is displayed for each parameter:

Key

The name of the parameter.

Value

The fully qualified path name of the package that contains the executors.

Executors

The executors that the application registered while scanning on server startup. The following information is displayed for each executor:

Name

The fully qualified path name of the executor.

Entity Names

The entities that are processed by the executor. This includes any subtypes of the targeted entity type. For example, if an executor targets the `Person` entity, this screen also lists `Vendor (Person)` and `User Contact`.

Priority

The priority of the executor, as set using the `@ExecutionPriority` annotation.

Validation in PolicyCenter

PolicyCenter has several types of validation:

- **Field validation** – Is part of the PCF rendering framework and uses regular expressions to ensure that entry is valid. For example, field validation checks that a phone number is in the correct format.
- **Class-based** – You can invoke Gosu class-based validation from PCF files, job processes, wizard steps, workflow steps, integration plugins, and from any Gosu code. The validation class checks that certain conditions are met. The class can raise errors or warnings that includes the validation level at which it failed. For example, after entering vehicles on a Personal Auto policy in the submission wizard, your class-based validation code can check that the policy has at least one driver. If not, the code raises an error at a particular validation level.
- **Rules-based** – On a database commit, for example when you click **Next** in wizard, PolicyCenter calls the validation rules on each validatable object in the bundle. The rules check that certain conditions are met on the object. If the conditions are met, the bundle can be saved. Otherwise, the rule raises a validation error or warning that includes the validation level at which it failed.

Choosing between validation and underwriting authority

Both validation and underwriting authority provide mechanisms to stop the progress of a policy transaction. In the base configuration, validation is checked before underwriting authority.

Use validation to prevent the progress of policy transactions that nobody can sign off on. Validation checks for:

- Missing or inconsistent data – Including things like a vehicle on a personal auto policy without a VIN at bind time, as well as conflicting coverages or terms.
- Structurally inappropriate policies – For example, some insurers do not ever permit a personal auto policy with no vehicles.

Use underwriting authority to restrict who can progress a policy transaction that at least somebody in the organization could sign off on. If only certain people within the insurer organization can approve writing a policy in California, then use an underwriting rule.

However, if the insurer will never write a policy in the state of California under any circumstances, then use a validation rule.

Field-level validation

Field validators handle simple validation for a single field. A validator definition defines a regular expression, which a data field must match to be valid. It can also define an optional input mask that provides a visual indication to the user of the data to enter in the field.

PolicyCenter field-level validation is not part of the class- or rule-based validation framework. Field-level validation does not involve Gosu business rules or classes. It is simply part of the PCF rendering framework.

The PolicyCenter rendering framework builds the user interface from sets of PCF files. During the rendering process, PolicyCenter performs any field-level checks that have been configured in the PCF files. For example, if you configure a specific field in a PCF as required, then if the user leaves that required field empty, the rendering framework generates a user error.

This type of field-level validation is independent of class- or rule-based validation. PolicyCenter performs this type of validation before policy validation and displays problems that occur at the PCF level before it runs policy validation or other processes.

For example, suppose that a user does not enter a value for the **Fleet** field on the (Submission wizard) **Policy Info** page. In this case, the rendering framework automatically generates a field-level validation error as the user attempts to move to the next step in the Submission wizard.

PolicyCenter generates the error message text string from the `MissingRequired` display key, substituting the field name for the `{0}` variable:

```
Java.Validation.MissingRequired = Missing required field "{0}"
```

In this particular instance, you specify that the **Fleet** field in the `BALineDV.pcf` PCF file is required. You do this by setting the `required` property to `true`.

Guidewire recommends that you also implement most of the field validation checks as class-based validation as well. For example, implement a class-based validation method for the BA line that ensures that all required fields have been set.

[See also](#)

- *Data Model Configuration*

Class-based data object validation

PolicyCenter performs class-based data object validation on policy and policy-related data objects. Class-based validation is implemented in Gosu classes. In a Gosu class, you specify how PolicyCenter validates a particular data object. A database commit does not automatically trigger class-based validation.

At any time, you can invoke Gosu class-based validation from PCF files, job processes, wizard steps, workflow steps, integration plugins, and from any Gosu code. The validation class checks that certain conditions are met. The class can raise errors or warnings that includes the validation level at which it failed. For example, after entering vehicles on a Personal Auto policy in the submission wizard, your class-based validation code can check that the policy has at least one driver. If not, the code raises an error at a particular validation level.

PolicyCenter runs full validation on the policy graph at critical points in the job process, such as before quoting or binding.

[See also](#)

- “Class-based validation” on page 35

Rules-based data object validation

PolicyCenter performs rules-based data object validation on non-policy objects. You define this kind of validation in business rules, in the Studio Rules editor. Committing a data bundles to the database triggers rules-based validation, if the bundle contains a validatable entity.

On a database commit, for example when you click **Next** in wizard, PolicyCenter call the validation rules on each validatable object in the bundle. The rules check that certain conditions are met on the object. If the conditions are met, the bundle can be saved. Otherwise, the rule raises a validation error or warning that includes the validation level at which it failed.

In the base configuration, the following entities implement the `Validatable` delegate and are validatable:

- Account
- Activity
- Contact
- Group
- Organization
- ProducerCode
- Region
- User

See also

- *Gosu Rules Guide*

Highlighting and linking to validation errors in PCF files

When a validation error occurs on a PCF page, you can highlight that error and create links that take you to the page with the error. You can find many examples of this in the base configuration. These topics describe how to do this for use cases that have fewer examples.

Highlighting coverage term field with validation error

If class-based validation fails on a coverage term, you can highlight the coverage term and link the error message to that coverage term. The `Result.reject` method can highlight the coverage term. This approach works for coverage terms, schedules, and other abstract parameters.

Note: In the error message, the link can only navigate to locations on the main screen. It cannot navigate to other tabs on the screen.

You can find an example in Personal Auto that highlights the **Broadened Collision** coverage term when you select both **Collision** and **Collision - Limited Coverage** coverages. In `PALineCoveragesValidator.gs`, the `Result.reject` call in the `mutuallyExclusiveCoverages` method specifies the path to the coverage term:

```
Result.reject(vehicle.ID, vehicle, "PACollisionCov.PACollisionBroadTerm.Value",
    ValidationLevel.TC_DEFAULT,
    DisplayKey.get("Web.Policy.PA.Validation.MutuallyExclusiveCollisionCov", vehicle.DisplayName),
    null, null, COVERAGES_WIZARD_STEP)
```

View validation error to coverage term

About this task

In Personal Auto for Massachusetts, you can create a validation error that highlights the **Broadened Collision** coverage term when you select two types of collision coverage: **Collision** and **Collision - Limited Coverage** coverages.

Procedure

1. Create a new account for a person who lives in Massachusetts.
2. From the account, create a new Personal Auto submission.
3. Add this person as the driver on the **Drivers** screen.
4. On the **Contact Detail** tab, enter the following:

Date of Birth

Enter a date

License #	Enter a license number
License State	Enter Massachusetts

- On the **Roles** tab, enter **Year First Licensed**.
- Create a vehicle on the **Vehicles** screen and set the **License State** to **Massachusetts**.
- On the **PA Coverages** screen, set **Collision** > **Broadened Collision** to **Yes**.
You have selected two types of collision coverage: **Collision** and **Collision - Limited Coverage**. This will cause a validation error.
- Select **Next** to trigger validation.
PolicyCenter displays an error message at the top of the screen.
- Click the error message to jump to the coverage term error.
PolicyCenter scrolls to the **Broadened Collision** coverage term.

Highlighting dynamic step with validation error

If validation fails on a step dynamically generated by a Wizard Step Iterator, the error message can link to that step.

You can find an example in Inland Marine, whose wizard adds steps dynamically when you add coverage parts. This example generates a page-level validation error. Through configuration, you can implement this as a field-level validation error. You can trigger this validation if you define more than one sign with the same values for location, description, sign type, and interior. However, because you cannot navigate away from this page until you fix the error, you cannot verify that the error links to the step.

To implement the page-level validation, you need to make the following changes:

- Add a method that validates the dynamic steps.
For an example, see the `handleValidationForDynamicSteps` method in `IMLineValidation.gs`.
- In the PCF file, set the `handlesValidationIssue` property to the method that validates the dynamic steps.
For an example, see `LineWizardStepSet.InlandMarine.pcf`. In the PCF file, highlight the `JobWizardStep` inside the `WizardStepIterator`. In **Advanced Properties**, the value of `handlesValidationIssue` property is the `handleValidationForDynamicSteps` method.
- Add a validation that gives the dynamic wizard step (the generic parent) as the step with the issue. For example, see `ContractorsEquipmentPartValidation.gs`:

```
Result.addError( _equipPart, TC_DEFAULT, <Display Key for error message>, "impartstep")
```

View link to dynamic wizard step with validation error

About this task

In Inland Marine, if page-level validation fails on a step dynamically generated by a Wizard Step Iterator, the error message links to that step. However, because you cannot navigate away from the step until you fix the error, you cannot verify that the link to the step works.

These steps create an unrealistic example that you can use to verify that the link from the error jumps to the invalid step. The example validates that a sign cannot have a description that matches the name of the primary insured. You can trigger the page validation when you quote.

Procedure

- In `IMSignPartValidation.gs`, add a method that validates that the sign description does not match the name of the primary insured.

```
private function signDescriptionNotPNIName() : void {
    Context.addToVisited(this, "signDescriptionNotPNIName")
}
```



```

for(sign in signPart.IMSigns) {
    if ((sign.Description != null) and (sign.Description.equalsIgnoreCase(sign.Branch.PrimaryInsuredName))) {
        Result.addError(_signPart, ValidationLevel.TC_DEFAULT,
            "Sign cannot have a description of Primary Insured Name: " + sign.Branch.PrimaryInsuredName,
            "impartstep")
    }
}
}
}

```

2. In the `validateImpl` method, add a call to the `signDescriptionNotPNIName` method.

```

override protected function validateImpl() {
    Context.addToVisited( this, "validateImpl" )
    atleastOneSign()
    signsMustBeUnique()
    signDescriptionNotPNIName()
}

```

3. Compile PolicyCenter and start the server.

View in PolicyCenter

4. Create an Inland Marine policy.
5. Advance to the **Signs** step.
6. Add a sign with description containing a name, such as "Bill Smith".
7. Go back to the **Policy Info** screen and change the name of the **Primary Named Insured** to "Bill Smith".
8. Click **Quote** to trigger validation.
In **Validation Results**, PolicyCenter displays a message that there are errors on the **Signs** page.
9. Click the error to jump to the **Signs** page.

Highlighting clause term in popup with validation error

You can link to and highlight a clause term, or other dynamically derived, field that fails class-based validation in a popup. The linking and highlighting works if validation is called when the popup is open.

For example, on a coverage, validation code checks that if the coverage term is selected, then at least one limit coverage term has a value. Because these fields are dependent upon one another, you cannot use field-level validation. If class-based validation fails, the method calls `Result.addFieldError`:

```

Result.addFieldError( _coverable, "LOBCov.LOBCovTerm.Value", TC_DEFAULT,
    <display key>, "LOBValidation")

```

The signature for `addFieldError` is:

```

function addFieldError(problematicBean : KeyableBean,
    fieldPath : String, level : ValidationLevel,
    reason : String, wizardStepId : String)

```

The method uses the `problematicBean` as the root of a field path, and appends the `fieldPath`. The path is the field to highlight in the PCF file.

The `problematicBean` parameter must be a concrete entity type, but coverages and coverage terms are abstract. Therefore use the `coverable`, `_coverable`, for the `problematicBean`.

If you have the coverage, or other clause, but not the coverable, you can get the coverable from the coverage.

Class-based validation

Class-based validation works on policy objects only. These objects must be part of the policy graph. PolicyCenter uses Gosu classes to define the validation logic. In these classes, you specify how and when to validate a particular object.

Note: The terms *object* and *entity*—while not absolutely identical—are used interchangeably throughout this topic. See the *Gosu Rules Guide* for a description of each.

In the base configuration, Guidewire provides class-based validation only for policy objects. The objects are:

-
- PolicyContact
 - Line-of-business entities
-
- PolicyLocation
 - Policy-specific entities
-
- PolicyPeriod
-

IMPORTANT: Guidewire does not recommend, nor support, the use of class-based validation as a feedback mechanism for user action. Instead, if you need to provide that feedback, use confirmation dialog boxes and user-viewable exceptions. Class-based validation is not appropriate in all contexts, nor in all situations—use it carefully and thoughtfully.

Some features of class-based validation are:

- PolicyCenter permits configuration of class-based validation on any data object, including those objects that you create by extending a base object. These objects do not need to implement the `Validatable` delegate.
- PolicyCenter performs validation as designated by the appropriate Gosu class, and when it commits data to the database.
- PolicyCenter displays validation error and warning messages in the user interface *only* if they apply to the validation level of the given context.

[See also](#)

- “Policy object validation levels” on page 37

Validation classes in the base configuration

PolicyCenter performs class-based validation on policy-related entities any time that you call for validation on the entity. In the base configuration, Guidewire provides validation classes that you can use with a specific entity. The name of the validation class contains the name of the entity to which it applies or its purpose. The validation classes

extend `PCValidationBase` or `PolicyLineValidation`, directly or through inheritance. Some examples of validation classes are listed below.

IMPORTANT: As of 2023.06, all lines of business reference implementations have been removed from PolicyCenter. Guidewire recommends that you download one of the GO products that are available on Guidewire Marketplace (<https://marketplace.guidewire.com/>), import it into Advanced Product Designer, and then use that as a starting point to define a new line of business product.

All Lines of Business

- `AnswerValidation`
- `FormPatternValidation`
- `InvariantValidation`

Personal Auto

- `PALineAssignmentValidator`
- `PALineCoveragesValidator`
- `PALineDriversValidator`
- `PALineQuickQuoteValidation`
- `PALineStepsValidator`
- `PALineValidation`
- `PALineVehiclesValidator`
- `PersonalVehicleValidation`

There is nothing particularly special about these entities. Thus, they are not `validatable` entities in the sense that rule-based validation entities are defined as `validatable`. Guidewire provides these validation classes to illustrate how to create validation classes for specific purposes. For example, the base configuration validation classes illustrate:

- How to create a Gosu class that validates an entity in the PolicyCenter data model. `PALineValidation.gs` is an example of a class that validates the `PersonalAutoLine` entity.
- How to create a Gosu class that validates a wizard step, such as the **Policy Info** step in the submission wizard. `WCPolicyInfoValidation.gs` and `BOPPolicyInfoValidation.gs` provide examples of this type of validation. See “Invoking class-based validation” on page 47 for an example of how to invoke validation from a wizard step.

You can write your own Gosu validation classes and implement validation logic on any entity that you choose. However:

- Each validation class that you create must implement the `PCValidation` interface. A convenient way to do this is to extend `PCValidationBase` or one of its subtypes.
- Each validation class that you create must provide an overridden `validate` function.

See also

- “Configuring class-based validation” on page 36

Configuring class-based validation

Through the Gosu validation classes, Guidewire provides methods to help you write checks to determine if one of the policy-specific entities is passing validation at a certain priority level.

For example, you can use the following code to check if the validation level being tested is at least at the `bindable` level:

```
if Context.isAtLeast(TC_BINDABLE)
```

Depending on the result of your validation checks, you can configure PolicyCenter to display an error or warning message to the user.

To illustrate, suppose that one class-based check validates that an underwriting company must be assigned before binding a policy. Thus, you define the following condition in a validation class to enforce the constraint and generate an error message in PolicyCenter if the condition is not met.

```
var atBindable = Context.isAtLeast(TC_BINDABLE)
if ( (Period.UWCompany == null) && (atBindable == true) {
    Result.addError(Period, "bindable", DisplayKey.get("Web.Policy.errorMessage")
}
```

PolicyCenter displays an error if both are true.

Policy object validation levels

In the base configuration, PolicyCenter defines a set of validation levels that you can use to verify how valid data is before continuing. You can also create your own validation levels. You can use these validation levels in your Gosu validation classes.

PolicyCenter defines a `ValidationLevel` typelist that you can use to set how valid data must be before continuing. You can create your own validation levels. PolicyCenter uses the `priority` attribute of the typekeys to order the validation levels. Low priority numbers are more restrictive. Thus, in the base configuration `bindable` with a priority of 6000 is more restrictive than `quotable` with a priority of 7000.

The validate-on-commit process always checks against the special validation level `default`, which is the second-least restrictive level (`priority=8000`).

In the base configuration, Guidewire defines three immutable validation levels. You cannot delete these validation levels because PolicyCenter code requires them.

Level	Priority	Description
loadsave 10000		The least restrictive validation level against which PolicyCenter validates policies entering PolicyCenter from an external application. All data must pass loadsave to be saved to the database.
default 8000		The default validation level against which PolicyCenter runs the ordinary validate-on-commit process. PolicyCenter executes the flowstep filter at this level.
quotable 7000		The level a policy must pass before it can be sent to a rating engine.

PolicyCenter provides other validation levels in the base configuration. However, as Guidewire defines these levels in the `ValidationLevel` typelist, it is possible to modify, remove, or even add to them using the Studio typelist editor.

Level	Priority	Description
bindable 6000		The level a policy must pass before it can be bound. Binding finalizes a given job and promotes an associated policy period to the main branch.
quickquotable 7200		The level a policy must pass before it is ready for a quick quote.
readyforissue 5000		The level a policy must pass before it can be issued. Issuance is the special job that occurs after submission, causing the policy to become legally in effect.

Adding new validation levels

It is possible to add additional validation levels to meet your business needs. In configuring validation levels, you can do the following:

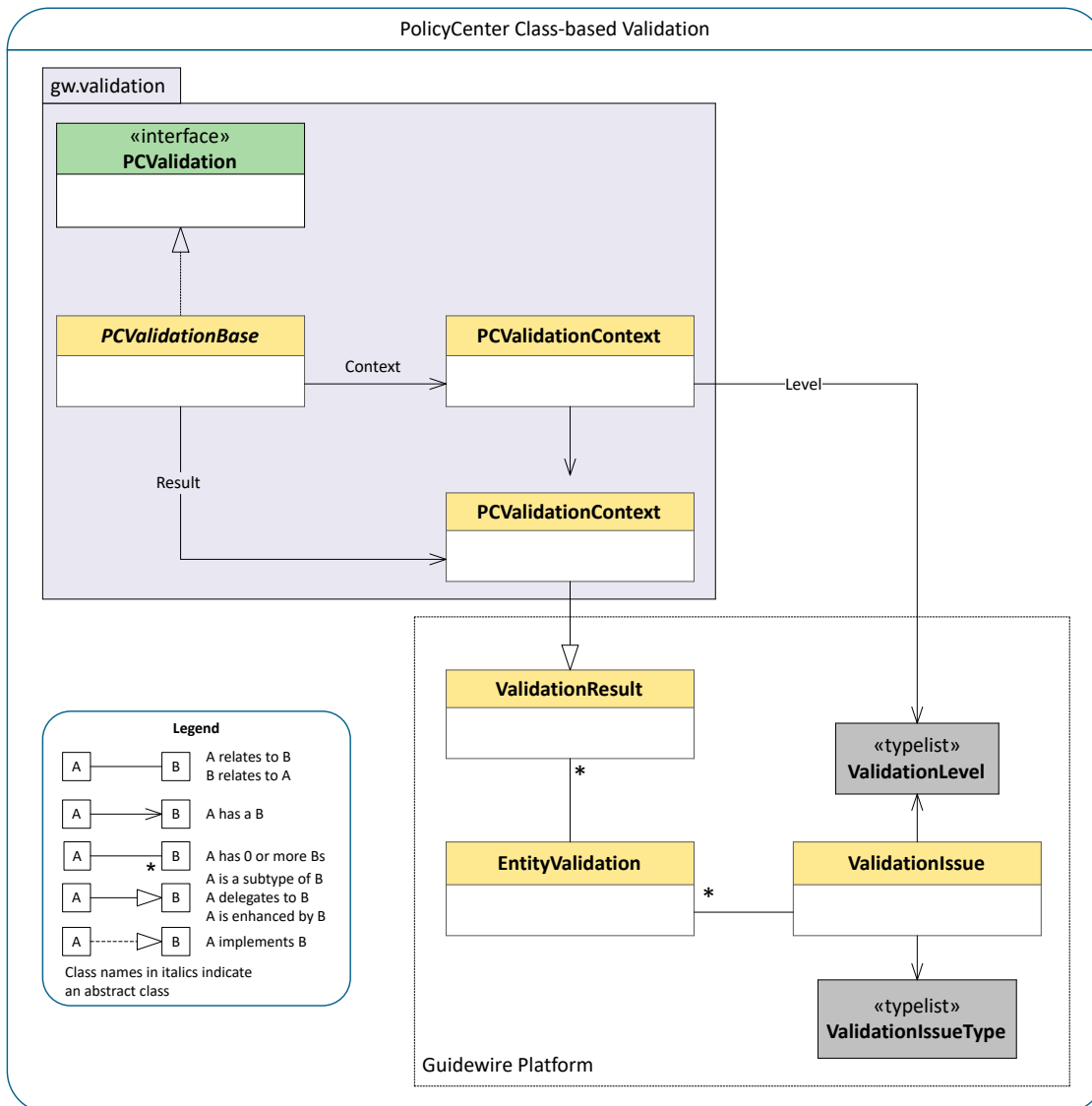
Validation level	Description
System	You can override the name and the priority of the three system validations levels: • loadsave • default • quotable
Non-system	You can modify or delete any non-system validation level at will.
Custom	You can add new validation levels to the <code>ValidationLevel</code> typelist. To do so: 1. Open the <code>ValidationLevel</code> typelist. 2. Click Add and fill in the required information.

Validation package

Guidewire provides the following validation-related classes and interfaces in the `gw.validation` package.

Class or Interface	Description
<code>PCValidation</code>	Interface that all validation classes must implement.
<code>PCValidationContext</code>	Class that takes a <code>ValidationLevel</code> and creates a new <code>PCValidationResult</code> object during initialization.
<code>PCValidationBase</code>	Abstract convenience class that takes a <code>PCValidationContext</code> instance.
<code>PCValidationResult</code>	Class that contains the object methods to use in generating warnings and errors.

The following object diagram illustrates these relationships.



PCValidation

All Gosu validation classes must implement the `PCValidation` interface. Any Gosu class (or any interface defined in Gosu) that implements `PCValidation` can perform validation logic. This interface contains a single `validate` method.

Classes that implement this interface can create methods that test for a single issue and call those method from their implementation of the `validate` method. For an example, see `PALineDriversValidator.gs`. Through object inheritance, the method `doValidate` does the validation:

```

override function doValidate() {
    qualifiedGoodDriver()
    appliedGoodDriverDiscount()
    licenseInfoRequired()
    ...
}

```

PCValidationBase

Class `PCValidationBase` is a convenience class that implements `PCValidation`. Its constructor takes a validation context (`PCValidationContext`) instance that holds the level at which to perform validation and the validation results (warnings and errors).

PCValidationBase provides a number of *getter* property methods. Some of the more important are:

```
property get Context() : PCValidationContext
property get Level() : ValidationLevel
property get Result() : PCValidationResult
```

PCValidationContext

Class PCValidationContext takes a ValidationLevel and creates a new PCValidationResult during initialization.

This class has several important methods for use in managing validation. The following list describes each briefly. However, for the most complete information, consult the *Gosu API Reference* documentation associated with the method.

Method	Description
addToVisited(validation, methodName)	Use to track a complete listing of the checks that PolicyCenter performed during the validation. The method returns false if the given validation.Name and methodName have been visited before. You can use this to determine that this particular validation method does not need to be checked again. This method only checks against the class name, not the validation object itself (using hasVisited). Thus, for example, if you are validating multiple vehicles, you cannot discern whether a method has been visited for a specific vehicle.
hasVisited(className, methodName)	Use to test whether the given combination of validation class name and method name have been seen before. If so, the method returns true.
isAtLeast(valLevel)	Use to perform a test to determine if the level specified by ValidationContext is at least valLevel. This method does not actually perform validation. Instead, you use the information to determine whether you want to perform validation.
showVisited()	Use to produce a string that lists the validation methods that were visited as validation was performed with the provided Context. You can then use this string for debugging.
resetVisited()	Resets the set of visited validation methods.
raiseExceptionIfProblemsFound()	Throws an EntityValidationException if either errors or warnings have been added to the validation context.
raiseExceptionIfErrorsFound()	Throws an EntityValidationException only if errors have been added to the validation context.

PCValidationResult

Class PCValidationResult holds the warnings and errors added by the validation implementation classes as problems are discovered. Warnings are non-blocking. The user can clear warnings and continue. In contrast, errors block further progress until the user resolves the problem.

PCValidationResult contains a number of object methods that you use in generating warnings and errors. The following list describes the more important ones briefly. However, for the most complete information, consult the *Gosu API Reference* documentation associated with the method.

Method	Description
addError	Use to add a general error message. This method takes the following arguments, of which the first three are mandatory and the last optional: - keyable bean (entity) that is the source of the error - validation level associated with the error - error reason to display to users

Method	Description
	• ID of the wizard step where the error can be corrected If you supply a wizardstepID, the method creates a link to the wizard step with ID wizardStepId if the error is not in the current step.
addFieldError	Use to add an error message specifically associated with a field on the given keyable bean as defined by the relative FieldPath. This method takes the following arguments: • keyable bean (entity) that is the source of the error • field path to the field that must be changed to resolve the error • validation level associated with the error • error reason to display to users • ID of the wizard step where the error can be corrected If you supply a wizardstepID, the method creates a link to the wizard step with ID wizardStepId if the error is not in the current step.
addWarning	Similar to the addError method, except that it generates a warning rather than an error.
addFieldWarning	Similar to the addFieldError method except that it generates a warning rather than an error.
PCValidationResult inherits several different forms of the reject method from the (platform) ValidationResult class (which it subclasses).	
Method form	Description
reject	Indicates a problem and provides an error message, but does not point to a specific field.
rejectField	Indicates a problem with a particular field, provides an error message, and indicates the correct field to fix.

Validation chaining

In the base configuration, the `gw.policy.PolicyPeriodValidation` class is an example of policy-graph validation in which one validation class successively calls (chains to) other validation classes to perform additional validation. For example, while running its validation checks for the policy period, `PolicyPeriodValidation` chains to:

- `gw.policy.PolicyContactRoleValidation` to validate the contact roles on the policy.
- `gw.policy.PolicyContactRoleForSameContactValidation` to validate multiple roles on the same contact.
- `gw.policy.PolicyLocationValidation` to validate each policy location.
- `gw.policy.PolicyLineValidation` to validate each of its lines of business.
- `gw.policy.ModifierValidation` to validate that all specified modifiers are valid modifier type codes.

Validation chaining is the process of one validation class calling another validation class to perform additional validation checks. Thus, to chain validation:

1. A `validate` method in one validation class first calls the validation methods defined in its class.
2. This `validate` method then invokes the `validate` method on another validation class (often by looping through a set of objects).
3. In turn, these classes chain to validations of entities they hold. For example, to validate all vehicles on a policy for a Personal Auto policy period, `PolicyPeriodValidation` chains to `PALineValidation` which then chains to `PALineVehiclesValidator`. To validate each individual vehicle, `PALineVehiclesValidator` chains to `PersonalVehicleValidation`. Other examples of chaining include `PALineValidation` calling `PALineCoveragesValidator`, `PALineDriversValidator`, and `PALineAssignmentValidator`.
4. After executing the `validate` methods in each of the related classes, control returns to the calling class. The calling class can then perform additional validation checks.

It is important to understand that validation chaining is not automatic. For a class to perform validation chaining, you must specifically construct it to do so. Guidewire specifically designed the `PolicyPeriodValidation` class to perform validation chaining.

Validation Chaining Example

The following graphic illustrates the concept of validation chaining. This is a simplified diagram—it does not show every validation method call in the actual implementation. The actual implementation provides an easy way to perform a full validation of the policy period.



Any change to a `PolicyPeriod` object triggers validation of that object (1 and 2 in the graphic). Initially, the `validateImpl` method defined in `PolicyPeriodValidation` (3 in the graphic) executes a number of validation methods defined in `PolicyPeriodValidation`, each of which tests for a certain condition. These methods (labeled 4 through 12) are:

- `checkPeriodDates`
- `checkQuoteNeededDate`
- `checkProductIsValid`
- `checkMinimumDataForQuote`
- `checkBillingInformation`
- `checkReportingPolicyIsCreatedWithFinalAudit`
- `checkOfficialIDs`
- `checkUnderwritingCompany`
- `checkAnswers`

After these checks complete—still in the `validateImpl` method—the code executes a loop through all of the `Location` objects in `PolicyLocations` (labeled 13). For each `Location`, it calls the `PolicyLocationValidation` `validate` method on that object (labeled 14). The validation methods within `PolicyLocationValidation` (labeled 15 through 17) are:

- `allAccountLocationsAreActive`
- `territoryCodeAgreesWithLocation`
- `checkAnswers`

The `PolicyPeriodValidation` `validateImpl` method continues by invoking the `validate` method for each policy line to perform line-specific validation:

```
Period.Lines.each(\ line -> validateLine(line, \ validator -> validator.validate() )
```

For example, if validating a Personal Auto line policy period, the code calls the `validate` method in `PALineValidation.gs` (labeled 19). This validation override first calls the local method `policyPeriodOneYearMax`. It then chains to `validate` methods in a series of other validation classes:

- `CoveragesValidator.doValidate`
- `AssignmentValidator.doValidate`
- `VehiclesValidator.doValidate`
- `DriversValidator.doValidate`

Each of these validation classes contains its own override of `doValidate`. Among these, only a few of the methods in `DriversValidator` are shown in diagram (labeled 21 through *n*). These methods perform specific checks for driver qualifications, including:

- `qualifiedGoodDriver`
- `appliedGoodDriverDiscount`
- `licenseInfoRequired`

Each of the validators loops as needed to validate all of the coverages, assignments, vehicles, and drivers that apply within the policy period being validated.

Finally, control returns to the original `validateImpl` method in `PolicyPeriodValidation` and calls additional local validation methods (labeled 24 through 29), including:

- `checkUniqueKeyBeansHaveNoDuplicates`
- `checkNamedInsuredIndustryCode`

- `checkPolicyLocationIndustryCode`

The following topics discuss parts of this example in more detail:

- “PolicyPeriodValidation: validateImpl method” on page 45
- “PolicyPeriodValidation validation checks” on page 45
- “Invariant validation checks” on page 46
- “Static validation checks” on page 47

For an explanation of Gosu block definition and syntax, refer to the *Gosu Reference Guide*.

PolicyPeriodValidation: validateImpl method

Ideally, the `validateImpl` method simply directs the flow of logic. Rather than check for problems itself, Guidewire recommends instead that the `validateImpl` method call other methods, each with a specific purpose. The `PolicyPeriodValidation.validateImpl` method demonstrates this approach. (The following is a simplified version of the method code.)

```
override protected function validateImpl() {
  if (not Context.addToVisited(this, "validateImpl")) {
    return
  }
  checkPeriodDates()
  checkQuoteNeededDate()
  checkProductIsValid()
  ...
  checkAnswers()
  ...
  Period.PolicyContactRoles.each(\ role -> new PolicyContactRoleValidation(Context, role).validate())
  accountContact.Values.each(\ roles -> new PolicyContactRoleForSameContactValidation(Context,
    roles).validate())
  locs.each(\ loc -> new PolicyLocationValidation(Context, loc, validatedTerritoryCodes).validate())
  Period.Lines.each(\ line -> validateLine(line, \ validator -> validator.validate()))
  checkUniqueKeyBeansHaveNoDuplicates()
  checkNamedInsuredIndustryCode()
  checkPolicyLocationIndustryCode()
  checkReinsuranceForBindablePeriod()
  checkPolicyAddress()
  modifiers.each(\ m -> new ModifierValidation(Context, m).validate() )
  new InvariantValidation(Context, Period).validate()
}
```

First, `validateImpl` registers the fact that it has been called. Notice how the `validateImpl` method perform no checks itself. Instead the `validateImpl` method calls other methods such as `checkPeriodDates` and `checkPolicyAddress`, all of which have a very narrowly focused purpose. Also notice that `validateImpl` chains to validations for entities held by `PolicyPeriod`:

- `PolicyContactRoleValidation`
- `PolicyContactRoleForSameContactValidation`
- `PolicyLocationValidation`
- `PolicyLineValidation`

PolicyPeriodValidation validation checks

The `PolicyPeriodValidation` check methods test for specific conditions. Guidewire strongly recommends that the method name summarize the test condition. The following code is for method `checkOfficialIDs`. This test ensures that if the validation level is at or above *bindable*, all required official IDs for the named insured have been set for all states on the policy.

Notice that the check method always registers itself with the `Context`, whatever the validation level. This is useful for debugging as it makes it clear that the method was called during the course of validation. This is true even if no message was added, which would be the case if no problems were discovered.

```
private function checkOfficialIDs() {
  Context.addToVisited(this, "checkOfficialIDs")
  var linePatterns = Period.Lines*.Pattern
```

```

    var states = Period.Lines*.CoveredStates.toSet()
    if (Context.isAtLeast(TC_BINDABLE)) {
        // check official IDs for each covered state
        for (covstate in states) {
            var checkOfficialIDs = Period.getNamedInsuredOfficialIDsForState
                (StateJurisdictionMappingUtil.getStateMappingForJurisdiction(covstate))
                .where(\id -> id.typeis PCOfficialID and linePatterns.contains(id.Pattern.PolicyLinePattern))
            for (officialID in checkOfficialIDs) {
                officialID.validateOfficialID(Period, Result)
            }
        }
    }
}

```

Invariant validation checks

The `validateImpl` method in the `gw.policy.PolicyPeriodValidation` class also calls the following code:

```
new InvariantValidation(Context, Period).validate()
```

This code initiates what Guidewire calls *invariant* checks. PolicyCenter uses these checks to determine whether a policy has been modified in ways that violate implied constraints to the data model.

The method call creates a new `InvariantValidation` object, whose `validate` method initiates the actual invariant tests and checks.

In the base configuration, these tests perform the following entity checks:

Entity	PolicyCenter checks that...
Coverage	- Required coverage terms are not null. - Direct coverage terms are within minimum and maximum constraints.
BusinessAutoLine	If the Business Auto Line exists, then the correct policy line pattern is applied.
CoverageSymbolGroup	- The coverage symbol group is compatible with policy line pattern. - The coverage symbol is compatible with coverage symbol group pattern. - The coverage symbol pattern does not have multiple instances. (Each coverage symbol must appear only once).
Modifier	The rate modifier value is within the allowed minimum and maximum values.
PolicyLine	- A policy line subtype exists for a policy line. - A policy line pattern exists for a policy line. - The policy line type matches the subtype of the policy pattern.
Product	The referenced policy line pattern is compatible with this product.
RateFactor	- The rate factor pattern is compatible with the schedule credit pattern. - The rate factor is within the allowable minimum and maximum values

Constraint Errors

If a check indicates that the user-defined product model violates one or more constraints, PolicyCenter generates an appropriate error message.

PolicyCenter stores the invariant errors in display keys. In Studio, in **Project** view, navigate to **configuration > config > Localizations > c_Resource Bundle 'display'**. Assuming that you are modifying the display key for U.S. English, open `display.properties` and search for the following text string:

```
Java.Invariant.*
```

For example:

```
Java.Invariant.Modifier.RateAboveUpperBound
```

contains the display key:

The rate modifier value “{0}” cannot exceed the allowed maximum “{1}”.

During product model verification, PolicyCenter replaces the variables {0} and {1} with the actual values.

Static validation checks

Within a validation class, you can define static methods that define the specific validation checks to perform at a specific wizard step, reducing the need for full-policy inspections. For example, the **Drivers** steps on the submission wizard for Personal Auto contains the following validation code (on the `beforeSave` property of the `LineWizardStepSet.PersonalAuto.pcf`):

```
gw.lob.pa.PALineStepsValidator.validateDriversStep(policyPeriod.PersonalAutoLine)
```

This code invokes the `validateDriversStep` method in `PALineStepsValidator` before committing the **Drivers** wizard step.

```
static function validateDriversStep(paline : PersonalAutoLine) {
    PCValidationContext.doPageLevelValidation(\ context -> {
        var validator = new PALineValidation(context, paline)
        validator.CoveragesValidator.validate()
    })
}
```

Notice the following:

1. Wrapper method `doPageLevelValidation` provides the exception reporting necessary if a validation check fails to pass. It is the `PCValidationContext.doPageLevelValidation` method that calls the `raiseExceptionIfProblemsFound` method if any of the `validator.DriversValidator` validation checks fail. The `raiseExceptionIfProblemsFound(context)` method instructs the PolicyCenter rendering framework to display the appropriate warning or error automatically if there is an exception condition.
2. Validation class `PALineDriversValidator` provides the validation checks for `validateDriversStep`. This validation class defines a number of validation checks, not all of which the `validateDriversStep` method invokes.
3. The actual validation takes place in the `doValidate` override located in the `CoveragesValidator` class.

For a discussion of how to use this validation method to raise exceptions within PolicyCenter, see “Example: Invoking validation in a job wizard step” on page 48.

Invoking class-based validation

PolicyCenter does not automatically invoke class-based validation by virtue of the database commit cycle. Instead, you must explicitly invoke class-based validation in configuration code. (The PolicyCenter user interface framework automatically handles the rendering of all errors and warnings raised through exceptions.)

PolicyCenter runs full validation on the policy graph at critical points in the job processes, such as before a quote and before binding. (At any time, you can also invoke class-based validation from PCF files, job processes, wizard steps, workflow steps, integration plugins, and from any Gosu code.)

Use the following to invoke class-based validation from a particular PolicyCenter location:

Location	Use
Wizard step	The <code>beforeSave</code> property for that step
Pop-up	The <code>beforeCommit</code> property for that pop-up

The PolicyCenter user interface framework automatically handles the rendering of all errors and warnings raised through exceptions.

Example: Invoking validation in a job wizard step

This example shows the process of invoking a static validation method from a job wizard step. Specifically, it illustrates how to link the various Gosu validation classes and methods to the **Vehicles** step in the Commercial Auto **Submission** wizard. The page fails a validation check because there is no value set for the **Length of Lease/Rental (months)**.

To reproduce the validation check, follow the instructions in “Invoke validation in a job wizard step” on page 48.

To see how the PCF files trigger the validation check, see “Trace the validation path” on page 48.

Invoke validation in a job wizard step

About this task

These steps walk through the process of invoking a static validation method from a job wizard step. Specifically, it illustrates how to link the various Gosu validation classes and methods to the **Vehicles** step in the Commercial Auto **Submission** wizard. The page fails a validation check because there is no value set for the **Length of Lease/Rental (months)**.

Procedure

1. Start a Commercial Auto submission. In these steps, create the submission from the Wright Construction account in sample data.
2. On the **Commercial Auto Line** screen, enter the following:

Product	Commercial Auto
Fleet	Fewer than 10 units

3. Advance to **Vehicles** and click **Create Vehicle**.
4. Fill in all required fields:

Garaged At	846 Yount Ln., Hollywood, CA
Vehicle Type	Passenger Vehicles
VIN	abc
Cost	9000
Class	0423
Leased or Rented	Yes
Length of Lease/Rental (months) <none>6 months or greaterLess than 6 months	Do not enter a value

5. Click **OK**.
In the **Validation Results** panel, PolicyCenter displays an error message that length of lease is required.

What to do next

“Trace the validation path” on page 48 in PCF files in Studio.

Trace the validation path

Before you begin

“Invoke validation in a job wizard step” on page 48 in PolicyCenter

About this task

The `beforeSave` property invokes validation on the **Vehicles Details** card of the **Vehicles** page. These steps trace the path through PCF files to find this property.

Procedure

1. In Studio, open the `SubmissionWizard` PCF file.
This is the main PCF file of the PolicyCenter Submission wizard.
2. In the PCF file, select the **Wizard Step Set Ref** with the **Shared section mode**
3. Set the **Shared section mode** to `BusinessAuto` to select the mode for Commercial Auto.
When you select `BusinessAuto`, Studio displays `LineWizardStepSet.BusinessAuto` PCF file nested in the **Wizard Step Set Ref**. This PCF file governs the Commercial Auto **Vehicles** page in PolicyCenter.
4. Double-click the **Wizard Step Set Ref** to open `LineWizardStepSet.BusinessAuto`.
5. In `LineWizardStepSet.BusinessAuto`, select **JobWizardStep: BusinessVehicles**.
6. At the bottom of the screen, scroll to view the `beforeSave` property in **JobWizardStep > Advanced Properties**.
The value of the `beforeSave` property invokes validation on the **Vehicles Details** card of the **Vehicles** page.

Configuring contingencies

PolicyCenter manages work associated with a creating or changing a policy through policy transactions. A policy transaction is completed when the policy is bound and issued, or when the policy transaction is withdrawn. However, the policy may still require additional work. You can use *contingencies* to manage this additional work and take actions, including policy change or cancellation transactions, if conditions are not met in a timely manner.

Through configuration, you can change the default behavior of contingencies. You can change the object model and the code that creates contingencies and associated documents, notes, and activities. You can modify the batch process that starts actions on contingencies.

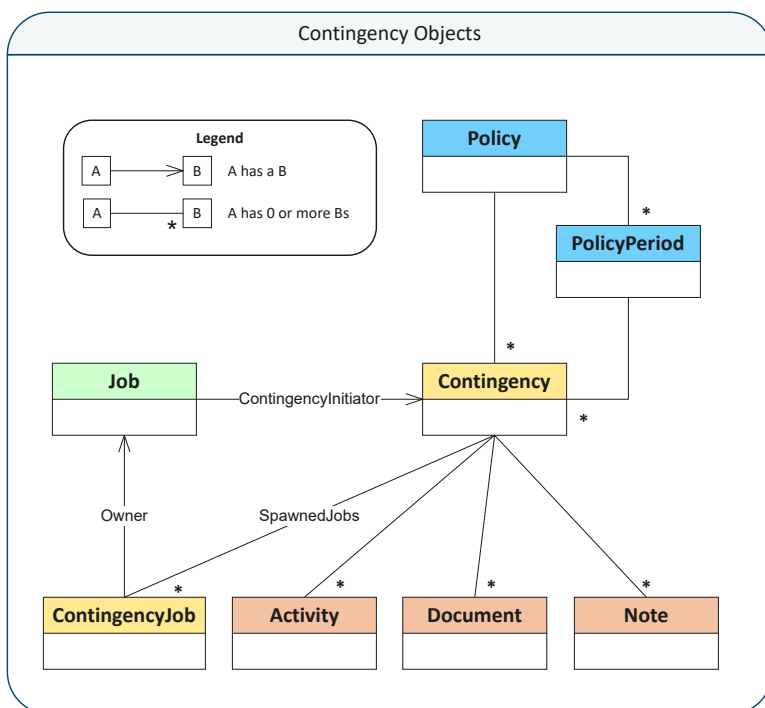
Through configuration, you can add contingency actions that start job types other than policy change and cancellation. You can add short rate calculation and custom calculation methods in addition to flat and prorated.

See also

- *Application Guide*

Contingency object model

The following illustration shows Contingency and related objects.



Contingency

Each Contingency has a number of properties, including:

Property	Description
Title	Brief description enabling a user to quickly understand the type of contingency.
Description	Longer description which can include specific details of this instance. Can include, for example, the conditions under which the contingency can be resolved.
Status	Type key indicating if this contingency is still active, has been successfully resolved, or failed to be resolved in a timely manner. After contingencies have been processed by the Handle Unresolved Contingency work queue, their status is ACTION_INITIATED. Values of FAILED or ACTION_INITIATED appear on the Contingency screen under Closed Without Satisfactory Resolution.
Action	What action to start if the contingency is not resolved prior to the action start date. The action can start a policy transaction.
ActionStartDate	When to initiate action if the contingency is not resolved.
DueDate	The due date for the contingency. If the contingency is not closed by this date, batch processing changes the status to failed, and closes the contingency.
CloseDate	When the contingency was closed.
CloseUser	The user who closed the contingency.
Policy	The policy associated with the contingency.
PolicyPeriod	Generally the policy period that caused the contingency to be associated with the policy. If the contingency was attached directly to the policy, policy period is null.

Contingency Job

If a contingency is not resolved in a timely manner, Unresolved Contingency batch processing initiates action on the contingency. ContingencyJob is the job initiated by the action.

Contingency Gosu class

The main configuration class for contingencies is `ContingencyEnhancement.gsx`. This file contains methods that set the action start date, define how notes and activities are added to a contingency, define how contingencies are resolved and waived.

Action Start Date

You can specify action start dates in the `updateActionStartDate` method. In the base configuration, the values are:

Policy transaction	Action start date
Policy change	5 day before the due date
Cancellation	30 days before the due date

Notes and Activities

Code that adds notes and activities to a contingency is in the `createContingencyActivity` and `createContingencyNote` methods. For activities, you can customize the owner, title, and description, among others. For notes, you can customize the subject and author, among others.

In the base configuration, notes and activities are related to the contingency through foreign keys. Through configuration, notes and activities can be related to other objects.

Contingency batch processing

The batch processes associated with contingencies are:

- **Handle Unresolved Contingency** – Initiates action on pending contingencies. See the *InsuranceSuite Server Administration*.
- **Recalculate Contingency Action Start Date** – Recalculates the action start date on all unresolved contingencies. See the *InsuranceSuite Server Administration*.

Configuring the Spotlight integration

The Spotlight integration in PolicyCenter provides access to interactive and risk assessment services available in Guidewire Spotlight. Spotlight provides a map in PolicyCenter. If you update the address, PolicyCenter refreshes the map. Spotlight provides risk information associated with a location.

In the base configuration, Spotlight is integrated with the commercial property line of business, and is accessible from locations. You can add Spotlight interactive and risk assessment services to other lines of business.

The risk assessment service provides data for the selected risk profile, configured specifically for the customer.

Note: For information about obtaining access to Spotlight, contact your Guidewire support representative.

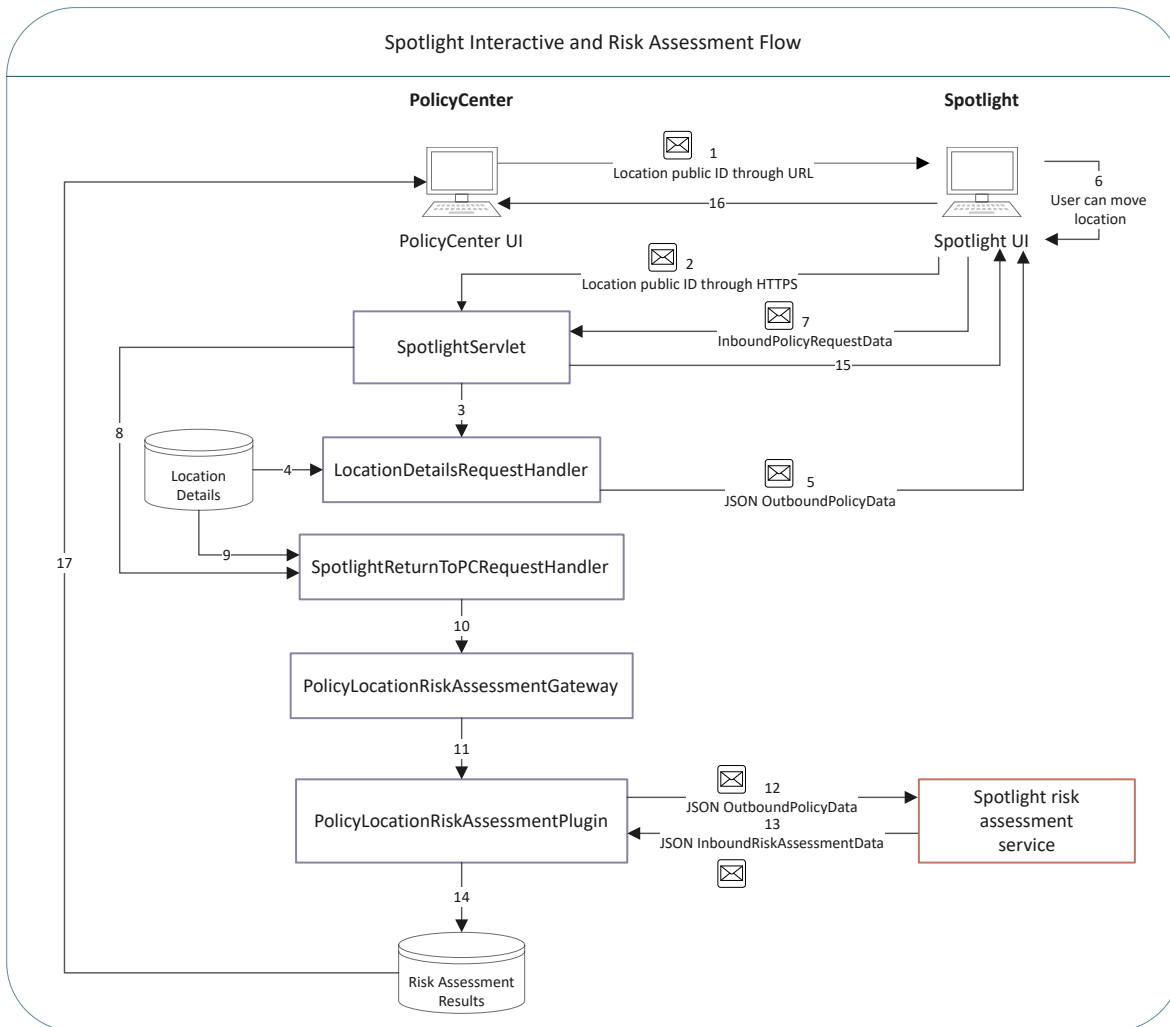
Communication with Guidewire Spotlight must be secured by serving PolicyCenter over HTTPS. Failure to do so may result in various browser warnings as well as potentially transmitting sensitive information over insecure channels. Therefore, implement HTTPS and disable HTTP on PolicyCenter servers.

For Spotlight services, in particular the interactive integration, to function properly and securely, the PolicyCenter application server must be run using HTTPS with a valid certificate.

Accessing risk assessment interactively

When accessing Spotlight interactive service, you can go to the Spotlight application from PolicyCenter by clicking **View in Spotlight** on the **Location Information** screen. In Spotlight, you can examine the location and its surroundings, and perform risk evaluations. You can also move the location pin for longitude and latitude. Spotlight risk assessment service provides risk data. Then you can return to PolicyCenter.

The following illustration shows the Spotlight interactive and risk assessment flow in PolicyCenter.



For interactive risk assessment flow, the following steps correspond to the numbers in the previous illustration.

1. **PolicyCenter** When the user clicks **Evaluate in Spotlight**, PolicyCenter sets a variable indicating that this button was clicked. Then PolicyCenter stores the current policy location data into the `OutboundLocationRiskAssessmentTempStore`, `OutboundPeriodRiskAssessmentTempStore`, and `OutboundReinsurableRiskAssessmentTempStore` temporary objects. PolicyCenter uses these objects to match the data that the risk assessment service returns.

For new locations that do not have latitude and longitude, PolicyCenter sends a formatted address. The `AddressFormatter` class formats the address into a globalized address. In your customization, pass the fields you require into the formatter. In the base configuration, the fields are stored in a static function called `GeoCodableAddressFields` in `SpotlightConfigParameters.gs`.

PolicyCenter passes three parameters to Spotlight through the URL. These parameters are used in requests to transfer JSON data from Spotlight to PolicyCenter.

- `policySystemServer` – Base URL which prepends the other two page parameters
- `encodedLocationDetailRequestPage` – Page for requesting location details from PolicyCenter including location public ID
- `encodedReturnPage` – Page for returning from Spotlight to PolicyCenter to display the risk assessment calculated by Spotlight

The URL is in the following format:

```
https://SpotlightURL/urlPrefix/policySystemServer/
paramsUrlFragment/encodedLocationDetailRequestPage/returnUrlFragment/encodedReturnPage
```

2. **Spotlight** After PolicyCenter sends the public ID of the `LocationForRiskAssessment` object to Spotlight, Spotlight sends a request to PolicyCenter to retrieve the details of that location. The request is made through the URL:

```
policySystemServer/encodedLocationDetailRequestPage
```

In PolicyCenter the request goes to `SpotlightServlet.gs`.

3. **PolicyCenter** The `SpotlightServlet` delegates to `LocationDetailsRequestHandler.gs`.
4. The `LocationDetailsRequestHandler` gets the location details.
5. The `LocationDetailsRequestHandler` responds to the request by sending JSON data from `OutboundPolicyData.gs`. Jackson is used to serialize and deserialize the JSON data. This class implements the `JacksonEnabledSerializable` interface so that Jackson works with Gosu.

Note: Guidewire recommends that you do not use polymorphic deserialization, or that you use it only with annotations on an individual class basis. Do not enable global default typing.
6. **Spotlight** At this point, the user can modify the location pin or risk profile in Spotlight, then click **PolicyCenter** to return to PolicyCenter. The Spotlight service uses the supplied latitude and longitude if present to identify the location, in preference to the address information.
7. **PolicyCenter** When the user initiates the return to PolicyCenter, a request is sent to the URL:

```
policySystemServer/encodedReturnPage
```

The request arrives at `SpotlightServlet.gs`.

The request includes the location data as JSON data in a request parameter which is parsed with Jackson and stored in `InboundPolicyRequestData.gs`.

8. The `SpotlightServlet` passes control to `SpotlightReturnToPCRequestHandler`.
9. `SpotlightReturnToPCRequestHandler` matches the JSON data with temporary objects in database.
10. The request handler calls `PolicyLocationRiskAssessmentGateway` with the matched temporary objects. The gateway ensures that the plugin methods are called correctly, checks permissions, and handles errors.
11. The gateway calls the `PolicyLocationRiskAssessmentPlugin`.
12. **Spotlight** `PolicyLocationRiskAssessmentPlugin` calls the Spotlight risk assessment service. The data is sent by `OutboundPolicyData`.
13. The risk assessment data is returned with `InboundRiskAssessmentData`.
14. **PolicyCenter** The data is persisted in `InboundLocationRiskAssessmentTempStore`. The handler returns control to `SpotlightServlet`.
15. **Spotlight** The servlet returns control to Spotlight.
16. **PolicyCenter** Spotlight directs the URL back to PolicyCenter.
17. In PolicyCenter, the **Location Information** screen displays the risk assessment data below the map. The risk assessment data includes **Latitude**, **Longitude**, and other fields.

When the user returns to PolicyCenter, the page reloads the `OutboundLocationRiskAssessmentTempStore` object which contains the risk assessment data in `InboundLocationRiskAssessmentTempStore`.

PolicyCenter reloads the thumbnail map to display the latitude and longitude picked in Spotlight if moved. When there is new risk assessment data, PolicyCenter displays a message below the thumbnail map saying **Spotlight returned new data**. You can choose whether or not to **Use this new data**. The data in `InboundLocationRiskAssessmentTempStore` object is added into the `LocationRiskAssessment` object if you choose to use the new data and click **Ok**. If you click

Cancel, the risk assessment data is not stored into the policy graph. You can view the location in Spotlight multiple times and move the location pin each time resulting in different risk assessment values. Only the last risk assessment value is preserved into the policy graph when you select **Ok**.

You also see the risk assessment details after visiting Spotlight and clicking **Evaluate**. Next to the label **Risk Profile: US Large Commercial**, click **Show** to view risk assessments and details.

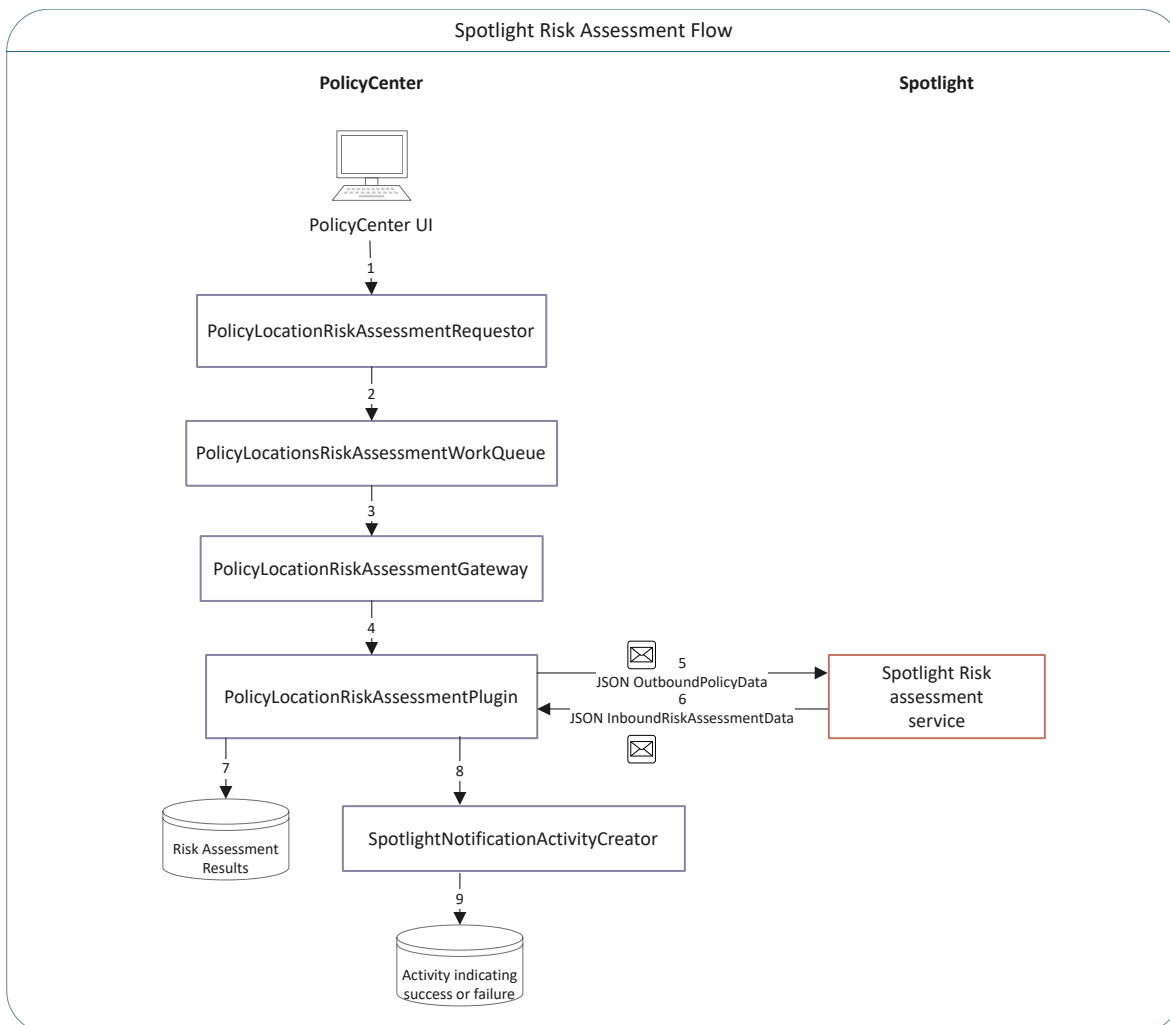
When the new risk assessment is accepted, a `LocationRiskAssessment` object is created and populated with risk assessment, including the hash of the data used to generate this risk assessment.

Accessing risk assessment non-interactively

You can access risk assessment non-interactively by clicking **Update Risk Evaluations** on the **Risk Analysis** screen. PolicyCenter runs risk assessment for all locations on the policy period.

When accessing risk assessment non-interactively, PolicyCenter calls the Spotlight risk assessment service, saves the results, and creates an activity indicating whether the risk assessment succeeded or failed. This occurs without providing the user access to Spotlight interactive services.

The following illustration shows the flow when accessing Spotlight non-interactively from PolicyCenter.



For non-interactive risk assessment flow, the following steps correspond to the numbers in the previous illustration.

1. The user selects **Update Risk Evaluations** and PolicyCenter calls the `PolicyLocationRiskAssessmentRequestor` class.

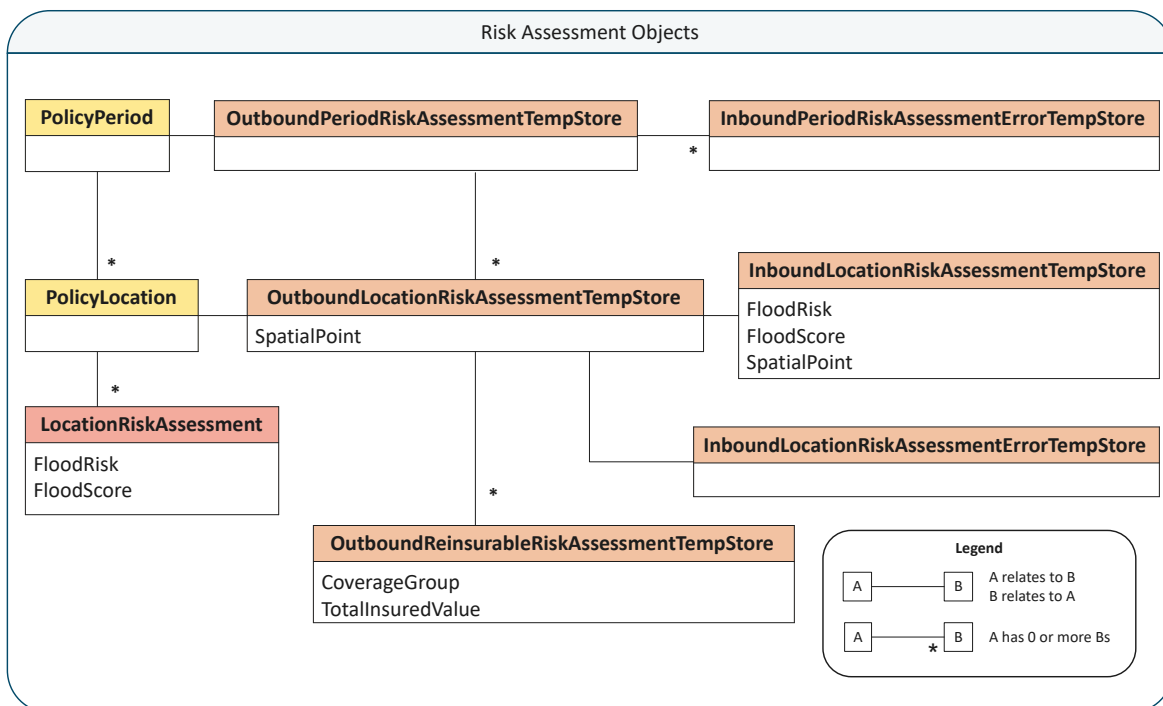
This class requests risk assessments for all the locations on a policy period, by calling the `enqueueForLocationsRiskAssessment` method. This method creates a `RiskEvaluationWorkItem` which refers to the `PolicyPeriod`. It also sets the `RequestingUser` field to the current user. The workItem is created and committed in the `PolicyPeriod` bundle.

The class also provides the static query `isLocationsRiskAssessmentInProgress` method to determine whether there is already an active `RiskEvaluationWorkItem` for this policy period.

2. Policy Locations Risk Assessment batch processing writes items to the Policy Locations Risk Assessment work queue. This work queue starts risk assessment for all locations on the policy.
This work queue does not have a finder. Therefore, batch requests are created explicitly by the `PolicyLocationsRiskAssessmentRequestor` class.
3. The work queue calls `PolicyLocationRiskAssessmentGateway` class. The gateway ensures that the plugin methods are called correctly, checks permissions, and handles errors.
4. The gateway class calls the `PolicyLocationRiskAssessmentPlugin` plugin.
5. The plugin calls the Spotlight risk assessment service. The data is sent by `OutboundPolicyData`.
6. The risk assessment data is returned with `InboundRiskAssessmentData`.
7. The data is persisted in `InboundLocationRiskAssessmentTempStore` objects.
8. For each location, the `SpotlightNotificationActivityCreator` class creates an activity indicating whether risk assessment succeeded or failed.
9. Users can view the activities in PolicyCenter.

Risk assessment object model

The following illustration shows some of the objects and properties associated with risk assessment.



The `PolicyLocation` provides access to risk assessment data for a policy location through an array of `LocationRiskAssessment` objects. The `PolicyLocation` also has a one-to-one relationship with risk assessment data to be sent to Spotlight in the `OutboundLocationRiskAssessmentTempStore` object.

Location risk assessment object

The LocationRiskAssessment object provides risk assessment data for a policy location. LocationRiskAssessment is an effective dated object.

Property	Description
AssessmentDate	Time that the assessment was given to PolicyCenter.
BasedOnValue	Pointer to the LocationRiskAssessment object on which this object is based.
Fixed	Pointer to this object across all branches and policy periods.
FloodRisk	The flood risk set to Unknown, Low, Medium, or High. This provides an example of parsed data from Spotlight.
FloodScore	A score for the flood risk. This provides an example of risk score from Spotlight.
SpatialPoint	Latitude and longitude of the risk evaluation.

Risk assessment temporary objects

Inbound objects store information that the Risk Assessment Service provides to PolicyCenter. Outbound objects store information that PolicyCenter sends to the Risk Assessment Service.

- OutboundPeriodRiskAssessmentTempStore
- OutboundLocationRiskAssessmentTempStore
- OutboundReinsurableRiskAssessmentTempStore
- InboundLocationRiskAssessmentTempStore
- InboundLocationRiskAssessmentErrorTempStore
- InboundPeriodRiskAssessmentErrorTempStore

Inbound location risk assessment temporary store object

The InboundLocationRiskAssessmentTempStore object stores information that the Risk Assessment Service provides to PolicyCenter. This object contains the latitude and longitude of the location and specific assessments configured to be parsed, such as flood data which is provided as an example.

Property	Description
AssessmentDate	Time that the assessment was given to PolicyCenter.
FloodRisk	The flood risk set to Unknown, Low, Medium, or High. This provides an example of parsed data from Spotlight.
FloodScore	A score for the flood risk. This provides an example of risk score from Spotlight.
RiskProfileCode	The code in the risk assessment service indicating how the location risk was assessed.
SnapshotReportURL	Reference to report of how risk assessment took place.
SpatialPoint	Latitude and longitude of this location.
UnparsedRiskAssessmentResponse	The raw text from the risk assessment.

Inbound location risk assessment error temporary store object

The InboundLocationRiskAssessmentErrorTempStore object holds an error associated with a location for a failed request to the risk assessment service.

Inbound period risk assessment error temporary store

The `InboundPeriodRiskAssessmentErrorTempStore` object holds an error associated with a failed request to the risk assessment service.

Outbound location risk assessment temporary store object

The `OutboundLocationRiskAssessmentTempStore` object store location information that PolicyCenter sends to Risk Assessment Service. This object contains the address and latitude and longitude of the location.

Property	Description
<code>AddressLine1</code>	The first line of the location address. This object also includes address properties, including City, County, State, and Country.
<code>SpatialPoint</code>	Latitude and longitude of this location.

Outbound period risk assessment temporary store

The `OutboundPeriodRiskAssessmentTempStore` object represents the risk assessment request which contains one or more locations for risk assessment. This object stores request level errors.

Outbound reinsurable risk assessment temporary store object

The `OutboundReinsurableRiskAssessmentTempStore` object store reinsurance information for the associated `OutboundLocationRiskAssessmentTempStore` object. PolicyCenter sends this information to the Risk Assessment Service.

Property	Description
<code>CoverageGroup</code>	Coverage group type for reinsurance.
<code>TotalInsuredValue</code>	The total insured value of the risk.

Configuring risk assessment in the Location Information screen

The **Location Information** screen displays risk assessment information and a thumbnail map.

Risk assessment information

In Commercial Property, the **Location Information** screen displays risk assessment information. When the user enters this screen, `CPLocationPopup.pcf`, the screen checks whether the current location data differs from what was used to generate the existing `LocationRiskAssessment`, if one exists. If there is a difference, the screen displays a warning indicating that the current risk assessment may be out-of-date, or stale. This warning also appears on the **Buildings and Locations** screen for any location with a stale risk assessment. The `LocationRiskAssessment.isRiskAssessmentStale` property indicates whether the risk assessment is stale or not.

To perform this check for stale risk assessments, a non-persisted `OutboundLocationRiskAssessmentTempStore` is created and populated with the current location data. The spatial point and hash value of this object are calculated and compared with the spatial point and hash value of the latest `LocationRiskAssessment`. The hash value is available in the `LocationRiskAssessment.InputChecksum` property.

The hash value is determined by a list of properties on the `OutboundLocationRiskAssessmentTempStore`. The list of properties is specified in the `RiskAssessmentInputDataHasher` property in the `RiskAssessmentInputDataHasher` class. The Location Risk Assessment plugin calls `RiskAssessmentInputDataHasher`.

The hash value is calculated using the following list of properties:

- Branch.PeriodStart
- Branch.PeriodEnd
- GeocodeableAddress
- City
- County
- PostalCode
- State
- Country

You can configure the list of fields in the hash to meet your needs.

Note: Do not include the `SpatialPoint` among the fields used for hashing. The spatial point on the `OutboundLocationRiskAssessmentTempStore` does not reflect changes in spatial point that occur while the user is in Spotlight.

Risk assessment thumbnail map

The **Location Information** screen in PolicyCenter displays a thumbnail map embedded in the `CPLocationPopup.pcf`. The map is provided by Spotlight and has a pin dropped on the current best-known location. If the location has latitude and longitude, then the pin is placed on that location. Otherwise, the address fields are used to place the pin. You can update the map to reflect changes to coordinates or address by clicking **Update Map**. As previously mentioned, address-only changes do not affect the map if the location already has coordinates.

The embedded map uses the template panel to link to the Spotlight client and enables PolicyCenter to display the embedded map. The `createSpotlightMapURL` method in `SpotlightConfigParameters.gs` creates the URL.

Note: The thumbnail map content returned by Spotlight is licensed and subject to change. You can render the content but do not parse it.

Deleting temporary risk assessment objects

The risk assessment integration creates temporary objects for each policy location when accessing Spotlight interactive and risk assessment services. PolicyCenter deletes these objects from the database in the following ways:

- **Bound policy** – During binding, right before it is bound, PolicyCenter deletes temporary objects that are associated with the policy locations. This is done in `PolicyPeriodBaseEnhancement.gsx` in the `cleanUpLocationForRiskAssessment` method.
- **Unbound Policy** – For policies that are withdrawn, not taken, or declined, PolicyCenter deletes the temporary objects. The code that deletes the objects is not accessible.
- **Bound policies and Spotlight interactive** – Selecting **Evaluate in Spotlight** in a bound policy creates temporary objects. These temporary objects are deleted by Purge Risk Assessment Temporary Store batch processing.

See also

- “Risk assessment temporary objects” on page 60
- *InsuranceSuite Server Administration*

Risk assessment configurable classes

The risk assessment integration includes a number of classes which you can modify. The following are some of the main classes in the integration.

Class	Description
SpotlightServlet.gs	This servlet is used to handle all HTTP requests between PolicyCenter and the Spotlight interactive service. This servlet has two request handlers: • URL contains <code>LocationDetails.do</code> – A request handler sends the formatted address and spatial point to Spotlight interactive service so that PolicyCenter displays the location on the Spotlight interactive map. • URL contains <code>SpotlightReturn.do</code> – A request handler takes the data assessed on the Spotlight Interactive side, and returns the new risk assessment data to PolicyCenter. The handler redirects Spotlight interactive back to PolicyCenter and the correct wizard step.
LocationDetailsRequestHandler.gs	Handles HTTP requests for <code>locationDetails</code> and provides location data to Spotlight.
SpotlightReturnToPCRequestHandler.gs	Handles HTTP requests for <code>SpotlightReturn</code> . Takes Spotlight location details, delegates to the Spotlight Policy Location Risk Assessment plugin gateway, then redirects back to PolicyCenter.
SpotlightConfigParameters.gs	Specifies configuration parameters for Spotlight.
SpotlightNotificationActivityCreator.gs	Users to whom an activity is sent. In the base configuration, the producer and underwriter associated with the policy transaction.
LocationRiskAssessmentBroker.gs	Handles temporary objects for policy location and location risk assessments in <code>CPLocationPopup.pcF</code> and toggles the risk assessment information display.
InboundPolicyRequestData.gs	Contains data Spotlight passes to PolicyCenter when longitude and latitude are adjusted in Spotlight.
OutboundPolicyData.gs	Contains data PolicyCenter passes to Spotlight to interactively display location details in Spotlight.
InboundRiskAssessmentData.gs	Contains data the Spotlight risk assessment service passes to PolicyCenter after calculating risk assessment values.
RiskAssessmentInputDataHasher.gs	Contains the list of fields that are used determine if a previous risk assessment was performed on the same location and under similar conditions.

Spotlight service authentication for risk assessment

The Spotlight Policy Location Risk Assessment plugin uses the `ClientLoginService` service, provided by `loginServiceAPIStandalone.jar`, using the credentials in `LocationRiskAssessmentPlugin` for authentication. The credentials are specified in the plugin in the `username` and `password` parameters. In the base configuration, the values for these parameters are provided using external server configuration. The parameters do not have default values. The authorization cookie returned by the service is stored in the `gwAuth` variable in `SpotlightConfigParameters.gs`. The plugin use that cookie to access the risk assessment service.

The Spotlight Policy Location Risk Assessment plugin uses the same `gwAuth` cookie for a specified period of time before it requires a new authorization cookie. The maximum login session time is configurable by setting `SPOTLIGHT_MAX_LOGIN_SESSION_MILLISEC` in `SpotlightConfigParameters.gs`. In the base configuration, the default login session time is 1 hour.

If the risk assessment service connection is refused with an HTTP status code 403 with the current `gwAuth` cookie, then the plugin resets the login session expiration time. The plugin obtains a new authorization cookie before making a subsequent call to the risk assessment service.

The plugin communicates with Spotlight using secure HTTPS communication.

[See also](#)

- *InsuranceSuite Security Administration*

Risk assessment error handling

The Spotlight integration includes a framework that handles error conditions between PolicyCenter and Spotlight. The `SpotlightErrorCollector.gs` class collects errors encountered while getting risk assessment details for a `PolicyLocation` from Spotlight. This class passes the error collector to the `RiskAssessmentErrorHandler` which has a method to handle the errors in the collector.

The error collector passes the errors to one of two error handler implementations:

`SpotlightInteractiveErrorHandler` and `SpotlightNonInteractiveErrorHandler`, for handling errors in interactive and non-interactive modes, respectively. Each error is associated by a type specified in the type list `RiskAssessmentError.tti`.

Request and location level errors

All errors are specified in the `RiskAssessmentError` typelist and have an associated category from the `RiskAssessmentErrorType` typelist. The error types are `Request` and `Location`. Request level errors are errors associated with the entire request, and not the locations. These are errors such as the connection was refused, insufficient permission, or something that causes the entire request to fail. Location level errors are errors associated with a specific location. An example of a location level error is an invalid address.

Request level errors are stored in `InboundPeriodRiskAssessmentErrorTempStore` temporary objects. Location level errors are stored in `InboundLocationRiskAssessmentErrorTempStore` temporary objects. These errors are linked to their corresponding `OutboundPeriodRiskAssessmentTempStore` and `OutboundLocationRiskAssessmentTempStore` objects so that these errors can be handled by the `RiskAssessmentErrorHandler`.

In interactive and non-interactive modes, some errors are written to the log file.

In interactive mode, the errors are stored in their respective temporary objects and appear when the locations are revisited in the user interface.

In non-interactive mode, an activity is created for each job and error type. The activity is assigned it to the underwriter and the producer user roles on the current job. You can configure the roles that the activities are assigned to in `SpotlightNotificationActivityCreator.gs`.

Enabling Spotlight services

Access to Spotlight services requires a credential, consisting of a username and password. The `SpotlightPolicyLocationRiskAssessmentPlugin` plugin signs into Spotlight using this credential. You specify the credential in `SpotlightCredentialProvider.gs`. In the base configuration, this file contains a placeholder.

IMPORTANT: To determine whether your Guidewire PolicyCenter license agreement includes Spotlight risk assessment services, contact your Guidewire sales representative. To obtain Spotlight URLs, user names, and credentials, contact your Guidewire support representative.

Risk assessment configuration parameters

Risk assessment parameters in `config.xml` enable you to set the URL for log in to Guidewire Spotlight and to access risk assessment services.

To enable access to Spotlight risk assessment services, set `RiskAssessmentIntegrationEnabled` to `true`, and then enable or disable the individual services by setting configuration parameters for those individual services. If `RiskAssessmentIntegrationEnabled` is `false`, then all Spotlight risk assessment services are disabled regardless of whether the individual service is enabled.

Set URLs to point to the risk assessment services in the `SpotlightRiskAssessmentServiceURL`, `SpotlightInteractiveServiceURL`, and `SpotlightThumbnailMapURL` configuration parameters.

For individual Spotlight risk assessment services, the configuration parameters are:

Risk assessment service	Parameter
Thumbnail map	RiskAssessmentThumbnailMapEnabled
Request for single evaluation	SingleLocationRiskAssessmentEnabled
Request for multiple location evaluation	MultipleLocationRiskAssessmentEnabled

If risk assessment access is disabled for a service, PolicyCenter provides alert messages. In the base configuration, the service is inaccessible from the user interface when the service is unavailable.

However, if a service is disabled and access is attempted through the user interface, PolicyCenter displays a message indicating that the feature is not enabled. If an attempt is made programmatically to access a disabled service, PolicyCenter sends an activity to users identified by a configured set of roles. The activity message reports a permanent failure and advises that the administrator can provide access.

Spotlight session time-out

You may need to adjust the session time-out to enable users to work in Spotlight and then return to PolicyCenter without being automatically logged out. Check the value of the `SessionTimeoutSecs` parameter in `config.xml`.

Configuring underwriting authority

Underwriting authority in PolicyCenter provides an underwriting infrastructure which you can modify or augment.

In PolicyCenter, underwriting authority includes underwriting rules which specify when to raise underwriting issues. Approving underwriting issues requires authority profiles. Administrative users can create underwriting rules in the PolicyCenter user interface. Through configuration, you can modify the functionality available in the user interface for creating underwriting rules. If you have underwriting requirements that the underwriting rules framework cannot accommodate, you can implement those requirements in Gosu code.

The base configuration contains authority profiles for agents, underwriters, and an underwriting manager. You can modify the authority profiles and create new ones.

Underwriting authority contains the following major components:

- **Underwriting Rule** – Includes the name of the rule and describes how the policy transaction is affected by underwriting, and how issues can be approved. The rule includes the condition under which it will raise an underwriting issue. PolicyCenter provides a user interface in **Administration > Business Settings > Business Rules** for defining underwriting rules. Alternatively, you can implement the underwriting rule in Gosu code instead rather than through the business rules interface.
- **Underwriting Issue** – A specific occurrence on a policy transaction where an underwriting condition was true.
- **Authority** – The ability to approve underwriting issues created by a rule, and to what degree.
- **Authority Profile** – A group of authorities that can be given to a user, allowing the user to approve underwriting issues that were created by a particular rule.

See also

- “Configuring underwriting authority and multicurrency” on page 238
- *Application Guide*

Configuring authority grants

Authority profiles consist of one or more authority grants that give users the authority to approve particular types of issues. Users can have more than one authority profile, and authority profiles can be shared among multiple users.

Like roles and permissions, authority grants are always additive. A user can approve an issue if the user has at least one grant that allows them to approve that issue. For example, a user has one profile that grants a user authority to approve total premium up to \$1000. This user also has another profile that grants them the ability to approve total premium up to \$1500. In this case, the \$1500 limit will be the one that is in effect.

An authority grant has an associated value if the comparator specifies a value type other than None. If the authority grant for a given issue has an associated value, the grant can either be contingent on the given value or can apply to any possible value. If the grant applies to any value then the `ApproveAnyValue` bit is set on the associated authority grant. Otherwise, PolicyCenter sets the `Value` field on the authority grant, and compares the grant value to the issue value to determine if the issue can be approved. In addition, PolicyCenter compares the grant value to the approval value, to make sure that the approval falls within the limits of the grant.

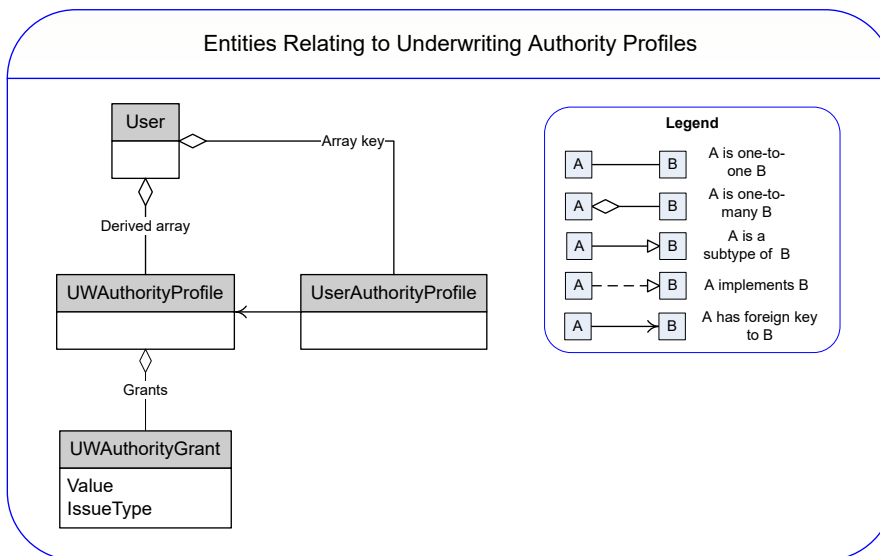
See also

- The *Application Guide* for more information about creating authority profiles in the PolicyCenter application and how to work with them.

Underwriting authority profile object model

Authority profiles consist of one or more authority grants. PolicyCenter stores authority profiles as `UWAuthorityProfile` entities and authority grants as `UWAuthorityGrants`. You can access authority profiles from the `User` entity through the `UserAuthorityProfiles` field. The relationship is many-to-many, such that users can have more than one profile, and multiple users can share the same profile.

The following illustration shows the relationship between key entities for underwriting authority profiles.



The following table describes entities associated with authority profiles.

Entity	Description
User	The PolicyCenter user.
UWAuthorityProfile	This entity contains a series of <code>UWAuthorityGrant</code> entities. This entity is similar to a <code>Role</code> entity which serves as a collection of <code>RolePrivilege</code> entities. Although this entity has a <code>Grants</code> array to <code>UWAuthorityGrant</code> , each profile can have at most one grant for a given issue type. Profile names must be unique.
UWAuthorityGrant	This entity represents a grant of authority to approve a particular type of issue. For issues that have an associated value type, that authority can either be restricted to a particular value or can be unrestricted.
UserAuthorityProfile	The authority profile contains a foreign key to <code>UWAuthorityProfile</code> .

Displaying authority grants in the user interface

The user configures authority grants on the **Authority Profiles** screen under the **Administration** tab. The second column displays value of the `ValueComparator` typelist.

Note: Guidewire chose to display the comparators as the strings **At most** and **At least** rather than as the $\leq$ and $\geq$ symbols (U+2264 and U+2265, respectively). Guidewire made this choice because databases that do not support Unicode characters may not store these characters correctly. These symbols may be used in Unicode-compliant installations.

See also

- *Application Guide*

Configuring underwriting rules

Underwriting rules and the underwriting issues the rules create are at the core of the underwriting authority feature. PolicyCenter raises underwriting issues at specific points in the job flow. The points are defined by the checking set. Some issues block progress at points in the job flow until they are approved. These points are defined by blocking points. Through configuration, you can add or remove checking sets and blocking points. You can also configure other features of underwriting rules and underwriting issues.

Underwriting rules defined in **Business Rules > Underwriting Rule** can be viewed by authorized users in PolicyCenter. They can edit, enable, and disable these rules. Using **Import** and **Export**, you can move rules between PolicyCenter instances.

If the underwriting rule is too complex to be implemented in the **Underwriting Rules** user interface, you can use an alternate implementation:

- If just the rule condition is too complex, use the **Underwriting Rule** user interface. In the **Rule Condition** call a custom utility function that implements the rule condition.

See “Custom rule utility functions” on page 96.

- Use a Gosu implementation if:
 - The rule is too complex to write within the **Underwriting Rules** framework.
 - Has outcomes that are more complex than just adding an underwriting issue. For example, create an underwriting issue and an activity or contingency.

See “Creating externally managed underwriting rules in Gosu code” on page 87.

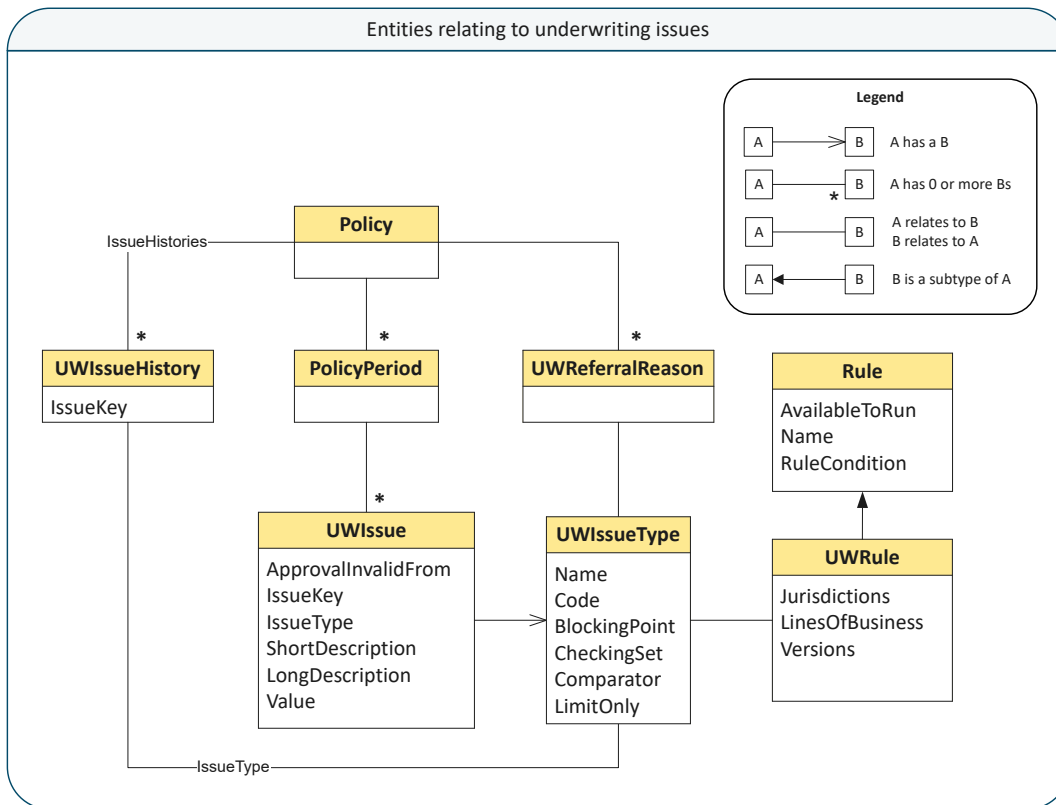
Configuring underwriting rules through business rules

Underwriting rules are a part of the business rules functionality, which is configurable. For more information on how to configure business rule functionality, see:

- “Business rules parameters” in *Configuration Parameters*
- “Configuring business rule editor functionality” on page 92 – Specify which methods and properties are visible in the rule condition builder
- “Business rule actions” on page 93 – Raises underwriting issues if conditions are met
- “Rule permission provider” – Define permissions for underwriting rules
- “Custom rule utility functions” on page 96 – Adding functions for use in the **Rule Condition** editor
- “Rule contexts and rule context definitions” on page 96 – Adding new context objects for use in rules
- “The business rules plugin” – For general information about the plugin that enables you to configure business rule functionality

Underwriting rules object model

The following illustration shows the relationship between some of the entities for underwriting rules.



The following table describes some of the entities associated with underwriting issues.

Entity or object	Description
PolicyPeriod	Stores information for a specific period of a policy. The <code>UWIssuesIncludingSoftDeleted</code> array provides access to all issues on this policy, including those marked inactive.
UWIssue	These are the issues that evaluator classes create. The issues link to issue types and can contain an approval. These entities can vary in effective time and are copied from branch-to-branch, just like any other effective-dated entity. Some fields on this entity are: • <code>IssueKey</code> and <code>IssueType</code> – These two fields uniquely identify the issue. • <code>ShortDescription</code> – Description of the issue that appears in the user interface. • <code>LongDescription</code> – Long description of the issue that appears in the user interface. Some users may have permission to view the short description but not the long description. • <code>Value</code> – The value, if any, associated with this issue. If present, the value is compared with authority grants to determine if the user can approve this issue. The value is also compared with approvals to determine if the approval still applies.
UWIssueHistory	This entity represents a history of issues and approvals on the policy. The <code>IssueKey</code> field is a reference to the <code>UWIssue</code> with that <code>IssueKey</code>. PolicyCenter maintains the history in a set of non-effdated entities on the policy and outside of the context of any particular job or policy period. PolicyCenter maintains this history for the following reasons: • The entity captures events that happen on branches that do not end up binding. • The entity persists after issues are removed or approvals expire. Since this entity attaches directly to the policy, it is retireable rather than effdated.
Rule	A rule defined in Administration > Business Settings > Business Rules .

Entity or object	Description
UWRule	This entity defines an underwriting rule as defined in Business Rules > Underwriting Rules . Has a foreign key to UWIssueType.
UWIssueType	This entity defines the characteristics of an underwriting issue type. Contains the Name and Code that are defined in Underwriting Rules .
UWReferralReason	This entity is similar to the UWIssue entity but is accessible from the policy rather than the policy period. This entity is used for marking a policy issue outside of a job. It identifies the details of a corresponding UWIssue that will be generated the next time the application checks for UWIssues. Because of the close relationship between a UWReferralReason and the associated UWIssue, the default user interface displays them in a similar way. For more information, see the <i>Application Guide</i> and “Configuring underwriting referral reasons” on page 78.

The actual link between User and UWAuthorityProfile is through the array key from User to UserAuthorityProfile then through the foreign key from that entity to UWAuthorityProfile. For convenience, there is a derived array from User to UWAuthorityProfile.

Adding a new checking set

The default configuration of PolicyCenter provides a number of predefined checking sets. Checking sets represents points in the job at which issues can be raised. PolicyCenter evaluates each checking set at specific blocking points for each job type and determines whether or not to raise an underwriting issue. For each checking set, you can modify when the checking set is evaluated. See “Triggering point mapping” on page 96.

However, you will need to add a new checking set if you have an underwriting rule that requires checking at a new combination of job type and blocking point.

The following table describes the checking set values in the base configuration.

Value	In UI?	Description
All		PolicyCenter checks this rule at all blocking points. PolicyCenter removes an existing underwriting issue if the evaluators do not trigger this type of issue. Issues created by this checking set may come from external systems in a way that is not necessarily synchronized with the job flow. For example, a claim system sends notice of excessive claims, or a billing system sends notice of a delinquent account. If you have an issue that cannot change once the policy is quoted (without re-editing the policy), do not use this checking set.
Manual	•	The underwriting issue is created when the user manually added an issue to the policy period. PolicyCenter displays Manual issues on the Risk Analysis screen.
MVR		For motor vehicle record underwriting issues. PolicyCenter checks for this issue at quote, quote release, bind, and issuance.
PolicyRenewalAPI	•	PolicyCenter checks this rule prior to calling policy renewal web services. Policy renewal web services provides methods for managing renewals in PolicyCenter.
PreBind	•	For underwriting issues that can only be detected immediately prior to binding the job. PolicyCenter checks this rule prior to binding.
PreIssuance	•	For underwriting issues that can be detected immediately prior to issuance. For submission policy transactions, PolicyCenter checks this rule prior to issuing the policy. If the policy transaction does not have an issuance step, PolicyCenter checks this rule prior to binding.
PreQuote	•	For underwriting issues that can be detected on the policy period even before quote, such as issues related to data entered in draft mode. PolicyCenter checks this rule prior to generating a quote.
PreRateRelease		PolicyCenter checks these rule underwriting rules after quoting and before the rate or quote is released.

Value	In UI?	Description
PreQuoteRelease	•	For underwriting issues that need information from a valid quote or that can be detected after a valid quote. For example, use this checking set for issues that depend on the premium. PolicyCenter checks this rule after a valid quote, but before releasing the quote.
Question		For underwriting issues is based on answers to a question set. PolicyCenter checks this rule before quote.
Referral		For underwriting issues is related to underwriting referral reasons on the policy. PolicyCenter checks this rule at all blocking points. PolicyCenter displays these issues in the UW Referral Reasons tab of the Risk Analysis screen. You can also add a referral reason to a policy by using the <code>addReferralReason</code> API.
RegulatoryHold		For underwriting issues related to policy holds with the Regulatory Hold hold type. PolicyCenter checks this rule at all blocking points. The user is notified each time the job advances to the next step.
Reinsurance		For underwriting issues related to reinsurance. For example, you can use this checking set to raise an issue if a policy has insufficient reinsurance. PolicyCenter checks this rule at all blocking points except quote.
Renewal	•	For underwriting issues related to renewals. PolicyCenter checks this rule in renewal policy transactions at quote and issuance.
Upgrade		In certain cases, Guidewire uses this value when upgrading PolicyCenter. Do not use this value for any other purpose.
UWHold		Use for underwriting issues related to policy holds with the Underwriting Hold hold type. PolicyCenter checks this rule at all blocking points. Therefore, the user is notified each time the policy transaction advances to the next step.

The In UI column indicates the values that appear in the Rule screen of the base configuration. The `TriggeringPointKey.TF_UWRULECHECKINGSETFILTER` filter narrows the choices in the user interface. Modify this filter to add hidden values such as **RegulatoryHold**.

Except for **All**, a rule can be associated with only one checking set.

Checking set values correspond to the `UWIssueType.CheckingSet` property. The `UWIssueCheckingSet` typelist defines these values.

The blocking point at which each checking set is evaluated is specified in the `JobProcessUWIssueEvaluator.gs` class. The code specifies which checking sets are evaluated at all blocking points. For each blocking point, the code specifies which additional checking sets are evaluated. Each job type can also specify when checking sets are evaluated.

For more information about checking sets and blocking points, see:

- “Checking sets and evaluators” on page 76
- “Blocking points” on page 77
- “Job interactions with underwriting issues” on page 79

Add a new checking set affecting all job types

About this task

If a checking set will be evaluated at the same blocking points for all job types, then you can change the defaults in `JobProcessUWIssueEvaluator`.

Procedure

1. In Studio, open the `UWIssueCheckingSet.ttx` typelist extension.
2. Add a new typecode entering a code, name, description, and optional priority.

3. Add the same typecode to the `TriggeringPointKey.ttx` typelist extension.
Next, link the checking set typecode to the triggering point key.
4. In `UWIssueCheckinSet.ttx`, add a category. Set typelist to `TriggeringPointKey` and code to the typecode value.
5. Open `gw.job.JobProcessUWIssueEvaluator.gs`.
6. Modify the static function that returns a `JobProcessUWIssueEvaluator` associated with the specific job type or types that you wish to affect. An example in the default configuration is the `forRenewal` method. This method overrides the built-in checking at the Quote and Issuance blocking points. (Both the `CheckedAtAllBlockingPoints` as well as the blocking-point-specific lists are examined.)
7. If `JobProcessUWIssueEvaluator` has been modified in a way that affects the `CheckedAt` properties or logic, then you may need to make modifications. Do a similar review of `JobProcess` and how it may call `JobProcessUWIssueEvaluator` or otherwise affect the `CheckedAt` behavior.

Add a new checking set affecting a single job type

Procedure

1. In Studio, open the `UWIssueCheckingSet` typelist.
2. Add a new typecode entering a code, name, and description.
3. Add the same typecode to the `TriggeringPointKey.ttx` typelist extension.
Next, link the checking set typecode to the triggering point key.
4. In `UWIssueCheckinSet.ttx`, add a category. Set typelist to `TriggeringPointKey` and code to the typecode value.
5. Open `gw.job.JobProcessUWIssueEvaluator.gs`.
6. In the `forJobtype` method, instantiate a new `JobProcessUWIssueEvaluator` with the appropriate `CheckedAt` properties set. Assign it to `this.JobProcessEvaluator`. If this code already exists, you can add a `CheckedAt` property.
For example, the `forSubmission` method in the default configuration is as follows:

```
static function forSubmission() : JobProcessUWIssueEvaluator {
    return new JobProcessUWIssueEvaluator() {
        :CheckedAtAllBlockingPoints = {"All"}
    }
}
```

Adding a new value comparator

In an underwriting rule, use the **Value Comparator** field to define how to compare the issue value. You can specify:

- If the value of an issue is within the authority granted to a user
- If the value of an issue is within the associated value of the approval

This comparator also specifies whether the value of an approval is within the authority granted to a user.

In the default configuration, the values are:

Value	Code	Description
At least	Numeric_GE	The issue value must be greater than or equal to the approval or authority grant value. Select this value if the issue has an associated value where a smaller number is associated with more risk, such as a deductible. Treat the value as a <code>BigDecimal</code> .

Value	Code	Description
At least (monetary)	Monetary_GE	The issue value must be greater than or equal to the approval or authority grant value. Select this value if the issue has an associated value where a smaller number is associated with more risk, such as a deductible. Treat the value as a MonetaryAmount.
At most	Numeric_LE	The issue value must be less than or equal to the approval or authority grant value. Select this value if the issue has an associated value where a larger number is associated with more risk, such as total premiums or total insured value. Treat the value as a MonetaryAmount.
At most (monetary)	Monetary_LE	The issue value must be less than or equal to the approval or authority grant value. Use this comparator for currency amounts. Select this value if the issue has an associated value where a larger number is associated with more risk, such as total premiums or total insured value. Treat the value as a MonetaryAmount.
In set	State_Set	Treat the authority grant or approval value as a set of jurisdictions, and the issue value must be within that set.
None	None	Has no associated value. Use for issues that either exist or do not exist.

The value comparator corresponds to the `UWIssueType.Comparator` property.

The `ValueComparator` typelist defines these values. This typelist defines the comparators that issue values can use. You can add to the `ValueComparator` typelist. Each `ValueComparator` typekey has an associated `UWIssueValueComparatorWrapper` Gosu class that performs the comparison.

You may need to add a new value comparator if:

- **The issue type needs to have an associated value** because different PolicyCenter users have authority to approve specific values or ranges of values. For example, issues of a certain type have an associated class code. PolicyCenter users can be authorized to approve issues within certain class code ranges. There is no existing value comparator for this type. Therefore, you need to create a new value comparator for this type, and implement Gosu code that determines whether one class code or range contains another class code range.
- **The existing comparators do not meet your needs** because:
 - **There is no comparator for this kind of data** – For example, you need a comparator that determines the manufacturer of a car based on the Vehicle Identification Number (VIN).
 - **There is no comparator that compares two values in the way that you require** – For example, there is a state set comparator. However, you need a comparator which determines if a jurisdiction is in the northern, southern, eastern, or western part of the country.

Comparators are used to determine whether one value associated with some `UWIssueType` is within the bounds set by some other value. The comparator is used for two different comparisons:

- Ensuring that an approval value is within a grant value.
- Ensuring that an issue value is within an approval value.

Each new value comparator has a typekey in the `ValueComparator` typelist and a wrapper. There must be exactly one `UWIssueValueComparatorWrapper` for each `ValueComparator` and one `ValueComparator` for each `UWIssueValueComparatorWrapper`. When you instantiate the wrapper, you specify the associated `ValueComparator`. There is a two-way association between typekey and wrapper.

See also

- “Comparing issue values” on page 83
- *Application Guide*

Add a value comparator

Procedure

1. In Studio, edit the `ValueComparator` typelist.
2. Add a new typekey entering a code, name, and description.
3. Instantiate a wrapper for your new typekey.

In the default configuration, the wrappers are instantiated in the `UWIssueValueComparatorWrapper` class located in the `gw.job.uw` package. Add code to instantiate a wrapper for your new typekey. Examples in the default configuration can be seen at the top of the file:

```
// The wrappers for the various OOTB ValueComparators
static final var _ge : UWIssueValueComparatorWrapper as readonly NumericGEWrapper
    = new NumericGEValueComparatorWrapper("Numeric_GE")
```

If you prefer not to instantiate the wrapper in `UWIssueValueComparatorWrapper`, you can instantiate the wrapper in the location of your choice. However, the wrapper must be instantiated in a class that loads before PolicyCenter calls `UWIssueValueComparatorWrapper.wrap`. For example, you can instantiate your wrappers in a startable plugin which runs after loading the database, but before serving any web pages. For more information, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

IMPORTANT: Failure to guarantee that PolicyCenter instantiates the wrapper before loading `UWIssueValueComparatorWrapper` may result in non-deterministic null pointer exceptions if `UWIssueValueComparatorWrapper` returns null for an extended typekey.

4. Instantiate a new extension to `UWIssueValueComparatorWrapper` for your typekey in the `gw.job.uw.comparators` package. You can use existing comparators as an example.

Adding a new value formatter

The base configuration of PolicyCenter provides formatters for currency, numbers, jurisdictions, age, units, and U.S. dollars. PolicyCenter even provides a formatter that does not format the data. If the formatter you need is not provided in the default configuration, you can add new value formatters. You can also modify the existing value formatters.

In the base configuration, the values are:

Value	In UI	Description
Age	•	Formats the value as an age followed by the abbreviation yrs.
Currency		Formats the value as the currency associated with the current locale.
Integer	•	Formats the value as an integer.
MonetaryAmount		Formats the value as a <code>MonetaryAmount</code> , which contains a value and a currency.
Number	•	Formats the value as a <code>BigDecimal</code> .
StateSet	•	Formats the value as a jurisdiction.
USD		Formats the value as a U.S. dollar (with cents).
USDBrief		Formats the value as a U.S. dollar (without cents).
Unformatted	•	Does not format the value.
Units	•	Formats the value as an integer followed by the word units .
TestFormatter		Used only for testing.

The In UI column indicates values that appear in Rule screen of the base configuration. If you add hidden values such as Currency, PolicyCenter supports those values without further configuration.

The value formatter type corresponds to the `UWIssueType.ValueFormatterType` property.

The `ValueFormatterType` typelist defines these values. The code is the same as the value or name. The formatter types allow the presentation of values in a localizable and consistent manner in the user interface. Only output uses formatters. You can configure formatters to meet a wide range of output needs.

See also

- *Application Guide*

Add a new value formatter

Procedure

1. In Studio, edit the `ValueFormatterType` typelist.
2. Add a new typekey entering a code, name, and description.
3. Instantiate the formatter and link it to your new typekey. Write code to format the value.

In the default configuration, the value formatters are instantiated in the `ValueFormatter` class located in the `gw.job.uw` package. Add code to instantiate the value formatter for your new typekey. Also add code to format the value. For example, the following is the code for the `Integer` value formatter in the `ValueFormatter` class:

```
final public static var FOR_INTEGER : ValueFormatter = new ValueFormatter(TC_INTEGER) {  
    override function format(value : String) : String {  
        return formatInteger(value)  
    }  
}
```

The code `ValueFormatter = new ValueFormatter(TC_INTEGER)` links the formatter to the `Integer` typekey. The `formatInteger` method formats the value.

If you prefer not to instantiate a value formatter in the `ValueFormatter` class, you can instantiate it in the location of your choice. However, the value formatter must be instantiated in a class that is guaranteed to load before PolicyCenter calls the `ValueFormatter.format` method. For example, you can instantiate your wrappers in a startable plugin which runs after loading the database, but before serving any web pages. For more information, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

IMPORTANT: Failure to guarantee that the value formatter is instantiated before loading `ValueFormatter` may result in non-deterministic null pointer exceptions if `ValueFormatter.format` returns null for an extended typekey.

Checking sets and evaluators

Checking sets represents points in the job at which issues can be raised. For each checking set, evaluator classes determine whether or not to create underwriting issues. Each underwriting rule and its corresponding `UWIssueType` type has a checking set property.

The `IPolicyEvaluationPlugin` triggers policy evaluation. For each underwriting rule, this plugin uses the checking set to determine whether to add underwriting issues on a policy period. In the base configuration, this plugin calls evaluator classes to raise issues. The plugin then removes orphaned issues. Orphaned issues are underwriting issues that were generated at a checking set but are no longer an issue when that checking set runs at a later time.

If you need to change this plugin, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

See also

- “Job interactions with underwriting issues” on page 79
- *Application Guide*

Evaluator Classes

The evaluator classes contain methods that raise underwriting issues based upon the checking set. In the base configuration, the default evaluator class, `DefaultUnderwriterEvaluator.gs`, raises underwriting issues for all lines of business. Each line of business implements its own evaluator class. For example, the evaluator for the personal auto line of business is `PA_UnderwriterEvaluator.gs`. These evaluator classes contain methods that raise underwriting issues for a checking set. For example, the `onRenewal` method in the default evaluator raises underwriting issues for the Renewal checking set.

Note: The evaluator classes ignore the **Enabled** setting of the underwriting rule defined in the Underwriting Rule user interface. Therefore, these classes raise underwriting issues regardless of the **Enabled** setting.

If you make code changes to the methods that raise underwriting issues, the code must adhere to the following:

- The method must raise issues only for that checking set.
- Only one checking set can raise a given underwriting issue type.

In the default configuration, the evaluators always raise auto-approvable issues regardless of whether the current (or other) user has sufficient authority to approve them. The evaluators always raise these issues because the issue needs to exist for the least-privileged user of the system. This approach separates the underwriting guidelines for things require approval from the approval levels assigned to users. For users with sufficient authority to approve the resulting issue, PolicyCenter usually does not display the issue in the user interface. (The user can easily view the issue by selecting different view options.)

You can configure the application to raise auto-approvable issues only when one or more users would be unable to approve them. One reason to do this would be to improve performance by reducing the number of auto-approvable issues being raised and silently approved.

See also

- “Creating externally managed underwriting rules in Gosu code” on page 87

Orphaned underwriting issues

Orphaned issues are underwriting issues that were generated at a checking set but are no longer an issue when that checking set runs at a later time. Therefore, issues that resulted from a previous version of the policy graph are removed if the data that caused them to be raised has been removed. For example, during a submission, PolicyCenter raised an issue for an expensive car on the policy. When the agent removes that car from the policy, the issue also disappears.

Note: PolicyCenter does not remove manual underwriting issues as orphans.

The `IPolicyEvaluationPlugin` removes orphaned issues after it runs the evaluators that create underwriting issues. The plugin removes or deactivates any issues for the current checking set that were not raised during that execution of the evaluators. Issues that currently have an approval or rejection associated with them are made inactive by setting the `Active` bit to `false`. Other issues are removed by setting the `ExpirationDate` to the slice date after which the conditions for the issue no longer exist. If an inactive issue later reappears, the `Active` bit on the issue is set back to `true`, and any unexpired approval or rejection present on the issue applies again.

Blocking points

Blocking points represent points in the job at which an issue can block progress of the job.

The blocking point values, listed in priority order from lowest to highest, are:

Value	Code	Description
Non-Blocking	NonBlocking	Sometimes referred to as informational issues. These issues do not block progress unless specifically rejected by a user.

Value	Code	Description
Blocks Issuance	BlocksIssuance	Prevents issuance for submission policy transactions. Prevents binding for policy transactions that do not have a separate issuance step.
Blocks Bind	BlocksBind	The issue blocks binding the policy.
Blocks Quote Release	BlocksQuoteRelease	Allows quoting to be performed, but unauthorized users cannot view the quote until an underwriter has the opportunity to review and approve it. The job cannot progress further.
Blocks Rate Release	BlocksRateRelease	Block releasing a policy period rate. If two-step quoting is enabled in the policy line, this prevents the user from accessing the Quote screen. For example, although the policy is rated, you may not want the user to access the Quote screen until the policy has underwriting approval. In one-step quoting, this blocking point blocks quote release. The BlocksRateRelease blocking point is treated slightly differently from other blocking points. At the BlocksRateRelease blocking point, PolicyCenter only checks the PreRateRelease checking set, and does not check the All checking set. This is done for performance reasons, to make the first step of two step quoting execute quickly. If necessary, you can modify this behavior by making a change in the JobProcessUWIssueEvaluator class.
Blocks Quote	BlocksQuote	Prevents quoting and steps leading up to quoting. Use for issues that require intervention before the rating engine is called. You can use this blocking point for other types of issues, as well. However, if a policy transaction has a Blocks Quote issue, PolicyCenter will not call the rating engine, and the policy transactions remains in draft state. This blocking point also blocks rating, which is the first step in two-step quoting. Do not use for issues that need information from a valid quote. For example, do not use for issues related to total premium.
Rejected	Rejected	The most restrictive value. Rejects the issue. A rejected issue prevents the policy transaction from crossing any blocking point.

The blocking point corresponds to the `UWIssueType.BlockingPoint` property.

Blocking points are defined in the `UWIssueBlockingPoint` typelist. This typelist represents the places at which an issue can block. The priority of the typecode orders the blocking point, with larger numbers indicating a more restrictive blocking point. The blocking point of an issue is based on the combination of the blocking point and approval, if present.

The job process classes evaluate whether to raise underwriting issues and check for blocking issues in the `evaluateAndCheckForBlockingUWIssues` method. For example, the job process calls this method before binding a policy change. Before binding, the `PolicyChangeProcess` evaluates the all, pre-bind, and pre-issuance checking sets and then checks to see if any issues block issuance. PolicyCenter determines the current point at which an issue blocks both by its type and any associated approval or rejection. Thus, the issue blocks operation if the priority of the current point is greater than or equal to the priority of the operation. Therefore, a rejected issue blocks everything, a non-blocking issue blocks nothing. An issue that blocks bind also blocks issuance but does not block quoting.

See also

- *Application Guide*

Configuring underwriting referral reasons

Underwriting referral reasons cause underwriting issues to be raised the next time a policy transaction is run on for that policy. The object for underwriting referral reasons is `UWReferralReason`. Referral reasons raise underwriting issue types that have the **Checking Set** equal to **Referral**.

In the base configuration, the policy evaluation plugin implementation always checks for referral reasons on the policy, regardless of checking set or line of business. `PolicyEvalPlugin.gs` implements the `IPolicyEvaluationPlugin` plugin interface. The `onReferral` method in the `DefaultUnderwriterEvaluator` class raises underwriting issues for referral reasons. This method creates a `UWIssue` based on that underwriting referral reason.

See also

- *Application Guide*
- *Plugins, Prebuilt Integrations, and SOAP APIs*

Job interactions with underwriting issues

At specific places in the job flow, the job classes raise issues at checking sets and stop job progress because of blocking issues.

You can view and modify the Gosu code for each job. The Gosu code is in the `gw.job` package in Studio.

The following topics provide the checking and blocking points for each job type for which underwriting is provided in the base configuration. Through configuration, you can extend underwriting to other job types, such as cancellations.

Submission

The `SubmissionProcess` class handles checking sets and blocking points at the following times:

Submission job	Checking sets	Blocking points
Before quoting	All	BlocksQuote
	Referral	
	PreQuote	
	Question	
Before quote release	All	BlocksQuoteRelease
	Referral	
	PreQuoteRelease	
During bind, when not issuing	All	BlocksBind
	Referral	
	PreBind	
During bind, when also issuing	All	BlocksIssuance
	Referral	
	PreBind	
	PreIssuance	

Renewal

Unlike other jobs, the `RenewalProcess` is often automated. As a result, renewal follows a different flow than the other jobs. There are a few primary differences in the automated flow:

- Checks are done and then repeated at various timeout points in the process.
- Blocking issues generate exceptions. When this occurs, the renewal is escalated to the underwriter.

The renewal job handles checking sets and blocking points at the following times:

Renewal job	Checking set	Blocking point
Before quoting	All	BlocksQuote
	Referral	
	PreQuote	
	Question	
Before quote release	All	BlocksQuoteRelease

Renewal job	Checking set	Blocking point
	Referral	
	PreQuoteRelease	
Before scheduling the first check	All	BlocksIssuance
	Referral	
	Renewal	
	PreBind	
	PreIssuance	
During the first check	All	BlocksIssuance
	Referral	
	Renewal	
	PreBind	
	PreIssuance	
During the final check	All	BlocksIssuance
	Referral	
	Renewal	
	PreBind	
	PreIssuance	
Before issuing the renewal	All	BlocksIssuance
	Referral	
	Renewal	
	PreBind	
	PreIssuance	

The code handles automated renewals differently from manual renewals. In automated renewals, all issues are run as *auto-approvable*. Issues are given automated approvals if the special renewal user has sufficient authority to progress the job. The `runMethodAsRenewalUser` method gets a user from the `getAutomatedRenewalUser` method in the `IPolicyRenewalPlugin`. In the default configuration, the user is `renewal_daemon`. The automated renewal runs the job as this user, using the authority profile of this user for approvals. This user approves the issues.

The `runMethodAsRenewalUser` method sets a flag on the `RenewalProcess` object indicating that the renewal is automated.

See also

- For more information about the `IPolicyRenewalPlugin`, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

Policy Change

The `PolicyChangeProcess` class handles checking sets and blocking points at the following times:

Policy change Job	Checking set	Blocking point
Before quoting	All	BlocksQuote
	Referral	
	PreQuote	
	Question	
Before quote release	All	BlocksQuoteRelease
	Referral	
	PreQuoteRelease	

Policy change Job	Checking set	Blocking point
During bind	All	BlocksIssuance
	Referral	
	PreBind	
	PreIssuance	

The code handles automated policy changes differently from manual policy changes. In automated policy changes, all issues are run as *auto-approvable*. Issues are given automated approvals if the special policy change user has sufficient authority to progress the job. In the default configuration, the user is `policychange_daemon`. The automated policy changes runs the job as this user, using the authority profile of this user for approvals. This user approves the issues.

Rewrite

The RewriteProcess and RewriteNewAccountProcess classes handle checking sets and blocking points at the following times:

Rewrite job	Checking set	Blocking point
Before quoting	All	BlocksQuote
	Referral	
	PreQuote	
	Question	
Before quote release	All	BlocksQuoteRelease
	Referral	
	PreQuoteRelease	
During bind	All	BlocksIssuance
	Referral	
	PreBind	
	PreIssuance	

Issuance

The IssuanceProcess class handles checking sets and blocking points at the following times:

Issuance job	Checking sets	Blocking points
Before quoting	All	BlocksQuote
	Referral	
	PreQuote	
	Question	
Before quote release	All	BlocksQuoteRelease
	Referral	
	PreQuoteRelease	
During bind, when also issuing	All	BlocksBind
	Referral	
	PreBind	
	PreIssuance	

Reinstatement

When you reinstate a canceled policy, approvals carry over from the canceled policy even if they were approved only until the next change. PolicyCenter does not consider reinstatement a change.

Cancellation

Cancellation jobs do no checks for underwriting issues.

Audit

Audit jobs do no checks for underwriting issues.

Issue keys

The underwriting rule and issue key uniquely identify each underwriting issue. For any given combination of underwriting rule and issue key, there can be only one issue on a policy period at a given time. When creating an issue, any pre-existing issue with the given type and key will be returned, and any inactive issue that matches will be activated and then returned.

On the underwriting issue, the `UWIssue.IssueType` property specifies the `UWIssueType` of the corresponding the underwriting rule. Issue key is specified in the `UWIssue.IssueKey` property. The `PolicyEvalContext` object automatically enforces this.

Guidewire recommends that you do not change the algorithm for forming the key for an issue after moving that algorithm to production. If you change the algorithm, current issues may be treated as orphaned. PolicyCenter will create new copies of the issues, requiring new approvals or rejections.

See also

- *Application Guide*

Issue Key Guidelines

Guidewire recommends these guidelines for forming issue keys.

For issues that affect the policy as a whole, such that there is only ever one such issue, use the code of the underwriting rule as the issue key. Total policy premium is an example of this. Thus, if the premium goes up or down, the same issues will be edited, rather than raising new issues each time.

For issues that relate to a particular entity, have the issue key encode the `FixedID` of the entity, or potentially some other unique identifier such as the VIN number. For example, use this method for an issue relating to the collision deductible on a given vehicle. This method allows that same issue to be raised for several different vehicles. It also ensures that the same issue will be modified if changes affect that issue.

If the issue relates to a jurisdiction, create one issue for each jurisdiction, with the jurisdiction encoded as part of the issue key. Since approvals for jurisdiction-based issues only apply to one jurisdiction at a time, rather than to a set of jurisdictions, each jurisdiction must have its own issue. An issue relating to a different jurisdiction would not reuse approvals or rejections. For example, an issue around the garage jurisdiction of a vehicle encodes both the `FixedID` or VIN of the vehicle as well as the jurisdiction. Changing the garage jurisdiction generates a new issue, rather than modifies the existing issue. This behavior also applies to set comparator types that you develop.

If the issue relates to a specific location, or a specific building at a certain location, encode the `FixedID` or other invariant identifier into the issue key. This encoding ensures that approving Building 2 in San Francisco to go without an alarm does not approve the same for Building 2 in Los Angeles.

Configuring underwriting issue history

Underwriting history preserves an audit trail of issues and approvals, even on abandoned branches. To keep track of this history, every issue creation or removal and every issue approval generates an `UWIssueHistory` entity. The `Policy` entity has an `IssueHistories` array. This array contains the history of changes to all underwriting issues associated with the policy. The `UWIssueHistory` entity is non-effdated.

A UWIssueHistory entity is created automatically every time an issue is approved or rejected. Issue histories are also generated when the issue is reopened, removed, or when the approval expires. See the `gw.policy.UWIssueEnhancement.gsx` class for details.

An UWIssueHistory entry is also generated whenever an issue is either created or made inactive.

Note: In the default configuration, a history event is not created if values on an issue change. For example, an issue exists on a policy because there are six cars. Later the policy only has three cars. The checking set checks the current number of cars but does not generate a history event.

UWIssueHistoryStatus typelist

The Status field on the UWHistory entity is an UWIssueHistoryStatus typelist. The UWIssueHistoryStatus typelist represents the type of action recorded by the history. Values are:

Value	Description
Approved	The issue was approved.
Created	The issue was created.
Expired	The issue approval expired.
Rejected	The issue was rejected.
Removed	The issue was removed.
Reopened	The issue was reopened by removing approval or rejection.

Comparing issue values

The values associated with underwriting issues (UWIssue objects) can be either numeric or non-numeric, as needed to describe level of risk or authority related to a specific issue type. The `ValueComparator` is used to determine if:

- The value of an issue is within the limits set on the existing approval
- The value of an issue is within the user's authority to approve
- A requested approval is within the user's authority to authorize

Additionally, the value itself is not strongly typed; its type is inferred from the `ValueComparator` assigned to the issue type.

The `ValueComparator` typelist contains the list of possible comparisons, such as *at least* and *at most*. The `UWIssueValueComparatorWrapper` class defines how to compare issue values. That class uses the `UWIssueValueType` class to define what the underlying value type is. The `UWIssueValueType` class defines how to convert the value that gets stored in the Value field on UWIssue or the `ReferenceValue` field on UWHistory. This class also defines how to validate raw string values. More than one comparator can use the same underlying value type.

The `UWIssueValueType` class has three methods: `deserialize`, `serialize`, and `validate`. This class defines two primary value types, represented as constants on the `UWIssueValueType` interface. The two value types are `BIG_DECIMAL` type, and `STATE_SET` type. The `BIG_DECIMAL` type converts and validates to and from a big decimal number. The `STATE_SET` is a comma-separated list of codes for jurisdictions. The `STATE_SET` type can either be an inclusive or exclusive set. If the list is preceded by *not*, a jurisdiction is part of the set if it is not in the list.

You can use `STATE_SET` as an example for writing comparators for other types of sets.

You can create comparators for other types of numeric values such as dates. Dates would need a greater than or equal comparator that operates on dates instead of numbers. The `UWIssueValueComparatorWrapper` is bound to a particular `UWIssueValueType`. The comparator wrapper implements a `compare(String, String)` method, which indicates whether or not the given issue value is within the bounds of the given grant value. The wrapper also implements a `calculateDefaultApprovalValue` method. The wrapper ties together an issue type to its underlying value type (if any). Every key in the `ValueComparator` typelist must have an associated subclass of `UWIssueValueComparatorWrapper`.

See also

- *Application Guide*

Configuring default values for assignment type and value offset amount

For an underwriting rule, **Default Value Assignment Type** is used if the value comparator is **At least**, **At least (monetary)**, or **At most**. In turn, if **Default Value Assignment Type** is **Offset Amount**, the **Default Value Offset Amount** and **Default Value Offset Currency** are used. These fields correspond to properties on the `UWIssueType` object.

You can extend this default setting behavior for other numeric types. Positive offsets reflect increased underwriting risk for the at least and at most comparators. For example, the **At most** comparator can be used for determining the risk associated with the value of a car. If the authority grant provides default approval at \$100,000 with 10% offset, the user can approve a car whose value is as high as \$110,000. The **At least** comparator can be used for determining the risk of a deductible. If the authority grant provides default approval for at least \$1000, with 10% offset, then the user can approve a deductible as low as \$900.

Underwriter issue type verifier class

The `UWIssueTypeValidationRules` Gosu rules enforce constraints on `UWIssueType` objects to prevent certain types of configuration errors. These rules are in **configuration > config > rules > Validation** in Studio.

In the base configuration, these rules enforce the following:

- Either the `UWIssueType.DefaultApprovalBlockingPoint` must be one level higher than the issue `UWIssueType.BlockingPoint`, or both must be non-blocking.
- The `DefaultValueAssignmentType` must be defined if the `Comparator` is `Numeric_LE`, `Numeric_GE`, `Monetary_LE`, or `Monetary_GE`. These correspond to **Value Comparator** set to **At least**, **At most**, **At least (monetary)**, **At most (monetary)** in the Underwriting Rule screen.
- The `DefaultValueOffsetAmount` must be set if the `DefaultValueAssignmentType` is `OffsetAmount` or `OffsetPercent`. These correspond to **Default Value Assignment Type** set **Offset Amount** or **Offset Percentage**.

See also

- The `UWIssueType` entity in “Underwriting rules object model” on page 69

Localizing underwriting issue details

As part of the base PolicyCenter product, the **Underwriting Rules** and associated screens provide support for multiple languages.

On the Underwriting Rule screen, the **Underwriting Issue Details** has **Short Description** and **Long Description** fields for descriptions in the display languages, such as **English (US)** or **Japanese (Japan)**. These descriptions will appear in the associated underwriting issue. Localization is implemented using multiple parameters, with one parameter for each language. Therefore, each field corresponds to a single parameter.

Configuring approvals

PolicyCenter creates an approval when you approve or reject an issue. The `UWIssueApproval` object represents an approval. To approve an issue, you must have an authority grant in an authority profile for that issue type. If the issue has a value, the authority grant must be valid for that value.

Note: In the default configuration, the **Approve** button is only available if the user has the authority to approve the issue to its default level. You can modify the configuration if you have a different use case.

Approvals can be generated either by a user clicking to approve or reject an issue, or automatically by the application. Automatic approval is most commonly seen for auto-approvable issues, but also occurs during unattended processing, such as in routine renewals or automated policy changes. You can identify automatically-generated approval through the `IsManual` property.

At any point in effective time, each `UWIssue` can have at most one approval. If you reapprove an issue, PolicyCenter removes any existing approvals from that point forward in effective time. Approvals always extend from the current slice the user is working with until the end of the period. An issue is eligible for approval if it currently blocks progress and has not been rejected. If the issue has already been rejected, it must be reopened before being approved. If the issue has been approved through issuance with a sufficient value, it must be reopened before being approved for a different value or different approval settings.

Rejecting an Issue

Rejections generate an approval, but with the `BlockingPoint` set to `Rejected`. A rejection prevents the job from crossing any blocking point. The user must have the `uwreject` permission. The user can reject an issue if:

- It is eligible for approval.
- It is a non-blocking issue and has not already been rejected.

Note: The user need not have sufficient authority to approve the issue to reject it.

Reopening an Issue

Reopening an issue removes an approval from the issue. The issue then blocks at the blocking point associated with the issue type. To reopen an issue, you must have the `uwreopen` permission or be able to approve the issue.

Approval expiration

Approvals can be set to expire after a certain point in time by specifying one or three years in the `DefaultDurationType` of the `UWIssueType`. When an approval of this type is created, the expiration date is calculated and written to the `ApprovalInvalidFrom` field.

Special approval permission

There is a special approval permission that allows a user to approve an issue for any value. This permission is `uwapproveall`. Guidewire provides this permission to approve issues that no user can approve. Guidewire recommends that you never use this permission unless there is a time-critical policy that cannot be advanced by any underwriter. An incomplete or misconfigured set of authority profiles can cause this situation. In the default configuration, only `su` is given this permission.

A user with the `uwapproveall` permission is presented with a **Special Approve** button that allows them to approve any issue that they could not otherwise approve. The user must confirm that they wish to use this permission before the application displays the screen that allows them to approve the issue.

Passing approval requests to underwriters

In the underwriting approval process, the policy period can be referred to users with higher or different authority profiles. These users receive activities notifying them they need to approve issues on the policy. A particular policy may be referred to several users before being approved. When the underwriter feels that the review of the policy is complete, whether issues have been approved or not, the underwriter clicks the **Release Lock** button. This action releases the policy from underwriting, and PolicyCenter sends an activity to the user who initiated the referral. The `InitialReferrer` role stores the name of this user. (The underwriter can modify the recipient and other fields in the activity.)

To facilitate passing policies between agents and underwriters, a given policy period can be flagged as *edit locked* and *quote hidden*. After an agent hands these policies to an underwriter, the agent usually cannot edit the policy or view the quote. The permissions for editing or viewing the quote while under review are usually reserved for underwriters.

Edit Locked Flag

When the `EditLocked` flag is `true`, only users with the `editlockoverride` permission can edit the policy. The intention is that this permission is granted to underwriters but not to producers. The `EditLocked` flag is set to `true` when the user clicks the **Request Approval** button to request approval from another user. If a policy has not been

formally submitted for underwriting review, the underwriter expects the quote to be hidden and the policy locked for editing. Therefore, the `EditLocked` flag is also set to true whenever a user with the `editlockoverride` permission quotes a policy. The `EditLocked` flag is set back to false when the underwriter sends the policy back to an agent by clicking the **Release** button on the associated activity. (Click **Release Lock** to get to the activity.) The **Release** button is available even if the policy has blocking issues. The `EditLocked` flag is also set to false when the policy is bound and/or issued. The `EditLocked` flag is always set to false when a job starts.

Note: The PCF files check the `EditLocked` flag to determine whether or not the policy can be edited. The `EditLocked` flag does not lock the `PolicyPeriod` object.

Quote Hidden Flag

When the `QuoteHidden` flag is set to true, only users with the `quotehideoverride` permission can see the quote for that `PolicyPeriod`. Similar to the `editlockoverride` permission, the intention is that the permission only be granted to underwriters.

The `QuoteHidden` flag is set to true when requesting approval if the job does not already have a valid quote. The flag is also set to true during quoting if the `PreQuoteRelease` blocking point has blocking issues or if the policy being quoted is marked as `EditLocked`. Thus, if an underwriter with `editlockoverride` quotes a policy, the policy will end up both edit locked and quote hidden. The `QuoteHidden` flag is set to false if a user, such as a producer, who does not have the `editlockoverride` permission approves all issues blocking quote release.

In the default configuration, when `EditLocked` is set to false, `QuoteHidden` is also set to false. Invalidating the quote also sets `QuoteHidden` to false. However, for policies that are under edit lock, `QuoteHidden` is immediately set to true on quoting. Thus any policy in underwriting review will have its quote hidden if it has ever had an invalid quote while under review.

Note: The PCF files for the job wizards determine whether to display the quote step by calling the `canViewQuote` method in the `JobProcess` class. The `canViewQuote` method checks the value of the `QuoteHidden` flag. The `QuoteHidden` flag does not lock the `PolicyPeriod` object.

Initial Referrer User Role

The `InitialReferrer` user role facilitates passing policies between underwriting and producers. The initial referrer is the user who referred the policy to underwriting. When requesting approval for issues on a policy period, if the `InitialReferrer` user role is not set, the role is set to the current user. When the underwriter releases the policy by clicking **Release**, the `InitialReferrer` role is unset. An activity is sent to the `InitialReferrer` on the period. You can customize this by using activity assignment as described in the *Gosu Rules Guide*. You can also customize the `InitialReferrer` user role.

The `InitialReferrer` user role is a transient role assigned to users in the context of their role in the job. Because it is a transient role, it is assigned by the job code rather than being assigned to the user in the **Administration** tab. Because it is a transient role, it is not in the `UserRole` typelist.

Handling underwriting issues in policy revisions

PolicyCenter handles underwriting issues in policy revisions. Policy revisions reflect changes to the policy over a period of time.

Handling underwriting issues in out-of-sequence policy changes

This topic describes how PolicyCenter handles underwriting issues in out-of-sequence policy changes. For detailed information on how PolicyCenter handles out-of-sequence changes, see the *Application Guide*.

PolicyCenter handles out-of-sequence policy changes by evaluating every future slice in the policy when it evaluates whether to raise underwriting issues. PolicyCenter first evaluates the slice. Then PolicyCenter checks to see if the slice is blocked and automatically approves auto-approvable issues. PolicyCenter evaluates whether to raise underwriting issues on the slice. The data can change such that the value of an issue changes in the future, or the issue can disappear and reappear in different slices. When this occurs, the issue is automatically split and removed as necessary. The `PolicyEvalContext` object pulls issues forward or backwards in time as necessary during evaluation. However, any

issue with the same type and key will have the same FixedID if it is not completely removed from the policy as of the period start date.

Approvals are effective as of the slice date on which they are approved until the end of the period. When an issue is reopened or reapproved as of a particular effective date, PolicyCenter removes all approvals for that issue from that date forward. The behavior is the same for reopened and reapproved issues because a reapproved issue is automatically reopened. Thus, an approval at time T1 will extend through T2 until the end of the period, and any approval already at T2 will be removed. If the approval given at T1 is insufficient for time T2, the user will need to reopen the issue at T1 and create a more lenient approval. Alternately, the user can move T2 in effective time and reapprove the issue at time T2. PolicyCenter expires the original approval at T2 and creates a new approval from T2 until the end of the period.

If underwriting issues block future slices, PolicyCenter uses the same slice selector that it uses for out-of-sequence validation failures. PolicyCenter displays the selector if the `Failed00SEvaluation` flag is set to true on the period. This flag indicates that the policy period is out-of-sequence. The flag is set if a future slice has blocking issues but the current slice is not blocking. The flag can also be set if any issues that are generated have values that vary over time (which includes the issue itself disappearing and reappearing).

In the default configuration, PolicyCenter adds a section sign, \$, to indicate that the issue varies in time. PolicyCenter displays the section sign if the `ValueVariesAcrossSlices` and `IssueBlocksAtSomeSlice` properties in `UWIssueEnhancement` are true. PolicyCenter displays the section sign in parentheses, (\$), if the value varies in time but is not currently blocking any slice. Lastly, the user interface displays a **Next Blocked Date** button that automatically takes the user to the next slice in effective time where issues are blocked. The **Next Blocked Date** button appears if there are blocking issues in future slices. The user can step through the slices, approving issues from that point forward until the policy is no longer blocked. To see how PolicyCenter displays this in the user interface, see the *Application Guide*.

Handling underwriting issues in preempted jobs

This topic describes how PolicyCenter handles underwriting issues in preempted jobs.

A preemption occurs if one job is in progress when another job on the same policy binds. For example, a job starts on a policy and creates policy period PP1, which is not yet bound. Then another job on the same policy binds, resulting in a bound policy period PP2 that preempts PP1. When the user handles the preemption on PP1, PolicyCenter creates a new policy period PP3. PolicyCenter handles underwriting issues as follows:

- All issues and approvals which are present in PP2 are carried over to PP3. Approvals from PP2 are not expired. Therefore, if PP2 includes an approval which is invalid from the next change, that approval remains in PP3.
- If PP1 includes a new issue that does not appear in PP2, that new issue does not appear in PP3.
- If PP1 includes changes to an issue or approval that is present in PP2, the issue appears in PP3 along with the approval from PP2. However, the issue history includes links to an invalid job. (The links still refer to PP1, which was preempted and is no longer valid.)

See also

- *Application Guide*

Creating externally managed underwriting rules in Gosu code

In PolicyCenter, **Business Rules > Underwriting Rules** provides a user interface to define underwriting rules. However, if your underwriting rule does not fit within the **Underwriting Rules** framework, you can implement the underwriting rule in Gosu code. These rules are *externally managed rules*, which have their rule condition defined outside business rules, often in Gosu code. This topic describes how to create an externally managed rule in Gosu code.

Use an externally managed rule if:

- The rule is too complex to write within the **Underwriting Rules** framework.
- Your underwriting rule has outcomes that are more complex than just adding an underwriting issue. For example, create an underwriting issue and an activity or contingency.

Note: After upgrading from PolicyCenter 7.0 or 8.0, legacy underwriting rules are initially implemented as externally managed rules. See “Upgrading Underwriting Authority” in the *Configuration Upgrade Guide*.

Underwriting issues can be raised by the **Rule Condition** in the **Underwriting Rules** user interface, or by an externally managed rule. Underwriting rules conditions written using the underwriting **Rule** screen can be viewed by authorized users in PolicyCenter. These rule conditions can be edited, enabled, and disabled, and can be efficiently moved between PolicyCenter instances. Externally managed underwriting rule conditions written in Gosu are not accessible in the user interface, however the fields associated with the underwriting issue type are. However Gosu code can handle more complex rules and, in some cases, improve system performance. In both cases, PolicyCenter creates **UWRule** and **UWIssueType** entities. These entities have foreign keys to each other.

This topic describes how to write Gosu code that defines the rule condition that raises underwriting issues.

IMPORTANT: To avoid unexpected results, specify the rule condition in either the user interface or in Gosu code, but not in both. PolicyCenter first executes the **Rule Condition** defined in the user interface and raises an underwriting issue when the condition evaluates to true. Then PolicyCenter executes the rule condition in Gosu.

See also

- “Configuring underwriting rules” on page 69
- *Application Guide*

Overview of Gosu implementation

Use the **Underwriting Rules** interface to define basic information about the underwriting rule, primarily the name and **UWIssueType** properties. Use Gosu code to define the rule conditions and raise the underwriting issue. In your Gosu implementation, you can define rule conditions that correspond to the **Start Date**, **End Date**, **Rule Condition**, **Applies to**, and **Rule Context** in the Underwriting Rules user interface.

If you implement an underwriting rule in Gosu:

- In the **Underwriting Rules** user interface, define the **UWIssueType** fields. See “**UWIssueType** fields in underwriting rule” on page 89.
- In the underwriting **Rule**, set **Enabled** to **No**.
- In the underwriting **Rule**, set the **Rule Condition** to **None**. You will define the rule condition in Gosu.
- On the **UWIssueType** entity instance, set the **ExternallyManaged** property to **true**. In the base configuration, the user interface does not have a button to set this property for a rule. You can write code that sets this property. You can add a button to the user interface that accesses this code.

However, be careful about when the button is available. Rules cannot be toggled between **ExternallyManaged** and not **ExternallyManaged**. Make this button available to set **ExternallyManaged** to **true** on a brand new rule that has no information beyond the underwriting issue type fields defined.

If **ExternallyManaged** is **true**, then the business rules framework does not run the rule.

- Provide the rule condition logic in the Gosu code. See “Creating underwriting issues with policy evaluation plugin” on page 89.
- Manage the rule execution in the Gosu code. See “Raise underwriting issues in Gosu code” on page 90.

A work queue called **CreateUnmappedUWRulesWorkQueue** creates an underwriting rule in the business rules framework for any **UWIssueType** entity instance that does not have a corresponding **UWRule**. You must create the rule in Gosu and associate its **uwIssueCode** with the same code of the new **UWIssueType**. When you run **CreateUnmappedUWRulesWorkQueue**, the rule is created and appears in the PolicyCenter application. To see an example, open the Gosu file **PA_UnderwriterEvaluator.gs** in Guidewire Studio and take note of the last line:

```
_policyEvalContext.addIssue("PAMVRAccidentsViolations",
    "Driver:${policyDriver.FixedId.toString()}",
    \-> shortDescription, \-> longDescription)
```

This call adds an underwriting rule with type **PAMVRAccidentsViolations**.

UWIssueType fields in underwriting rule

If you implement an externally managed underwriting rule in Gosu code, you must define the `UWIssueType` properties in the underwriting **Rule** user interface. The interface enables you to modify the `UWIssueType` object. The following table shows the fields in the underwriting **Rule** user interface with the corresponding `UWIssueType` property. For example, entering **Name** in the underwriting **Rule** sets **Name** on the corresponding `UWIssueType` object.

Underwriting Rule user interface	UWIssueType property
Rule Details tab	
Name	Name
Code	Code
Checking Set	CheckingSet
Blocking Point	BlockingPoint
Default Duration	DefaultDurationType
Rule Description	Description
Advanced tab	
Auto-approvable	AutoApprovable
Default Edit Before Bind	DefaultEditBeforeBind
Default Approval Blocking Point	DefaultApprovalBlockingPoint
Value Comparator	Comparator
Value Formatter Type	ValueFormatterType
Default Value Assignment Type	DefaultValueAssignmentType
Default Value Offset Amount	DefaultValueOffsetAmount

In an externally managed rule, values in the other underwriting **Rule** fields have no effect.

Note: After upgrading from PolicyCenter 7.0 or 8.0, legacy underwriting rules have a corresponding externally managed underwriting Rule. You can modify the `UWIssueType` properties listed above in the user interface.

Creating underwriting issues with policy evaluation plugin

The `IPolicyEvaluationPlugin` is the starting point for the rule which creates an underwriting issue. In the base configuration, the `IPolicyEvaluationPlugin` interface is implemented by the `PolicyEvalPlugin` plugin. This plugin creates a `PolicyEvalContext` object and then evaluates (`evaluator.evaluate` method) whether to raise issues for the current checking set and policy lines associated with the policy period. The `evaluate` method first evaluates the underwriting rules defined in the user interface. Then the `evaluate` method evaluates underwriting issues types implemented in Gosu code.

The plugin uses a default class (`DefaultUnderwriterEvaluator.gs`) and line-of-business-specific classes (`PA_UnderwriterEvaluator.gs` for example) to determine whether or not to raise underwriting issues.

When you evaluate each checking set, your implementation must raise issues and refresh existing issues with current values. Your implementation must also remove orphaned issues. Guidewire recommends that your implementation use the `PolicyEvalContext` class to create and remove orphaned issues because these are complex and error-prone tasks.

See also

- *Plugins, Prebuilt Integrations, and SOAP APIs*

Raise underwriting issues in Gosu code

About this task

This topic describes how to raise underwriting issues using Gosu code.

Procedure

1. To raise underwriting issues for all lines of business, modify the default evaluator class, `gw.lob.common.DefaultUnderwriterEvaluator.gs`. To raise underwriting issues for a particular line of business, modify the evaluator class for that line of business, `gw.lob.pa.PA_UnderwriterEvaluator.gs`, for example.
2. Find the appropriate method for the checking set. For example, to raise underwriting issues for the Renewal checking set, add code in the `onRenewal` method of the evaluator class. You can view and modify the correspondence between checking set and method in the `AbstractUnderwriterEvaluator` class.
3. Use the `PolicyEvalContext.addIssue` method to add underwriting issues. To raise underwriting issues, you need to write Gosu code which examines the `PolicyPeriod` and determines whether to raise one or more issues of this type. For example, the code may raise multiple issues if the `PolicyPeriod` entity contains multiple items of the same type, such as cars or buildings.

- a) Add an `addIssue` method (using one the available method signatures). For example:

```
addIssue( issueType, key, shortDescriptionBlock, longDescriptionBlock, value)
```

- b) Provide values for `issueType` and `key`:

issueType Code from the **Underwriting Rule Details** in the underwriting **Rule** definition. Becomes `UWIssue.Code`.

key **Issue Key** in the **Underwriting Issue Details** on the underwriting **Rule** definition. Becomes `UWIssue.IssueKey`.

- c) Provide a block for the `shortDescription` and `longDescription` parameters (one each). This allows the method to localize the description text for all available locales. (PolicyCenter performs the localization as it evaluates the display key.) Use this format:

```
shortDescriptionBlock = \ -> shortDescriptionDisplayKey
longDescriptionBlock = \ -> longDescriptionDisplayKey
```

- d) Enter the value, which is the value that PolicyCenter checks to determine whether to raise the issue.
4. Use the `PolicyEvalContext.removeOrphanedIssues` method to remove orphaned underwriting issues. The `removeOrphanedIssues` method is called in the `PolicyEvalContext` plugin.

Configuring business rules

Guidewire provides a business rules plugin, the `IBizRulesPlugin` plugin, that you can configure to manage certain business rule functionality. For example, it is possible to modify the default plugin implementation to do the following:

- Create new rule contexts and add context symbols
- Add, modify, or delete rule contexts, rule actions, and rule triggers
- Modify the set of functions and entity properties that are visible in the **Rule Condition** editor in the **Business Rules** screens
- Create methods on the `Util` class that are available in autocompletion in the **Rule Condition** editor
- Change the rule permission provider

Guidewire uses a different implementation class for the `IBizRulesPlugin` plugin depending on the installation configuration.

Base configuration

In the base configuration, the implementation class for the `IBizRulesPlugin` plugin is Gosu class `BizRulesPlugin`.

In all cases, make any modifications or customizations to the business rules in class `BizRulesPlugin`.

Business rule subtypes

Guidewire PolicyCenter provides business rules for underwriting only. Guidewire does not recommend, nor does it support, the creation of additional business rule subtypes.

Modifying the business rules functionality

To make modifications to business rules, you make modifications to the following object types in the `ADSBizRulesPlugin` class, the implementation class for the Business Rules plugin.

Method	Function
<code>BlackListedMethods</code>	Provides the list of methods not permitted for non-symbol types that might be present in the context definition. In the base configuration, PolicyCenter permits all methods on non-symbol types present in the context definition.
<code>BlacklistedProperties</code>	Provides a list of not permitted properties on rule context symbols. In the base configuration, PolicyCenter permits all properties on context symbols. This action makes the properties available in

Method	Function
	the Rule Condition builder in the business rule editor. Adding a property to the list hides the property in autocomplete in the Rule Condition builder. Note: Array properties such as <code>claim.Activities</code>, do not allow dereferencing. For example, PolicyCenter does not allow <code>claim.Activities[0]</code>. However, PolicyCenter does permit array properties such as <code>claim.Activities.Count</code>.
RuleActions	Provides the class that defines the valid rule actions. In the base configuration, this is Gosu class <code>AddUWIssueRuleAction</code> . It is possible to modify or replace this Gosu class.
TriggeringPointMap	Maps between triggering points and the associated rule context definitions for each triggering point. It is not possible to add new triggering points.
WhiteListedMethods	Provides a list of methods permitted on the rule context symbols. In the base configuration, PolicyCenter limits methods on context symbols to ensure that invoking a rule does not change the state of an entity. Adding a method to the list permits the symbol method to show on autocomplete in the Rule Condition builder.
LookupEnabled	Override Guidewire default setting to enable or disable the use of lookups in business rule variable expressions.

Working with the business rules plugin class

Configuring business rule editor functionality

It is possible to configure which methods and properties are visible in the business rule editor by modifying the implementation class for the `IBizRulesPlugin` plugin:

- Setting an entity method or property as blocked makes that item not visible in the **Rule Condition** builder.
- Setting an entity method or property as permitted makes that item visible in the **Rule Condition** builder.

It is also possible to enable or disable the use of lookup expressions in business rules in PolicyCenter through configuration of this same implementation class.

Blocking an entity method in the rule condition builder

To block an entity method is to make that method unavailable for use in the **Rule Condition** builder in the business rules editor. The default implementation of the `IBizRulesPlugin` plugin does not block any entity methods.

The following code sample illustrates how to block an entity method.

```
override property get BlackListedMethods(): Set<IMethodReference> {
    return {
        PolicyPeriod#activateDraftFacAgreements(),
        UWIssueType#calculateDefaultValue(String)
    }
}
```

Blocking an entity property in the rule condition builder

To block an entity property is to make that property unavailable for use in the **Rule Condition** builder in the business rules editor. The default implementation of the `IBizRulesPlugin` plugin does not block any entity properties.

The following code sample illustrates how to block an entity property.

```
override property get BlackListedProperties(): Set<IPropertyReference> {
    return {
        PolicyPeriod#Active,
        PersonalAutoLine#AllClauses,
        PersonalAutoLine#AllExclusions
    }
}
```

Permitting an entity method in the rule condition builder

To permit an entity method is to make that method available for use in the **Rule Condition** builder in the Business Rules editor. The default implementation of the `IBizRulesPlugin` plugin provides an example of a business rule allowed method.

The following code sample illustrates how to allow an entity method.

```
override property get WhiteListedMethods(): Map<IMethodReference, String> {
    return {
        Object#toString() -> "converts instance to readable text",
        PolicyContextDefinitionLibrary#hasGoodDriverDiscount(PolicyDriver) ->
            "determines whether a driver qualifies for GoodDriverDiscount"
    }
}
```

Disabling lookup expressions

By default, Guidewire sets the return value of method `isLookupEnabled` to `false` in the non-editable `DefaultBizRulesPlugin` class, which the `IBizRulesPlugin` implementation class extends. The `false` return value disables the use of lookups in business rule expressions.

`PolicyCenter` then overrides the default value of `false` through an override of the method in the default implementation class of the `IBizRulesPlugin` plugin:

```
override property get LookupEnabled() : boolean { return true }
```

To again disable the use of lookups in `PolicyCenter` business rules, change the overridden method so that it returns `false`. After you do so, `PolicyCenter` removes the **Lookup** option from the drop-down picker in the business rule variable and condition builders.

Business rule actions

The default implementation class of the business rules plugin contains an `AddUWIssueRuleAction` action:

```
override property get RuleActions(): Set<IRuleAction> {
    return {new AddUWIssueRuleAction()}
}
```

Whenever `PolicyCenter` executes a rule, it evaluates the rule condition against a `PolicyPeriod` object. If the condition meets the specified criteria, `PolicyCenter` calls the `execute` method on the following class to create an underwriting issue:

```
gw.bizrules.provisioning.AddUWIssueRuleAction
```

This is the main functionality of the `AddUWIssueRuleAction` class. The `CommandParameterDefinitions` property in that class contains parameters needed during execution.

Modifying an existing action

To modify the default implementation of the `AddUWIssueRuleAction` action in plugin implementation class, create an override of the `execute` method of class `AddUWIssueRuleAction` in the implementation class for the `IBizRulesPlugin` plugin, for example:

```
override function execute(command: IRuleCommand) {
    //add your code here
}
```

Adding a new action

Adding a new underwriting issue action is a multi-step process. See “Add new rule action” on page 94 for detailed information.

Add new rule action

Implement the following steps to add an additional action to an existing base configuration rule.

About this task

By default, PolicyCenter shows only a single action for each underwriting rule. This procedure outlines how to modify PolicyCenter to support multiple actions within PolicyCenter business rules.

Procedure

1. Open PolicyCenter Studio and navigate to the following location in the Studio **Project** window:

config > Extensions > Typelist

2. Create a **RuleActionKey** typelist extension, if one does not exist.

- a) Add the following typecode to the typelist extension.

```
code AddCustomIssue
name AddCustomIssue
desc Create a custom issue
```

- b) Manually add an **UWRule** category to the new typecode.

As the Rule typelist is a generated typelist (from `Rule.eti`), you need to enter the category information manually using the **RuleActionKey** typelist **Text** tab.

```
<category code="UWRule" typelist="Rule"/>
```

The resulting typecode definition looks similar to the following code.

```
<typecode code="AddCustomIssue"
  desc="Create a custom issue"
  name="AddCustomIssue">
  <category code="UWRule" typelist="Rule"/>
</typecode>
```

3. In the Studio **Project** window, navigate to `gsrc` and create a package to hold your work.
4. Add a new `AddCustomizedRuleAction` class to your work package and have that class implement the `IRuleAction` interface.

For example, create a class that is similar to the following code.

```
package ...

uses gw.bizrules.CommandParameterDefinition
uses gw.bizrules.IRuleAction
uses gw.bizrules.IRuleCommand

class AddCustomizedRuleAction implements IRuleAction {

  override property get Key() : RuleActionKey {
    return RuleActionKey.TC_ADDCUSTOMISSUE //new RuleActionKey typecode created in a previous step
  }

  override property get CommandParameterDefinitions() : Map<String, CommandParameterDefinition> {
    return null //add your own implementation here
  }

  override function execute(command : IRuleCommand) {
    //custom action code
  }

  override function describe(ruleCommandDefinition : RuleCommandDefinition) : String {
    return null //add new custom implementation here
  }
}
```

IMPORTANT: For this class to be functional, you must add your own implementation code to the class.

5. Open the implementation class for the IBizRulesPlugin plugin.

You need to register your new AddCustomizedRuleAction class in the plugin code.

- a) Find the RuleActions property, which, in the base configuration looks similar to the following code:

```
override property get RuleActions() : Set<IRuleAction> {
    return {
        new AddUWIssueRuleAction()
    }
}
```

- b) Add your new AddCustomizedRuleAction class to this property.

For example, do something similar to the following code.

```
override property get RuleActions() : Set<IRuleAction> {
    return {
        new AddUWIssueRuleAction(),
        new AddCustomizedRuleAction()
    }
}
```

Notice that this change now triggers the execution of two underwriting rule actions.

6. Open class UWRuleBuilder.

- a) Find the constructor that begins with the following code:

```
construct (excludeHead : boolean) {
    ...
}
```

- b) Modify this constructor to add a new populator for RuleActionKey.TC_ADDCUSTOMISSUE.

For example, add something similar to the following code to the constructor.

```
construct (excludeHead : boolean) {
    //existing code..

    //add new populator for the RuleActionKey.TC_ADDCUSTOMISSUE
    addPopulator(\bean -> {
        var builder = new RuleCommandDefinitionBuilder()
        .withRule(bean as UWRule)
        .withOrderNumber(1)
        .withRuleActionKey(RuleActionKey.TC_ADDCUSTOMISSUE)
        for (name in _commandParams.Keys) {
            var param = _commandParams.get(name)
            switch (param.Second) {
                case formula:
                    builder.addRuleCommandParameter(name,
                        ExpressionFragmentBuilders.forCodeExpression(param.First))
                    break
                case template:
                    builder.addRuleCommandParameter(name,
                        ExpressionFragmentBuilders.forGosuTemplateExpression(param.First))
                    break
                default:
                    throw new IllegalArgumentException("Illegal parameter mode: ${param.Second}")
            }
        }
        builder.create(bean.Bundle)
    })
    //existing code...
}
```

Notice also that the code increments the order number by 1 as well.

7. Open class UWRulesPageHelper.

You need to modify class UWRulesPageHelper to include the new TC_ADDCUSTOMISSUE typecode.

- a) Search for static property UIConfigs.
- b) Add the TC_ADDCUSTOMISSUE typecode.

For example, add code similar to the following to this property.

```
static property get UIConfigs(): Map<RuleActionKey, CommandDefinitionUIConfig> {
    return {
        RuleActionKey.TC_ADDUWISSUE->new AddUWIssueCommandDefinitionUIConfig(),
        RuleActionKey.TC_ADDCUSTOMISSUE->new AddUWIssueCommandDefinitionUIConfig()
    }
}
```

What to do next

To actually view multiple rule actions in PolicyCenter, you also need to modify PCF file `UWRuleDetail.pcf` to use `RuleCommandDefinitionsListDetailView.pcf` instead of `UWIssueCommandDefinitionDV.pcf`.

Triggering point mapping

In the implementation class for the `IBizRulesPlugin` plugin, Guidewire provides the following trigger point mapping to supported context definitions.

```
override property get TriggeringPointMap(): Map<TriggeringPointKey, IRuleContextDefinition[]> {
    return TriggeringPointKey.TF_UWRULECHECKINGSETFILTER.TypeKeys
        .mapToKeyAndValue{\t -> t, \t -> _uwContexts}
}
```

You can add, delete, or modify the default underwriting rule triggering points.

Custom rule utility functions

In the Business Rules editor, the **Rule Condition** builder permits the definition of expressions that contain custom utility functions. It is possible for a custom function to return a string formatted as a money amount in the given currency if given a big decimal value and a currency type.

To access a custom function in the expression builder, use the following syntax, with `xxxx` being the name of the custom method:

```
Util.xxxx()
```

Gosu class `PolicyContextDefinitionLibrary`, in the `gw.bizrules.provisioning.contexts` package, defines the available custom functions. In the base configuration, this class contains such functions as `hasGoodDriverDiscount` and `monetaryAmount`.

To add new functions to this class, use the existing functions as examples of how to define a new function. In this way, you can make a utility function accessible as a context symbol. You can also exclude a utility function from a rule context.

Rule contexts and rule context definitions

A rule context and a rule context definition are not identical business rule constructs. It is important that you understand the difference between the following concepts.

Context definition

PolicyCenter uses the rule context definition to define the symbol table that provides the list of symbols available in the business rules editor while creating or editing a rule during rule development.

Context

PolicyCenter uses the rule context as a runtime construct to populate the context definition symbols with actual data from the rule that triggered the current action. It then uses this symbol data during execution of the triggered rule.

PolicyCenter business rule context definition

Guidewire defines rule contexts in the following business rules package:

```
gw.bizrules.provisioning.contexts
```


This package contains context definition classes such as the following, plus many more:

- CommercialPropertyUWContextDefinition
- GenericUWRuleContextDefinition

A rule context definition can contain one or more context symbols. For example, Gosu class CommercialPropertyUWContextDefinition contains the following addSymbol definitions in the class constructor:

```
construct() {
    addSymbol(PARAM_CPLINE, CommercialPropertyLine, \ec -> ec.Period.CPLine)
    addSymbol(PARAM_CPLOCATIONS, CPLocation[], \ec -> ec.Period.CPLine.CPLocations)
    addSymbol(PARAM_CPBUILDINGS, CPBuilding[], \ec -> ec.Period.CPLine.CPLocations.
        arrays("Buildings").toArray(new CPBuilding[0]))
}
```

Add new underwriter rule context definition

Procedure

1. Open PolicyCenter Studio.
2. In the Studio **Project** window, navigate to **configuration > gsrc > gw > bizrules > provisioning > contexts**.
 - a) In the contexts package, create a class that extends the GenericUWRuleContextDefinition class.
 - b) In a similar manner to the existing symbol definitions, edit the class constructor:
 - For each new context symbol, call the addSymbol method for that symbol.
 - If this is to be an iterable rule context definition, call exactly one addIterativeSymbol method.
 - c) Define the applicability of the new rule context definition to the PolicyCenter policy lines in function appliesToPolicyLines.
3. In the Studio **Project** window, navigate to **configuration > config > Extensions > Typelist**.
 - a) Create a RuleContextDefinitionKey typelist extension, if one does not exist.
 - b) Add a typecode to this typelist for the rule context definition that you created in the previous step.
4. In the Studio **Project** window, navigate to **configuration > gsrc > gw > plugin > bizrules.impl**.
 - a) Open the implementaton class for the IBizRulesPlugin plugin.
 - b) Add an entry to the class constructor for the new rule context definition. Use an existing entry as an example.
5. In the Studio **Project** window, navigate to **configuration > gsrc > gw > bizrules > provisioning > triggeringpoints**.
 - a) Open class UnderwriterEvaluatorTriggeringPoint.
 - b) In function supportedContexts, add an entry for the typecode that you added to the RuleContextDefinitionKey typelist. Use an existing entry as an example.
6. Restart the application server for these changes to become effective.
7. Open Guidewire PolicyCenter and navigate to **Administration > Business Settings > Business Rules > Underwriting Rules**.
 - a) Create a new Underwriter rule.
 - b) Verify that the new context symbol shows in the Context drop-down of the rule editor.
 - c) Experiment with the different policy lines that show in the Applies To section of the rule editor.

Configuring applicability criteria

PolicyCenter application logic triggers the execution of a business rule if the business rule meets all of the following conditions:

- The trigger and context of the triggering point match that of the rule.
- The applicability criteria, as set in the **Applies To** section of the rule definition screen.

Base configuration applicability criteria

In the base configuration, PolicyCenter defines the following types of rule applicability criteria.

Applicability criteria	Array name	Array entity
Jurisdictions	Jurisdictions	AppCritJurisdiction
Policy lines	LinesOfBusiness	AppCritLineOfBusiness
Policy transactions	PolicyTransactions	AppCritLineOfBusiness

The definition for these applicability criteria exist in the non-editable `UWRule.eti` type metadata. To work with rule applicability criteria, perform your work in `UWRule.etx`, which is an editable extension of `UWRule`.

As part of the applicability criteria test, PolicyCenter also tests whether the rule execution date (today's date) is between the defined start and end dates for the rule.

Single- and multi-value applicability criteria

Applicability criteria can encompass a single value or a point in time. Or, applicability criteria can be multi-valued. For example, encompassing both the states of California and Nevada.

The business rule data model handles single- and multi-value applicability criteria differently.

Single-value rule criteria

The business rule data model stores single applicability values as a property on the `UWRule` entity type. To use a single-value criterion, add properties for single values to the `UWRule` entity type. Then, update the PCF file that displays the rule.

Multi-value rule criteria

The business rules data model stores multiple applicability values in their own entity type with a foreign key to the `UWRule` entity type. These linked entities need to implement the `RuleVersionDependent` interface. Implementing this interface ensures that PolicyCenter imports and exports these entity properties as part of the rule.

Filtering single- and multi-value applicability criteria

Guidewire provides Gosu class `UWRuleApplicabilityCriteriaPredicate` to filter the execution of the business rules based on applicability criteria. This class contains a test method that takes in a `RuleVersion` object and returns true or false, depending on whether the associated rule matches the defined applicability criteria. Each class that implements the `ITriggeringPoint` interface uses the `UWRuleApplicabilityCriteriaPredicate` class to determine how to filter the rules for execution whenever application logic triggers the business rules framework.

Add a new PolicyCenter applicability criterion

About this task

Adding a new applicability criterion to the set of availability criteria available in the PolicyCenter base configuration is a multistep process.

Procedure

1. Create a new data entity type for your applicability criterion.
Use the existing applicability criteria entity types as examples:
 - `AppCritLineOfBusiness`
 - `AppCritJurisdiction`

- AppCritPolicyTransation

Use the same naming conventions as the existing examples.

2. Add the applicability criterion to the `UWRule.etx` entity type extension as an array of the entity type that you created in the previous step.

Use an existing array definition as an example.

3. Extend class `UWRuleApplicabilityCriteriaPredicate` and override the default behavior of the `test` method to add, modify, or remove filtering restrictions.
4. Create new `ITriggeringPoint` implementation classes that use the `Predicate` class that you created in the previous step.
5. Update PCF `UWRulePanelSet` to include the new input sets in the `ApplicabilityCriteria` pane.

The PCF `ApplicabilityCriteria` pane shows as the **Applies To** section of the business rule definition screen.

6. (Optional) Update PCF `UWRules` so that the list view shows a column for the new applicability criterion.
7. (Optional) Update Gosu class `UWRuleFilterCriteria` and the associated `UWRuleFilterCriteriaDV` PCF to improve the search filtering view on the **Underwriting Rules** screen.

You access the search functionality of the `UWRuleFilterCriteriaDV` PCF pane by clicking **Show Filters** on the **Underwriting Rules** screen.

Configuring policy data spreadsheet import/export

The base PolicyCenter configuration includes three classes that allow you to import and export policy data to a spreadsheet:

- APDExportDataWebFactory.gs
- APDExportDataWebFactoryLOBProvider.gs
- APDExportDataWebFactoryLOBProviderFactory.gs

This policy data could include buildings, locations, or any other entity type. PolicyCenter also provides import/export PCF files that you can reference in your PolicyCenter configuration.

To set up import/export for an entity, you must follow the steps below:

1. Register the export data web factory class.
2. Create an XML file that lists the entity columns to import and export.
3. Define column data resolvers.
4. Create an import strategy for each new data column resolver.
5. Register the import strategy class.

Register the export data web factory class

Procedure

Create a subclass of APDExportDataWebFactoryLOBProvider. The class must follow the naming convention LOBCodeInUppercase + ExportDataWebFactory, and it must be placed in the gw.exportimport.web package. For example:

```
package gw.exportimport.web

uses gw.exportimport.EntityInfo
uses gw.exportimport.ExportData

class ACAExportDataWebFactory implements APDExportDataWebFactoryLOBProvider {

    override function createLOBBasedExportDataCreators() : Map<Type, APDExportDataWebFactory.APDExportDataCreator> {
        var toACAVehicleCreator = new APDExportDataWebFactory.APDExportDataCreator(\period : PolicyPeriod, maskSelection :
ExportMaskSelection -> {
            var entityInfo = APDExportDataWebFactory.APDExportDataCreator.createEntityInfo(maskSelection) as
```

```
EntityInfo<ACAVehicle>
    var vehicles = (period.Lines[0] as ACAGoCommercialAuto).ACAVehicles.toList()
    return new ExportData<ACAVehicle>(entityInfo, vehicles)
})

return {
    ACAVehicle -> toACAVehicleCreator
}

override property get LOBKey() : InstalledPolicyLine {
    return InstalledPolicyLine.TC_ACA
}
}
```

Create an XML file describing columns to import and export

About this task

For each entity that you want to export and import, create an XML file that describes the columns to export and import.

Note that for APD-managed products there are limitations on which entities can be imported or exported. These limitations are explained in the steps below.

Procedure

1. Create a new XML file that describes the fields to export from and import into the entity. Name the file *EntityNameFlow.xml* and save it in **configuration > config > resources > exportimport** in Studio.
2. Add the following root element to the file:

```
<EntityInfo
  EntityType="entity.EntityName"
  ParentEntityType="entity.parentEntityName"
  ParentEntityColumnPath="EntityName.ID">
```

Attribute	Required?	Description
EntityType	Required	The name of the entity preceded by: entity.
ParentEntityType	Optional	If the parent entity is also to be imported and exported, the name of the parent entity preceded by: entity.
ParentEntityColumnPath	Optional	The path that identifies the unique ID column of the parent entity, if a parent entity is specified.

3. Within the EntityInfo element, add a collection of ColumnInfo child elements. Each ColumnInfo element specifies one column to be exported to and imported from the spreadsheet. The order of the ColumnInfo elements determines the order of the columns in the exported spreadsheet.

```
<ColumnInfo
  ColumnType="pathTypeName"
  ExcludeFromTemplate="true|false"
  FlagsAction="true|false"
  FlagsEntityId="true|false"
  Header="displayKeyPath"
  id="columnName"
  Locked="true|false"
  Path="beanPathToColumnValue"
  RequiredForImport="true|false">
```

Attribute	Required?	Description
ColumnType	Optional	The full path type name of the column's data value. If the column type cannot be handled by one of the predefined data column resolvers, you must define a new data

Attribute	Required?	Description
		column resolver for each such type. Then adjust the <code>ColumnDataResolverFactory</code> class to select the new type when appropriate. Default: <code>String</code> .
<code>ExcludeFromTemplate</code>	Optional	Whether to exclude the column when exporting a template. Default: <code>false</code> .
<code>FlagsAction</code>	Optional	Whether the column specifies the action for the spreadsheet row. Only one column can have this attribute set to true, and this is the Action column, typically exported as the first column in the spreadsheet. All other attributes except Header are ignored. Default: <code>false</code> .
<code>FlagsEntityId</code>	Optional	Whether the column uniquely identifies the entity. Default: <code>false</code> .
<code>Header</code>	Required	The display key path that contains the string that is output as the column header. Using a display key enables the column headers to be localized. Specify the Header value for the <code>Action</code> column as <code>Export.Action</code> . PolicyCenter writes the column display keys under <code>Export > Entity</code> , though a display key for that name may already exist at some other location. You can quickly look at all exported column data under the <code>Export</code> node.
<code>Id</code>	Optional	The name identifier of the column upon which other columns are dependent. Used only when another <code>ColumnInfo</code> specifies a Required element with this <code>ColumnInfo</code> as its target.
<code>Locked</code>	Optional	Whether the column value cannot be modified. Locked columns are set to read-only and shaded gray in the exported spreadsheet. Default: <code>False</code> .
<code>Path</code>	Optional	Required except when <code>FlagsAction</code> is true. The bean path that accesses the column value of the entity.
<code>RequiredForImport</code>	Optional	Whether the column value must be specified when the spreadsheet is imported. Default: <code>false</code> .

APD-managed products support the bulk export and import of large schedules, however the following limitations apply:

- Fields of type Date-Time, Involved Party, and Other are not supported.
- For risk fields of type Location, `flow.xml` generates field names from `PolicyLocation` as hardcoded values (AddressLine1, City, State, for example) under `config/resources/exportimport`. If you have customized field names for `PolicyLocation`, you must adjust the mappings manually in `flow.xml`.

4. If the entity you are defining is a child of another entity that is to be exported and imported, add a `Parent` element within the `EntityInfo` element. Within the `Parent` element, add a map of the parent's `ColumnInfo` elements.

For example:

```
<EntityInfo ...>
.
.
.
  <Parent>
    <ColumnInfo
      Path="CPLocation.PublicID"
      Header="Export.CPBuilding.LocationID"
      Locked="true"
      ColumnType="java.lang.String" />
    <ColumnInfo
      Path="CPLocation.Location.LocationName"
      Header="Export.CPBuilding.LocationName"
      ColumnType="java.lang.String" />
    ...
  </Parent>
.
.
.
</EntityInfo>
```

5. Optionally, add dependent elements.

A `ColumnInfo` element can also contain an optional map of `Dependent` child elements, each of which specifies another column on which the containing `ColumnInfo` is dependent. Each `Dependent` element takes a single attribute, `purpose`, that is a key to the id of a `ColumnInfo` on which the containing `ColumnInfo` is dependent. For example:

```
<!-- Column that can have dependent columns because it has an id attribute -->
<ColumnInfo
  Path="CPLocation.Location.State"
  Header="Export.CPBuilding.State"
  ColumnType="typekey.State"
  id="stateColumn"
  RequiredForImport="true" />
.
.
.
<!-- Column that depends on another column -->
<ColumnInfo
  Path="ClassCode"
  Header="Export.CPBuilding.ClassCode"
  ColumnType="entity.CPClassCode"
  RequiredForImport="true" >
  <Dependent purpose="State">stateColumn</Dependent>
</ColumnInfo>
```

Example

```
<?xml version="1.0"?>
<EntityInfo
  EntityType="entity.ACAVehicle">
  <ColumnInfo
    Header="Export.Action"
    FlagsAction="true" />
  <ColumnInfo
    Path="PublicID"
    Header="Export.ACAVehicle.VehicleID"
    Locked="true"
    FlagsEntityID="true"
    ExcludeFromTemplate="true" />
  <ColumnInfo
    Path="ACAVehicleLicState"
    Header="Export.ACAVehicle.ACAVehicleLicState"
    ColumnType="typekey.LicenseState"/>
  <ColumnInfo
    Path="ACAVehicleType"
    Header="Export.ACAVehicle.ACAVehicleType"
    ColumnType="typekey.ACAVehicleType"/>
  <ColumnInfo
    Path="ACAVehicleCostNew"
    Header="Export.ACAVehicle.ACAVehicleCostNew"
    ColumnType="java.math.BigDecimal"/>

  <ColumnInfo
    Path="ACAVehicleVIN"
    Header="Export.ACAVehicle.ACAVehicleVIN"
    ColumnType="java.lang.String"/>

  <ColumnInfo
    Path="ACAVehicleYear"
    Header="Export.ACAVehicle.ACAVehicleYear"
    ColumnType="java.lang.Integer"/>

  <ColumnInfo
    Path="ACAVehicleMake"
    Header="Export.ACAVehicle.ACAVehicleMake"
    ColumnType="java.lang.String"/>

  <ColumnInfo
    Path="ACAVehicleNumber"
    Header="Export.ACAVehicle.ACAVehicleNumber"
    ColumnType="java.lang.Integer"/>

  <ColumnInfo
    Path="ACAVehicleModel"
    Header="Export.ACAVehicle.ACAVehicleModel"
    ColumnType="java.lang.String"/>

  <ColumnInfo
    Path="ACAVehicleBodyType"
    Header="Export.ACAVehicle.ACAVehicleBodyType"
    ColumnType="typekey.ACABodyType"/>

  <ColumnInfo
    Path="ACAVehicleUse"
```



```

    Header="Export.ACAVehicle.ACAVehicleUse"
    ColumnType="typekey.VehicleUse"/>

    <ColumnInfo
      Path="ACAVehicleSeatingCapacity"
      Header="Export.ACAVehicle.ACAVehicleSeatingCapacity"
      ColumnType="typekey.SeatingCapacity"/>

    <!-- no support for boolean types?
    <ColumnInfo
      Path="ACAVehiclePartOfFleet"
      Header="Export.ACAVehicle.ACAVehiclePartOfFleet"
      ColumnType="java.lang.Boolean"/>!-->

    <ColumnInfo
      Path="ACAVehicleDriveToWork"
      Header="Export.ACAVehicle.ACAVehicleDriveToWork"
      ColumnType="typekey.DriveToWorkSchool"/>

    <ColumnInfo
      Path="ACAVehicleRadius"
      Header="Export.ACAVehicle.ACAVehicleRadius"
      ColumnType="typekey.Radius"/>

    <ColumnInfo
      Path="ACAVehicleFarthestTerminal"
      Header="Export.ACAVehicle.ACAVehicleFarthestTerminal"
      ColumnType="typekey.FarthestTerminal"/>

    <ColumnInfo
      Path="ACAVehicleGrossVehWeight"
      Header="Export.ACAVehicle.ACAVehicleGrossVehWeight"
      ColumnType="java.lang.Integer"/>

    <ColumnInfo
      Path="ACAVehicleGrossCombinedWeight"
      Header="Export.ACAVehicle.ACAVehicleGrossCombinedWeight"
      ColumnType="java.lang.Integer"/>

    <ColumnInfo
      Path="ACAVehicleStatedAmount"
      Header="Export.ACAVehicle.ACAVehicleStatedAmount"
      ColumnType="java.math.BigDecimal"/>

    <ColumnInfo
      Path="ACAVehicleTerritory"
      Header="Export.ACAVehicle.ACAVehicleTerritory"
      ColumnType="java.lang.String"/>

    <ColumnInfo
      Path="ACAVehiclePrimaryClassCode"
      Header="Export.ACAVehicle.ACAVehiclePrimaryClassCode"
      ColumnType="java.lang.Integer"/>

    <ColumnInfo
      Path="ACAVehicleSecondaryIndClassCd"
      Header="Export.ACAVehicle.ACAVehicleSecondaryIndClassCd"
      ColumnType="java.lang.Integer"/>

  </EntityInfo>

```

Define column data resolvers

Each type (class) other than String to be exported and imported must have its own `ColumnDataResolver` class. This class specifies how to convert a value to a string on export and convert the string back into the correct value type on import. PolicyCenter defines the following `ColumnDataResolvers`:

- `NullColumnDataResolver` – Imports nothing. Typically used for the **Action** column and for any informational column that is processed during import or export operations.
- `SimpleColumnDataResolver` – Exports and imports strings and numbers. It is so named because processing strings and numbers is simple compared to other types.
- `TypeKeyColumnDataResolver` – Exports a resolved type key as a string and imports a string as a type key.
- `CoverageColumnDataResolver` – Exports the display value of a coverage term as a string. Imports the localized coverage term value using the dotted path to resolve the coverage pattern and coverage term pattern.
- `TerritoryCodeColumnDataResolver` – Exports a territory code value as a string and imports a string as a territory code value.

- `TaxLocationColumnDataResolver` – Exports a tax location value as a string and imports a string as a tax location value.
- `ClassCodeColumnDataResolver` – Exports a class code value as a string and imports a string as a class code value.
- `AbstractColumnDataResolver` – An abstract class that uses iterative reflection to resolve the dotted bean path to the value that needs to be exported and imported. Typically you extend this class when defining new column data resolvers.
- `ColumnDataResolver` – An interface that is responsible for importing and exporting cell data values. Implemented by `AbstractColumnDataResolver`.

If you need to export and import a type (class) that is not handled by one of the provided column data resolvers, you must implement a new one. In Studio navigate to **configuration** > **gsrsrc** and define a new class in `my_domain_name.exportimport.resolver` where `my_domain_name` is the package for your company's files. Give the class a name appropriate to the column type it is designed to resolve.

Typically it is sufficient to leverage the `AbstractColumnDataResolver` and implement the method `#calculateCellValue`. This method constructs and returns a new object of a type that matches the `ColumnInfo` type.

For example, you can implement a new `TaxLocationColumnDataResolver` using `CPCClassCodeColumnDataResolver` as a model. If the `ColumnDataResolver` needs more context to resolve the value, you can define dependent data for the `ColumnInfo`. In this case, the `TaxLocationColumnDataResolver` requires state information to calculate the jurisdiction and find the `TaxLocation`. This dependency must be completed by using the `Dependent` element and `id` attribute in the `EntityInfo` XML file.

See also

- For more information about dependent elements, “Create an XML file describing columns to import and export” on page 102

Update the `ColumnDataResolverFactory` class

About this task

For each new column data resolver, edit the `ColumnDataResolverFactory` class to select the new type when appropriate.

Procedure

1. To open this class, use Studio to navigate to **configuration** > **gsrsrc** and open the class `gw.exportimport.resolver.ColumnDataResolverFactory`.
2. Map each new column data resolver class to an appropriate type in the `TYPE_RESOLVER_MAP`.

Create an import strategy for each new data column resolver

About this task

The import strategy defines how (and whether) to create and delete entities during an import operation. To create an import strategy:

Procedure

1. In Studio, navigate to **configuration** > **gsrsrc** and define a new class in `gw.exportimport.entityimport`. Name the class to reflect the entity you are configuring. For example, if you are configuring the `PersonalVehicle` entity for import and export, name the new class `PersonalVehicleImportStrategy`.
2. Implement the `deleteEntity` and `createEntity` methods as needed, using return values that are appropriate to the entity.
3. Specify a true or false return value for the `AllowCreate` and `AllowDelete` properties. If the entity you are configuring has complex deletion requirements, consider prohibiting delete during import.

Examine the `CPLocationImportStrategy.gs` and `CPBuildingImportStrategy.gs` as models for creating new import strategies.

- `CPLocationImportStrategy` illustrates how to allow creation of new entities and prevent deletion. Deleting locations is prohibited. PolicyCenter does not allow deleting a location that is in use, and there is no easy way to determine if a location is in use during an import operation.
- `CPBuildingImportStrategy.gs` illustrates a somewhat more complex scenario of creating buildings within a location as well as the scenario of deleting a building.

Register the import strategy class

Procedure

1. Create a subclass of `ImportStrategyRegistryLOBProvider`. The class must follow the naming convention `LOBCodeInUppercase + ImportStrategyRegistry`, and it must be placed in the `gw.exportimport.entityimport` package. For example:

```
package gw.exportimport.entityimport

class ACAImportStrategyRegistry implements ImportStrategyRegistryLOBProvider {
    override function createLOBBasedImportStrategy() : Map<Type, EntityImportStrategy> {
        return {
            ACAVehicle -> new ACAVehicleImportStrategy()
        }
    }

    override property get LOBKey() : InstalledPolicyLine {
        return InstalledPolicyLine.TC_ACA
    }
}
```

2. Create a subclass of `EntityImportStrategy` for the entity that you want to import/export. For example:

```
package gw.exportimport.entityimport

class ACAVehicleImportStrategy implements EntityImportStrategy<ACAVehicle, KeyableBean> {
    override function deleteEntity(vehicle : ACAVehicle) {
        vehicle.GoCommercialAuto.removeFromACAVehicles(vehicle)
    }

    override function createEntity(period : PolicyPeriod, parentEntity : KeyableBean) : ACAVehicle {
        return period.GoCommercialAutoLine.createAndAddACAVehicle()
    }

    override property get AllowCreate() : boolean {
        return true
    }

    override property get AllowDelete() : boolean {
        return true
    }
}
```


Changing the spreadsheet protection password

The worksheets within the exported spreadsheets are protected and designed to be used while protected. In the base configuration, the password is set to 1234.

The spreadsheet protection password is not intended to provide security. It exists to help spreadsheet users maintain the integrity of the data in the spreadsheet. If the data in your exported spreadsheets is sensitive, you must implement appropriate measures to ensure the required level of security.

Change the worksheet protection password

Procedure

1. In Studio, navigate to `gw.exportimport` and open `ExcelExporter.gs`.
2. In the `ExcelExporter` class, find the `SpreadsheetPassword` variable.
3. Change the value of `SpreadsheetPassword`.
4. Restart PolicyCenter to deploy the change.

Configuring locations

You can configure locations in various ways.

- `PolicyLocation`, `AccountLocation`, and line of business locations are extendable, as are all of the entities in the default configuration.
- You can configure which fields are revisioned on `PolicyLocation`.
- You can configure at what points and on which policy transactions location information is synchronized.

Creating a new account location

To add a new `AccountLocation` to the account, call the `newLocation` method in the `Account` class.

Creating a new policy location

If the associated account location is not yet defined on the account:

- A new account level location is created on the account, and a new associated policy level location is created on the policy.
- The policy level location has a foreign key to the account level location.

If the associated account location is already defined on the account:

- A new associated policy level location is created on the policy.

To add a new `PolicyLocation` from a policy period, call one of the methods in the `PolicyPeriod` class:

- `newLocation()` – Creates a new `PolicyLocation` and a new `AccountLocation`.
- `newLocation(accLocation : AccountLocation)` – Creates a new `PolicyLocation` linked to the existing `AccountLocation` passed into the method.

Adding a revisioned field to a location

This topic describes how to add revisioned fields between account and the `Location` entity.

This topic provides general instructions and step-by-step instructions for adding a revisioned field. Follow the steps to add a `FloodPlain` revisioned field to the `AccountLocation` entity. This Boolean field indicates whether the location is on a flood plain.

See also

- *Data Model Configuration*

Define revisioned field on account location

About this task

Define the field on the account. In the default configuration for locations, the revisioned fields are on the AccountLocation entity. If the field already exists, you do not need to define it. If the field does not exist, add the field to an extension entity for AccountLocation.

The step-by-step instructions add a FloodPlain field to the AccountLocation entity.

To add the revisioned field to the account location, you first define the revisioned field for AccountLocation in an extension entity.

Procedure

1. In Studio, navigate to **configuration > config > Extensions > Entity**.
2. Right-click **Entity**, and select **New > Entity Extension**.
3. Select AccountLocation and click **OK**.
4. In AccountLocation.etx enter a column with the following values:

Name	Value
name	FloodPlain
type	bit
desc	Whether the location is on a flood plain.

What to do next

“Define revisioned field on policy location” on page 112

Define revisioned field on policy location

Before you begin

“Define revisioned field on account location” on page 112

About this task

In the default configuration for locations, the revisioned fields are on the PolicyLocation entity. Define an internal field on PolicyLocation in an extension entity. Define the field as follows:

- Append Internal to the field name to signal an internal field. This convention is a Guidewire best practice for internal field names. All revisioned account syncable fields in the base configuration follow this naming convention.
- If this is a new field, append _Ext to the field name to avoid name conflicts with base PolicyCenter entities. For more information, see *Data Model Configuration*.
- Set the value of the setterScriptability and getterScriptability attributes to doesNotExist. This value makes the field internal and not visible to Gosu code. The field will not appear in the data dictionary. For more information about these attributes, see *Data Model Configuration*.

To define the revisioned field on the policy location:

Procedure

1. In Studio, navigate to **configuration > config > Extensions > Entity** and open PolicyLocation.etx.
2. Add a column to the entity with the following values:

Name	Value
name	FloodPlainInternal_Ext
desc	Whether the location is on a flood plain
getterScriptability	doesNotExist
setterScriptability	doesNotExist
type	bit

What to do next

“Define revisioned field as syncable on account location” on page 113

Define revisioned field as syncable on account location

Before you begin

“Define revisioned field on policy location” on page 112

About this task

Add the new field to the `gw.account.AccountLocationToPolicyLocationSyncedField.gs` class. Define the new field as revisioned and syncable on the account.

The `AccountLocationToPolicyLocationSyncedField` constructor assumes that the field name is appended by `Internal`. If the field name is appended by `Internal`, copy one of the existing the variable definitions and modify it for the new field. For example, you can copy and modify `AddressLine1`:

```
public static final var AddressLine1 : AccountLocationToPolicyLocationSyncedField<String>
    = new AccountLocationToPolicyLocationSyncedField<String>("AddressLine1")
```

If the field name is appended with `Internal` and an extension, `Internal_Ext` for example, copy one of the existing the variable definitions and modify it for the new field. Then add a second parameter for the name of the field on the policy location. You must also define a constructor with two arguments for the `accountEntityFieldName` method.

The instructions describe how to define the field if it is appended by `Internal_Ext`.

To define the revisioned field appended with `Internal_Ext`:

Procedure

1. In Studio, open `gw.account.AccountLocationToPolicyLocationSyncedField.gs`.
2. Define a constructor for `accountEntityFieldName` with two arguments:

```
construct(accountEntityFieldNameArg : String, policyEntityFieldNameArg : String) {
    super(accountEntityFieldNameArg, policyEntityFieldNameArg)
}
```

3. Add the `FloodPlain` field to the `AccountLocationToPolicyLocationSyncedField` class:

```
class AccountLocationToPolicyLocationSyncedField<T> extends
    AbstractAccountSyncedFieldImpl<PolicyLocation, T> {
    ...
    public static final var FloodPlain : AccountLocationToPolicyLocationSyncedField<String> =
        new AccountLocationToPolicyLocationSyncedField<String>("FloodPlain",
            "FloodPlainInternal_Ext")
    ...
}
```

What to do next

“Define revisioned field as syncable on policy location” on page 114

Define revisioned field as syncable on policy location

Before you begin

“Define revisioned field on account location” on page 112

About this task

Define the revisioned field as syncable on the policy location by adding it to the `ACCOUNT_SYNCED_FIELDS` variable in the `gw.policylocation.PolicyLocationAccountSyncableImpl` Gosu class.

Procedure

1. In Studio, navigate to **configuration > gsrc** and open `gw.policylocation.PolicyLocationAccountSyncableImpl.gsu`.
2. Add FloodPlain to the `ACCOUNT_SYNCED_FIELDS` variable:

```
static final var ACCOUNT_SYNCED_FIELDS = ImmutableSet.of({
    ...
    AccountLocationToPolicyLocationSyncedField.FloodPlain,
    ...
})
```

What to do next

“Add get and set methods to policy location” on page 114

Add get and set methods to policy location

Before you begin

“Define revisioned field as syncable on policy location” on page 114

About this task

The `gw.policylocation.PolicyLocationEnhancement` has get and set methods for syncable fields. To find these methods, open the file and search for `Shared` and `revisioned`. You can copy one of the existing methods and modify the code for your field.

Procedure

1. In Studio, open `gw.policylocation.PolicyLocationEnhancement.gsu`.
2. Search for `Shared` and `revisioned`.
3. Copy the get and set methods for `AddressLine1`, and paste it in the file.
4. Modify the pasted code. The get and set methods for `FloodPlain` are:

```
/**
 * Shared and revisioned flood plain.
 */
property get FloodPlain() : String {
    return AccountLocationToPolicyLocationSyncedField.FloodPlain.getValue(this)
}

/**
 * Shared and revisioned flood plain.
 */
property set FloodPlain(arg : String) {
    AccountLocationToPolicyLocationSyncedField.FloodPlain.setValue(this, arg)
}
```

What to do next

“Add revisioned field to PolicyCenter user interface” on page 115

Add revisioned field to PolicyCenter user interface

Before you begin

Complete the step “Add get and set methods to policy location” on page 114 before you perform this step.

Procedure

To see this field in PolicyCenter, add it to the `PolicyFileLocations`, `AccountLocationPopup`, `AccountLocationDetailInputSet`, and other PCF pages.

Configuring geocoding

PolicyCenter provides geocoding, which assigns a latitude and longitude to an address, for locations. The default configuration provides Gosu methods to search for locations centered around an address. You can extend the search criteria to meet your business needs.

The `Address` entity contains fields for geocoding. The geocoding fields on `Address` are `Latitude`, `Longitude`, and `GeocodeStatus`. Successful geocode statuses include `exact`, `postalcode`, `street`, and `city`. Each status indicates the degree of precision in identifying the geographic location of an address.

Geocoding is performed when you create or edit an `AccountLocation`. There is also a **Geocode Writer** batch process which runs periodically to update geocoding information on locations.

Geocoding is implemented with the `GeocodePlugin`. In the default installation, the `GeocodePlugin` is implemented by an `AzureMapsPlugin` class, which integrates with the Azure Maps Geocode Service. You must obtain a valid Azure Maps application key from Microsoft to use this implementation. Alternatively, the deprecated Microsoft Bing Maps is still supported. In this case the `BingMapsPluginRest` class is used, and a Microsoft Bing Maps application key is also required.

Policy location

Policy location geocoding methods are located in the `gw.policylocation` package.

`PolicyLocationSearchCriteriaEnhancement.gsx` specifies search criteria and allows you to search for locations matching those criteria. This enhancement allows you to search for locations in policies:

- By line of business
- In the same account
- In bound policies
- In open submission policy transactions
- Within a specified radius

Implementing your own unused location removal

In the base configuration, PolicyCenter removes unused locations from policies in a line of business.

You can choose to implement your own code that removes unused locations for a line of business. This is only recommended if your line of business handles unused location removal through other means such as a location management screen.

If you implement your own code to remove unused locations, you can stop PolicyCenter from performing location cleanup on that policy line. Override the Boolean `PolicyLineMethods.isUnusedLocationCleanupEnabled` method to return `false` for that policy line.

Configuring the Team tab

The Team Screens batch process generates the **Summary** statistics that show in the screens accessible from the PolicyCenter **Team** tab. In the default configuration, this batch process runs once an hour. If you have the requisite access and permission, you can run this batch process manually.

Note: Through configuration, you can create additional categories for statistics. However, the **Team Screens** batch process will not generate statistics for those categories. The **Team Screens** batch process only calculates statistics for activities, submissions, renewals, and other policy transactions.

Setting the window size for team statistics

The window sets the time boundary for submissions, activities, or other reporting categories that appear in the **Team** tab. The window applies to all columns except the **Open** column. The **Calculated on** field is the time when the batch process ran and is the end of the window. The start of the window is calculated backwards from that time.

In the default configuration, the reporting categories for activities and policy transactions are for one week.

You can set the window for reporting categories to the following sizes:

Window size	Description
This week	The window starts at the beginning of the current business week until the most current run of the Team Screens batch process. The start of current business week is the end of the last business week. In the base configuration, the end of a business week is configured as Friday at 5:00 p.m. Therefore, the start of the current business week is 5:00 pm of the Friday prior to the current day. If that day is a holiday, then PolicyCenter goes back to the last know business day.
This month	The start of the current month until the most current run of the Team Screens batch process. The start of the current month is the first day of the current month at 12:00 a.m.
N days	The window is the last N days of activity, including the current day. The start of a day is 24 hours prior to the most current run of the Team Screens batch process. Therefore, for a one day window, if the batch process runs at July 11 10:54 a.m., the start of the window is July 10 10:54 a.m.

You can configure the statistics window for all reporting categories except activities.

In Studio, you configure the window size in `config.xml` by modifying the following parameters:

Parameter name	See also
ActivityStatisticsWindowSize	"ActivityStatisticsWindowSize" in <i>Configuration Parameters</i>
OtherWorkOrdersStatisticsWindowSize	"OtherWorkOrdersStatisticsWindowSize" in <i>Configuration Parameters</i>
RenewalsStatisticsWindowSize	"RenewalsStatisticsWindowSize" in <i>Configuration Parameters</i>
SubmissionsStatisticsWindowSize	"SubmissionsStatisticsWindowSize" in <i>Configuration Parameters</i>

How PolicyCenter calculates reporting categories

The following tables describes how PolicyCenter calculates which activities, jobs, or policy transactions appear in each reporting category depending upon the window. PolicyCenter calculates the status when it runs the **Team Screens** batch process. The **Team** tab displays the time that the batch process ran in the **Calculated on** field.

Note: You cannot change how PolicyCenter calculates the status of reporting categories.

Activities with a One Day Window Example

The Calculation column in the following table describes how PolicyCenter calculates the status for each activity in the **Activities** reporting category.

The Example column describes which activities will be included. The example assumes that the statistics were last calculated at July 11 10:54 a.m. and that the activities window is one day.

Status	Calculation	Example
Open	Activity with open status	Includes any open activity. The window does not apply.
Overdue	Activity with open status and target date on or before the start of the window.	Includes an open activity due on or before July 10 10:54 a.m.
Completed	Activity with completed status and close date on or after to the window start date.	Includes an activity completed between July 10 10:54 a.m. and calculating the statistics.

Submissions with a This Week Window Example

The default window size for activities and policy transactions is this week.

The Calculation column in the following table describes how PolicyCenter calculates the status for each submission in the **Submission by User** reporting category.

The Example column describes which submissions will be included. The example assumes that the statistics were last calculated at July 11 10:54 a.m. and that the submissions window is **this week**. The July 11 date is a Friday. In the base configuration, the end of the business week is Friday at 5:00 p.m. Therefore, the business week started on Friday, July 4 at 5:00 p.m.

Status	Calculation	Example
Open	The job close date field is not set.	Includes any open submission. The window does not apply.
New	The job close date is not set, and the job create time is on or after the window start date.	Includes a submission if it was created between July 4 5:00 p.m. and calculating the statistics.
Bound	Any policy period associated with the job has a non-null model number. The job close date is on or after the window start date.	Includes a submission bound between July 4 5:00 p.m. and calculating the statistics.

Renewals with a This Month Window Example

The Calculation column in the following table describes how PolicyCenter calculates the status for each renewal in the **Renewals by User** reporting category.

The Example column describes which renewals will be included. The example assumes that the statistics were last calculated at July 11 10:54 a.m. and that the renewal window is **this month**. The month started on July 1 at 12:00 a.m.

Status	Calculation	Example
Open	The job close date field is not set.	Includes any open renewal for this user. The window does not apply.
New	The job close date is not set, and the job create time is on or after the window start date.	Includes a renewal created between July 1 12:00 a.m. and calculating the statistics.
Renewed	Any policy period associated with the job has a non-null model number. The job close date is on or after the window start date.	Includes a renewal issued between July 1 12:00 a.m. and calculating the statistics.
Non-Renewed	Any policy period on the job is locked and the status field is non-renewed. Also, the job close date is on or after the window start date.	Includes a renewal non-renewed between July 1 12:00 a.m. and calculating the statistics.
Not-Taken	Any policy period on the job is locked and the status field is not taken. Also, the job close date is on or after the window start date.	Includes a renewal not taken between July 1 12:00 a.m. and calculating the statistics.

Other Policy Transactions

The **Other policy transactions** category includes cancellations, reinstatements, policy changes, rewrites, rewrite new accounts, and audit jobs.

The Calculation column in the following table describes how PolicyCenter calculates the status for each policy transaction in the **Other policy transactions** reporting category.

Status	Calculation
Open	The job close date field is not set.
New	The job close date is not set, and the job create time is on or after the window start date.
Approved	Any policy period associated with the job has a non-null model number. The job close date is on or after the window start date.

Setting the maximum number of rows on the Team screens

You can use the TeamScreenTabVisibilityRowCountCutoff configuration parameter to set the maximum number of rows that the **Activities**, **Submissions**, **Renewals**, and **Other Policy Transactions** screens shows on the **Team** tab. Increasing this parameter potentially increases the amount of time that PolicyCenter takes to render the **Team** screens. If the results exceed the value of this parameter, then PolicyCenter displays a message on that screen and does not display the results.

If you select an individual user, PolicyCenter always displays the filtered results even if the number of results exceeds the cutoff parameter.

Using submissions as an example, view the **Summary** screen. If the total number of **Open**, **New**, or **Bound** submissions is greater than value of this parameter, then PolicyCenter does not display any submissions on the **Submissions** screen. PolicyCenter displays a message on the **Submissions** screen.

See also

- "TeamScreenTabVisibilityRowCountCutoff" in *Configuration Parameters*

Configuring earned premium

Earned premium is the portion of premium that applies to the expired part of the policy period. In other words, earned premium is the amount of premium that has been earned as of the current date. This topic describes how to configure the earned premium which appears on the **Earned Premium** field on the policy **Summary** screen.

For reporting policies, the calculation includes the earned-but-unreported (EBUR) amount prior to final audit. For package policies, PolicyCenter displays earned premium for each line of business.

[See also](#)

- *Application Guide*
- *Plugins, Prebuilt Integrations, and SOAP APIs*

How PolicyCenter calculates earned premium

The earned premium calculation examines all financial transactions on the policy. The calculation checks a series of conditions, most of which are based on the calculation, or *as-of*, date. For reporting policies where EBUR may be included, it also uses the date of the last premium report period. The calculation evaluates a transaction as follows:

1. If the as-of date is prior to either the posted date or the written date of the transaction, then the earned amount for the transaction is 0.
2. Else if the amount type on the related cost is not premium, for example taxes or surcharges, then the earned amount for the transaction is 0.
3. Else if the transaction is to be accrued (`ToBeAccrued` is `true`), then:
 - a. If the as-of date is after the expiration date of the transaction, then the whole transaction amount counts toward earned premium.
 - b. Else if the as-of date is prior to the effective date of the transaction, then the earned amount for the transaction is 0.
 - c. Else, the only other possibility is that the as-of date is between the transaction's effective and expiration dates. The earned amount is calculated as a prorated portion based on the as-of date.
4. Else, the only transactions left are transactions that are not to be accrued (`ToBeAccrued` is `false`). These transactions are of two types:
 - a. If the transaction is earned on the effective date of the policy, The whole transaction amount counts toward earned premium. Transactions for flat-rated costs are an example.
 - b. Else if the transaction is for a cost that is subject to reporting, the transaction must meet both of the following criteria.

1) The calculation includes EBUR.

2) And the as-of date is after the date of the last premium reporting period.

For these transactions, the earned amount is the prorated portion between the last premium report date and the as-of date.

See also

- *Application Guide*

Methods that calculate earned premium

An enhancement on the Transaction entity, `TransactionEPEnhancement.gsx`, defines methods to calculate earned premium. You can modify these methods.

Calculate Earned Premium

The earned method calculates the earned premium for the transaction as of the current date. The method signature is:

```
earned(lastReportedDate: Date, includeEBUR: boolean): BigDecimal
```

This method calls the `earnedAsOf` method with the `asof` parameter set to the current date. The parameters are described in that method.

Calculate Earned Premium As of Date

The `earnedAsOf` method calculates the earned premium as of a specific date passed in as a parameter.

```
EarnedAsOf(asof: Date, lastReportedDate: Date, includeEBUR: boolean)
```

- `asof` – The as-of date for determining the earned premium.
- `lastReportedDate` – The latest date for which a premium report was processed (the end date of the report). This parameter is only applies to reporting policies.
- `includeEBUR` – Set to `true` to calculate EBUR on the current transaction. This parameter only applies to reporting policies.

The calculations for earned premium include or exclude EBUR based on the `includeEBUR` parameter. In the base configuration, PolicyCenter EBUR is configured as follows:

- The PCF that calculates earned premium automatically sets `includeEBUR` to `true` for reporting policies where the final audit is not complete. For more information, see “PCF files that display earned premium” on page 122.
- For all other policies `includeEBUR` is set to `false`.

Through configuration, you can change whether PolicyCenter includes EBUR in the calculation.

PCF files that display earned premium

The `Policy_Summary_EarnedPremiumDV.pcf` displays earned premium. For reporting policies, this screen shows whether or not the earned premium includes EBUR. The **Earned As Of** date is a link that enables you to change the as-of date for the calculation.

Configuring quote cloning for business intelligence

Some insurance companies have a need to capture information from quotes that users and other processes generate through the course of the day. Quote cloning provides one approach for retaining some of this information long enough to capture it for business intelligence (BI) needs.

Through configuration, you define the processing of the cloned quotes. When you have gathered the data from the clones and no longer need them, you can run Purge Quote Clones batch processing to purge the cloned quotes.

[See also](#)

- *Application Guide*

Overview

Quote cloning provides a framework for creating cloned copies of policy period quotes. Clones can include quotes that never result in a bound policy. Cloned quotes can be generated from submission, policy change, reinstatement, renewal, and rewrite policy transactions.

This framework is disabled in the base configuration.

In your implementation, you define these characteristics:

- Which quotes to clone. You define this in your implementation of the Quote Clone plugin interface.
- How to process the quote clones. You define the processing of the clones in your implementation of the Quote Clone plugin interface. Alternately, you can choose not to use the Quote Clone plugin interface and implement your own integration or plugin to process the clones.
- How often to purge processed clones. According to a schedule you define, Purge Quote Clones batch processing purges cloned quotes.
- Which processed clones to purge. You can add criteria to the query that Purge Quote Clones batch processing uses for selecting purge candidates.

Quote cloning does not have an end-user interface. It provides a framework that automated processes can use to generate BI data. Quote cloning is not intended to provide additional end-user functionality. The captured data does not appear in the PolicyCenter user interface.

IMPORTANT: Before enabling quote cloning, evaluate the impact on system performance, including the impact to memory and database.

Quote cloning object model

The PolicyPeriod object has two properties for quote cloning:

- **TemporaryClone** – If true, this policy period is a temporary clone. A clone does not appear in the PolicyCenter user interface. This is a virtual property that PolicyCenter derives.
- **TemporaryCloneStatus** – The status of the cloned policy period. If Unprocessed, the policy period has not been processed. If Purgeable, the cloned policy period is no longer needed and can be purged.

Enable quote cloning

About this task

In the base configuration, quote cloning is disabled. To enable quote cloning, you must do the following in Studio:

Procedure

1. Create an implementation of the PolicyPeriodQuoteClonePlugin plugin interface that implements your functionality. See “Quote clone plugin” on page 124.
2. In the plugins registry for PolicyPeriodQuoteClonePlugin, enable the plugin and specify your implementation as the Gosu class.
3. Enable Purge Quote Clones batch processing in scheduler-config.xml by uncommenting the ProcessSchedule entry for PurgeQuoteClones.
4. Modify the schedule of Purge Quote Clones batch processing to meet your needs.

Quote clone plugin

The Quote Clone plugin provide methods for you to define and customize how PolicyCenter creates and processes cloned quotes. The plugin also provides a method for adding criteria to the query that selects cloned quotes for purging.

In the base configuration, the DefaultPolicyPeriodQuoteClonePlugin Gosu class is an example implementation of the PolicyPeriodQuoteClonePlugin plugin interface.

If the plugin is enabled, PolicyCenter calls the following methods in the order listed whenever it creates a policy period that can be cloned:

Method	See...
shouldCloneQuote	“Determining whether to clone a quote” on page 124
cleanupAfterCloningPolicyPeriod	“Cleaning up after cloning policy period” on page 125
processClonedPolicyPeriod	“Processing the cloned policy period” on page 125

The plugin has one method for modifying Purge Quote Clones batch processing:

Method	See...
addToPurgeClonedPeriodsQuery	“Adding to the Purge Quote Clones query” on page 125

Determining whether to clone a quote

PolicyCenter calls the shouldCloneQuote Boolean method after a policy period has finished quoting in a policy transaction.

In the `shouldCloneQuote` method, include code that returns `true` to flag the policy period for cloning. For example, the following implementation flags all submissions for cloning:

```
override function shouldCloneQuote (period: PolicyPeriod): boolean {  
    return period.Job.Subtype == TC_SUBMISSION  
}
```

When this method returns `true`, PolicyCenter creates a cloned policy period and sets its `TemporaryClone` property to `TC_UNPROCESSED`.

Cleaning up after cloning policy period

After creating the cloned policy period, PolicyCenter calls the `cleanupAfterCloningPolicyPeriod` method.

IMPORTANT: If this method is not properly implemented for your data model, loss of data or functionality may occur.

The intent of this method is to fix up any problems caused by cloning. For example, the clone and the originating policy period may have foreign keys pointing to the same object. If PolicyCenter attempts to delete the originating policy period, PolicyCenter generates an error when it encounters the common object because the cloned policy period still points to it. To avoid this, you can do a deep copy of the common object and point the foreign key on the cloned policy period to the copy of the common object. If the common object has other relationships, you must duplicate those relationships in copy of the common object and the clone. For example, if the common object has a foreign key pointing back to the originating policy period, you must duplicate that relationship.

In the base configuration, if you have enabled and implemented a multicurrency system, this problem occurs with `PolicyFXRate` objects. After cloning, the original policy period's `PolicyFXRate` objects are also linked to from the clone. In the example plugin implementation, the `cleanupAfterCloningPolicyPeriod` method provides code that fixes the links to the `PolicyFXRate` object. The method does a deep copy of the `FXRate` and links the cloned policy period to the `PolicyFXRate` copy.

Processing the cloned policy period

After cleaning up the cloned policy period, PolicyCenter calls the `processClonedPolicyPeriod` method. In this method, add code that processes the cloned policy period. Alternately, you can leave the `processClonedPolicyPeriod` method empty and create your own integration or plugin that processes the data and extracts it to an external BI system.

In either case, after processing, mark the cloned policy period as purgeable by setting the `TemporaryCloneStatus` property to `Purgeable`:

```
clonedPeriod.updateTemporaryCloneStatus(PolicyPeriodCloneStatus.TC_PURGEABLE)
```

Adding to the Purge Quote Clones query

Purge Quote Clones batch processing calls this method.

In the `addToPurgeClonedPeriodsQuery` method, you can add criteria to the query that Purge Quote Clones batch processing uses to select cloned quotes for purging. For example, you can specify that purgeable policy periods must be at least 2 hours old.

After returning from this method, the process uses the query to create the list of items to be purged.

Purge Quote Clones batch processing

Quote cloning creates copies of policy period quotes, known as quote clones. In PolicyCenter, the Purge Quote Clones work queue deletes quote clones that are marked as processed from the PolicyCenter database.

By default, PolicyCenter comments out the entry for this work queue in file `scheduler-config.xml` in the base configuration. To schedule this work queue, remove the comment marks from around its entry in `scheduler-config.xml`.

To find candidate cloned quotes for purging, Purge Quote Clones batch processing uses a query to select policy periods where the following are true:

- `TemporaryClone` property is `true`
- `TemporaryCloneStatus` property is `Purgeable`

You can modify the query that Purge Quote Clones batch processing uses to select candidates for purging. Modify the query in the `addToPurgeClonePeriodsQuery` method of the `PolicyPeriodQuoteClonePlugin` plugin.

[See also](#)

- “Adding to the Purge Quote Clones query” on page 125
- *InsuranceSuite Server Administration*

Configuring quoting and rating performance

To improve quoting and rating performance, you can use and configure asynchronous quoting, parallel rating, parallel product model synchronization, and two-step quoting.

See also

- “Configuring asynchronous quoting ” on page 15
- *Application Guide*
- *Plugins, Prebuilt Integrations, and SOAP APIs* for information about configuring parallel rating

Configuring parallel product model synchronization

Parallel product model synchronization can be configured through configuration parameters. If you require further customization, you can modify Gosu code.

Parameters

Parallel product model synchronization can be configured through configuration parameters.

- `ParallelizedProductModelSyncEnabled` – Enable parallel product model synchronization.
- `MaxProductModelSynchThreadPoolSize` – Maximum number of threads spawned per request during parallel product model synchronization.
- `MinNumberOfCoverablesToParallelizeProductModelSync` – The minimum number of coverables on a policy period required to trigger parallel product model synchronization.
- `ParallelProductModelSyncTimeoutPerCoverable` – The number of milliseconds per coverable to wait for parallel product model synchronization to complete before PolicyCenter reports an error.

For more information about these parameters, see "Product model parameters" in *Configuration Parameters*.

Gosu code

The default implementation of `shouldParallelizeProductModelAvailabilitySync` enables synchronization for any commercial product with at least the minimum number coverables specified by the `MinNumberOfCoverablesToParallelizeProductModelSync` parameter.

In the base configuration default, each thread operates over a coverable and its clauses of a particular type. For example, one thread operates on the coverages on vehicle1, another on the conditions on vehicle1, another thread does the same for vehicle2. Through configuration, you can change both when parallel product model synchronization is triggered and the granularity of what each thread operates on.

If you require different granularity as to when parallel product model synchronization occurs, you can modify the `shouldParallelizeProductModelAvailabilitySync` method in `PolicyPeriodBaseEnhancement`. For example, if the requirements for triggering synchronization vary by product, you can define a method for these requirements in the `PolicyLineMethods` class. Define the requirements for each product in `PolicyLineMethods` inheritor classes.

If you require different granularity as to what each thread operates on, you can customize the `createProductModelCheckTasks` method in the `JobProcess` class. This method performs the work of each thread. You can customize this method and implement new tasks.

Configuring two-step quoting

Two-step quoting is enabled for the commercial property line of business. Side-by-side quoting in personal auto uses two-step quoting. High volume quote requests use two-step quoting for all lines of business. You can easily enable two-step quoting for other lines of business.

Enabling two-step quoting

Configuration parameters and properties control whether two-step quoting is enabled in the PolicyCenter user interface and control its availability for each line of business in the base configuration.

The PCF files for the lines of business in the base configuration are already configured for two-step quoting. Two-step quoting is enabled in the commercial property line of business. You can enable two-step quoting for the other lines of business in the base configuration.

Enable two-step quoting in PolicyCenter

To enable two-step quoting in PolicyCenter, set the `TwoStepQuotingEnabled` parameter to `true` in `config.xml`. In the base configuration, this parameter is `true` by default.

Make two-step quoting available in a line of business

For lines of business in the base configuration, the `TwoStepQuotingAvailable` property controls whether two-step quoting is available during policy transactions for that line. To make two-step quoting available, add code that sets the `TwoStepQuotingAvailable` property to `true` in the `xxPolicyLineMethods` class, where `xx` is the abbreviation for the line of business. For an example, see the `TwoStepQuotingAvailable` property getter in `BAPolicyLineMethods.gs`.

If you develop your own lines of business, use commercial property as an example for adding two-step quoting to your implementation. Modify your code to handle the `PolicyPeriodStatus` of `Rated` and the distinction between `ValidRate` and `ValidQuote` properties.

Enable two-step quoting for side-by-side quoting in PolicyCenter

To enable two-step quoting for side-by-side quoting in the PolicyCenter application, set the `SideBySideTwoStepQuotingEnabled` parameter to `true` in `config.xml`. In the base configuration, this parameter is `true` by default. The personal auto line of business has side-by-side quoting in the base configuration.

Two-step quoting in side-by-side quoting does not require that `TwoStepQuotingEnabled` is `true` in `config.xml`.

Quoting process steps

The quote process class (`QuoteProcess`) performs the following steps:

Step	Actions
Before rating	1. Clean up the policy period branch. For example, remove unused locations and unused answers. 2. Validate the policy period branch. 3. Merge out-of-sequence conflicts. 4. Run segmentation to select the underwriting company. 5. The quote process is at <code>BlocksQuote</code>. Evaluate whether all underwriting issues that block quote can be removed. 6. Prepare account syncable objects for quoting. 7. Remove rating worksheets.
During rating	1. Rate the policy period. 2. Calculate transactions. 3. Denormalize denormalized fields. 4. Set policy period branch to <code>Rated</code> status.
Finish rating	The quote process is at the <code>BlocksRateRelease</code> blocking point. Evaluate whether all underwriting issues raised at <code>PreRateRelease</code> can be removed.
After rating	1. Infer forms. 2. Set policy period branch to <code>Quoted</code> status.
After quote	1. Set the payment information. 2. Handle reinsurance, if applicable. 3. Evaluate whether underwriting issues that block quote release can be removed.

Configuring quote purging

Quote purging provides batch processes to purge jobs that did not result in bound policies and prune alternate policy periods created through multi-version quoting and side-by-side-quoting. Quote purging also removes orphaned policy periods. Purging and pruning removes these jobs, policy periods, and associated objects from the PolicyCenter database. Quote purging is not a reversible operation. Quote purging is disabled in the base configuration.

For performance considerations related to how you schedule and configure quote purging, see [Guidewire Cloud Standards](#).

See also

- *Application Guide*

Quote purging configuration overview

Quote purging is accessed through the **Purge** and **Purge Orphaned Policy Periods** batch processes. These batch processes make use of configuration parameters and a purge plugin. When enabled in the default configuration, quote purging considers submission and policy change job types for purging.

You can configure quote purging by making modifications to the following:

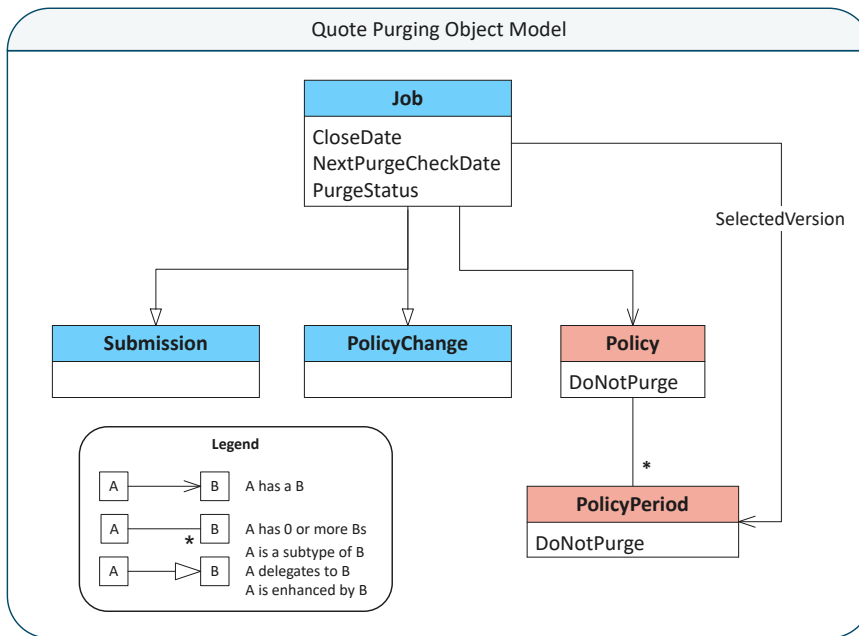
- **Batch processes** – Specify how often, if ever, to run the batch processes that remove jobs and policy periods from the database. For more information, see “Quote purging batch processes” on page 134.
- **Configuration parameters** – You can use these parameters to specify the number of days that must pass before purging a job or pruning a policy period. You can enable renewal and rewrite jobs for purging. For more information, see “Quote purging configuration parameters” in *Configuration Parameters*.
- **Purge plugin** – Modify quote purging through Gosu code. By configuring the purge plugin, you can control which jobs are purged or pruned. You can also choose to retain certain information related to those jobs. For more information, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

In addition, you can access quote purging through:

- **Web services** – From an external system, set a flag to prevent a job or policy period from being purged or pruned. For more information, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

Quote purging object model

The following diagram shows some of the objects and properties related to quote purging.



If the DoNotDestroy flag on Policy is true, then quote purging does not purge the Job associated with this Policy or prune PolicyPeriod objects.

If the DoNotDestroy flag on PolicyPeriod is true, then quote purging does not prune this PolicyPeriod.

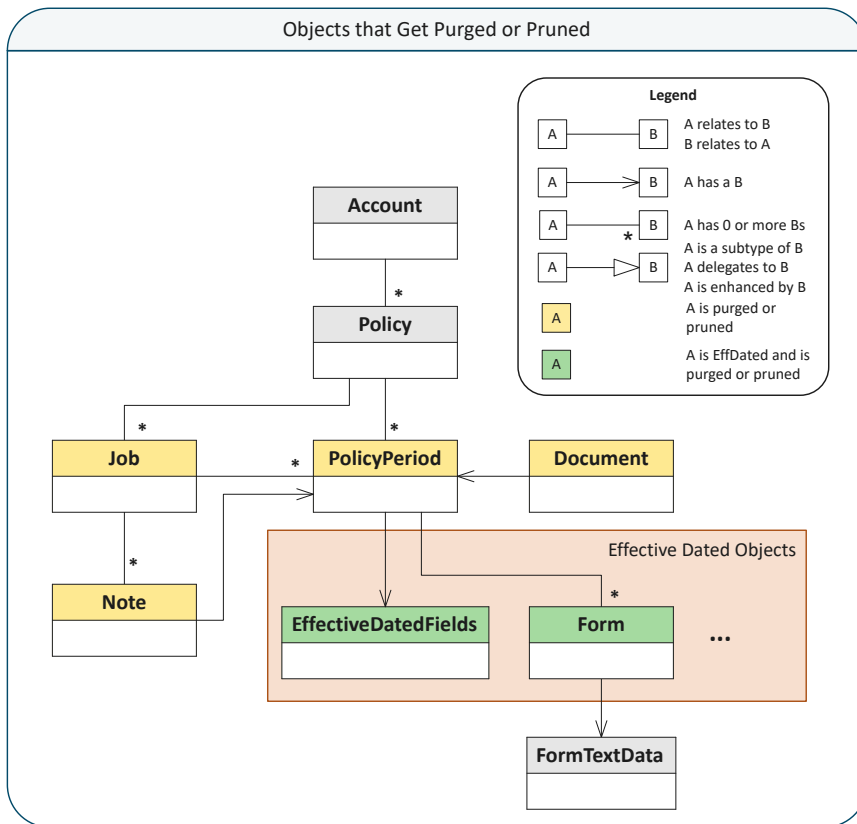
You can manage the DoNotDestroy property in Gosu code and in the PurgeAPI web service. To set the DoNotDestroy property, you must have the **Enable purging for a policy period** and **Disable purging for a policy period** permissions. The codes for these permissions are `purgeenable` and `purgedisable`, respectively.

[See also](#)

- *Plugins, Prebuilt Integrations, and SOAP APIs*

Objects that get purged or pruned

The following illustration shows some of the objects that get purged or pruned.



Purging and pruning remove the policy period and all effective dated objects in the policy period branch.

Objects where the branch is equal to the policy period are also removed. These objects, which include notes, documents, and activities, directly relate to the policy period and would otherwise point to a removed object. Purging and pruning does not remove objects not directly related to the policy period.

Purging and pruning also remove forms.

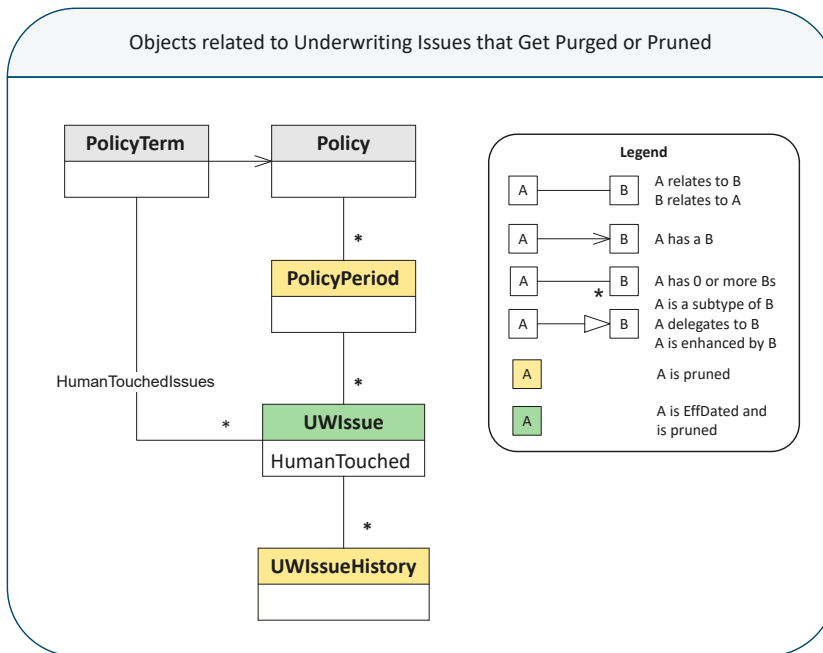
Form text data is not removed because form text data can be shared between policy periods. As a result, form text data can be orphaned as a result of quote purging. The Form Text Data Delete batch process removes orphaned form text data.

In addition to removing policy periods, purging also removes the job. If purging removes all jobs associated with a policy, it also removes the Policy object. For example, purging removes a Policy if it only exists because of an unbound submission. Pruning only removes policy periods; pruning does not remove the job.

If you added custom objects that attach to policy period, these objects will be removed if they meet the criteria mentioned above. Through configuration, you can remove objects that do not meet this criteria. The way that forms are removed may provide some help in your implementation.

Underwriting issues

The following illustration shows objects related to underwriting issues that get purged or pruned.



Purging and pruning also remove objects related to underwriting issues on the policy period. Human-touched underwriting issues are not purged or pruned. A human-touched UWIssue has the HumanTouched property set to true.

Quote purging batch processes

Quote purging purges jobs and prunes policy periods, and purges orphaned policy periods. You access quote purging through batch processes. The quote purging batch processes are:

Name	Code	Description
Purge	Purge	Purges jobs and prunes policy periods that meet the purge and prune criteria. This batch process deletes jobs and other entities from the database. For more information, see “Purge batch process” on page 135.
Purge Orphaned Policy Periods	PurgeOrphanedPolicyPeriod	Purges policy periods orphaned as a result of preemption. This batch process deletes policy periods and other entities from the database. For more information, see “Purge Orphaned Policy Periods batch process” on page 138.
Reset Purge Status and Check Dates	ResetPurgeStatusAndCheckDates	Reset the purge status and purge or prune dates on the jobs. Run this batch process if and when you have a need to reset the purge status and purge date. For each job, this batch process: • Sets the Job.PurgeStatus property to Unknown. • Sets the Job.NextPurgeCheckDate to null. For example, if a job has a PurgeStatus of NoActionRequired or Pruned, the batch process resets the PurgeStatus to Unknown.

IMPORTANT: The Purge and Purge Orphaned Policy Periods batch processes remove jobs and other entities from the database. These batch processes are not reversible operations. The Reset Purge Status and Check Dates batch process is also not reversible.

Enable quote purging batch processes

About this task

In the base configuration, the Purge and Purge Orphaned Policy Periods batch processes are disabled. To enable these batch processes, you must do the following in Studio:

Procedure

1. In `config.xml`, set the following parameters to `true` to enable running the batch processes from the **Server Tools Batch Process Info** screen in PolicyCenter:

Batch process	Parameter
Purge	PruneAndPurgeJobsEnabled
Purge Orphaned Policy Periods	PurgeOrphanedPolicyPeriodsEnabled

2. To enable the batch process to run on a regular schedule, uncomment the following `ProcessSchedule` entries in `scheduler-config.xml` and modify the schedule to meet your needs:

Batch process	Process
Purge	Purge
Purge Orphaned Policy Periods	PurgeOrphanedPolicyPeriod

Batch processes to run prior to running the purge batch process

A policy period cannot be purged if it is associated with one or more workflows. To more effectively purge and prune policy periods, clean up workflows before purging or pruning by running the following batch processes:

- Purge Workflow
- Purge Workflow Logs
- Workflow

Purge batch process

The Purge batch process purges jobs and prunes policy periods that meet the purge and prune criteria.

Jobs for which all of the following apply are placed into the Purge work queue:

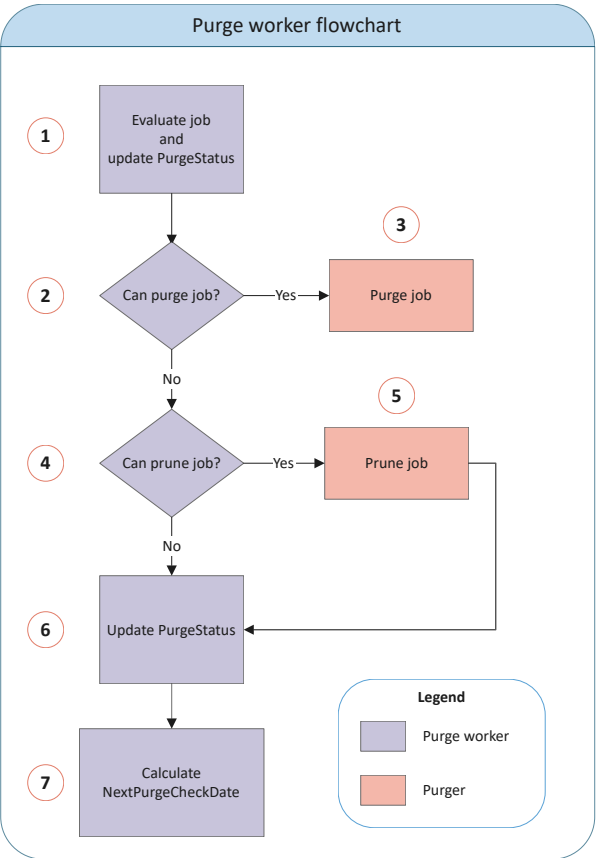
Condition	Configurable in Purge plugin?	Configurable in config.xml?
Job subtype is allowed for purging or pruning.	•	
The batch process calls the following property getters in the <code>PurgePlugin</code> plugin interface: • <code>AllowedJobSubtypesForPurging</code>		
The Job is closed, and the <code>CloseDate</code> is at least <code>PurgeJobsRecentJobCompletionDays</code> or <code>PruneVersionsRecentJobCompletionDays</code> in the past.		
The <code>NextPurgeCheckDate</code> on the job has passed.		
The <code>PurgeStatus</code> has a value other than <code>NoActionRequired</code> .		

See also

- “Enable quote purging batch processes” on page 135
- *Plugins, Prebuilt Integrations, and SOAP APIs*

Purge worker

The Purge worker picks up a job from the Purge work queue and purges or prunes jobs that meet the criteria. The following flow chart shows how the purge worker processes the job. Any job that emerges from step 7 will be considered for purging by the batch process on or after the NextPurgeCheckDate.



The numbered items in the flow chart are described below.

1. In this step, the job is evaluated according to the following criteria:

Condition	Configurable in Purge plugin?	Configurable in config.xml?
Is the PurgeStatus not equal to NoActionRequired?		
On the policy period that is the SelectedVersion, is the Status not bound, making the job a candidate for purging?		
Does the job have more than one policy period, making the job a candidate for pruning?		

Then the worker updates the PurgeStatus.

2. This decision point evaluates whether the job and all of its policy periods can be purged. A job can be purged if all of the following apply:

Condition	Configurable in Purge plugin?	Configurable in config.xml?
The policy is not bound.		
The job subtype is a subtype for purging.	•	
Calls the AllowedJobSubtypesForPurging getter in the PurgePlugin plugin interface.		
The DoNotDestroy flag is set to false on the policy and all policy periods on the job.		
The purge date returned by the getPurgeJobDate method is less than or equal to the current date.	•	•
The purge date is calculated by comparing the job close and policy end dates with configuration parameters. For more information, see "Quote purging configuration parameters" in <i>Configuration Parameters</i> .		
Calls the getPurgeJobDate method in the PurgePlugin plugin interface.		
The job has no open activities.		
There are no workflows on any policy period in the job.		
For information about a process that purges workflows, see the <i>InsuranceSuite Server Administration</i> .		

3. In this step, the purger purges the job in two separate bundles:

- a. The first bundle calls the createContext and prepareForPurge methods in the PurgePlugin plugin interface.
For each policy period in the job, the purger calls the skipPolicyPeriodForPurge method in the PurgePlugin plugin interface. This bundle then purges the job and the related objects.
- b. The second bundle calls the postPurge method in the PurgePlugin plugin interface.

IMPORTANT: Guidewire recommends that you put business critical operations in the prepareForPurge method so that the first bundle does the purge and business critical operations. Do not put business critical operations in the second bundle. If by chance the system fails while the purger is processing the job, the second bundle may not occur. If the first bundle does not complete, the bundle transaction consisting of the prepareForPurge and the purge-from-database action is rolled back. The job will be again be a candidate for purging or pruning the next time the batch process picks it up.

4. This decision point evaluates whether the policy periods on a job can be pruned. The policy periods on a job can be pruned if all of the following apply:

Condition	Configurable in Purge plugin?	Configurable in config.xml?
The job subtype is a subtype for pruning.	•	
Calls the AllowedJobSubtypesForPruning getter in the PurgePlugin plugin interface.		
The prune date returned by the getPruneJobDate method is less than or equal to the current date.	•	•
The prune date is calculated by comparing the job close and policy end dates with configuration parameters. For more information, see "Quote purging configuration parameters" in <i>Configuration Parameters</i> .		
Calls the getPruneJobDate method in the PurgePlugin plugin interface.		
The job has no open activities.		

If the previous conditions are met, then each individual policy period is evaluated for pruning. The policy period is sent to the purger if the following apply:

Condition	Configurable in Purge plugin?	Configurable in config.xml?
The policy period is not bound.		
The DoNotDestroy flag is set to false on the policy period.		
There are no workflows on the policy period.		

5. In this step, the purger prunes policy periods on the job in two separate bundles:

- a. The first bundle calls the `prepareForPurge` method in the `PurgePlugin` plugin interface.
For each policy period in the job except the selected version, the purger calls the `skipPolicyPeriodForPurge` in the `PurgePlugin` plugin interface. This bundle then purges the policy periods and the related objects.
- b. The second bundle calls the `postPurge` method in the `PurgePlugin` plugin interface.

IMPORTANT: Guidewire recommends that you put business critical operations in the `prepareForPurge` method so that the first bundle does the purge and business critical operations. Do not put business critical operations in the second bundle. If by chance the system fails while the purger is processing the job, the second bundle may not occur. If the first bundle does not complete, the bundle transaction consisting of the `prepareForPurge` and the purge-from-database action is rolled back. The job will be again be a candidate for purging or pruning the next time the batch process picks it up.

6. This step updates the `PurgeStatus` to one of the following values:

- `NoActionRequired` – This is the most common case. For example, `PurgeStatus` gets this value when the only remaining period is bound.
- `Pruned` – When there is one and only one policy period version remaining.
- `Unknown` – If the purge status is unknown. For example if a version that is not the selected version is marked `DoNotDestroy`, then pruning leaves that version as well as the selected version.

7. This step calculates the next purge check date. This step calls the `calculateNextPurgeCheckDate` method in the `PurgePlugin` plugin interface.

When the batch process runs again, this job will be reevaluated for purging or pruning.

Purge Orphaned Policy Periods batch process

The **Purge Orphaned Policy Periods** batch process finds *orphaned* policy periods, which are policy periods not associated with a job. Orphaned policy periods are created in the following scenarios:

- A job is preempted.
- The effective date is modified on a policy change.

The `PurgeOrphanedPolicyPeriodsEnabled` parameter in `config.xml` must be set to true.

For each orphaned policy period, this batch process calls the following method in the `PurgePlugin` plugin interface:

- `skipOrphanedPolicyPeriodForPurge` method that signals not to purge an orphaned policy period.

Note: If you do not wish to purge orphaned policy periods, do not enable the **Purge Orphaned Policy Periods** batch process. Customizing the `skipOrphanedPolicyPeriodForPurge` method is not an effective way to disable purging orphaned policy periods.

See also

- To enable this batch process, see “Enable quote purging batch processes” on page 135.

Using events to notify external system of purged or pruned entities

PolicyCenter generates an event when quote purging or pruning removes a PolicyPeriod, Job, or Policy entity instance. You can use messaging to send this event to external systems that track PolicyCenter entities or data. The events that PolicyCenter generates for quote purging are:

- JobPurged
- PolicyPurged
- PolicyPeriodPurged

See also

- *Plugins, Prebuilt Integrations, and SOAP APIs*

Purging test tool

The **Purging Test** tool is provided to help you see the effect of purging or pruning on one job. Use this tool during development to see the effect of the Purge batch process on a selected job. This tool also has a link to run the Purge batch process.

To access this tool, press ALT+SHIFT+T and select **Internal Tools > Purging Test**.

Flushing other work queues with the purging test tool

A policy period cannot be purged if it is associated with one or more workflows. To more effectively purge and prune policy periods, clean up workflows before purging or pruning.

- In **Flush other work queues (Purge workflows, etc.)**, click **Run**.

This command cleans up workflows by running the following batch processes:

- Purge Workflow
- Purge Workflow Logs
- Workflow

Purge or prune a job with the purging test tool

About this task

Follow these step-by-step instructions to purge or prune a particular job with the **Purging Test** tool.

Procedure

1. Enter the policy transaction number of the job you wish to purge.
2. Click **Find/Refresh Job** to fetch the job from the database.

Field	Description
Job Number	The number of the job to purge or prune.
Subtype	The job subtype.
Close Date	The job close date.
DoNotDestroy flag (Policy)	The value of the DoNotDestroy flag on Policy.

Field	Description
Policy Period(s)	The number of policy periods associated with this job. The Prune -- Purge Policy Period(s) section displays more information about each policy period.
Coverage End	The coverage end date of the policy.

PolicyCenter displays a **Purge or Prune Job** section. This section displays the following information about the job:

You can select to purge or prune the selected job.

- To purge or prune the job without checking to see if the job is eligible for purging or pruning, select **Skip configurable Purging checks** or **Skip configurable Pruning checks**.

If you did not choose to skip checks, then the job is purged or pruned based on the current configuration, including the `PurgePlugin` implementation.

If you choose to skip checks, then the job is purged or pruned regardless of the **Subtype**, **Close Date**, or **Coverage End**.

If you choose to skip checks, a job can be purged or pruned only if the following are true:

- The job's `CloseDate` is not null.
- The `DoNotDestroy` flag is not set on the policy associated with the job.
- The job has no open activities.

Additionally, if you choose to skip checks, the following criteria also apply. For purging, each policy period on the job must meet this criteria. For pruning, the policy period being pruned must meet this criteria:

- The `PolicyPeriod` is not bound.
- The `DoNotDestroy` flag on the `PolicyPeriod` is not set.
- The `PolicyPeriod` has no workflows.

- Click **Purge Job** or **Prune Job** to purge or prune the selected job. PolicyCenter does not run the Purge batch process but performs equivalent actions to purge or prune the selected job. The purge or prune accesses the configuration parameters and runs the `PurgePlugin` implementation.

Prune policy periods with the purging test tool

About this task

After selecting a job with the **Purging Test** tool, the policy periods attached to the selected job appear in the **Prune -- Purge Policy Period(s)** section. This section displays the following information about each policy period:

Field	Description
Policy Period	The version number of the policy period, and whether it is the selected policy period.
Period Start	The policy period start date.
Period End	The policy period end date.
Period Status	The policy period status.
DoNotDestroy (period)	The value of the <code>DoNotDestroy</code> flag.
Flip DoNotDestroy bit	A button to toggle the <code>DoNotDestroy</code> flag on this policy period.

Procedure

- To prune policy periods without checking to see if the policy period is eligible for pruning, select **Skip checks for purging policy period**.

If you choose to skip checks, then the policy period is pruned regardless of the **Subtype**, **Close Date**, or **Coverage End**.

If you did not choose to skip checks, then the policy period is pruned based on the current configuration, including the **PurgePlugin** implementation.

2. Click **Purge** in the row of the policy period you wish to prune.

The **Purge** button is enabled for deselected policy period versions.

PolicyCenter does not run the Purge batch process, but performs equivalent actions to prune the selected policy period. The purge or prune accesses the configuration parameters, and runs the **PurgePlugin** implementation.

Run the Purge batch process from the purging test tool

About this task

You can run the Purge batch process from the **Purging Test** tool.

Procedure

In **Run Purge Batch Process**, click **Run**.

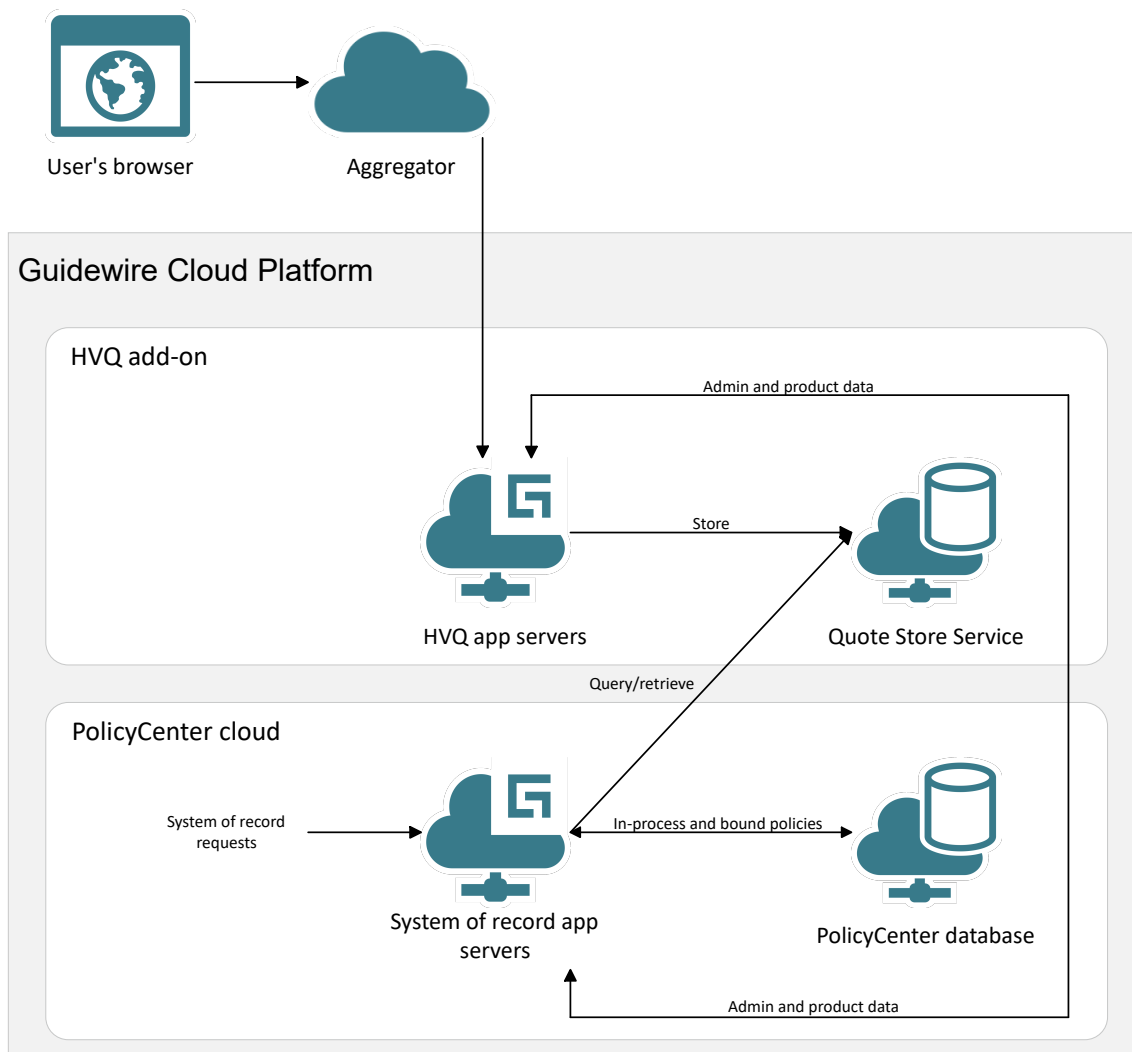
This is the same as running the batch process from the **Server Tools > Batch Process Info** screen. The batch process purges jobs and prunes policy periods in the PolicyCenter database.

High Volume Quoting

Some insurers need to handle high volumes of quote requests generated by external applications, such as the web sites of *comparative raters* and *quote aggregators*. A comparative rater's web site enables its agents to view and choose between quotes from multiple insurers. A quote aggregator's web site enables consumers to view and choose quotes from multiple insurers. Users of these applications or web sites also expect faster response times.

High Volume Quoting (HVQ) uses the following architecture to handle large volumes of quotes efficiently:

- A set up with multiple instances of PolicyCenter where all instances leverage the same PolicyCenter version and configuration. This includes one or more quote-only PolicyCenter instance to process quotes and a separate PolicyCenter system-of-record to complete submissions, issue policies, and perform subsequent actions on the policy.
- Since most of these high volume requests do not result in a policy, the quote-only instance of PolicyCenter stores quotes in an external database managed by the Quote Storage Service (QSS), a service that is external to PolicyCenter and managed by Guidewire. Access to Quote Storage Service (QSS) is abstracted through the included Java API which provides quotes that are stored outside the PolicyCenter instance. Guidewire does not support the ability to connect PolicyCenter to custom external database implementations.



Guidewire provides an example API and interface for generating quotes with HVQ. For more information, see “Overview of a High Volume Quoting implementation” on page 145 and the *Application Guide*.

High Volume Quoting (HVQ) requirements

Before you enable High Volume Quoting, consider the following requirements:

- Your license agreement must include High Volume Quoting. For more information, contact your Guidewire sales representative.
- PolicyCenter must use Guidewire Rating Management or an external rating engine. For information, see “Configuring Rating Management” on page 161.

Enable High Volume Quoting (HVQ)

Complete the following steps to configure HVQ:

1. Submit a ticket to Guidewire to [Enable HVQ](#).
2. Verify the completion of feature adoption steps for any version that you upgraded from. For more information, see *Update Guide*.
3. As needed, you can also configure the following:

- Configure the validation level, rating style, and underwriting criteria that blocks quoting. For more information, see “Configure validation levels, rating style, and blocking issues” on page 149.
- Configure additional DataDog metrics. For more information, see “Observing High Volume Quoting metrics” on page 151.
- Configure the number of days the QSS retains quotes. The default is 90 days. For more information, see “Configuring time to live for quotes in Quote Storage Service” on page 147.

After HVQ is enabled, you can see the **API HVQ URL** in Guidewire Cloud. For more information, see the [Cloud Platform](#) documentation.

Overview of a High Volume Quoting implementation

Components

A quote-only instance of PolicyCenter supports the high volume generation of quotes and generates quotes in a storable XML format. The system of record PolicyCenter converts a quote from XML format to a bound policy. To support these systems, use or implement the following components:

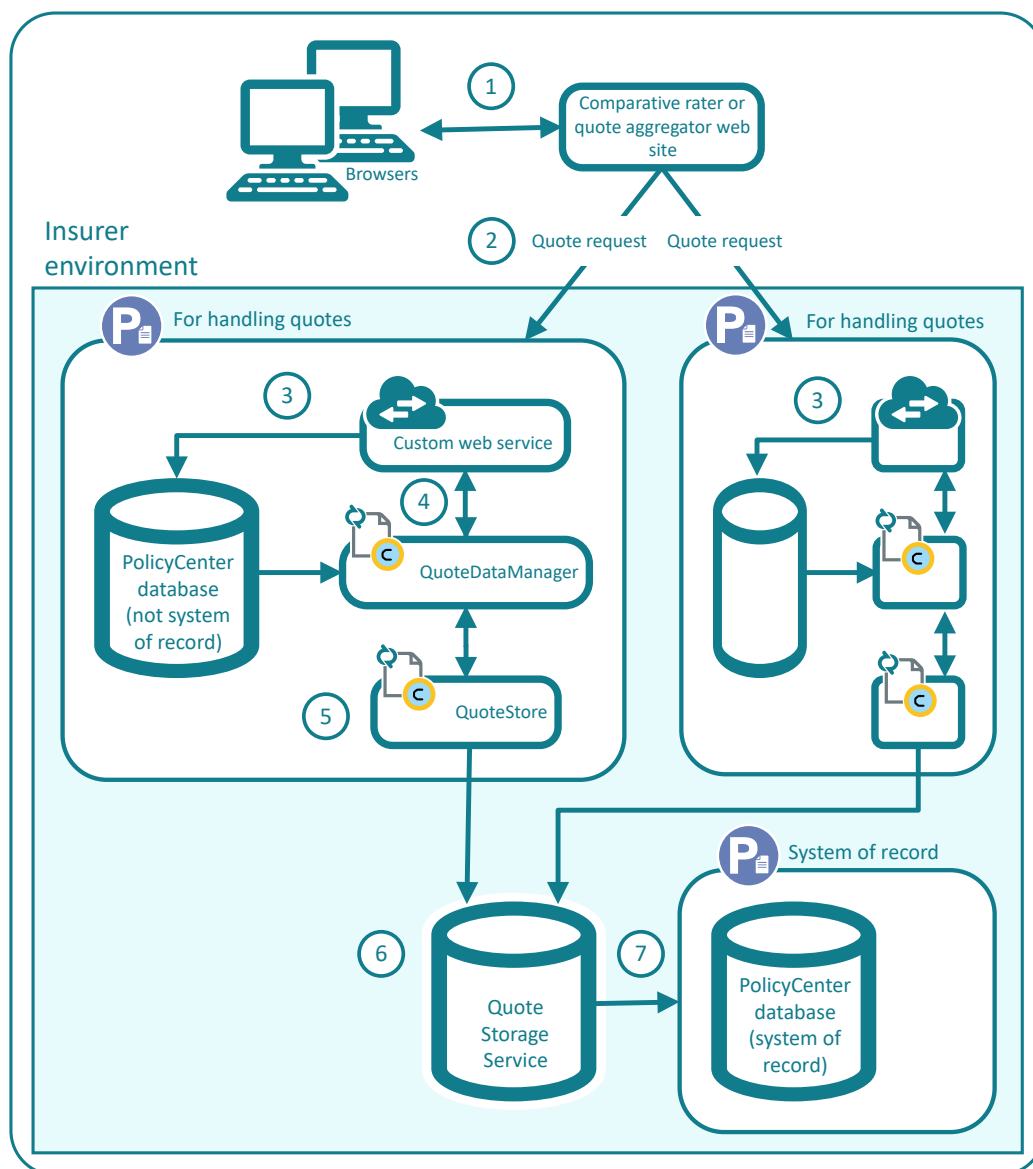
Component	Required on	Demonstration example
Web service that the comparative rating or quote aggregator system calls. This web service receives quote requests on the Quote-only instance from external applications and sends them to the quote data manager. The system of record PolicyCenter receives quote retrieval requests. You write this web service and publish it from PolicyCenter.	Quote-only instance and system of record	Provided via the PC-HVQ-Example GPM Package
Quoting factory that creates quote data manager, quote converter, and quote store objects.	Quote-only instance and system of record	DefaultQuotingFactory implements QuotingFactory
Quote data manager that calls quote converter and quote store methods to save and retrieve quote data in XML format.	Quote-only instance and system of record	DefaultQuoteDataManager implements QuoteDataManager
Quote converter that converts a quote between PolicyCenter entity instances and XML format.	Quote-only instance and system of record	ArchiveXmlQuoteConverter implements QuoteConverter
Quote representation that stores all information about a quote in XML format.	Quote-only instance	ArchiveXmlQuoteRepresentation implements QuoteRepresentation
Quote representation that reconstitutes all information about a quote from XML format to an entity instance.	System of record	ArchiveXmlRestoreQuoteRepresentation extends ArchiveXmlQuoteRepresentation implements RestoreQuoteRepresentation
Quote store that saves and retrieves quotes in XML format using the external database.	Quote-only instance and system of record	ExternalXmlQuoteStore implements QuoteStore
Quote search criteria that specify conditions to find a particular quote on the external database or the system of record database.	Quote-only instance and system of record	QuoteSearchCriteria extends SearchCriteria

Example package

PolicyCenter includes a PC-HVQ-Example GPM Package, which provides a sample HVQ API for the Personal Auto line of business. The package shows a working example of the HVQ API that you can use as a foundation for your company's specific requirements. To download the PC-HVQ-Example GPM Package, visit the [Guidewire Support site](#).

Process flow for high volume quotes

The following illustration presents a simplified model of how an insurer can use API classes to handle quote requests.



1. The web site of a comparative rater enables potential customers to view policy quotes gathered from multiple insurers. A potential customer on this web site enters basic policy information and requests quotes. One of these insurers uses PolicyCenter to handle these requests
2. The comparative rater web site sends the policy information to multiple insurers. One of these requests goes to a insurer with PolicyCenter. The insurer can have multiple instances of PolicyCenter to handle these requests.
3. Your web service formats the request, creates a policy period (PolicyPeriod) that contains the information on which to quote, and verifies the data. The base configuration does not contain an example of this type of web service, However, a sample implementation is provided in the PC-HVQ-Example GPM Package.

Generating the policy period and quote access is done by the PolicyCenter database attached to the PolicyCenter instance for handling quotes. This database is the system of record. However, in this implementation neither the policy period nor the quote are committed to the system\ of record PolicyCenter database

4. Your web service then calls the `generateAndExportQuote` method in a Gosu object that implements the Quote Data Manager interface (`QuoteDataManager`) to generate a quote. The `generateAndExportQuote` method calls the `convertQuotePeriod` method in a Gosu object that implements the Quote Converter interface (`QuoteConverter`). The `convertQuotePeriod` method formats the data into an XML model of a `PolicyPeriod` and an additional XML structure that contains information about entities to which the policy period refers.
5. The `generateAndExportQuote` method calls the `storeQuote` method in a Gosu object that implements the Quote Store interface (`QuoteStore`).
The external quoting database, described in the next step, is optimized for storing the quoting information.
6. The `storeQuote` method stores the XML in an external database and generates and returns a quote reference number (`QuoteKey`). The quoting database can be partitioned so that it can concurrently handle requests from multiple instances of PolicyCenter.
7. If the potential customer decides to purchase the insurer's policy, the PolicyCenter system of record retrieves the XML representation of the quote from the external quoting database. The PolicyCenter system of record can use the quote reference number to retrieve the XML. The policy can be bound and saved to the PolicyCenter database.

Keeping quick quotes outside the main database

In a submission, you can choose to do a Quick Quote rather than a Full Application. In the base configuration, Quick Quote stores the quotes in the PolicyCenter system of record database. If your bind rate on these policies is low, you can modify Quick Quote to store the quote outside the system of record database until the quote is bound. Do this by modifying the Quick Quote wizard to provide the quote and then call the Quoting Data plugin to write the quote to the quoting table.

Quote-only instances of PolicyCenter

The quote-only PolicyCenter generates the same quote pricing as the system of record PolicyCenter. To function effectively with the system of record, the quote-only instance uses the same configuration information and database as the system of record.

When setting up a system to handle high volume quotes, consider the following guidance:

- All quote-only instances of PolicyCenter access the same Quote table in the external quoting database for storage and retrieval. Therefore, you can add additional instances as necessary to support peak loads. Each instance operates independently. To request additional instances, contact Guidewire Customer Support.
- Access to quote data in the external quoting database is restricted through authentication per planet environment, and a planet environment's data cannot be accessed by another planet environment's PolicyCenter instances. To keep the size of the Quote table manageable, PolicyCenter purges the table of all quotes older than 90 days by default.
- Maintenance on the PolicyCenter system of record impacts the operation of the quote-only instances.
- A PolicyCenter system can be only either a quote-only instance or a system of record, not both. Archiving operates only on a system of record and is disabled on a quote-only instance. PolicyCenter uses configuration parameters to determine which type of system it is. If you set conflicting configuration parameters, PolicyCenter does not start.

Configuring time to live for quotes in Quote Storage Service

In environments that implement high volume quoting, you can configure the number of days that the Quote Storage Service (QSS) retains quotes.

By default, quotes in the QSS have a *time to live* value of 90 which means quotes expire after 90 days. You can configure when the quote expire in the following ways:

- “Configure QSS time to live in the storeQuote function” on page 148
- “Configure QSS time to live in config.properties” on page 148

Determining time to live for quotes in the QSS

To determine the *time to live* value for a given quote, PolicyCenter evaluates the following settings until it finds an applicable value:

1. The value of the `ttl` parameter in `storeQuote` function. The `ExternalXMLQuoteStore.gs` file includes a call to the `storeQuote` function.
2. The value of the `quote.store.service.hvq.ttl.productCode.baseState` property in `config.properties`. PolicyCenter evaluates the *time to live* settings in `config.properties` to find the correct *time to live* setting for the quote:
 - First, it looks for a `quote.store.service.hvq.ttl.productCode.baseState` setting associated with the base state and product code of the quote.
 - Then, it looks for `quote.store.service.hvq.ttl.productCode` setting associated with the product code of the quote
 - Finally, it looks for a `quote.store.service.hvq.ttl` setting.
3. When there is no *time to live* setting defined, PolicyCenter uses the default value of 90 days.

Configure QSS time to live in the storeQuote function

In the `storeQuote` function, you can set the number of days that the Quote Storage Service (QSS) retains quotes with a custom calculation.

About this task

You can configure the *time to live* for quotes in the QSS with the `storeQuote` function.

Procedure

1. In PolicyCenter Studio, open `modules/configuration/gsrc/gw/quoting/impl/ExternalXMLQuoteStore.gs`.
2. Locate the `storeQuote` function.
3. Create a utility function that takes the `policyPeriod` as input and calculates the *time to live*.
4. Add a `ttl` variable that gets its value from the utility function you created and then add `ttl` as a parameter of the `saveQuote` function.

In this example, the `ttl` variable gets its *time to live* value from a utility function named `calculateTtl`.

```
...
override function storeQuote(data : QuoteRepresentation, policyPeriod : PolicyPeriod) : QuoteKey {
    ArgCheck.nonNull(data, "QuoteDataRepresentation")
    var additionalSearchFields = getAdditionalSearchFields(policyPeriod)
    var externalQuoteId = getExternalQuoteId(policyPeriod)
    var ttl : Integer = calculateTtl(policyPeriod)
    return runExceptionally(\-> _quoteStoreHandler.saveQuote(ttl, externalQuoteId, policyPeriod, data,
        additionalSearchFields))
}
...
```

Configure QSS time to live in config.properties

In the `config.properties`, you can set the number of days that the Quote Storage Service (QSS) retains quotes.

About this task

You can configure the *time to live* value for all quotes, quotes with a specific product code, or quotes with a specific product code and base state.

Procedure

1. In the PolicyCenter Studio `modules/configuration/config/config.properties` file, add the following template syntax:

```
quote.store.service.hvq.ttl.productCode.baseState
```

2. Update the following variables in the syntax per your requirements.

Configuration Setting	Description
productCode and baseState	Indicates the product and base state that the <i>time to live</i> value applies to. For example, this configuration applies to Personal Auto quotes in California: <code>quote.store.service.hvq.ttl.PA.CA = 100</code>
productCode	Indicates the product that the <i>time to live</i> value applies to. For example, this configuration applies to Personal Auto quotes that do not have a specific <i>time to live</i> value for their base state: <code>quote.store.service.hvq.ttl.PA = 80</code>
<no optional settings>	Indicates the <i>time to live</i> value that applies to all quotes without a more specific <i>time to live</i> value. For example, this configuration applies to all quotes that do not have a specific <i>time to live</i> value for their product or base state: <code>quote.store.service.hvq.ttl = 80</code>

Configure validation levels, rating style, and blocking issues

You can configure validation levels, rating styles, and blocking issues for high volume quoting processes in PolicyCenter.

After reading this topic, you'll be able to:

- Configure the validation level and rating style for products that use local job processes.
- Configure the validation level and rating style for products that use cloud job processes.
- Define criteria that blocks high volume quotes due to underwriting issues.

Procedures

Configure validation levels and rating style for local job process

The local job process invokes a `requestRate` method that accepts the validation level and rating style. PolicyCenter provides an implementation of `QuoteDataManager` (`gw.quoting.impl.DefaultQuoteDataManager`) that calls the `requestRate` method within the `requestQuoteAndHandleUWIssues` function.

In this example, `TC_QUICKQUOTABLE` is the validation level and `TC_QUICKQUOTE` is the rating style:

```
private function requestQuoteAndHandleUWIssues(jobProcess : JobProcessImpl) {
    var prevEvaluator = jobProcess.JobProcessEvaluator
    try {
        // Create a JobProcessEvaluator that checks for UWIssues, but no blocking until bind.
        jobProcess.JobProcessEvaluator = new JobProcessUWIssueEvaluator() {
            override function evaluateAndFindBlockingUWIssues(branch : PolicyPeriod, bp : UWIssueBlockingPoint) :
            entity.UWIssue[] {
                return {}
            }

            override function findBlockingIssues(branch : PolicyPeriod, bp : UWIssueBlockingPoint) : entity.UWIssue[] {
                return {}
            }
        }
        jobProcess.requestRate(ValidationLevel.TC_QUICKQUOTABLE, RatingStyle.TC_QUICKQUOTE)
    } finally {
        jobProcess.JobProcessEvaluator = prevEvaluator
    }
}
```

Configure validation levels and rating style for cloud job process

About this task

The cloud job process invokes a `requestRate` method that expects the validation level and rating style configuration to be set in `config.properties`.

Procedure

1. In PolicyCenter Studio, open the `modules/configuration/config/config.properties` file.
2. Configure the following properties:

Property	Description
<code>quote.highvolume.validation.enabled</code>	Enables validation for high volume quoting. Set to <code>true</code> to enable validation. The default is <code>false</code> which means it is not enabled.
<code>quote.highvolume.validation.level</code>	The validation level for high volume quoting. You can set the validation to any level available in <code>ValidationLevel.ttl</code> .
<code>quote.highvolume.rating.style</code>	The rating style for high volume quoting. You can set the rating style to any level available in <code>RatingStyle.ttl</code> .

3. Restart the PolicyCenter server to apply the changes.

Define when underwriting issues block high volume quotes

By default, the cloud job process and local job process do not block the creation of high volume quotes due to underwriting issues.

The local job process and the cloud job process each have their own instance of the `requestQuoteAndHandleUWIssues` function. Each instance of the `requestQuoteAndHandleUWIssues` function includes two override methods that you can use to define when to block quotes due to underwriting issues.

Location of the local job process code

Within private function `requestQuoteAndHandleUWIssues(jobProcess : JobProcessImpl)`, update the return statements of the `evaluateAndFindBlockingUWIssues` and `findBlockingIssues` functions. By default, the return statement does not return a value. The default appears as follows:

```
private function requestQuoteAndHandleUWIssues(jobProcess : JobProcessImpl) {
    var prevEvaluator = jobProcess.JobProcessEvaluator
    try {
        // Create a JobProcessEvaluator that checks for UWIssues, but no blocking until bind.
        jobProcess.JobProcessEvaluator = new JobProcessUWIssueEvaluator() {
            override function evaluateAndFindBlockingUWIssues(branch : PolicyPeriod, bp : UWIssueBlockingPoint) :
entity.UWIssue[] {
                return {}
            }

            override function findBlockingIssues(branch : PolicyPeriod, bp : UWIssueBlockingPoint) : entity.UWIssue[] {
                return {}
            }
        }
        jobProcess.requestRate(ValidationLevel.TC_QUICKQUOTABLE, RatingStyle.TC_QUICKQUOTE)
    } finally {
        jobProcess.JobProcessEvaluator = prevEvaluator
    }
}
```

Location of the cloud job process code

Within private function `requestQuoteAndHandleUWIssues(jobProcess : CloudJobProcessImpl)`, update the return statements of the `evaluateAndFindBlockingUWIssues` and `findBlockingIssues` functions. By default, the return statement does not return a value. The default appears as follows:

```
private function requestQuoteAndHandleUWIssues(jobProcess : CloudJobProcessImpl) {
    var prevEvaluator = jobProcess.JobProcessEvaluator
    try {
        // Create a JobProcessEvaluator that checks for UWIssues, but no blocking until bind.
        jobProcess.JobProcessEvaluator = new JobProcessUWIssueEvaluator() {
            override function evaluateAndFindBlockingUWIssues(branch : PolicyPeriod, bp : UWIssueBlockingPoint) :
```

```
entity.UWIssue[] {
    return {}
}

override function findBlockingIssues(branch : PolicyPeriod, bp : UWIssueBlockingPoint) : entity.UWIssue[] {
    return {}
}
}
jobProcess.requestRate() // can also use jobProcess.requestRate(false)
} finally {
    jobProcess.JobProcessEvaluator = prevEvaluator
}
}
```

Observing High Volume Quoting metrics

Guidewire recommends that you add sufficient logging to the High Volume Quoting configuration to help with debugging and analysis. Guidewire also recommends that you use the Observability Plugin to add metric emitters. You can use existing metric emitters, such as the Submission Quote Designated Function emitters.

The following code example shows how you can use the Observability Plugin with existing metrics:

```
class HVQuoteAPIHandler {
    protected var _observabilityPlugin : ObservabilityPlugin

    construct() {
        _observabilityPlugin = Plugins.get(ObservabilityPlugin)
    }

    function createHVQuote(body : JsonObject) {
        var context = new ObservabilityContext()
        _observabilityPlugin.beforeProcessStart("gw.pc.submission.requestquote", context)
        try {
            // create HVQ Quote
            _observabilityPlugin.afterProcessComplete("gw.pc.submission.requestquote", context)
        } catch (e : Exception) {
            _observabilityPlugin.onProcessError("gw.pc.submission.requestquote", e, context)
        }
    }
}
```

The sample code emits the following metrics, which you can see in Datadog:

- gw.pc.submission.requestquote.complete
- gw.pc.submission.requestquote.complete.duration
- gw.pc.submission.requestquote.errors
- gw.pc.submission.requestquote.failed
- gw.pc.submission.requestquote.failed.duration

For more information about Datadog metrics and using the Observability Plugin, see *Administration*.

High volume quote requests API classes and XML

PolicyCenter provides Gosu API interfaces that support conversion of a `PolicyPeriod` entity instance to and from XML format. PolicyCenter provides demonstration classes that implement these interfaces in the PC-HVQ-Example GPM package.

The enclosed topics describe the XML structures and API classes for handling high volume quote requests with PolicyCenter.

Quoting factory

A quoting factory manages the classes that implement quote management, storage, and retrieval. A quoting factory implements the `gw.quoting.QuotingFactory` interface. PolicyCenter provides a demonstration implementation of this interface at `gw.quoting.impl.DefaultQuotingFactory`. Use this Gosu class as a basis for developing your own implementation of the following methods:

- `createQuoteDataManager` – Creates a quote data manager object, which controls conversion to XML, storage, retrieval, and conversion from XML of quote data
- `createQuoteConverter` – Creates a quote converter object, which creates XML quote data from a policy period and associated entity instances and reconstitutes entity instances from the XML.
- `createQuoteStore` – Creates a quote store object, which stores, locates, and retrieves XML quote data in an external database

XML structures and classes for quotes

A quote representation requires two types of XML structure. An Archive XML structure represents the policy period entity instance for the quote. PolicyCenter can interpret this type of XML structure after a system upgrade, without requiring customization even if entity structures change in the data model.

The Archive XML includes fields that contain identifier (ID) values that refer to other entities, such as a producer code or underwriting issue codes. Identifiers for these entities can differ between the quote-only and system-of-record PolicyCenter instances. A second XML structure, known as a sidecar, contains denormalized metadata information about the other entities that the quote requires. The information in the sidecar XML enables the SOR PolicyCenter to assign the correct references to the quote that the comparative rater's customer wants to bind. PolicyCenter also uses the sidecar information to search for quotes if the user does not have the reference number of the quote. The sidecar XML contains information about the following entities:

- Account holder (AccountContact)
- Group (Group)
- Industry code (IndustryCode)
- Organization (Organization)
- Producer code (ProducerCode)
- User (User)
- Underwriting company (UWCompany)
- Underwriting issue (UWIssue)

The following XML lines show an example of sidecar data:

```
<?xml version="1.0" encoding="UTF-8"?>
<AdditionalEntity xmlns="http://guidewire.com/quoting/sidecar"
  xmlns:gw="http://guidewire.com/quoting/sidecar">
  <AccountHandler>
    <AccountSearchCriteria FirstName="Gregory" LastName="House"/>
  </AccountHandler>
  <ProducerCodeHandler>
    <OrganizationName name="ACV Property Insurance" producerCodeID="pc:16"/>
    <ProducerCode code="301-008578" producerCodeID="pc:16"/>
    <OrganizationName name="Armstrong and Company" producerCodeID="pc:6"/>
    <ProducerCode code="100-002541" producerCodeID="pc:6"/>
  </ProducerCodeHandler>
  <IndustryCodeHandler>
    <IndustryCode code="0740" domainName="SIC" effectiveDate="1990-01-01 00:00:00.000"
      industryCodeID="SIC:0740"/>
  </IndustryCodeHandler>
  <UserHandler>
    <userInSystem userID="default_data:1" userName="su"/>
    <userInSystem userID="pc:123" userName="carkle"/>
    <userInSystem userID="pc:37" userName="enyugen"/>
  </UserHandler>
  <GroupHandler>
    <OrganizationName groupID="pc:151" orgName="ACV Property Insurance"/>
    <GroupInSystem groupID="pc:151" groupName="Producers"/>
    <OrganizationName groupID="pc:41" orgName="Enigma Fire & Casualty"/>
    <GroupInSystem groupID="pc:41" groupName="Minneapolis Branch UW"/>
    <OrganizationName groupID="systemTables:1" orgName="Enigma Fire & Casualty"/>
    <GroupInSystem groupID="systemTables:1" groupName="Enigma Fire & Casualty"/>
  </GroupHandler>
  <UWCompanyHandler>
    <UWCompany code="1111_1111" uwCompanyID="uwc:1"/>
  </UWCompanyHandler>
  <UWIssueTypeHandler>
    <UWIssueType code="PAGarageState" uwIssueTypeID="587b5e62-4f9c-4393-a3bd-a130cd495427"/>
    <UWIssueType code="PACompDeductible" uwIssueTypeID="674b77e1-fbec-4b8d-884b-301a33cec228"/>
  </UWIssueTypeHandler>
</AdditionalEntity>
```



```
<UWIssueType code="PACollisionDeductible" uwIssueTypeID="87a76d80-1588-4f3b-9a15-5a418bb85d08"/>
</UWIssueTypeHandler>
</AdditionalEntity>
```

PolicyCenter uses two classes to support the creation and retrieval of quote data in XML format:

- A class for storing quotes. This class implements the `gw.api.quoting.QuoteRepresentation` interface. PolicyCenter provides an implementation of this interface at `gw.quoting.impl.ArchiveXmlQuoteRepresentation`.
- A class for retrieving and reconstituting quotes. This class implements the `gw.api.quoting.RestoreQuoteRepresentation` interface and extends the class that implements the `gw.api.quoting.QuoteRepresentation` interface. PolicyCenter provides an implementation of this interface at `gw.quoting.impl.ArchiveXmlRestoreQuoteRepresentation`.

The `ArchiveXmlQuoteRepresentation` interface provides the following properties:

- `QuoteInfo` – A read-only property of type `String`, containing the quote information in Archive XML format.
- `AdditionalInfo` – A read-only property of type `String`, containing the sidecar XML.

The `RestoreQuoteRepresentation` interface extends the `QuoteRepresentation` interface by providing the read-only property, `QuoteKey`. `QuoteKey` is an object of type `gw.api.quoting.QuoteKey`. This object stores the reference number of the quote as a unique identifier. The constructor takes a `String` parameter that is the reference number value. The `PolicyQuoteId` read-only `String` property provides the value of the reference number.

`ExternalQuoteStoreHandler` uses `ArchiveXmlQuoteRepresentation` and `ArchiveXmlRestoreQuoteRepresentation` for `saveQuote` and `retrieveQuote` functionality, respectively.

Quote data manager

A quote data manager Gosu object controls conversion to XML, storage, retrieval, and conversion from XML of quote data. The quote data manager implements the `gw.quoting.QuoteDataManager` interface. This interface provides methods to store a policy quote in XML format in an external database and to reconstitute a policy period from the XML. PolicyCenter provides a demonstration implementation of `QuoteDataManager` (`gw.quoting.impl.DefaultQuoteDataManager`), which uses the Quote Store Service to store quotes. Use this class as a basis for your own implementation of the `QuoteDataManager` interface.

Quote data manager methods

The `QuoteDataManager` interface provides the following methods:

- `createAccountSearchCriteria`
- `generateAndExportQuote`
- `reconstituteQuote`
- `removeQuote`
- `retrieveQuote`

Quote data manager process flow

Calls to quote data manager methods create XML quote data on the quote-only instance and retrieve XML data on the system of record in the following typical sequence:

On the PolicyCenter instance that provides high volume quoting

1. The web service that you implement creates a policy period entity instance from the information that the comparative rater site sends to PolicyCenter.
2. The web service calls the quoting factory `createQuoteDataManager` method to create the quote data manager. The web service calls the quote data manager `generateAndExportQuote` method.

3. The `generateAndExportQuote` method:
 - Requests a quote from the job entity instance for the policy period.
 - Calls the quoting factory `createQuoteConverter` method to create a quote converter object.
 - Calls the quote converter `convertQuotePeriod` method to create the XML representation of the quote.
 - Calls the quoting factory `createQuoteStore` method to create a quote store object.
 - Calls the quote store `storeQuote` method to store the XML representation of the quote in the external quote storage database. This method returns a `QuoteKey` object that contains the reference number of the quote.
 - Sets the `QuoteIdentifier` property of the policy period to the quote reference number.
4. The web service sends the quote reference number with other quote information that the comparative rater requires to the comparative rater system.

On the PolicyCenter SOR

1. The PolicyCenter user interface calls the quoting factory `createQuoteDataManager` method to create the quote data manager.
2. The user interface calls the quote data manager `createAccountSearchCriteria` method to locate an existing account for a quote that a comparative rater's customer wants to accept. The parameter for this method is the sidcar XML of a quote from the external quote database. The account search uses the account holder information in the XML data to locate matching accounts. The user interface provides additional search fields to refine the list of accounts that the search action returns.
3. A quote search criteria object identifies a quote on the HVQ database.
4. The user interface calls the quote data manager `retrieveQuote` method.
5. The `retrieveQuote` method:
 - Calls the quoting factory `createQuoteStore` method to create a quote store object.
 - Calls the quote store `retrieveQuote` method to retrieve the XML representation of the quote from the external quote storage database.
6. The quote data manager `retrieveQuote` method returns that XML data object to the user interface.
7. The user interface calls the quote data manager `reconstituteQuote` method to convert the XML data to a policy period (`PolicyPeriod`) entity instance.
8. The `reconstituteQuote` method:
 - Calls the quoting factory `createQuoteConverter` method to create a quote converter object.
 - Calls the quote converter `restoreQuotePeriod` method to create a policy period from the XML representation of the quote. The quote converter `restoreQuotePeriod` method checks the sidcar XML for any additional entities that relate to the quote. These types of entities include an account for the insured, a producer, an underwriting issue type, and so on. If PolicyCenter does not have matching records, you must create them. You must ensure that the foreign keys in the policy period refer to the correct existing or newly created records. Do not assume that the public ID values of any records in the SOR PolicyCenter are the same as those in the quote-only instance.
 - Sets the status of the policy period to quoted and performs post-quote processes. The quote-only instance does not perform this step. These post-quote processes include inferring the forms for the quote, setting payment information, attaching reinsurance agreements, and raising underwriting issues. The quote on this policy period is now valid and can be bound.
 - Calls the quote store callback method, `onReconstitute`. Use this method to check that the SOR succeeded in creating the quote. Update the quote record in the external quote database with the submission ID and account ID of the policy period. The quote-only instance can use this information to determine that the SOR created a quote from the data.

Quote data manager logging

The quote data manager logs messages about quoting jobs (policy transactions) to the `PCLoggerCategory.QUOTING` category.

Quote converter

A quote converter Gosu object converts Policy Period entities to and from the XML representation of a quote in the external quote storage database. The quote converter implements the `gw.quoting.QuoteConverter` interface. This interface provides methods to create, locate, and retrieve a quote in XML format. PolicyCenter provides a demonstration implementation of `QuoteConverter`, `gw.quoting.impl.ArchiveXmlQuoteConverter`. Edit this class or use this class as a basis for your own implementation of the `QuoteConverter` interface.

If you extend the policy graph in your PolicyCenter implementation, you must take these extensions into account when you convert a quote between entity and XML formats. In particular, you must create archive upgrade triggers for these extensions to convert the quote from Archive XML format to a quote in the PolicyCenter system of record.

The `QuoteConverter` interface provides the following methods:

- `convertQuotePeriod` – Converts a policy period (`PolicyPeriod`) on the quote-only instance to XML format. This method takes a `PolicyPeriod` entity instance as a parameter and returns a Gosu object that implements the `QuoteRepresentation` interface.
- `createAccountSearchCriteria` – Creates a search criteria object to locate accounts that match the values in the sidecar XML for a quote. This method takes a `gw.api.quoting.QuoteRepresentation` object as a parameter and returns an `AccountSearchCriteria` object. The PolicyCenter user interface displays these accounts so that the user can see whether an account for the quote already exists.
- `restoreQuotePeriod` – Converts a quote in XML format to a policy period (`PolicyPeriod`) on the system of record. This method takes a quote representation (`QuoteRepresentation`) parameter and an optional `Account` parameter. If the account for the quote already exists, you pass a value for the account parameter. This method returns a `PolicyPeriod` entity instance.

See also

- “XML structures and classes for quotes” on page 152
- “Quote data manager process flow” on page 153

Quote store

A quote store Gosu object converts quote data to and from XML. The quote store implements the `gw.quoting.QuoteStore` interface. This interface provides methods to store, locate, retrieve, and remove a policy quote in XML format in an external database. This interface also provides a callback method that the system of record PolicyCenter calls after successfully reconstituting a quote from XML format to a policy period entity instance. PolicyCenter provides a demonstration implementation of `QuoteStore`, `gw.quoting.impl.StandAloneQuoteStore`, which emulates an external database as hash maps in memory. Use this class as a basis for your own implementation of the `QuoteStore` interface.

The `QuoteStore` interface provides the following methods:

- `onReconstitute` – A callback method that you use to make changes to the quote record in the external quote database. The system of record PolicyCenter calls this method after successfully reconstituting a quote from XML format to a policy period entity instance. The types of changes that this method makes include setting the submission number from the system of record to show that the quote was accepted. For example, the quote-only instance can use this submission number value when searching for quote records to purge from the external quote database.
- `removeQuote` – Removes a quote record specified by a `QuoteKey` parameter from the external quote database and returns a Boolean value to indicate success or failure.
- `retrieveQuote` – Retrieves a quote specified by a `QuoteKey` parameter from the external quote database as an object that implements the `RestoreQuoteRepresentation` interface.

- `search` – Uses quote search criteria to locate matching quotes in the external quote database. This method takes an object that implements `QuoteSearchCriteria` as a parameter and returns a List of `gw.quoting.QuoteSummarySearchResult` objects.
- `storeQuote` – Stores a quote on the external quote database. This method takes an object that implements the `QuoteRepresentation` interface and a `PolicyPeriod` entity instance as parameters and returns a `QuoteKey` object.

The `ExternalXmlQuoteStore` implementation of the `QuoteStore` interface implements the supported interface methods to integrate with the Quote Store Service using the internal API provided by the `ExternalQuoteStoreHandler`. While `ExternalXmlQuoteStore` is customizable, Guidewire recommends that you extend this class to customize the behavior. An example implementation has been provided in the `HVQExternalXmlQuoteStore` class in the PC-HVQ-Example GPM package.

In addition to the interface methods provided by the `QuoteStore` interface, `ExternalXmlQuoteStore` also provides the method `getAdditionalSearchFields`, which is a callback method that adds new searchable fields to the Quote Search metadata.

See also

- “XML structures and classes for quotes” on page 152
- “Quote data manager process flow” on page 153

Quote search criteria

PolicyCenter uses a combination of information such as a quote identifier or the customer’s name to locate the quote. The `gw.quoting.QuoteSearchCriteria` class provides properties that a user enters in the PolicyCenter user interface to search for a quote. If the `HVQFilter` property is set to `true`, the search only accesses quotes on the external quote database. To see the properties that this class provides for searching for a quote, examine the code in the Studio editor. You can edit this class to enhance its functionality.

The `QuoteSearchCriteria` class can be extended to map additional searchable fields to search results. An example implementation of has been provided in the `HVQQuoteSearchCriteria` class of the PC-HVQ-Example GPM package.

Product Model Synchronization (V2.0)

PolicyCenter provides an improved product model synchronization architecture that reduces the time it takes to synchronize coverables when quoting policy change jobs. Product Model Synchronization (V2.0) achieves this performance improvement by syncing coverables only if any value referenced by the coverable in an APD rule or in a lookup table has changed. When enabled, Product Model Synchronization (V2.0) only runs at quote time when the entire policy is synchronized. Product Model Synchronization (V2.0) is an improvement over the preexisting product model synchronization, which synchronizes all coverables and coverages even when quoting a policy change with changes on a subset of coverables. The preexisting Product Model Synchronization will continue to work when navigating between PolicyCenter screens and also when you import or export policy data.

Product Model Synchronization (V2.0) achieves the most performance gains for large commercial policies where modifications are limited to a small set of coverables.

Product Model Synchronization (V2.0) can be configured for one or more APD lines, or for all APD lines. Product Model Synchronization (V2.0) is not supported for non-APD lines.

Enabling Product Model Synchronization (V2.0)

Product Model Synchronization (V2.0) must be explicitly enabled using configuration properties. You can set these configuration properties in Helios.

productmodel.sync.v2.<LineCode>.feature.enabled

Enables Product Model Synchronization (V2.0) for the specific line identified by <LineCode> (which may be an APD line).

productmodel.sync.v2.apdlines.feature.enabled

Enables Product Model Synchronization (V2.0) for all APD lines.

productmodel.sync.v2.feature.enabled

Enables Product Model Synchronization (V2.0) for all lines (note that Product Model Synchronization (V2.0) is not supported for non-APD lines).

The feature flags are hierarchical and PolicyCenter checks them in the order shown above. This means that for example that if `productmodel.sync.v2.<LineCode>.feature.enabled` is set to `true`, then this feature flag takes precedence over the other feature flags.

Limitations

Product Model Synchronization (V2.0) only comes into effect if the following conditions are met. If any one of these conditions is not met, then all coverables will be synchronized when the policy change job is quoted (this is the behavior of the preexisting Product Model Synchronization).

- Product Model Synchronization is only intended for cloud products with an active cloud edition. It is incompatible with database-backed or empty editions.
- The specific cloud edition used by the policy change job must remain unchanged from initialization to completion.
- Product Model Synchronization (V2.0) is exclusively for jobs that have a based-on period. This means that it cannot be used with policy submissions or policy renewals.
- Product Model Synchronization cannot be used with policy change reversal jobs.
- Upgraded periods are excluded from Product Model Synchronization (V2.0).
- All underlying property paths must be accurately defined in the lookup table for the line that requires Product Model Synchronization (V2.0). Any modifications to these paths during an active policy change job will disqualify the policy period from Product Model Synchronization (V2.0).
- The product model patterns, including the child patterns, must not contain availability scripts. This condition must remain in effect for the lifecycle of the policy change job. If an availability script is detected or added during the policy job, then the preexisting base configuration Product Model Synchronization takes effect.
- PolicyCenter configuration must remain unchanged during the lifecycle of the policy change job. Any configuration changes will cause PolicyCenter to revert to the base configuration Product Synchronization Model.

Log messages

When the PolicyCenter server starts, the `ProductModelSyncV2ConfigValidator` class verifies the lines of business enabled for Product Model Synchronization (V2.0) (through the configuration properties) do not have any Gosu availability scripts defined in the product model. The `ProductModelSyncV2ConfigValidator` class logs the following messages (all in the `Configuration.ProductModel` category).

Log Level	Log Message	Description
DEBUG	Product model sync v2 configuration validator started.	This appears once on every server startup to indicate that Sync V2-enabled lines of business will be checked for availability scripts.
DEBUG	Product model sync v2 feature not enabled. Product model sync v2 configuration validator not loaded.	No lines of business are enabled for Sync V2 through configuration properties, in which case none of the log messages below will appear either. If you did enable a line of business for Sync V2, then check that the config property was set correctly in the appropriate environment.
DEBUG	Policy line pattern '<code>' is not enabled for product model sync v2 - returning empty list.	The specified line of business is not enabled for Sync V2 so the check for availability scripts is not needed and will be skipped for this line. If you did enable this line of business for Sync V2, then check that the configuration property was set correctly in the appropriate environment.
WARN	Policy line pattern '<code>' enabled for sync v2 has an invalid product model configuration. See console log for details.	The specified line of business is enabled for Sync V2 but has product model availability scripts defined, so Sync V2 cannot be used.
DEBUG	<displayName> [field : <fieldName>] : - Availability script defined with Product Model Sync V2 enabled for Policy Line '<lineCode>'.	If this warning message appears, then each product model element that has an availability script defined will be logged to help you locate the problematic configuration.
INFO	Policy line pattern '<code>' enabled for sync v2 has a valid product model configuration.	The specified line of business is enabled for Sync V2 and it passes the availability script check successfully.

Log Level	Log Message	Description
DEBUG	Product model sync v2 configuration validator stopped.	This message appears when the validator service is stopped, which typically happens only when PolicyCenter is shut down.

When the PolicyCenter server starts, the `LookupTableFieldManager` class verifies that lines of business enabled for Product Model Synchronization (V2.0) have valid `valuePath/underlyingPropertyPaths` attributes for all of their lookup table dimensions. This class logs the following messages (all in the `Configuration.ProductModel` category).

Log Level	Log Message	Description
DEBUG	Lookup tables field manager started.	This message appears once on every server startup to indicate that the lookup table dimensions for Sync V2-enabled lines of business will be verified.
DEBUG	Product Model Sync v2 feature not enabled. Lookup tables field manager not loaded.	No lines of business are enabled for Sync V2 through configuration properties, in which case none of the log messages below will appear either. If you did enable a line of business for Sync V2, then check that the configuration property was set correctly in the appropriate environment.
DEBUG	Policy line pattern '<code>' is not enabled for product model sync v2 - returning empty list.	The specified line of business is not enabled for Sync V2 so the check for lookup table dimensions is not needed and will be skipped for this line. If you did enable this line of business for Sync V2, then check that the config property was set correctly in the appropriate environment
DEBUG	Policy line pattern '<code>' enabled for sync v2 has an invalid lookup table configuration. Lookup table maps will not be loaded for this policy line.	One or more of the lookup table dimensions for the line of business are invalid, so the line of business cannot use Sync V2 even though it is enabled. The invalid dimensions will be logged with the messages below to help you locate the problematic configuration.
	Missing underlyingPropertyPaths for dimension <field> in lookup table <code>: valuePath '<path>' cannot be used.	The valuePath for the specified lookup table dimensions does not resolve to an entity property, and no underlyingPropertyPaths attribute exists.
	Invalid underlyingPropertyPaths '<path>' for dimension <field> in lookup table <code>: <exception message>	The underlyingPropertyPaths attribute for the specified lookup table dimension contains an invalid path.
	Invalid underlyingPropertyPaths '<path>' for dimension <field> in lookup table <code>: Path root does not match lookup table root/ appliesTo	The root of an underlyingPropertyPaths for the specified lookup table dimension does not match the lookup table's root or appliesTo type.
	Invalid underlyingPropertyPaths '<path>' for dimension <field> in lookup table <code>: Path leaf is not an entity property	The underlyingPropertyPaths for the specified lookup table dimension does not resolve to an entity property.
DEBUG	Loading lookup table maps for policy line '<code>'.	The specified line of business is enabled for Sync V2 and it passes the lookup table dimensions check successfully.
DEBUG	Lookup tables field manager stopped.	This will appear when the field manager service is stopped, which typically happens only when PolicyCenter is shut down.

At commit or quote time, log messages will indicate when Product Model Synchronization (V2.0) cannot be used. These messages are in the `Application.JobProcess` log category. Each message begins with `PolicyPeriod '<displayName>' can not be synced via Sync V2` followed by one of the reasons below.

Log Level	Log Message	Description
DEBUG	Period does not have a based-on.	Sync V2 cannot be used for jobs without a based-on period (such as submissions or conversion renewals), or if previous periods were archived or purged.
DEBUG	A null lock value suggests that the period was likely upgraded and dirty tracking did not occur.	Sync V2 cannot be used for jobs that were created before Sync V2 was enabled.
DEBUG	Policy line is a manual line.	Sync V2 is not supported for APD visualized products.
DEBUG	Period belongs to policy change reversal job.	Sync V2 is not supported for policy change reversals.
DEBUG	Period window doesn't overlap with its based-on period.	Sync V2 is not supported if the period window changes from the based-on job, such as for renewal or rewrite jobs.
DEBUG	Invalid lookup table or product model pattern files detected.	Either the lookup tables for the LOB contain invalid valuePath/underlyingPropertyPaths attributes, or availability scripts are defined for the line of business.
DEBUG	Config has changed.	The PolicyCenter configuration changed after the job was started.
DEBUG	No applicable cloud product edition found.	Sync V2 relies on information in product editions.
DEBUG	Failed to encode product edition checksum.	There was a problem recording the product edition used for the job (this is an internal error that will not usually occur).
DEBUG	Product edition content has changed.	The product edition was changed, or a different edition is in use (after the job started).
DEBUG	Sync V2 is not enabled for line '<line>'.	No configuration property that enables Sync V2 for the line of business was found.

Configuring Rating Management

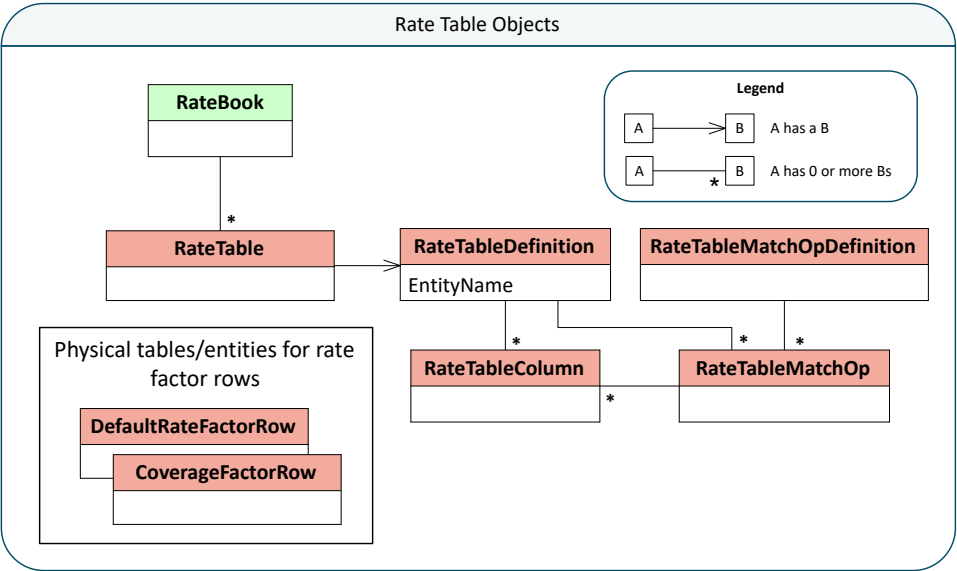
Guidewire Rating Management has been designed to be extended through configuration if required. However, major configuration changes may not be required.

IMPORTANT: To determine whether your Guidewire PolicyCenter license agreement includes Guidewire Rating Management, contact your Guidewire sales representative. Using Rating Management within PolicyCenter requires an additional license key. For instructions on obtaining and installing this key, contact your Guidewire support representative.

Rating Management data model

Rate table data model

The following illustration shows some of the object relationships for rate tables.



The `RateTable` entity represents a rate table. The rate table has a pointer to a `RateTableDefinition`.

The `RateTableDefinition` entity represents a rate table definition. The `EntityName` property contains the name of the physical table or entity of a rate factor row. The rate factor row entity describes the data types that you can assign to parameters and factors.

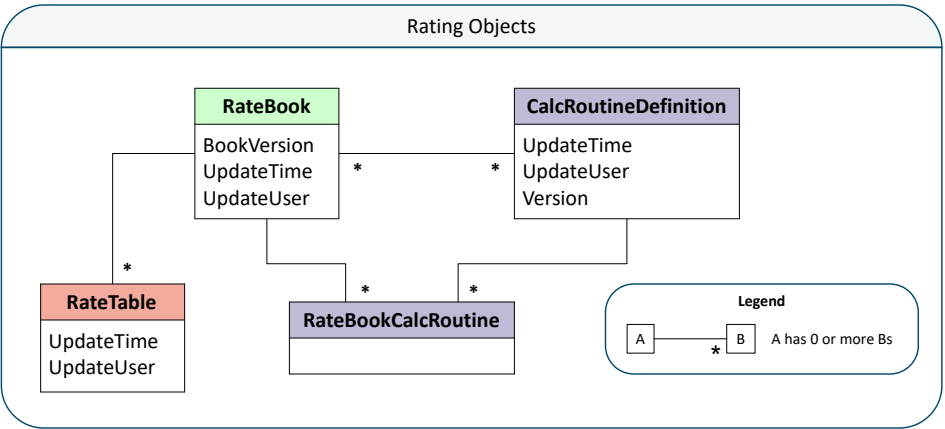
In the default configuration, the rate factor row entities are `DefaultRateFactorRow` and `CoverageRateFactor`.

See also

- *Application Guide*

Rate routine data model

The following illustration shows some of the object relationships for rate routines.

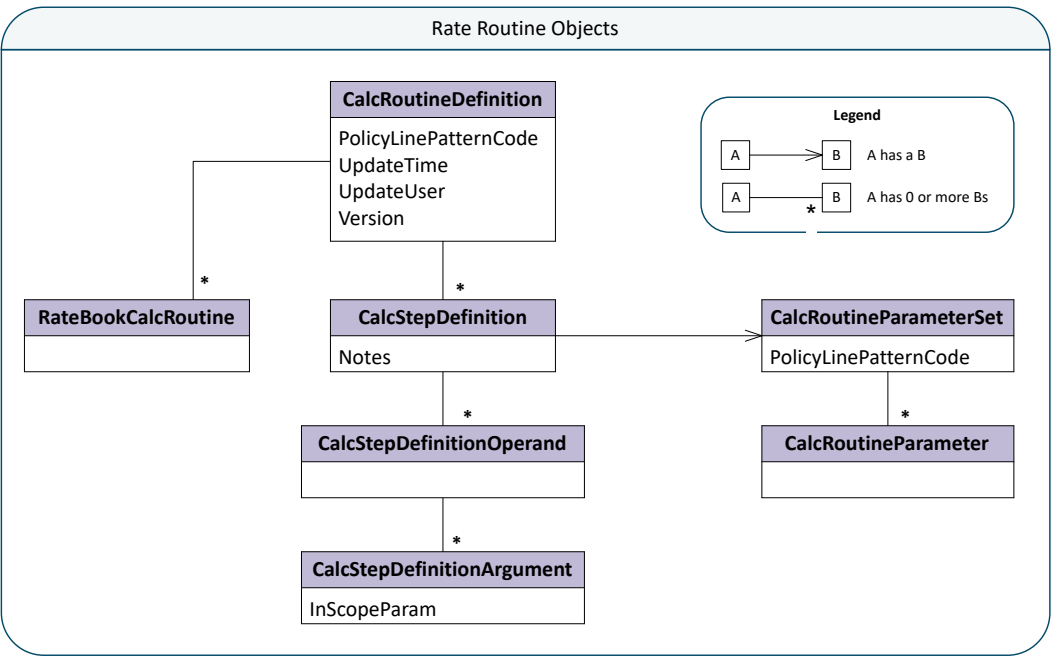


The `RateBook` object represents a rate book and contains rate tables and rating routines.

The `RateTable` object represents a rate table.

The `CalcRoutineDefinition` object represents a rate routine definition. In PolicyCenter, you define rate routines on the **Administration > Rate Routines** screen.

The `RateBookCalcRoutine` object represents a rate routine included in a rate book. In PolicyCenter, rate routines included in a rate book appear on the **Included Rate Routines** tab on the Rate Book Details screen. The following illustration shows the objects for rate routine steps and parameter sets.



A rate routine is composed of multiple steps, represented by the `CalcStepDefinition` object.

Rating Management performance

Improving startup and first access to Rating Management components

Guidewire Rating Management provides two features that can improve startup time and first access to Rating Management components. These two features can be used together. Both features are configurable.

Preload Rating Management components

Ensures that the first access to Rating Management components is quicker because they are loaded when the system starts. When used alone, this feature increases startup time. This feature is enabled in the base configuration.

Precompile rate routines JAR file

Improves system startup time. Use this feature if you have so many rate routines that startup time is unacceptably long. Rate routines are precompiled into a JAR file. Then the JAR file is built into the PolicyCenter application. This feature is not enabled in the base configuration.

Rate routines are stored in the database and must be converted to executable code. When rate book preloading is on, the conversion is done at start up which can cause very long start up times. Additionally, the conversion must be done by each and every PolicyCenter instance. The rate routines JAR feature addresses these issues. The conversion is done just once on a test/stage environment and the converted classes are combined into a JAR. On production systems, the conversion is avoided altogether and the rate routines are simply read from the JAR.

This feature requires that you adhere to the rate book lifecycle as described in *Application Guide*.

Preload Rating Management components

In the base configuration, PolicyCenter preloads all rate books and their included rate tables and rate routines into cache on system startup. Generally preloading Rating Management components at startup is desirable. Otherwise, components are loaded when the first quote request accesses them, causing the first quote request to be slower.

Preloading Rating Management components is recommended for production systems where an increase in server start time is less important than having faster first quote times. In development mode, where developers often restart PolicyCenter and wait for tests to run, it may be preferable to turn preloading off to speed up server start time while absorbing slower first quote times. These tradeoffs may be more noticeable in systems with complex rating content.

Preloaded Rating Management components are resident in cache, therefore preloading increases the memory requirements of the application. Preloading may load Rating Management components that are never used.

RateBookPreloadEnabled parameter

The `RateBookPreloadEnabled` configuration parameter controls whether Rating Management components are preloaded on system startup. This parameter is set to `true` in the base configuration.

Note: Unrelated to whether this parameter is `true` or `false`, PolicyCenter always preloads some parts of rate books that load quickly. After this, PolicyCenter writes this message to the log file:

```
INFO Application.Rating.RateTableManagement Preloading all ratebooks
```

Therefore, this message is not an indication of the `RateBookPreloadEnabled` parameter setting.

OverriddenTableArgsLabelSimple parameter

You can speed up rate table preloading by setting the configuration parameter `OverriddenTableArgsLabelSimple` to `true`. (The `RateBookPreloadEnabled` parameter must also be `true`.)

When `true`, when viewing a rate routine on the Rate Routine screen, parameters in rate routine steps may appear as ellipses (...).

Customizing how Rating Management components are preloaded

Through configuration, you can customize which Rating Management components are preloaded. For example, you can modify the code to only preload frequently-used rate books. In the base configuration, Active rate books are preloaded.

To change which rate books are preloaded, you can modify the `RateBooksToPreload` getter in the `RateBookPreloadPlugin` plugin implementation.

To preload Rating Management components, the `RatebookPrimer.prime` method calls the `primeForAllRateBooks` method in `RTMLoadActions` class. This class is not modifiable. You can create your own class and call it instead of `RTMLoadActions` from `RatebookPrimer.prime`. Use `RTMLoadActions` as a model. For example, consider checking the `RateBookPreloadEnabled` parameter as the `RTMLoadActions.primeForAllRateBooks` method does:

```
if (PCConfigParameters.RateBookPreloadEnabled.getValue()) {  
    ... // preload Rating Management components  
}
```

In the `RatebookPrimer.prime` method, you can also preload other Rating Management components, such as classes outside of rate books. This can speed up the first visit to Rating Management screens.

Loading parameter sets

The code that loads parameter sets is in the `prime` method in the `RatebookPrimer` class. Independent of the setting of `RateBookPreloadEnabled` parameter, this code always preloads parameters sets when the server is in development mode.

Precompile rate routines JAR file

Startup time in a PolicyCenter installation with a large number of rate routines can be very long. You can improve system startup time by precompiling rate routines into a JAR file and building the JAR file into the PolicyCenter application. This also improves performance when first accessing rate routines. This feature is not enabled in the base configuration.

When you use this feature, you generate and download the rate routines JAR file in the stage/test environment. In the base configuration, the JAR file contains rate routines from all Active rate books. You download the JAR file from the stage/test environment and include it in the production build of PolicyCenter.

Note:

The precompile rate routine JAR file feature is not supported on JBoss application servers.

Requirements

The precompile rate routines JAR feature requires that you adhere to the following:

- You must adhere to the rate book lifecycle as described in *Application Guide*. In the lifecycle, the rate book moves to Approved status in the stage/test environment. The rate book is imported to the production environment and then updated to Active status.
- All rate routines used in rating must be in one of the rate books returned by the `RateBooksToExportToJAR` getter of the rate book preload plugin. PolicyCenter throws an exception when it attempts to use a rate routine that is not in the list of rate books.

For example, a rate book contains a rate routine, and the rate book is not in the list of rate books returned by the plugin. If rating attempts to use that rate routine, PolicyCenter throws an exception, even if the rate book is in Active status. PolicyCenter relies on the list of rate book returned by the `RateBooksToExportToJAR` getter.

Incremental updates to the JAR file on the production system

PolicyCenter keeps the JAR file up-to-date by responding to changes in rate books. In the base configuration, the JAR file is updated when a rate book is imported from XML or when a rate book is promoted to Active status. (The JAR file is updated only if the rate book's rate routines are not in the JAR file.) This enables you to make changes to rate books without having to regenerate the JAR file and rebuild and redeploy PolicyCenter.

Note: If the production environment contains Active rate books that are not in the stage/test environment, then those rate routines are initially missing from the JAR file created in the stage/test environment. When the production system starts up, PolicyCenter updates the JAR file with the missing rate routines. However, this increases startup time.

Propagating major changes to the production system

If you are making major changes that require rebuilding the PolicyCenter application, then generate and download the rate routines JAR file on the stage/test system as part of the rebuilding process. Then include it in the production system.

Another reason for regenerating the JAR file is that rate books can expire. PolicyCenter never removes any routines that are already in the JAR, it only adds new routines. So the JAR file may contain rate routines from expired rate books. These rate routines, which will never be used for rating, will be preloaded.

Enable and configure the precompile rate routines JAR file feature

About this task

Follow these instructions to enable and configure precompiling the rate routines JAR file on the stage/test and production systems.

Procedure

Enable

1. In Studio, open `config.xml`.
2. Set the value of the `EnableRateRoutinesJar` parameter to `true`.

Setting this parameter to `true`:

- Causes the **Rate Books** screen to display the **Generate JAR File** and **Download JAR File** buttons and other user interface elements for creating the rate routines JAR file. If you wish to hide these user interface elements in the production system, you can modify the `RateBookSearchScreen.pcf` file.
 - Enables preloading rate routines from the rate routines JAR file.
3. In the `RateRoutinesJarFile` parameter, specify the path where the rate routines JAR file will be located in PolicyCenter. The recommended name and location is `modules/configuration/plugins/Gosu/lib/rateroutines.jar`. See *Plugins, Prebuilt Integrations, and SOAP APIs* for allowed locations.
 4. If you are running on Windows Server:
 - a) Specify the rate routines update JAR file in the `RateRoutinesUpdateJarFile` parameter.
On Windows Server, PolicyCenter updates this JAR file when changes occur in rate books. If you do not specify this parameter, the JAR file is not updated when changes occur. If you run PolicyCenter from Studio, you get an exception and a message that the file is locked by another process.
 - b) Modify the startup process as described in "RateRoutinesUpdateJarFile" in *Configuration Parameters*.

Configure

5. Review the base implementation of the `RateBookPreloadPlugin` plugin implementation and decide whether you will use the base implementation or create your own. You can use the base implementation as a model.
6. If you only modified configuration parameters, restart PolicyCenter to pick up these changes. If you changed the `RateBookPreloadPlugin` plugin, then recompile and restart PolicyCenter.

What to do next

“Generate and download the rate routines JAR file” on page 166

Generate and download the rate routines JAR file

Before you begin

“Enable and configure the precompile rate routines JAR file feature” on page 165

About this task

When rate books are ready to be deployed to the production system, then generate and download the rate routine JAR file on the stage/test system.

Procedure

1. On the stage/test system, go to the **Rate Books** screen and click **Generate JAR File** to create the rate routines JAR file. This action is asynchronous, so you can navigate to other screens.

If you make changes to rate books, redo this step.

2. When you have finalized the JAR file, click **Download JAR file** on the **Rate Books** page. This button becomes active after you generate the JAR file. If **Download JAR file** is not active, navigate away from and back to this page.

The downloaded JAR file is saved to a location specified by your browser and operating system. For example, on Windows, the file may be saved to the **Downloads** folder.

What to do next

“Deploy the rate routines JAR to the production system” on page 166

Deploy the rate routines JAR to the production system

Before you begin

- “Generate and download the rate routines JAR file” on page 166 on the stage/test system

About this task

Follow these instructions to build the rate routine JAR file into the production system. Perform these steps in the PolicyCenter directory where you are building the production system.

Procedure

1. In the PolicyCenter application directory for the production system, copy the downloaded `rateroutines.jar` to the directory you specified in the `RateRoutinesJarFile` configuration parameter.
2. Build PolicyCenter.

Rate book preload plugin

The preload Rating Management components feature and precompile rate routines JAR feature use the `RateBookPreloadPlugin` plugin.

The default plugin implementation is in the `RateBookPreloadPluginImpl` class. You can use this implementation as an example for creating your own plugin implementation.

- **Preload Rating Management components** – You may decide to preload one set of rate books, and compile a different set of rate books into the JAR file.
- **Precompile rate routines JAR file** – In the default implementation, the `RateBooksToExportToJar` getter returns the list of rate books that are in Active status. Your implementation can update the rate routines JAR file for other rate book events.

Preload Rating Management components feature

The preload Rating Management components feature calls the `RateBooksToPreload` getter to get the list of rate books to preload on system startup.

In the default implementation, the `RateBooksToPreload` getter returns the list of rate books that are in Active status. The rate books must meet the following criteria:

- For all Active rate books with the same code, the rate book with the most recent effective date.
- All Active rate books with effective date in the future. The assumption is that the rate book will soon be in use.

Precompile rate routines JAR file feature

The precompile rate routines JAR file feature calls the `RateBooksToExportToJar` getter to get the list of rate books whose rate routines will be included in the JAR file. PolicyCenter calls this getter when you generate the rate routines JAR file by clicking **Generate JAR file** on the **Rate Books** screen. PolicyCenter then compiles the rate routines in the list of rate books into the JAR file.

Note:

All rate routines used in rating must be in one of the rate books returned by the `RateBooksToExportToJAR` getter of the rate book preload plugin. PolicyCenter throws an exception when it attempts to use a rate routine that is not in the list of rate books.

For example, a rate book contains a rate routine, and the rate book is not in the list of rate books returned by the plugin. If rating attempts to use that rate routine, PolicyCenter throws an exception, even if the rate book is in Active status. PolicyCenter relies on the list of rate book returned by the `RateBooksToExportToJAR` getter.

In the default implementation, the `RateBooksToExportToJar` getter returns the list of rate books that are in Active status.

IMPORTANT: For the precompile rate routines JAR file feature, including rate books whose status is not Active may have undesirable results and is not advised.

Rate routines in non-Active rate books are editable. You have a rate routine in a non-Active rate book. You generate the JAR and the rate routine is in it. You start the server and the rate routine is preloaded. Now you edit the rate routine, and promote the rate book to Active, or you reimport the rate book with the modified rate routine. This causes a problem because the JAR file cannot be updated with the modified rate routine.

This feature calls the `update` method when PolicyCenter needs to update the rate routines JAR file. The `update` method uses the `RateBooksToExportToJar` getter.

In the default implementation, the JAR file is updated when:

- A rate book is promoted to Active status
- A rate book is imported

The input parameter to the `update` method is an event as defined by `RateBookEvent` enumerated list:

Create

Rate book created with Draft status

Stage

Rate book changed to Stage status

Approve

Rate book changed to Approved status

Activate

Rate book changed to Active status

Import

Rate book imported from XML

Export

Rate book exported to XML

Creating your own plugin implementation

You can use the default implementation, `RateBookPreloadPluginImpl`, as an example for creating your own plugin implementation. In your implementation, you may decide to preload one set of rate books, and compile a different set of rate books into the JAR file. Your implementation can update the JAR file for other rate book events.

Use case for precompiling rate books in Approved status

The precompile rate routines JAR feature is designed to work with Active rate books, where rate routines are not editable. If you include rate books in other statuses, and you edit rate routines in those books, rating may not work as expected.

With this in mind, there is a use case for including rate books in Approved status provided that the rate routines remain unchanged.

There are Approved rate books in the stage environment that you plan to move to production and make Active there. In addition to Active rate books, you wish to include Approved rate books in the JAR file. When the rate book is made Active on production, all rate routines are already in the JAR file. If you do not include Approved rate books in the return list, then the JAR will not contain the routines included in those rate books. When you make those books Active in production, the server needs to update the JAR file by compiling and adding the rate routines in the promoted rate books to the JAR file.

You can make this change in the `RateBooksToExportToJar` getter. Modify the list of rate books returned to include rate books in Approved status, in addition to Active status. The result is that rate routines in Active and Approved status rate books are compiled into the rate routines JAR file.

IMPORTANT: For the precompile rate routines JAR file feature, including rate books whose status is not Active may have undesirable results and is not advised.

Rate routines in non-Active rate books are editable. You have a rate routine in a non-Active rate book. You generate the JAR and the rate routine is in it. You start the server and the rate routine is preloaded. Now you edit the rate routine, and promote the rate book to Active, or you reimport the rate book with the modified rate routine. This causes a problem because the JAR file cannot be updated with the modified rate routine.

Configuration parameters and loading Rating Management components

You can use the preload Rating Management components feature and precompile rate routines JAR feature together or separately.

Preload Rating Management components only

In the base configuration, the preload Rating Management components feature is enabled. The precompile rate routines JAR feature is disabled. The configuration parameters are set to these values:

- `RateBookPreloadEnabled` to `true`
- `EnableRateRoutinesJar` to `false`

When you start the server with only the preload Rating Management components feature enabled:

- The `RateRoutinesJarFile` parameter is ignored.
- Rating Management components, including rate routines, are loaded into cache on system startup.

Preload Rating Management components and precompile rate routines JAR file

To use both the preload Rating Management components feature and the precompile rate routines JAR feature, set the configuration parameters as follows:

- RateBookPreloadEnabled to true
- EnableRateRoutinesJar to true

When you start the server with both features enabled:

- If the RateRoutinesJarFile parameter points to a valid JAR file:
 - Then all rate routines in the JAR file are preloaded.
 - If the RateBooksToExportToJar method in the rate book preload plugin returns rate books whose rate routines are not in the JAR file, the JAR file is recompiled. Then the JAR file is preloaded. Because this slows startup time, you are encouraged to avoid this situation.
- If the RateRoutinesJarFile parameter points to an invalid JAR file:
 - PolicyCenter throws an exception and does not start.

Precompile rate routines JAR file only

To use only the precompile rate routines JAR file feature, set the configuration parameters as follows:

- RateBookPreloadEnabled to false
- EnableRateRoutinesJar to true
- RateRoutinesJarFile

When you start the server with only this feature enabled:

- Rate routines are not preloaded.
- The file specified by RateRoutinesJarFile is created if it does not exist. If it exists, it is updated if it is a valid JAR file.

Disable both features

To disable both the preload Rating Management components feature and the precompile rate routines JAR file feature, set the configuration parameters as follows:

- RateBookPreloadEnabled to false
- EnableRateRoutinesJar to false

When you start the server with both features disabled:

- Rate routines are not preloaded.
- The rate routines JAR file is not used to load rate routines and is not updated.

Improving rate table performance

For each rate table, you can choose whether to load the table into memory or access it from the database. By default, PolicyCenter loads rate tables into memory. Loading the rate table in memory can provide quick access. Accessing the table on disk is slower, but loading a large table into memory may slow system performance even more.

To decide whether or not to load the table into memory, consider how large the table will be and how often it will be referenced. Consider database lookup if:

- The table has multiple thousands of rows and is referenced infrequently. For example, referenced only on a small percentage of quotes.
- The table is very large (multiple tens of thousands of rows) and is only referenced once or twice per quote.

For anything smaller, or for a table referenced numerous times to quote a policy, use memory lookup, if possible. Database lookup can result in significant slowdown.

Rating management consults the lookup setting when promoting the rate book status from Draft to Stage. Therefore, you can only change the lookup setting when the rate book is in Draft status.

A rate table can be included in multiple rate books but can have only one setting for lookup. You can only change the lookup setting in the first rate book that included the rate table, if it is in Draft status.

Performance depends on the rate table content, not just the row count. For a table that is small enough to fit in memory, in-memory will usually outperform a call out to the database. For database-queried tables, if the query tends to hit the correct row every time, database performance is better than if the query frequently goes through the relaxing process. A query hitting on the first try usually corresponds to a table which is mostly filled out, with very few null values. On the other hand, you can expect relaxation to occur often if the table has many null values.

See also

- *Application Guide*

Improving performance with a covering index

Doing repeated database queries against a generic physical table (such as `DefaultRateFactorRow`) can cause performance issues.

If a rate table is large enough to warrant running the queries in the database, configure the physical table to have a custom row entity with a *covering index*. The covering index on the row entity includes the foreign key to the `RateTable`, the `Retired` property, and all parameters and factors.

For example, you have a custom row entity, `CustomFactor`. This row contains properties for parameters named `OptCode` and `Jurisdiction`. The index for this row begins with the foreign key to rate table and `Retired`, followed by the parameters and factors:

```
<index
  name="factorlookup"
  desc="Speeds lookup queries against this table.">
  <indexcol
    name="RateTable"
    keyposition="1"/>
  <indexcol
    name="Retired"
    keyposition="2"/>
  <indexcol
    name="OptCode"
    keyposition="5"/>
  <indexcol
    name="Jurisdiction"
    keyposition="6"/>
    name="Factor"
    keyposition="7"/>
</index>
```

See also

- *Application Guide*

Improving performance of rate tables loaded in memory

To improve rate table lookups, these configuration parameters influence the behavior of the rate table row cache. You can modify the values in `config.xml`.

Cache entries are refreshed when they have exceeded the configured entry lifetime, and for some caches the entire cache is refreshed at periodic intervals. If a change to a rate book, rate table definition, or rate table is always followed by a server restart, the entry lifetime and update intervals can be set to their maximum allowed values. Conversely, if such changes are made without restarting the server and changes are required to take effect promptly, these parameters must be set to smaller values. However, setting these parameters to smaller values results in greater overhead associated with keeping the cache synchronized with the underlying entities.

A non-active rate book is a rate book not in Active status.

Parameter	Description	Default
<code>RateBookCacheUpdateIntervalSecs</code>	The time in seconds after which the rate book cache is rebuilt: all non-active rate books are refreshed and any	86400

Parameter	Description	Default
	rate books added after the last update are added to the cache.	
RateBookCacheEntryLifetimeSecs	The maximum time in seconds that a non-active rate book can stay in the rate book cache without being refreshed.	3600
RateTableCacheConcurrency	Provides guidance on the maximum number of concurrent operations on the rate table cache.	8
RateTableCacheEntryLifetimeSecs	The maximum time in seconds for which a rate table in a non-active rate book can stay in the rate table cache without being refreshed.	600
RateTableCacheMaxSize	The maximum number of rate tables that can be in the rate table cache. For no limit, set to -1. To disable rate table caching, set to 0.	-1
RateTableDefinitionCacheUpdateIntervalSecs	The time in seconds after which the rate table definition cache is cleared.	86400

Improving rate table cache performance

PolicyCenter maintains a rate table cache to improve the performance of rating and quoting. The rate table cache is loaded when PolicyCenter starts up, however if there are a large number of rate table rows then the preloading may take from thirty minutes to an hour to finish. This process increases the startup time of the PolicyCenter server.

PolicyCenter includes a feature that can significantly reduce the amount of time it takes to preload the cache for large number of rate table rows.

You must carry out the following steps to use this feature:

1. Generate a set of files containing a serialized version of the rate table cache. You must generate these files on a non-production system. The non-production system must have substantially the same active rate books as the system to which the serialization files are to be deployed. This process is similar to the way product model files are generated in Advanced Product Designer.
2. Deploy the files generated in the previous step to a production system.
3. Modify the code that preloads the rate tables (if necessary).

Notes:

1. This feature cannot be used with cloud rating.
2. This feature depends on the public IDs being properly maintained and synchronized between the non-production environment used to generate the serialization and the production system to which the files are later deployed. Specifically, the same rate table in both environments must have the same public ID.
3. This feature only improves preload times for rate tables that belong to active rate books.
4. We recommend that you periodically update the serialization files when new active rate books are promoted or imported, and when rate table definitions are modified. It is not possible for PolicyCenter automatically keep the serialization files synchronized with changes to rate table definitions and the addition of active rate books. Even if PolicyCenter automatically updated the serialization files in the deployed image, those changes would be lost during redeployment. The deployed image must be updated and that can only be done by manually repeating the serialization.
5. The serialization files are not required to be always completely synchronized with the set of rate tables belonging to the active rate books. It is acceptable if some rate tables belonging to active rate books are missing from the serialization files.
6. Serialization files are always generated for the entire rate table cache (it is not incremental to the rate tables that were changed or promoted since the last serialization).

Enable rate table cache improvements in PolicyCenter

When the rate table cache serialization is enabled through the Boolean configuration parameter `RateTableCacheSerializationEnabled` (see “Rate table cache configuration files” on page 173), the button **Optimize Rate Tables Preload** appears on the **Rate Table Definitions** search screen.

When you click on **Optimize Rate Tables Preload**, a dialog appears that allows you to specify the Git branch where the serialization files will be checked in. You can then click on **Finish** to generate the serialization files, check them in, and push the branch (see “Generate and deploy the serialization files on GWCP” on page 172) for the complete sequence of events.

Note: In a configuration with a large number of rate table rows it may take a long a time to generate the serialization files.

Generate and deploy the serialization files on GWCP

Generate serialization files on GWCP

1. (PolicyCenter) User clicks on **Optimize Rate Tables Preload**.
2. (PolicyCenter) User enters branch name.
3. (PolicyCenter) User clicks on **Finish**.
4. (GWCP BitBucket) PolicyCenter creates the branch.
5. (GWCP BitBucket) PolicyCenter commits the generated code to the branch.
6. (GWCP BitBucket) PolicyCenter pushes the branch to the repository.

Deploy serialization files on GWCP

1. (GWCP BitBucket) PolicyCenter pushes the branch to the repository.
2. (GWCP TeamCity) Trigger build to create docker image.
3. (Guidewire Home) Select branch and build and deploy your changes.

Modify preload code for rate table cache

Rate routines, rate tables and other rating-related artifacts are preloaded through `RatebookPrimer#prime()`. In the base configuration this function in turn calls `RTMLoadActions#primeForAllRateBooks()`. The base configuration file `RTMLoadActions.gs` is only provided as an example. If you have created your own modified version or subclassed `RTMLoadActions`, then you must manually update your version of `RTMLoadActions` so that the rate tables are loaded just as in the base configuration `RTMLoadActions.gs`. The relevant function `loadRatingRoutines()` is shown below:

```
public static function loadRatingRoutines() {
    final var rateBooksToPreload = lookupRateBooksToPreload()

    /*
     * Load rate routines and rate tables sequentially.
     */
    if (RateRoutinesJarUtil.RateRoutinesJarIsEnabled) {
        loadCalcRoutinesForRateBooksFromJar(lookupRateBooksToExportToJAR())
    } else {
        loadCalcRoutinesForRateBooks(rateBooksToPreload)
    }

    if (PCConfigParameters.RateTableCacheSerializationEnabled.Value) {
        RateTableCacheSerializerDeserializer.deserialize(rateBooksToPreload) // This will only get immutable tables

        // Get the rest of the tables referenced by given rate books
        var tables = StreamSupport.stream(new ArrayLoader<RateBook, RateTable>(RateBook.RATETABLES_PROP.get(),
            rateBooksToPreload).spliterator(), false)
            .filter(table -> table.Owned and table.QueryStrategy == FactorQueryStrategy.TC_MEMORY and
                // Not previously loaded by deserialization
                not RATE_TABLE_CACHE.containsKey(table))
            .collect(Collectors.toList<RateTable>())
        RateTablesLoader.loadRateTables(tables)
    } else {
        RateTablesLoader.loadRateTables(rateBooksToPreload)
    }
}
```

```
}  
}
```

Update serialization files for promoted rate books

When a rate book is imported as active or promoted to active in the production deployment, the serialized data of the cache entries of the rate tables owned by those rate books will be missing from the serialization files in the deployment. If a rate table definition is changed, the serialization data for all of the rate tables that reference that definition will be out of date because the cache entry for a table depends on the definition. Therefore it is necessary to periodically regenerate the serialization files and redeploy them.

Note that it is acceptable for the some tables to be missing from the serialization files, or for the serialization files for some tables to be out of date because the corresponding rate table definition has changed. The performance gain for other tables still remains.

Promoting new rate books to active

Promote the new rate book(s) to active and regenerate the serialization files in the non-production environment. Create a new deployment that includes the regenerated serialization files. Import the new rate books as active (rather than importing as staged and then promoting to active) into the production environment.

Modifying rate table definitions

You must only make changes to rate table definitions in a non-production environment. After you change the rate table definitions you can then regenerate the serialization files. Then change the rate table definitions in the production environment (at which point the serialized data will be out of date). Redeploy to production with the regenerated serialization files.

Rate table cache configuration files

RateTableCacheSerializationEnabled

If this parameter is set to false, the feature is completely hidden. It is possible to enable the feature and later disable it without any adverse impact: PolicyCenter will behave as if the feature had never been enabled and there will be no impact on performance.

RateBookPreloadEnabled

You must either set this parameter to true or load the rate table cache belonging to tables in active rate books before you generate the serialization files.

Cloud deployment properties

PolicyCenter reuses APD properties when it accesses the git repository.

Namespace: `pc.apd.codegen`

git.url

HTTP/S URL to the git repository.

git.https.username

The username for accessing the Git repository.

git.https.password

The password for the Git username.

Rate table cache log data

Log data for cache serialization operations is recorded on the `Application.Rating.RateTableManagement` logger (at the info level). Sample log data is shown below.

Optimize rate tables preload: serialization files generation

```

13:19:31,037 INFO Application.Rating.RateTableManagement Serialization: Started serializing rate table cache with 23
entries to directory '/home/sramamoorthy/projects/pc/ISPC-4568-h/modules/configuration/config/content/ratetables/
serialized'
Context={correlationId=582ccfd6-d11f-44bd-a6e6-75fdd4f46d8d, session=node013nbvqv5qrws9xtpa8it6331,
traceabilityID=582ccfd6-d11f-44bd-a6e6-75fdd4f46d8d, user=3, userID=su, userName=Super User}
13:19:31,040 INFO Application.Rating.RateTableManagement SerializationL Started serializability determination for all
23 cache entries.
Context={correlationId=582ccfd6-d11f-44bd-a6e6-75fdd4f46d8d, session=node013nbvqv5qrws9xtpa8it6331,
traceabilityID=582ccfd6-d11f-44bd-a6e6-75fdd4f46d8d, user=3, userID=su, userName=Super User}
13:19:31,051 INFO Application.Rating.RateTableManagement Serialization: Finished serializability determination for all
23 cache entries with 23 serializable entries in 11 ms
Context={correlationId=582ccfd6-d11f-44bd-a6e6-75fdd4f46d8d, session=node013nbvqv5qrws9xtpa8it6331,
traceabilityID=582ccfd6-d11f-44bd-a6e6-75fdd4f46d8d, user=3, userID=su, userName=Super User}
13:19:31,067 INFO Application.Rating.RateTableManagement Serialization: Started writing 1 entries from cache to file '/
home/sramamoorthy/projects/pc/ISPC-4568-h/modules/configuration/config/content/ratetables/serialized/rate-
tables-0.ser.zip'
Context={}
13:19:31,067 INFO Application.Rating.RateTableManagement Serialization: Started writing 1 entries from cache to file '/
home/sramamoorthy/projects/pc/ISPC-4568-h/modules/configuration/config/content/ratetables/serialized/rate-
tables-20.ser.zip'
Context={}
13:19:31,067 INFO Application.Rating.RateTableManagement Serialization: Started writing 1 entries from cache to file '/
home/sramamoorthy/projects/pc/ISPC-4568-h/modules/configuration/config/content/ratetables/serialized/rate-
tables-24.ser.zip'
Context={}
13:19:31,067 INFO Application.Rating.RateTableManagement Serialization: Started writing 1 entries from cache to file '/
home/sramamoorthy/projects/pc/ISPC-4568-h/modules/configuration/config/content/ratetables/serialized/rate-
tables-9.ser.zip'
Context={}
...
13:19:31,124 INFO Application.Rating.RateTableManagement Serialization: Finished writing 1 entries of 1 with total size
2 from cache to file '/home/sramamoorthy/projects/pc/ISPC-4568-h/modules/configuration/config/content/ratetables/
serialized/rate-tables-12.ser.zip' in 55 ms
Context={}
13:19:31,124 INFO Application.Rating.RateTableManagement Serialization: Finished writing 1 entries of 1 with total size
6 from cache to file '/home/sramamoorthy/projects/pc/ISPC-4568-h/modules/configuration/config/content/ratetables/
serialized/rate-tables-2.ser.zip' in 56 ms
Context={}
13:19:31,133 INFO Application.Rating.RateTableManagement Serialization: Finished writing 1 entries of 1 with total size
8 from cache to file '/home/sramamoorthy/projects/pc/ISPC-4568-h/modules/configuration/config/content/ratetables/
serialized/rate-tables-4.ser.zip' in 65 ms
Context={}
13:19:31,133 INFO Application.Rating.RateTableManagement Serialization: Finished serializing rate table cache with 23
entries with 477 total rows to 32 files '/home/sramamoorthy/projects/pc/ISPC-4568-h/modules/configuration/config/content/
ratetables/serialized/rate-tables*.ser' in 100 ms
Context={correlationId=582ccfd6-d11f-44bd-a6e6-75fdd4f46d8d, session=node013nbvqv5qrws9xtpa8it6331,
traceabilityID=582ccfd6-d11f-44bd-a6e6-75fdd4f46d8d, user=3, userID=su, userName=Super User}

```

Server run with rate table cache serialization

```

19:19:36,231 INFO Application.Rating.RateTableManagement Deserialization: Started building map of immutable rate tables
via PublicID
Context={server=borges.guidewire.com}
19:19:36,249 INFO Application.Rating.RateTableManagement Deserialization: Finished building map of size 23 of immutable
rate tables via PublicID in 17 ms
Context={server=borges.guidewire.com}
19:19:36,249 INFO Application.Rating.RateTableManagement Deserialization: Started deserializing into rate table cache
from directory '/home/sramamoorthy/projects/pc/ISPC-4568-h/modules/configuration/config/content/ratetables/serialized'
Context={server=borges.guidewire.com}
19:19:36,261 INFO Application.Rating.RateTableManagement Deserialization: Started reading entries into cache from file
'/home/sramamoorthy/projects/pc/ISPC-4568-h/modules/configuration/config/content/ratetables/serialized/rate-
tables-8.ser.zip'
Context={}
19:19:36,261 INFO Application.Rating.RateTableManagement Deserialization: Started reading entries into cache from file
'/home/sramamoorthy/projects/pc/ISPC-4568-h/modules/configuration/config/content/ratetables/serialized/rate-
tables-12.ser.zip'
Context={}
...
19:19:37,015 INFO Application.Rating.RateTableManagement Deserialization: Finished reading 1 entries (0 outdated) with
total rows 78 into cache from file '/home/sramamoorthy/projects/pc/ISPC-4568-h/modules/configuration/config/content/
ratetables/serialized/rate-tables-31.ser.zip' in 750 ms
Context={}
19:19:37,037 INFO Application.Rating.RateTableManagement Deserialization: Finished reading 1 entries (0 outdated) with
total rows 151 into cache from file '/home/sramamoorthy/projects/pc/ISPC-4568-h/modules/configuration/config/content/
ratetables/serialized/rate-tables-0.ser.zip' in 774 ms
Context={}
...
19:19:37,056 INFO Application.Rating.RateTableManagement Deserialization: Finished deserializing 23 entries with total
477 rows into cache from directory '/home/sramamoorthy/projects/pc/ISPC-4568-h/modules/configuration/config/content/
ratetables/serialized' in 806 ms

```

Out-of-sync serialized entry warning

```
21:04:25,911 INFO Application.Rating.RateTableManagement Deserialization: Finished reading 1 entries (1 outdated) with total rows 5 into cache from file '/home/sramamoorthy/projects/pc/ISPC-4568-h/modules/configuration/config/content/ratetables/serialized/rate-tables-23.ser.zip' in 540 ms
```

Notes for testing

If you start PolicyCenter with an empty database then you will not be able to generate the serialization files. This is because no data exists at server startup and the rate table cache is empty. You must restart the server if sample data is created on the initial run, or create data containing rate tables that belong to active rate books. On this second run, sample data or other data will exist which means the rate table cache is being populated.

Parameters and properties for Rating Management

Minimum rating level parameter

Rating Management has a configuration parameter to set the minimum rating level.

In production, only rate books in **Active** status are used for policy rating. However, since active rate books can never be updated, it is useful to be able to test with rate books not yet in **Active** status. Then you can make changes in the current version instead of creating another version of the rate book for the next cycle of testing.

Rating Management has a **RatingLevel** parameter to support rating with rate books not yet in **Active** status in a development or test environment. This parameter controls the minimum rate book status that the rating query considers as a candidate. The default level is **Active**, which is the required level for production environments. A lower level can be set for development or test environments.

The rating level is set in a parameter on the **IRatingPlugin** plugin in `gw.plugin.policyperiod`. The parameter can have one of the following **RateBookStatus** values (in decreasing order):

- **Active**
- **Approved**
- **Stage**
- **Draft**

The **RatingLevel** is passed to the query as part of the filter. The query considers all books that are of equal or higher status than the minimum rating level.

Note: For rating to work in a production environment, the **RatingLevel** parameter in the plugin must be set to **Active**.

See also

- *Plugins, Prebuilt Integrations, and SOAP APIs*

Rate table normalization configuration parameters

In a rate table that contains a range parameter and any other parameter, the table may have ranges that overlap. For tables that fall within specified configuration parameters, PolicyCenter normalizes the rate table to speed up rate factor lookups.

In `config.xml`, two parameters control whether or not PolicyCenter normalizes a rate table by default. These parameters are:

- **RateTableManagementNormalizationRowThreshold**
Default: 1000

If the number of rows in the normalized table would exceed this threshold, the user is given the option to store a non-normalized version of the table.

- RateTableManagementNormalizationRowLimit

Default: 10000

PolicyCenter determines whether to normalize a rate table if the table contains this many rows or more.

If the number of rows in the normalized table would exceed this limit, the table is automatically marked as non-normalizable. PolicyCenter stores the non-normalized version of the table.

[See also](#)

- *Application Guide*

Logging properties for Rating Management

Rating Management defines the following logging categories:

Category	Description
Application.Rating	Logging for messages related to Guidewire Rating Management.
Application.Rating.RateTableManagement	Logging for messages related to rate table management.
Application.Rating.Rateflow	Logging for messages related to creating and executing rate routines.

Third party libraries for Rating Management

Rating Management has the following third-party libraries.

POI library

The POI library supports reading and writing rate tables to and from Microsoft Excel workbooks. POI is an Apache licensed library. This library provides Java APIs for manipulating various file formats based upon the Office Open XML standards (OOXML) and Microsoft's OLE 2 Compound Document format (OLE2).

User authority and permissions for Rating Management

The base configuration includes a starter set of permissions and roles related to Guidewire Rating Management. Review and update these as required for your specific implementation. See the *Application Guide* for details on roles and permissions.

The base configuration has the following permissions and roles.

Rating Management permissions

The permissions related to Rating Management are:

- ratebookview
- ratebookedit
- ratebookapprove
- editrateasofdate
- usereditlang
- rateimpacttesting

Rating Management roles

The roles related to Rating Management are:

- rating_analyst
- rating_supervisor

Assignment of permission to roles

Permissions are assigned to roles in the default configuration as follows:

Permission	Code	Assigned to roles
View rate books and tables	ratebookview	rating_analyst rating_supervisor underwriter underwriter_supervisor
Edit rate books and tables	ratebookedit	rating_analyst rating_supervisor
Approve rate books and tables	ratebookapprove	rating_supervisor
Edit Rate as of Date	editrateasofdate	underwriter underwriter_supervisor
Edit user language	usereditlang	rating_analyst rating_supervisor underwriter underwriter_supervisor
Rate policies for rate impact testing	rateimpacttesting	rating_analyst rating_supervisor

Custom physical tables for Rating Management

The base configuration includes two physical tables that can be used as-is and as models for other custom tables that may be required for a particular implementation. These tables are:

- A default, generic physical table that can be used to store many of the rate tables required for most lines of business. This table is the `DefaultRateFactorRow` entity.
- A sample custom physical table for simple coverage-based rate tables. This table is the `CoverageRateFactor` entity.

If neither table is appropriate, you can configure a new custom physical table for one or a group of similar rate table definitions.

[See also](#)

- *Application Guide*

Configuring a new custom physical table

To configure a new custom physical table, you must create an `.eti` file for a system entity. You can use the `CoverageRateFactor` entity as a model for your new custom entity.

The following requirements must be satisfied when creating a custom entity for rate table factors:

- Must have a foreign key to `RateTable` entity.
- The name of the foreign key must be `RateTable`.
- Must define at least two columns to accommodate at least one parameter and at least one factor.

- Must have at least one column of either String, Integer, Decimal, Boolean, or Date type. These are the types supported for parameters and factors.

The custom entity must be created in the `modules/configuration/config/extensions` directory.

The custom entity can be referenced in the **Physical Table** field in the **Basics** tab of the **Rate Table Definition Editor**.

Configuring value providers

The base configuration includes the following types of value providers:

- Typelist value provider
- PolicyCenter product model value provider
- Reference factor value provider

If none of these is appropriate, a new custom value provider can be configured.

All classes related to value providers are located in the `gw.rating.rtm.valueprovider` package. A value provider may use an optional list of string arguments. Value providers are instantiated through the `RateTableCellValueProviderFactory` factory class.

This topic describes the value providers in the default configuration, and how to configure a new value provider.

[See also](#)

- The *Application Guide* for user-level information

Typelist value provider

The typelist value provide is:

- Implemented in `TypelistValueProvider` class.
- Tied to a typelist defined in the system.
- Does not support any optional attributes.
- A list of values is sorted according to typelist priorities.

PolicyCenter product model value providers

This topic includes descriptions of the PolicyCenter product model value providers.

Note: Availability of coverages, terms and options is not checked by any of these value providers.

Coverage value provider

The coverage value provider, `CoverageValueProvider`:

- Extends the abstract `AbstractProductModelValueProvider` class.
- Provides a list of policy line coverages for a particular policy line.
- Returns:
 - A list of coverage codes for values
 - A composite label consisting of coverage category name and coverage name based on a string template defined in the `Web.Rating.CoverageValueProvider.DisplayLabel` display key
- The list is sorted by coverage category priority first and then by coverage priority.
- Does not support any optional attributes.

Coverage term value provider

The coverage term value provider, `CovTermValueProvider`:

- Extends abstract `AbstractProductModelValueProvider` class.
- Provides a list of coverage terms for specific policy line coverage.
- Returns a list of coverage term codes sorted by priority.
- Supports one optional argument: coverage code.
- Use for rate table columns satisfying one of the following conditions:
 - A rate table column depends on another column that contains coverage code.
 - An argument is specified for this value provider.

Coverage term option value provider

The coverage term option value provider, `CovTermOptionValueProvider`:

- Extends the abstract `AbstractProductModelValueProvider` class indirectly through the `AbstractCovTermOptionValueProvider` class.
- Provides a list of coverage term options for a specific coverage term of a specific coverage.
- Returns a list of coverage term option codes sorted by priority.
- Supports two optional parameters: coverage code and coverage term code
- Works with either none or both parameters.
- Use for rate table columns satisfying one of the following conditions:
 - A rate table column depends on another rate table column that contains coverage term code.
 - Both arguments are specified for this value provider.

Termless coverage value provider

The termless coverage value provider, `TermlessCoverageValueProvider`:

- Is similar to `CoverageValueProvider` but returns only those coverages that have no terms.

Reference factor value provider

The reference factor value provider:

- Is implemented in the `ReferenceFactorValueProvider` class.
- Retrieves all distinct values used in a column of a rate table (source rate table).
- Must have two arguments configured:
 - Rate table code – Code of a source rate table.
 - Column name – Name of the column in the source rate table
- Returns a list of distinct values from a specific column (second argument) of the source rate table (first argument).
- Does not return any specific labels, just the values themselves.
- Implements internal cache (a hash map): (argument 1 + argument 2) -> ({set of values}).
 - Initialized when a value provider is constructed.
 - Populated for each combination of argument values on each call to `ReferenceFactorValueProvider#getValues()` method for those combinations of arguments that are not found in the cache.

- The lookup of the source rate table is done in the same rate book as the table specifying this value provider.

Creating a new value provider

If none of the default value providers is appropriate, you can configure a new custom value provider.

You create new value providers by extending the `RateTableCellValueProvider` abstract class. Create new value providers in the `gw.rating.rtm.valueprovider` package. The user interface has the following restriction: the package is hard-coded and does not display in the rate table definition user interface.

Performance considerations have to be taken into account when creating new value providers. Values may be retrieved from value providers multiple times when the **Rate Table Editor** screen loads. Therefore, caching responses that the value provider gets from certain sources may improve performance of the user interface. For an example of caching in value providers, see any of the product model value providers.

For the new value provider to appear in the list of custom value providers in the **Rate Table Definition Editor**, you must add a new typecode to the `ValueProvider` typelist. The Code of the added typecode must be the same as the name of the value provider Gosu class.

Configuring matching rule operations

In a rate table, you define a matching rule for each parameter. You set the matching rule to one of the match operations defined in the application.

If none of the default match operations is appropriate, you can configure a new custom match operation.

[See also](#)

- *Application Guide*

Configure a new match operation

[About this task](#)

To configure a new match operation:

[Procedure](#)

1. In Product Designer, navigate to the `rtm_matchop_defs.xml` system table.
2. In **RateTableMatchOpDefinition**, add an entry to the system table. You must specify the following:
 - **OpCode** – A code that uniquely identifies this match operation.
 - **OpName** – Appears in PolicyCenter as a match operation choice when the user clicks **Add** in the **Parameters** tab of the **Rate Table Definition Editor** screen.
 - **NumberOfParameters** – The number of parameters.
 - **ImplClass** – The fully qualified name of the new match operation factory class. For example, you can define the class as `gw.rating.rtm.matchop.OpCodeMatchOpFactory`.
3. Create the following new classes:
 - A factory class that extends abstract `gw.rating.rtm.matchop.MatchOperationFactory`. See “Extending the match operation factory class” on page 181.
 - A class that extends abstract `gw.rating.rtm.matchop.StatelessMatchOperator`. See “Match operation implementation” on page 181.
4. Optionally, create a custom validator for the match operation.
 - A class that extends `gw.rating.rtm.validation.MatchOpValidator`. See “Match operation validator” on page 182.

Extending the match operation factory class

The match operation factory class, `MatchOperationFactory`, is an abstract base class that provides a static method to get match operation factory by name: `getFactoryByName(String)`. You must create a class that extends `gw.rating.rtm.matchop.MatchOperationFactory`. For an example, see `gw.rating.rtm.matchop.LessThanOrEqualMatchOpFactory.gs`.

Your new match operation factory implementation must implement at least the following two methods.

Method That Creates a Stateless Match Operator

This method signature is:

```
function createStatelessMatchOperator(entity.RateTableMatchOp): <S extends StatelessMatchOperator>
```

This method must create and return a new instance of the match operation implementation class as described in “Match operation implementation” on page 181.

Method That Creates a Validator

This method signature is:

```
function createValidator():MatchOpValidator
```

This method must create and return a new instance of the match operation validator class as described in “Match operation validator” on page 182.

Match operation implementation

You must create a class that extends abstract `gw.rating.rtm.matchop.StatelessMatchOperator`. For an example, see `gw.rating.rtm.matchop.StatelessLessThanOrEqualMatch`.

The match operation implementation constructor accepts the following argument:

- An instance of `RateTableMatchOp` entity. This instance is part of the rate table definition.

Your new match operation implementation can provide implementations of methods from the base class, including the following methods.

Filter That Evaluates Row Under Consideration

This filter method signature is:

```
function filter(List<OrderedPersistenceAdapter>, Comparable, boolean):List<OrderedPersistenceAdapter>
```

This in-memory filter evaluates all rows still under consideration, and returns only those rows which satisfy the conditions for the match operation. This filter is only called when the match operation is active. A corresponding relax method is not needed, because the list of results to be filtered are pre-partitioned according to whether the row would match a filter or a relax. The input results must be fully ordered and stay that way upon output. For more information about relaxing, see also the *Application Guide*.

A stateless match operation doing in-memory filtering expects its input to be in ascending order with respect to the field or fields being matched by the match operation. For example, a range match operation matches the fields `int1` (minimum) and `int2` (maximum). This operation assumes that the rows are ordered according to increasing `int1` value, with a tie being broken by comparing the `int2` values.

The output of the stateless match operation must be a list that is properly formatted for the next match operation. Therefore, the output must be in ascending order according to the field or fields matched by that next match operation.

To meet this requirement, PolicyCenter sorts the rate table when it is first loaded, using order by/then by sorting. The rate table is sorted by the first match operation’s columns. Then each region with equal values for the first match

operation is sorted for the second match operation's columns, and so on. A match operation controls how its rows are ordered by overriding the method:

```
compareRowValues(a : Comparable, b : Comparable) : int
```

If a match operation does not naturally produce correctly-ordered output from this form of input, it needs to reorder the rows as part of its filtering process. For most match operations, an easy way to do this is to override the property `needsSubrangeMerge` so that it returns `true`. The `StatelessLessThanOrEqualMatch` provides an example of this.

Filter That Adds a Constraint to the Database Query

This filter method signature is:

```
function filter(Query<KeyableBean>, Comparable<Object>)
```

Adds a constraint to the database query and returns the adjusted query.

Entity instances that are passed into the above functions represent rate factor entities such as `RateFactorRow`, `CoverageRateFactor`, or any other custom entity.

Method Which Computes a Score

This method signature is:

```
function getScore(OrderedPersistenceAdapter, Comparable) : Comparable
```

Used to compute a score for the input argument, `Comparable`, in relation to rate table row value, `OrderedPersistenceAdapter`. This score is used during rate table matching to determine the best match.

Match operation validator

The match operation validator validates the parameter inputs to the match operation. To implement a validator, you must create a class that extends abstract `MatchOpValidator` in the `gw.rating.rtm.validation` package. For an example validator, see `RangeMatchOpMaxInclValidator` class in the `gw.rating.rtm.validation` package.

Note: If the match operation does not require a custom validator, you can use the `NoOpValidator` when creating the `MatchOperationFactory` for the new match operation. For an example, see `ExactMatchOpFactory` in the `gw.rating.rtm.matchop` package.

A new match operation validator must implement the following methods.

Validate Method

This method signature is:

```
function validate(List<Comparable>) : boolean
```

Returns `true` if all values are valid from the point of view of this match operation. For example, an exact match accepts any value, so it always returns `true`. See “step 4” in “Configure a new match operation” on page 180.

A range match operation accepts a list of two values where either value can be null or the first value is less than the second value. Otherwise, return `false`.

Validate Pair Method

This method signature is:

```
function validatePair(List<Comparable>, List<Comparable>) : boolean
```

Validates two sets of values against each other and returns `true` if both sets are valid with regards to each other.

This method is mainly used to identify overlaps and other value conflicts. For example, for the range match operation validator, this function checks if two numeric (or date) ranges have no overlap.

Configuring rate book matching

When selecting the most appropriate rate book, if an exact match is not found, PolicyCenter *relaxes* the optional attributes by looking for null in the optional attributes. The relaxing rules build an ordered list of rate books by relaxing the optional attributes.

The main entry point for rate book matching code is the `RatingQueryFacade` Gosu class. Some of the methods and objects that this class provides are:

- The `matchers` method finds the most appropriate rate book.
- The `matchersForHierarchy` method defines the cascading hierarchy for rate book matching when a rate table or rate routine is not found in the most appropriate rate book. Cascaded lookup must be enabled.
- Query for a rate book based on values in a `RateBookQueryFilter`. Uses the `RateBookQueryFilter` class.
- Query for a factor in the requested table based on `RateQueryParam` values. Uses the `RateBookQueryParam` class.

You can change the rate book matching logic by editing the `RatingQueryFacade` class and the associated `RateBookMatcher`, `RateBookQueryParam`, and `RateBookQueryFilter` classes.

See also

- *Application Guide*

Configuring rate routines

Configuring the rating engine to execute the rate routine

You can add a rate routine in the PolicyCenter **Rate Routines** screen. However, that rate routine is not part of rating a policy until you configure the rating engine to execute the rate routine.

In Rating Management, the built-in rating plugin implementation is `PCRatingPlugin`. This class implements rating only for the personal auto and commercial property lines of business. The sample data in the base configuration contains rate routines for these lines of business. The rate routines provided in the base configuration are already configured in the rating engine.

This topic describes how to extend the `PCRatingPlugin` class to execute new rate routines that you add.

See also

- The *Plugins, Prebuilt Integrations, and SOAP APIs* for instructions on how to enable the `PCRatingPlugin` plugin and extend it to other lines of business.
- *Application Guide*

Extend the rating engine to execute new rate routines

About this task

If you add new rate routines in PolicyCenter, you must extend the rating engine for the line of business specified in the rate routine. The `PCRatingPlugin` class calls the rating engine for each line of business. In the base configuration, the rating engine classes that use rating management are `PARatingEngine` and `CPRatingEngine`.

IMPORTANT: When adding code that executes a rate routine, be aware that simply executing a rate routine consumes both time and resources, whether or not the rate routine performs any actions. Therefore, only execute the rate routine when necessary. For example if a rate routine applies to a specific coverage on a policy, then make sure your code executes the rate routine only when the policy contains that coverage. The `executeCalcRoutine` method executes the rate routine.

To extend the rating engine to execute new rate routines:

Procedure

1. In the rating engine, add code that calls the `RateBook.executeCalcRoutine` method. For example, to execute a new rate routine in personal auto, modify the `PARatingEngine` class in the `gw.lob.pa.rating` package.

The method signature for `executeCalcRoutine` method is:

```
function executeCalcRoutine(code : String, costData : CostData<Cost, PolicyLine>,
    worksheetContainer : WorksheetEntryContainer,
    paramSet : Map<CalcRoutineParamName, Object>)
```

The arguments are:

- `code` – The rate routine code that uniquely identifies the rate routine, as a `String`.
 - `costData` – The cost data object which contains properties you can use to calculate a rate, as a `CostData`.
 - `worksheetContainer` – Holds information for rating worksheets. Rating worksheets describe the series of calculations to generate a rate, as a `WorksheetEntryContainer`. For more information, see the *Application Guide*.
 - `paramSet` – A map representing the parameters in the parameter set for the rate routine, as a `java.util.Map`.
2. If the rate routines does not calculate properties on the cost, set the `costData` parameter to `null` and the `worksheetContainer` to a `NoCostWorksheetContainer`.

For example, the following code in `PARatingEngine.gs` executes the `pa_assign_driver_style1_rr` rate routine:

```
RateBook.executeCalcRoutine("pa_assign_driver_style1_rr", null,
    new NoCostWorksheetContainer(), assignDriverParameterMap)
```

The `PARatingEngine` class is in the `gw.lob.pa.rating` package. The `NoCostWorksheetContainer` stores data about what the rate routine executed. However, the default implementation of rating worksheets does not save this information to the database nor display it on the **Rating Worksheet** screen.

See also

- *Application Guide*

Example of how the rating engine executes the rate routine

The sample data for the personal auto line of business contains a **PA Vehicle Coverage Premium Algorithm** rate routine. This rate routine provides a cost for vehicle level coverages on the policy. The code for this rate routine is `pa_veh_cov_premium_rr`.

The `PCRatingPlugin` class calls the `PARatingEngine` class to rate a personal auto policy. The `PARatingEngine` class is in the `gw.lob.pa.rating` package.

In the `PARatingEngine` class, the `rateVehicleCoverage` method rates vehicle-specific coverages such as a coverage with the `PALiabilityCov` code. If the policy has this coverage, the code executes the `pa_veh_cov_premium_rr` rate routine.

Adding a new rate routine function

A rate routine function is a predefined function that you can include in a rate routine step. Rate routine functions are defined in Gosu classes. You can define your own functions by adding a Gosu method to these classes. The function can return a value of any type. Functions with a return value appear in the **Operand** column of a rate routine step. Void functions appear in the **Instruction** column. Void functions do not return a value. An example of a void function is **logAmount**.

The `SharedRatingFunctions` class in the `gw.rating.flow.util` package contains rating functions for use across product lines.

For line-specific functions, create a class that extends the class `RatingFunctionSource`. By extending this class, PolicyCenter calls your rate routine function class. Functions in this class appear on the policy lines specified in the

availableForLine method. This method takes a policy line code String and returns true if the class supports that policy line. This class must implement the availableForLine method.

For example, the PARatingFunctions class in the gw.lob.pa.rating package implements rate routine functions only for the personal auto line.

A rate routine function cannot be a parameterized method. If you define a rate routine function as a parameterized method, it does not appear in the list of functions on the **Select a Function** screen. PolicyCenter sends a warning to the log file. For more information about parameterized methods, see the *Gosu Reference Guide*.

See also

- “Logging rate routine functions in a rating worksheet” on page 197

Configuring rounding operators

In the default configuration, you can use the following rounding operators in a rate routine:

- **R** – Round half up. Round towards the nearest number according to scale. If both numbers are equidistant, round away from 0.
- **RD** – Round down. Round down to the nearest number according to scale. Always round towards 0.
- **RU** – Round up. Round up to the nearest number according to scale. Always round away from 0.
- **RE** – Round half-even. Round towards the nearest number according to scale, unless both numbers are equidistant. If equidistant, round towards the even number.

Through configuration, you can make the following optional rounding operators available in rate routines:

- **RC** – Round ceiling. Round up to positive infinity.
- **RF** – Round floor. Round down to negative infinity.
- **HD** – Round half-down. Round towards nearest number according to scale, unless both numbers are equidistant. If equidistant, round towards 0.
- **NR** – Unnecessary. The requested operation is expected to produce an exact result. Throws an exception if rounding is necessary.

See also

- *Application Guide*

Make optional rounding operators available in rate routines

About this task

This topic describes how to configure PolicyCenter to make the optional rounding operators available for use in rate routine steps. In the default configuration, PolicyCenter the optional rounding operators, **RC**, **RF**, **HD**, and **NR**, are not available.

In Studio, the rounding operators are in the rounding category. Optional rounding operators are in the optrounding category. This topic describes how to make the **RF** optional rounding operator defined by the floor typecode available.

Procedure

1. In Studio, open the CalcStepOperatorType.tti typelist. The **Typelist** tab lists the typecodes in the typelist.
2. Click the **Outgoing Categories** tab.
3. Expand the optrounding category.
This category contains the optional rounding operators. The floor operator is in this category.
4. Return to the **Typelist** tab.
5. In the **Element** hierarchy, expand the floor typecode to reveal the category contained within.

For the floor operator, the category value is optrounding.

6. In the **Element** hierarchy, click **category** in the floor typecode.
7. Change **Value** of the **code** to **rounding**.
8. Click the **Categories** tab to verify that floor now appears in the rounding category.

Configure prefixes that identify types in rate routine steps

About this task

For some instruction and operand types, you can add or change the prefix that appears in the rate routine step. For example, in the default configuration, **table:** precedes the name of the rate table in a step. When changing the prefix, be aware that the prefix takes up space in the rate routine step.

In the default configuration, Studio includes display keys for prefixes on properties on the cost, function, rate table, and variable types.

To change the type prefix for properties on the cost, functions, rate tables, and variables:

Procedure

1. In Studio, localize the display key as described in the *Globalization Guide*.

For each type, the following table contains the name of the display key, the default prefix, alternate prefix, and the unicode value of the alternate prefix. The alternate prefix is provided as a suggestion only. You can define the prefix to a string of your choice. You can enter the unicode character directly into the Studio editor.

Instruction and operand type	Display key	Default prefix	Alternate prefix	Unicode value of alternate
Properties on the cost	Web.Rating.Flow.CalcRoutine.CostDataStoreLabel	None		
Function	Web.Rating.Flow.CalcRoutine.RateFunctionLabel	None	f	\u222B
Rate table	Web.Rating.Flow.RateTableLabel	table:		\u229E
Variable	Web.Rating.Flow.CalcRoutine.RateVariableLabel	None	v:	

If you add a variable prefix, the prefix appears for user defined variables and multiple factor variables.

2. In PolicyCenter, navigate to a rate routine to view the new prefix.

See also

- “Using the Display Keys editor”
- *Application Guide*

Configuring variant identifiers for a rate routine

A rate routine can have variant rate routines. The rate routine and its variant rate routines have the same code. In the default configuration, the jurisdiction variant is an example of a variant identifier. Through configuration, you can extend or change the definition of variants to have other identifiers. For example, you can configure a variant identifier to other features of the policy such as the underwriting company. Through configuration, you can remove the jurisdiction variant. For more information, see the *Application Guide*.

This topic describes how to configure a variant identifier for a rate routine.

Add the variant identifier to the data model

Procedure

1. In Studio, add a column with the name of the variant identifier to `CalcRoutineDefinition.eti` and include the name in the index. For detailed instructions, see *Data Model Configuration*.

For example, add a column for the variant identifier named `NewVariantIdentifier` defined as follows:

Name	Value
name	NewVariantIdentifier
type	shorttext
nullok	true
desc	A new variant identifier
localization	a_table_name

2. Add a `NewVariantIdentifier` index to `CalcRoutineDefinition` defined as follows:

Name	Value
name	NewVariantIdentifier
keyposition	4

3. Change the Retired index keyposition from 4 to 5. The `NewVariantIdentifier` index is inserted before Retired.

What to do next

Now you can go to the next step, “Add the rate routine variant identifier to PCF files” on page 188.

Add the rate routine variant identifier to the rate routine plugin

Before you begin

Before you begin, you must complete “Add the variant identifier to the data model” on page 187.

About this task

The built-in implementation of the rate routine plugin (`IRateRoutinePlugin`) is `RateRoutinePlugin.gs` in the `gw.plugin.rateflow` package. The getter for the `BranchingFields` property returns an ordered list of zero or more variant identifiers.

Procedure

In the implementation of the rate routine plugin, add the name of the variant identifier to the `fields` variable of the getter for the `BranchingFields` property.

For example, add `NewVariantIdentifier` to the `fields` variable as follows:

```
override property get BranchingFields() : IEntityPropertyInfo[] {  
    var fields : ArrayList<String> = {"Jurisdiction", "NewVariantIdentifier"}  
    ...  
}
```

What to do next

Now you can go to the next step, “Add the rate routine variant identifier to PCF files” on page 188.

Add the rate routine variant identifier to PCF files

Before you begin

Before you begin, you must complete “Add the rate routine variant identifier to the rate routine plugin” on page 187.

About this task

In Studio, add the rate routine variant identifier to the following PCF files.

Procedure

1. Add **New Variant Identifier** to the **Rate Routine Details** and **New Rate Routine** screens.

- a) Add a `Web.Rating.RateRoutines.NewVariantIdentifier` display key and set the value to `New Variant`. You can add the display key by adding it to the `display_LanguageCode.properties` file in **configuration > config > Localizations**.
- b) In `RateRoutineDV.pcf`, add a basic input after the **Jurisdiction** input. For example, from the **Toolbox**, go to **Basic Inputs** and insert an **Input** with the following properties:

Property	Value	Description
<code>editable</code>	<code>canEditIdentifyingFields or copyVersionType == BRANCH</code>	Whether the input is editable.
<code>id</code>	<code>NewVariantIdentifier</code>	A unique identifier for this widget.
<code>label</code>	<code>displaykey.Web.Rating.RateRoutines.NewVariantIdentifier</code>	The input label.
<code>value</code>	<code>rateroutine.NewVariantIdentifier</code>	Set value to the <code>NewVariantIdentifier</code> property on the <code>CalcRoutineDefinition</code> entity instance.

2. On the **Rate Routine Details** screen, change the name of the **Create Jurisdiction Variant** button to something more descriptive such as **Create Variant Rate Routines**. This button will now create variants where you can set **Jurisdiction** and **New Variant**.
3. Add a **New Variant** column in the **Search Results** on the **Rate Routines** screen.

In `RateRoutinePanelSet.pcf`, add a basic cell just after the **Jurisdiction** cell. For example, go to **Basic Cells** and insert a **Cell** with the following properties:

Property	Value	Description
<code>id</code>	<code>RateroutineNewVariantIdentifier</code>	A unique identifier for this cell.
<code>label</code>	<code>displaykey.Web.Rating.RateRoutines.NewVariantIdentifier</code>	The cell label.
<code>value</code>	<code>rateroutine.NewVariantIdentifier</code>	Set value to the <code>NewVariantIdentifier</code> property on the <code>CalcRoutineDefinition</code> entity instance.

4. On the **Rate Book Details** screen, add **New Variant** to the **Included Rate Routines** tab.

In `RateBookDetailsScreen.pcf`, add a **TextCell** just after **Jurisdiction** with the following properties:

Property	Value	Description
<code>id</code>	<code>NewVariantIdentifier</code>	A unique identifier for this cell.
<code>label</code>	<code>displaykey.Web.Rating.RateRoutines.NewVariantIdentifier</code>	The cell label.
<code>value</code>	<code>rateroutine.NewVariantIdentifier</code>	The cell value.

5. Add **New Variant** to other rate routine screens where the **Jurisdiction** variant appears.

See also

- “Rate routine plugin methods” on page 201
- *Application Guide*

Configuring the rate routine plugin

When PolicyCenter executes a rate routine, PolicyCenter calls the rate routine plugin defined by the `IRateRoutinePlugin` interface. The rate routine plugin:

- Enables rating worksheets in the plugin.
- Sets up cost data wrappers that determine which properties on the cost appear when you select an instruction or operand in a rate routine step.

For more information, see “Rate routine plugin” on page 200.

Using Gosu to get the rate factor from the rate table

You can get the rate factor from a rate table in a rate routine. In certain circumstances however, getting the rate factor may involve more complex logic than rate routines support. This topic describes how to get the rate factor from the rate table using Gosu code. You might add this code in the rating engine or in a rate routine function.

This topic provides an example of how to accomplish this using workers’ compensation as an example.

Create a class for getting the rate factor

Procedure

Create a class to contain methods that get the rate factors from all rate tables in this line of business. For workers’ compensation, you can define a `WCRateFactorSearch.gs` class in the `gw.lob.wc.rating` package.

What to do next

You can now move to the next step “Define a method to get the rate factor” on page 189.

Define a method to get the rate factor

Before you begin

Before you begin, you must complete “Create a class for getting the rate factor” on page 189.

About this task

Follow these steps to define a `getFactor` method that gets the rate factor from a rate table. This method is general purpose and is not specific to a particular line of business. If you are adding Rating Management to multiple lines of business, you could choose to put this method in its own class.

Procedure

1. In the public method for the class, add private variables for storing the policy period and rate book status.

```
class WCRateFactorSearch {  
    private var _period : PolicyPeriod as readonly Period  
    private var _minRatingLevel : RateBookStatus as MinimumRatingLevel  
    construct() {  
    }  
}
```

2. Insert uses statements after the package statement at the top of the file. The method that you define in the following steps uses these classes.

```
package gw.lob.wc.rating

uses java.math.BigDecimal
uses gw.rating.rtm.query.RateQueryParam
uses java.lang.Comparable
uses java.util.Date
uses gw.rating.rtm.query.RateBookQueryFilter
uses gw.job.RenewalProcess
uses gw.rating.rtm.query.RatingQueryFacade

class WCRateFactorSearch {
```

3. After the class constructor, add the `getFactor` method that gets the rate factor.

```
construct() {
}
private function getFactor<Q extends Comparable>(jurisdiction : Jurisdiction,
inputParams : List<RateQueryParam<Q>>,
tableCode : String,
ratingDate : Date) : BigDecimal
{
    /* Constructs a new rateBookQueryFilter object and sets properties such as Jurisdiction */
    /* and UWCompany. */
    var filter = new RateBookQueryFilter(ratingDate, Period.RateAsOfDate,
Period.WorkersCompline.PatternCode)
    {
        :Jurisdiction = jurisdiction,
        :UWCompany = Period.UWCompany,
        :Offering = Period.Offering.CodeIdentifier,
        :MinimumRatingLevel = MinimumRatingLevel,
        :RenewalJob = Period.JobProcess typeis RenewalProcess
    }
    var result = new RatingQueryFacade().getFactor<Q, BigDecimal>(filter, tableCode, inputParams)
    return !result.Empty ? result.Factor : BigDecimal.ONE
}
```

The input parameters to the `getFactor` method are:

- `jurisdiction` – The jurisdiction of the policy location.
- `inputParams` – Specify values for each parameter in the rate table.
- `tableCode` – The code of the rate table from which to obtain the rate factor. This is the `TableCode` property on the `RateTableDefinition` object. In the user interface, `tableCode` is the **Code** on the **Basics** tab of the **Rate Table Definition** screen.
- `ratingDate` – The date for which to obtain rates. The date is compared against the `RateBook.EffectiveDate` or `RateBook.RenewalEffectiveDate` properties depending upon the job type. In the user interface, `ratingDate` is the **Policy Effective or Coverage Reference Date** or **Renewal Effective Date** in **Policy Criteria** on the **Rate Book Details** screen.

This method return the rate factor as a `BigDecimal`. If no rate factor is found, the method return the value 1.

What to do next

Now you can go to the next step, “Define methods that get the rate factor” on page 190.

Define methods that get the rate factor

Before you begin

Before you begin, you must complete “Define a method to get the rate factor” on page 189.

About this task

In this series of steps, you define a method that gets the rate factor. The rate factor is used to calculate rates for ratable objects such as coverables. Repeat these steps to define a method for each ratable object. This example gets the rate factor for a workers’ compensation class code.

Procedure

1. If you must get the jurisdiction from a policy location, add a `uses` statement at the top of the file after the package name.

The method that gets the jurisdiction for a particular ratable object uses methods in this class.

```
uses gw.api.util.JurisdictionMappingUtil
```

2. In the public method for the class, add a private variable for storing the code of the rate table. The code is passed to the `getFactor` method in the `tableCode` parameter.

```
class WCRateFactorSearch {
    private var WC7_CLASS_CODE_TABLE = "WorkersComp_ClassCode_Rate"
```

The private variable is not strictly necessary. Instead of using this private variable, you could pass the `tableCode` parameter directly in the call to the `getFactor` method.

3. Add a method that gets the rate factor for a particular ratable object. This method calls the `getFactor` method. Name the method appropriately for the type of ratable object.

In this example, the method is `getClassCodeRate`. This method gets the rate for a covered employee's class code from the rate table.

```
function getClassCodeRate(covEmp : WCCoveredEmployee, ratingDate : Date) : BigDecimal {
    var jurisdiction
    JurisdictionMappingUtil.getJurisdictionMappingForPolicyLocation(covEmp.LocationWM)
    var params = {
        new RateQueryParam<String>("CLASSCODE", covEmp.ClassCode.Code)
    }
    return getFactor(jurisdiction, params, WC7_CLASS_CODE_TABLE, ratingDate)
}
```

The input parameters to this method vary depending upon the ratable object. The input parameters to the `getClassCodeRate` method are:

- `covEmp` – The `WCCoveredEmployee` object from which to get the class code.
- `ratingDate` – The date for which to get the rate.

This method returns the rate factor as a `BigDecimal`.

Configuring new parameters in parameter sets

This topic describes how to create a new parameter and make the parameter available for inclusion in a parameter set. The new parameter is available in PolicyCenter, and you can insert the new parameter in rate routine steps. For example, in **Administration > Rating > Parameter Sets**, select a parameter set. In the **Parameters** panel, click **Edit**. To add a new parameter, click **Add**. In the **Name** column, drop-down list contains the list of parameters. This topic describes how to add a parameter to the list.

In PolicyCenter, parameter sets are associated with rate routines and rate table definitions. In addition, you must add the parameter to the rating engine code so that the parameter can be used in rating.

To configure a parameter:

- **In the PolicyCenter user interface, create the parameter for inclusion in a parameter set.** Define the parameter and specify a default type, such as `entity.PersonalVehicle` for a personal vehicle entity. Then you can include the parameter in a parameter set. In the parameter set, you can override the parameter's default type. Each rate routine that uses that parameter set can access that parameter in the rate routine steps.

For instructions, see "Creating a new parameter for inclusion in parameter sets" on page 192.

- **In Studio, modify the rating engine so that it rates the object or entity instance that the parameter specifies.** For each rate routine that includes the parameter set, add a mapping from the parameter to an object type. The object type could be the primary vehicle on a personal auto policy. When the rating engine executes the rate routine,

the parameter is used to reference the actual run-time instance or value. The run-time instance could be the primary vehicle on Solomon's personal auto policy.

For instructions, see "Adding a new parameter to the rating engine" on page 193.

- **If the parameter uses a wrapper, create the wrapper code.**

For instructions, see "Configuring new wrappers for parameter sets" on page 194.

The parameter definitions must meet the following criteria:

- **Parameter types must be compatible.** In the parameter set, the parameter's type must be compatible with the Gosu object that the rating engine passes to the method that executes the rate routine. If the user overrides the parameter's default type in the parameter set, the override takes precedence. The rating engine must pass an object that is the same type or a subtype of the parameter type. PolicyCenter throws an exception if the rating engine passes in an object that is not compatible with the parameter associated with the executing rate routine.
- **In the rating engine, the parameter map must be a superset of the rate routine parameters.** When the rating engine executes a rate routine, one of the arguments is a map of the rate routine's parameters. In the rate routine parameter map, each parameter is mapped to an object or entity instance, such as a vehicle on Solomon's policy. The rate routine parameter map must contain all of the names in the parameter set defined in PolicyCenter for the rate routine being executed. The parameter set may contain the names of additional parameters. The rating engine throws an exception if it executes a rate routine and the parameter map is missing any parameters in the parameter set.

See also

- "Adding a new parameter to the rating engine" on page 193
- *Application Guide*

Creating a new parameter for inclusion in parameter sets

These instructions describe how to create a new parameter for inclusion in parameter sets and accessible in rate routines in the PolicyCenter user interface. To create a new parameter, you must complete the following instructions.

Add the parameter to the parameter name typelist for rate routines

About this task

Create a new typekey for the parameter. In a parameter set, the parameter **Name** appears in the **Name** column on the **Parameter Sets > Parameters** tab in PolicyCenter.

Procedure

1. In Studio, open CalcRoutineParamName.ttx in **configuration > config > Metadata > Typelist**.
2. Click **typelist** at the top of the **Element** hierarchy.
3. Click the drop-down list next to  and choose **typecode**. This adds a typecode for the new parameter to this typelist. For example, you can add a PATest parameter defined as follows:

code	name	desc
patest	PATest	Personal Auto Test

What to do next

You can now move to the next step, "Enter the default type for the parameter" on page 193.

Enter the default type for the parameter

Before you begin

Before you begin, you must complete “Add the parameter to the parameter name typelist for rate routines” on page 192.

About this task

Enter the default type which appears in the **Type** column on the **Rating > Parameter Sets > Parameters** tab in PolicyCenter. If you do not specify a default type, the **Type** column is blank.

Procedure

1. In Studio, open `RatingParameterSetScreenHelper.gs` in the `gw.pcf.rating.flow` package.
2. In the `mapFromCodeToType` static variable, enter a mapping for the code of the new parameter.
Enter the code in capital letters prefixed by `CalcRoutineParamName.TC_`. Enter the default type as the fully qualified object type. For example:

```
CalcRoutineParamName.TC_PATEST -> "entity.PersonalAutoLine"
```

What to do next

Now you can move to the next step, “Include the parameter in a parameter set in PolicyCenter” on page 193.

Include the parameter in a parameter set in PolicyCenter

Before you begin

Before you begin, you must complete “Enter the default type for the parameter” on page 193.

Procedure

In the PolicyCenter user interface, include the parameter in one or more parameter sets.

When you add the parameter, you can override the default type. Keep in mind that the object the rating engine passes must be compatible with the type of object that rate routine expects. For instructions and more information, see the *Application Guide*.

Adding a new parameter to the rating engine

These instructions describe how to add a new parameter to the rating engine. When the rating engine runs a rate routine, the rating engine maps each parameter to an object or entity instance. The map is of type `java.util.Map`. The rating engine uses this map to pass run-time objects and entity instances to the method that executes the rate routine. This object or entity instance must be compatible with the type specified in the parameter set associated with the rate routine being executed. A compatible type is the same type or a subtype. If the object or entity instance is not compatible, then PolicyCenter throws an exception.

Note: For instructions on how to add a new parameter for use in PolicyCenter, see “Configuring new parameters in parameter sets” on page 191.

In these steps, you include the new parameter in Gosu code that executes each rate routine associated with this parameter set.

The rating engine code executes each rate routine by calling the `RateBook.executeCalcRoutine` method. One of the parameters to this method is a map of parameters that the rate routine uses. This map must be a superset of the parameter set in PolicyCenter. By definition, a superset contains at least all of a set.

Include the parameter in the rating engine code

About this task

For each rate routine associated with the new parameter's parameter set, you must add the new parameter to the map. The parameter map is in the rating engine class in Studio.

Procedure

1. In Studio, open the rating engine class for the line of business that uses this parameter. For example, open `PARatingEngine.gs` if this parameter is for the personal auto line. The `PARatingEngine` class is in the `gw.lob.pa.rating` package.
2. Modify the `executeCalcRoutine` method call for each rate routine associated with a parameter set that includes this parameter.
The following example assumes that you added a parameter to the **PIP NJ Parameter Set** parameter set. The rate routine with the `pa_pip_nj_basic_rr` code uses this parameter set.

The code that executes this rate routine is:

```
RateBook.executeCalcRoutine("pa_pip_nj_basic_rr",
                           :costData = data,
                           :worksheetContainer = data,
                           rateRoutineParameterMap)
```

The `rateRoutineParameterMap` argument is a map defined as:

```
var rateRoutineParameterMap : Map<CalcRoutineParamName, Object> =
    {TC_POLICYLINE ->_line,
     TC_VEHICLE    ->vehicle,
     TC_PAPIPNJ    ->cov}
```

In the map, the `Object` must be compatible with the type for the parameter set being used by the executing rate routine. You specify the type in the **Type** column on the **Parameter Sets > Parameters** tab in the PolicyCenter user interface.

In the map, the `Object` must be compatible with the type for the parameter set being used by the executing rate routine. You specify the type in the **Type** column on the **Parameter Sets → Parameters** tab in the PolicyCenter user interface.

3. In the `rateRoutineParameterMap` variable, add a map entry for your new parameter.
4. Start or restart PolicyCenter to view your changes.

Configuring new wrappers for parameter sets

You may have parameters sets that are essentially the same, but each line of business uses a different coverage, for example. Using *wrappers*, you can combine these parameters sets into one parameter set that uses a wrapper to select the coverage by line of business.

The wrapper calls Gosu code that selects a coverage based upon characteristic of the policy. You must first create the wrapper before you use it in a parameter set.

In Studio, create a new Gosu class that implements `gw.rating.flow.CoverageWrapper`. For an example, see `CPCoverageWrapper.gs` in the `gw.lob.cp.rating` package.

See also

- *Application Guide*

Example with Wrappers

In the base configuration, the **CP Coverage Parameter Set With Wrapped Coverages** parameter set (`CPCoverageWrapperParamSet`) uses a wrapper. The **Coverage** parameter uses the wrapper

`gw.lob.cp.rating.CPCoverageWrapper` to select either the **Business Personal Property Coverage** (CPBPPCov) or the **Building Coverage** (CPBldgCov). In Product Designer, you can view these coverages in the **Commercial Property Line**.

The **CP Coverage Premium Algorithm** rate routine (`cp_cov_premium_rr`) uses the **CP Coverage Parameter Set With Wrapped Coverages** parameter set. In this rate routine, the `Basis` is assigned the value of `Coverage.Limit`. The wrapper calculates the value of the limit based upon whether the coverage is **Business Personal Property Coverage** or **Building Coverage**.

Configuring rating worksheets

In the default configuration, rating worksheets are configured for the personal auto and commercial property lines of business. To configure rating worksheets for other lines of business, you must do the following:

- **Configure rating worksheets in the rate routine plugin** – For more information, see “Enabling rating worksheets in the rate routine plugin” on page 201.
- **Configure the user interface** – Implement the `getWorksheetRootNode` method in the policy-line-methods class for the line of business. This method controls how the worksheets are organized in the user interface. See `PAPolicyLineMethods.gs` class as an example. In this example, the code that traverses the cost version list is similar to the code that displays the costs in the **Policy Premium** tab of the **Quote** screen.

The user must have the **View rating worksheet** permission to view a rating worksheet. The code for this permission is `ratingworksheetview`.

See also

- *Application Guide*

Configuring extract and purge rating worksheets

Every time a user generates a rating worksheet, a copy of that rating worksheet gets stored in the PolicyCenter database. To improve PolicyCenter performance, PolicyCenter provides processes for extracting rating worksheet data and removing rating worksheet containers from the database. This is not an end user feature and is only accessible through **Server Tools**. The Extract Worksheets batch process extracts the rating worksheet data to files and marks worksheet containers for purging. The Purge Rating Worksheets batch process removes the worksheet containers from the database. You must enable this batch process.

This topic describes how to configure these two batch processes and the associated plugins.

See also

- *Application Guide*

Extract Rating Worksheets batch process

Code – extractworksheets

The Extract Rating Worksheets batch process extracts rating worksheet data from worksheet container (`WorksheetContainer`) objects to files in a specified directory on the batch server. The batch process also marks `WorksheetContainer` objects for purging.

In the base configuration, this batch process is not scheduled to run on a regular basis. You can run it manually or schedule it.

In the base configuration, the batch process extracts worksheet data to files to an `extracted_worksheets` directory off the file system root. Extract Rating Worksheets processes `WorksheetContainer` objects attached to a Policy on which all jobs are closed. Closed jobs have a non-null `Job.CloseDate`. The batch process does not process `WorksheetContainer` objects that have already been extracted.

In Studio, you can change the extract destination directory by modifying `WorksheetExtractPlugin.gwp`. Modify the value of the `WorksheetExtractDestination` parameter.

In Studio, you can modify the behavior of the batch process by modifying the worksheet extract plugin. For more information, see “Worksheet Extract plugin” on page 196.

Purge Rating Worksheet batch process

Code – purgeworksheets

The Purge Rating Worksheets batch process removes worksheet container (`WorksheetContainer`) objects that meet both of the following criteria:

- Are marked for purging
- That are on policies whose jobs have been closed more than a specified number of days (90 days in the base configuration).

In the base configuration, the Extract Rating Worksheets batch process marks the worksheets container objects for purging.

In the base configuration, this batch process is disabled. After you enable it, you can run it manually or schedule it.

A `WorksheetContainer` is marked for purging if its `CanPurge` flag is `true`.

The `RatingWorksheetContainerAgeForPurging` parameter in `config.xml` specifies the minimum number of days after a job is closed before the `WorksheetContainer` associated with its policy is eligible for purging.

In Studio, you can modify the behavior of the batch process by modifying the worksheet purge plugin.

See also

- “Worksheet Purge plugin” on page 197

Enable Purge Rating Worksheets batch process

About this task

In the base configuration, the Purge Rating Worksheets batch process is disabled. To enable this batch process, you must do the following in Studio:

Procedure

1. In `config.xml`, set the `PurgeWorksheetsEnabled` parameter to `true`.
2. In `scheduler-config.xml`, to enable the batch process to run on a regular schedule, uncomment the `ProcessSchedule` entry for `PurgeWorksheets` and modify the schedule to meet your needs.

Worksheet Extract plugin

The `WorksheetExtractPlugin` plugin interface identifies which rating worksheet to extract and extracts the worksheet data to files.

PolicyCenter includes a built-in implementation of this plugin interface. See `PCWorksheetExtractPlugin` in the `gw.plugin.purge.impl` package.

Identifying worksheets to extract

PolicyCenter calls the `buildWorksheetCandidatesQuery` method to determine which worksheets to extract.

In `PCWorksheetExtractPlugin`, this method identifies `WorksheetContainer` objects attached to a `Policy` on which all jobs are closed. Closed jobs have a non-null `Job.CloseDate`. The method ignores `WorksheetContainer` objects that have already been extracted.

Extracting worksheet data

PolicyCenter calls the `extractWorksheets` method to extract the worksheet data into a file and flag the worksheet container for purging.

In `PCWorksheetExtractPlugin`, the worksheet data is extracted to XML and compressed into Gzip format. The format of the file name is:

```
PolicyNumber_TermNumber_JobNumber_BranchPublicID.gz
```

Can purge extracted worksheets

PolicyCenter calls the `isExtracted` method. The method returns a value which indicates whether data has been extracted from a worksheet container.

In `PCWorksheetExtractPlugin`, this method retrieves the value of the `CanPurge` Boolean property on the worksheet container. The `CanPurge` flag is false by default. This flag is true if:

- Data has been extracted
- There is no data to extract

The Purge Worksheet Containers batch process uses the `CanPurge` flag.

Worksheet Purge plugin

The `WorksheetPurgePlugin` plugin interface configures how PolicyCenter purges rating worksheets.

PolicyCenter includes a built-in implementation of this plugin interface. See `PCWorksheetPurgePlugin` in the `gw.plugin.purge.impl` package.

The `buildWorksheetCandidatesQuery` method builds the list of worksheet containers that meet the requirements for the Worksheet Purge batch process to purge.

In `PCWorksheetPurgePlugin`, the `buildWorksheetCandidatesQuery` method finds worksheet containers that have the `CanPurge` property set to true, and whose jobs were completed more than `RatingWorksheetContainerAgeForPurging` days ago. The `RatingWorksheetContainerAgeForPurging` parameter is defined in `config.xml`.

PolicyCenter calls the `canPurgeWorksheetContainer` method to determine if the rating worksheet can be purged. In the base implementation, this method just returns the value of the `CanPurge` flag on the `WorksheetContainer`. You can include additional logic in this method.

Logging rate routine functions in a rating worksheet

In the default configuration, logging to the rating worksheet occurs automatically for each step in the rate routine.

You can add logging to the rating worksheet for a rate routine function by using methods in the `WorksheetLogger` class in the `gw.rating.worksheet` package.

In the default configuration, the `capPremiumByPercent` method in the `SharedRatingFunctions` class provides an example of how to use the `WorksheetLogger` methods. The `SharedRatingFunctions` class is in the `gw.rating.flow.util` package.

Note: In the default configuration, the personal auto and commercial property lines of business have logging for rating worksheets. For information about configuring rating worksheet for other lines of business, see “Configuring rating worksheets” on page 195.

See also

- “Adding a new rate routine function” on page 184

Configuring impact testing

This topic describes how to configure impact testing. You can enable impact testing for a line of business and modify the functionality of impact testing. In addition, there are batch processes associated with impact testing.

See also

- *Application Guide*

Impact testing warnings and recommendations

Impact testing is designed to work in a test environment on a copy of production data. Impact testing is accessible only when the server is in development or test mode. Because impact testing affects system performance and creates test policy periods that persist in the database, impact testing is not accessible in production mode.

When you run impact testing, impact testing creates test policy periods in the database. In the test environment, observe the following warnings and recommendations:

- The work of users can interfere with impact testing results. During impact testing, Guidewire recommends that only the single user performing impact testing be logged into PolicyCenter.
- Disable integrations to other systems. If you run impact testing with integrations enabled, the integration can send test data to these production systems. For example, disable the free-text search integration that uses the search engine Solr. The same applies to other integrations such as an integration with a billing system. To avoid unnecessary costs, disable integrations to systems that charge access fees.
- While running impact testing batch processes, do not make changes in PolicyCenter that impact rating such as doing a policy change, canceling, or changing rate books. For example, the baseline creation batch process or the test period quote batch process is running. During this time, quoting test periods fails if a user changes or cancels a policy which has a baseline.
- Do not include expired policies in the impact testing dataset. Specify an **In Force On** date to filter out expired policies.
- Stop the Job Expire batch process to prevent PolicyCenter from unexpectedly expiring policies during or between impact testing runs. To manage batch processes on a test server, press ALT + SHIFT + T to display the **Server Tools** page, then select **Batch Process Info** from the **Server Tools** tab.

Enabling impact testing for a line of business

In the default configuration, impact testing is enabled for the personal auto and commercial property lines of business. To enable impact testing for another line of business, you must implement a getter for the `Coverable` property in the cost adapter for that line of business. For more information, see “Cost adapter methods” on page 208.

Rate renewals as renewal jobs in impact testing

About this task

In the base configuration, impact testing rates the baseline and test policy periods for all job types as submission jobs. You can configure impacting testing to rate a renewal as a renewal job rather than a submission job. For example, rate routines may include special steps for renewal jobs. Rate tables may contain factors based on job type.

In the base configuration, PolicyCenter prevents a renewal on a policy period which has a open renewal. If you make this configuration change, PolicyCenter can create a renewal on a policy period which has an open renewal. Making this configuration change impacts how PolicyCenter handles renewals in impact testing and in other areas of the product.

To rate renewals and renewal jobs in impact testing, change the `IPolicyPlugin` in Studio.

Procedure

1. In Studio, open `IPolicyPlugin.gwp` in **configuration > config > Plugins > registry**.
2. Change **Gosu Class** from:

```
gw.plugin.policy.impl.PolicyPlugin
```

to:

```
gw.plugin.rateflow.ImpactTestingPolicyPlugin
```

The `ImpactTestingPolicyPlugin` plugin extends the `PolicyPlugin` plugin. The `ImpactTestingPolicyPlugin` plugin only contains an override of the `canStartRenewal` method.

IMPORTANT: The `ImpactTestingPolicyPlugin` plugin is only intended for use in a development environment where you are performing impact testing. Do not use this plugin in a production environment. If you enable the `ImpactTestingPolicyPlugin` plugin, PolicyCenter allows the creation of a renewal on a policy period with an open renewal. In the default configuration, PolicyCenter prevents a renewal on a policy period that has an open renewal. Enabling the `ImpactTestingPolicyPlugin` changes how PolicyCenter handles renewals in impact testing and in other areas of the product.

Modifying the functionality of impact testing

In the PolicyCenter default configuration, impact testing creates the test periods by creating policy renewals. You can customize how impact testing creates the test periods. For example, you may want to create test periods in another way, such as by copying submissions or spinning off policies. You can also change how to select alternate rate books in impact testing.

You can make these changes by writing your own implementation of the impact testing plugin interface.

[See also](#)

“Impact testing plugin” on page 204

Work queues for impact testing

In Guidewire Rating Management, impact testing uses the following work queues:

Work queue name	Code	Description
Impact Testing Test Case Preparation	<code>ImpactTestingTestPrep</code>	This work queue generates baseline policy periods on the selected policies when you click Create Baselines from the Create Baseline screen.
Impact Testing Test Case Run	<code>ImpactTestingTestRun</code>	This work queue generates test policy periods rated using the selected rate books when you click Quote Test Periods from the Testing Periods screen.
Impact Testing Export	<code>ImpactTestingExport</code>	This work queue exports test periods to an Excel file when you click Create Excel Export File on the Impact Results screen.

You can access impact testing in PolicyCenter by navigating to **Administration > Rating > Impact Testing**.

Configuring the impact testing plugin

You can use impact testing to see the impact that changing the rate book has on the policy premium.

The impact testing plugin (`IImpactTestingPlugin`) generates the baseline and test periods for impact testing. The plugin also gets and sets the rate books for quoting the test periods.

The built-in plugin implementation, `gw.plugin.rateflow.ImpactTestingPlugin.gs`, creates test periods by creating policy renewals. You can create your own implementation of this plugin. For example, you may want to create test periods in another way, such as by copying a submission or spinning off a policy. You can also change how the alternate rate books are selected.

For more information, see “Impact testing plugin” on page 204.

Configuring rate book export to spreadsheet and XML

On the Rate Book screen, you can export the rate book details and its included rate tables and rate routines to spreadsheet or XML format. There are several ways in which you can configure export:

- **Spreadsheet appearance** – You can configure the spreadsheet appearance by modifying `ExcelAutoSizer.gs` in the `gw.rating.rtm.excel` package.
- **Purge downloads** – Purge Rate Book Export Result batch processing removes rate books exported to spreadsheets and XML.

See also

- *Application Guide*
- *InsuranceSuite Database Administration*

Rating Management plugins and interfaces

Rate routine plugin

PolicyCenter provides the `IRateRoutinePlugin` to let you modify the processing of rate routines in PolicyCenter. The base configuration of PolicyCenter uses a built-in implementation of the plugin, `RateRoutinePlugin.gs`. You can create your own implementation of this plugin. To edit the registry for `IRateRoutinePlugin`, In Studio, open `IRateRoutinePlugin.gwp` in **configuration > config > Plugins > registry**.

When PolicyCenter executes a rate routine, PolicyCenter calls the rate routine plugin (`IRateRoutinePlugin`). The rate routine plugin:

- Enables rating worksheets in the plugin.
- Sets up cost data wrappers that determine which properties on the cost appear when you select an instruction or operand in a rate routine step.

See also

- “Configuring the rating engine to execute the rate routine” on page 183
- *Application Guide*

Rate routine plugin implementation

PolicyCenter includes a built-in implementation of the rate routine plugin. This implementation contains classes for the personal auto and commercial property lines of business. In Studio, the built-in plugin implementation is `RateRoutinePlugin.gs` in the `gw.plugin.rateflow` package. To customize the rate routine plugin logic, you can create your own implementation of this plugin.

The `RateRoutinePlugin.gs` passes the processing to line-of-business-specific configuration classes, if a class exists. The `RateRoutinePlugin.gs` looks for the line-of-business configuration classes in the `gw.lob` package. To find these configuration classes, the plugin maps the policy line code to the product abbreviation as follows:

Policy Line Code	Product Abbreviation	Configuration Class
PersonalAutoLine PA		<code>gw.lob.pa.rating.PARateRoutineConfig</code>
CPLine	CP	<code>gw.lob.cp.rating.CPRateRoutineConfig</code>

If the product abbreviation does not fit this mapping, the `makeAbbrevMap` and `makeConfigClassName` methods make adjustments to the mapping. In the `RateRoutinePlugin.gs`, these methods make an adjustment for the business auto line. If you need to make other adjustments, you must create your own version of the rate routine plugin.

The line-of-business configuration classes implement the `IRateRoutineConfig` interface.

Enabling rating worksheets in the rate routine plugin

The rate routine plugin has settings that enable the rate routine plugin to generate rating worksheets for a line of business. If rating worksheets are enabled for a line of business, the **Show Rating Worksheet** button appears on the **Quote** screen.

Enable rating worksheets in the plugin by making the following changes:

- Set the `WorksheetsEnabled` parameter to `true` in the `IRateRoutinePlugin` registry.
- In the `LOBRateRoutineConfig.gs` class, modify the `worksheetsEnabledForLine` override method to return `true`. For example, you can turn on rating worksheets for a line of business by modifying the line-of-business configuration classes with an override to the `worksheetsEnabledForLine` method. For example, the following code in the `CPRateRoutineConfig` class enables rating worksheets for commercial property:

```
override function worksheetsEnabledForLine(linePatternCode : String) : boolean {  
    return true  
}
```

In the `RateRoutinePlugin` class, the `worksheetsEnabledForLine` method calls the line-specific implementation of `worksheetsEnabledForLine`.

See also

- “Configuring rating worksheets” on page 195 for additional steps to enable rating worksheet
- *Application Guide*

Cost data in the rate routine plugin

The rate routine plugin contains methods that set up cost data wrappers for the rate routine to use. These wrappers are used to define which properties on the cost appear when you select an instruction or operand in a rate routine step. The plugin must implement the `getCostDataWrapperType` and `makeCostDataWrapper` methods. In the built-in implementation, the rate routine plugin simply transfers processing to line-of-business configuration classes, if they exist.

The `makeCostDataWrapper` method returns a cost data wrapper for the given cost data. The wrappers in the sample implementation are:

- `CalcRoutineCostData`
- `CalcRoutineCostDataWithAmountOverride`
- `CalcRoutineCostDataWithPremiumCap`
- `CalcRoutineCostDataWithTermOverride`

These wrappers are in the `gw.rating.flow.domain` package.

The `CostData` class is located in the `gw.rating` package. You can examine this class in Guidewire Studio to see the available properties. Descriptions for several of the properties appear in the source code. For most properties, the name is self-explanatory. Note that the properties that appear in the Structure pane in Guidewire Studio do not have the correct capitalization. Refer to the code for correct capitalization.

Rate routine plugin methods

The rate routine plugin implementation must contain these methods.

The built-in implementation of the rate routine plugin is `RateRoutinePlugin.gs` in the `gw.plugin.rateflow` package.

Set Parameters

The `setParameters` method gets the values of the `WorksheetsEnabled` parameter defined in the plugin registry and sets the values on internal variables. The value defined in the plugin registry overrides the plugin setting. The method signature is:

```
override function setParameters(params : Map<Object, Object>)
```

Get Cost Data Wrapper Type

The `getCostDataWrapperType` method gets the cost data wrapper type. The method signature is:

```
override function getCostDataWrapperType(paramSet : CalcRoutineParameterSet) : IType
```

In the built-in implementation, `RateRoutinePlugin` class transfers processing to the method of the same name in the line-specific rate configuration class, such as `PARateRoutineConfig`.

In the line-specific configuration class, the `getCostDataWrapperType` method gets the wrapper type that will be used by rate routines for the given parameter set. The rate routine editor accesses a `CostData` instance using this wrapper type. The rate routine editor uses the wrapper to display the desired set of properties related to `CostData` in virtual form. For example, when entering an **Instruction** or **Operand** in a rate routine step in PolicyCenter, you can select a property on the cost such as **AdjustedRate**. The wrapper implementation maps **AdjustedRate** to the correct underlying `CostData` property: `StandardAdjRate`, `OverrideAdjRate`, or `ActualAdjRate`. For example, if the `CostData` supports overrides, this could be the override or standard property. If the `CostData` does not support overrides, it is almost certainly the standard property.

The parameter is:

`paramSet` The parameter set for which to return the corresponding wrapper type

The method returns the `IType` corresponding to the instance that `makeCostDataWrapper` would return for the given `paramSet`.

Make Cost Data Wrapper

The `makeCostDataWrapper` method makes the cost data wrapper. The method signature is:

```
override function makeCostDataWrapper(
    paramSet : CalcRoutineParameterSet,
    costData : CostDataBase,
    defaultRoundingLevel : Integer,
    defaultRoundingMode : RoundingMode) : ICostDataWrapper
```

In the built-in implementation, `RateRoutinePlugin.gs` transfers processing to the method of the same name in the line-specific rate configuration class, such as `PARateRoutineConfig.gs`.

In the line-specific configuration class, the `makeCostDataWrapper` method makes a wrapper for the given `CostData` instance.

The parameters are:

<code>paramSet</code>	The parameter set for which to return the corresponding wrapper
<code>costData</code>	An instance of the <code>CostData</code> subclass
<code>defaultRoundingLevel</code>	The default rounding level of the <code>CostData</code>
<code>defaultRoundingMode</code>	The default rounding mode of the <code>CostData</code>

The method returns a wrapper instance of the `IType` that `getCostDataWrapperType` returns for the `paramSet`.

Enable Worksheets for a Policy Line

The `worksheetsEnabledForLine` method returns true if worksheets are enabled for a policy line. The method signature is:

```
override function worksheetsEnabledForLine(linePatternCode : String) : boolean
```

In the built-in implementation, `RateRoutinePlugin.gs` enables worksheets for a policy line if the `worksheetsEnabledForLine` method in the line-specific configuration class returns true.

The parameter is:

`linePatternCode` Pattern code for a line of business

The method returns true if worksheets are enabled for the line of business with the given pattern.

For more information, see “Add an implementation to a Plugins Registry item”.

Treat Type as Primitive

You can call the `treatTypeAsPrimitive` method to determine whether to treat a type as a primitive (atomic) or whether to expose its fields. When traversing the rating entity graph, PolicyCenter calls this method. If the type is primitive, stop traversal of that branch.

The parameter is:

`t` The type under consideration.

The method returns true to treat the supplied type as a primitive.

Get Branching Fields

The property getter for `BranchingFields` returns an array which is an ordered list of zero or more branching fields which represent variant identifiers of a rate routine. The fields together with `Code` and `Version` on the `CalcRoutineDefinition` entity uniquely identify a rate routine. A rate routine and its variant identifiers have the same code. The signature is:

```
override property get BranchingFields() : IEntityPropertyInfo[]
```

In the default configuration, jurisdiction is an example of a rate routine variant identifier. In the PolicyCenter user interface, the jurisdiction variant is referred to as the *jurisdiction variant*.

If you add a new variant identifier, you must add the name of the variant identifier to the `fields` variable of the getter for the `BranchingFields` property.

For more information about adding a variant identifier, see “Configuring variant identifiers for a rate routine” on page 186.

Include Property

The `includeProperty` method is a filter on which properties on **Parameters** the user can select in an **Instruction** or **Operand** of a rate routine step. Return true to make a property available. Return false to make a property unavailable. Return null to retain the default value of this property. It is recommended that this method end with a catch-all return null, so that any property which is not explicitly handled gets default behavior.

The signature is:

```
override function includeProperty(policyLinePatternCode : String, prop : IEntityPropertyInfo) : Boolean
```

The parameters are:

`policyLinePatternCode` The code for the policy line.

`prop` The descriptor for the property to include.

This method operates during the traversal of the policy graph, and controls whether a reference from one class to another will be traversed. (In contrast, the `filterIrrelevantItems` method runs after the traversal of the policy graph.)

For example, `PolicyLocation` has a foreign key to an `AccountLocation`. In the built-in implementation, `PolicyLocation.AccountLocation` is not traversed. To change this behavior, you might add the condition:

```
if (prop.OwnersType.isAssignableFrom(entity.PolicyLocation) and prop.Name == "AccountLocation") {
    return true
}
```

Filter Irrelevant Items

Similar to the `includeProperty` method, the `filterIrrelevantItems` method is a filter on which properties on **Parameters** the user can select in an **Instruction** or **Operand** of a rate routine step. However, this method runs after the traversal of the policy graph.

This filter runs after the traversal of the policy graph.

The signature is:

```
function filterIrrelevantItems(input : List<InScopeUsageBase>, policyLinePatternCode : String) :
    List<InScopeUsageBase>
```

In the built-in implementation, the `RateRoutinePlugin` class transfers processing to the method of the same name in the line-specific rate configuration class, such as `PARateRoutineConfig`. The built-in implementation calls the `ItemFilter` class which performs the default filtering. You can modify this class and add subclass helpers for specific implementations.

In the built-in implementation, `ItemFilter` considers the following types as irrelevant items:

- Ignored paths – List of paths to filter out (defined in the `_ignoredPaths` variable). For example, the built-in implementation filters out `.EffectiveDate` and `.ExpirationDate`. See the plugin implementation for the complete list.
- Ignored types – List of object types to filter out (defined in the `_ignoredTypes` variable). For example, the built-in implementation filters out policy period and policy line pattern. See the plugin implementation for the complete list.
- Arrays or maps of types. You can optionally filter by policy line pattern code.

Impact testing plugin

You can use impact testing to see the impact that changing the rate book has on the policy premium.

The impact testing plugin (`IImpactTestingPlugin`) generates the baseline and test periods for impact testing. The plugin also gets and sets the rate books for quoting the test periods.

PolicyCenter provides the `IImpactTestingPlugin` plugin interface so that you can customize impact testing logic in PolicyCenter. The default configuration of PolicyCenter uses a built-in implementation of the plugin, `ImpactTestingPlugin.gs`. You can implement your own instance of this plugin by copying and customizing this plugin implementation.

To edit the plugin registry for `ImpactTestingPlugin` in PolicyCenter Studio, open `IImpactTestingPlugin.gwp` in **configuration > config > Plugins > registry**.

See also

- “Configuring impact testing” on page 197
- *Application Guide*

Impact testing plugin implementation

In Studio, the built-in implementation of this plugin is the `ImpactTestingPlugin.gs` class in the `gw.plugin.rateflow` package.

This plugin creates test periods by creating policy renewals. You can customize the implementation of this plugin directly. For example, you may want to create test periods in another way, such as by copying a submission or spinning off a policy. You can also change how the alternate rate books are selected.

Impact testing plugin methods

The impact testing plugin must implement these methods.

Create Baseline Policy Period

The `createBaselinePeriod` method creates a baseline policy period from the original bound policy period. The baseline policy period is rated using the selected rate books. For more information on rate book selection, see the *Application Guide*.

Impact testing compares the base policy period with the test policy period. The method signature is:

```
public PolicyPeriod createBaselinePeriod(PolicyPeriod original)
```

The parameter is:

Parameter	Description
<code>original</code>	The original bound policy period from which to create a baseline policy period.

This method returns the baseline policy period.

Create Test Policy Period

The `createTestPeriod` method creates a test policy period from the original bound policy period. The test policy period is rated using the alternate rate books. The method signature is:

```
public PolicyPeriod createTestPeriod(PolicyPeriod original, PolicyPeriod baseline)
```

The parameters are:

Parameter	Description
<code>original</code>	The original bound policy period from which the baseline policy period was created.
<code>baseline</code>	The baseline policy period created by <code>createBaselinePeriod</code> .

This method can create the test policy period using either the original or baseline policy period, or a combination of both.

This method returns the test policy period.

Request a Quote on the Baseline Policy Period

The `requestBaselineQuote` method request a quote on the baseline policy period. The quote rates the policy using the default rate books. The method signature is:

```
public void requestBaselineQuote(PolicyPeriod period)
```

The parameter is:

Parameter	Description
period	The baseline policy period

This method has no return value.

Request a Quote on the Test Policy Period

The `requestTestQuote` method requests a quote on the test policy period. The quote rates the policy using the alternate rate books specified in “Impact testing plugin methods” on page 205. The method signature is:

```
public void requestTestQuote(PolicyPeriod period)
```

The parameter is:

Parameter	Description
period	The test policy period.

This method has no return value.

Flag to Include the Policy Period

The `shouldIncludePeriod` flag indicates whether to include or exclude the policy period in impact testing. The method signature is:

```
public boolean shouldIncludePeriod(PolicyPeriod period)
```

The parameter is:

Parameter	Description
period	The policy period to be considered for inclusion or exclusion

If the return value is `true`, then include the policy period in the test policy periods.

Get Alternate Rate Books

The getter for the `AlternateRatebooks` property returns the rate books selected for rating the test policy periods. In the **Impact Testing, Select Rate Books** screen in PolicyCenter, the alternate rate books appear under **Rate Books for Impact Testing**. The getter signature is:

```
property get AlternateRatebooks() : List<ImpactTestingRateBook>
```

This getter returns a list of alternate rate books.

Set Alternate Rate Books

The setter for the `AlternateRatebooks` property sets the alternate rate books for rating the test policy periods. You specify the alternate rate books in the `books` parameter. When querying for rate books, impact testing uses the alternate rate books instead of the default rate books. The setter signature is:

```
property set AlternateRatebooks(books : List<ImpactTestingRateBook>)
```

The parameter is:

Parameter	Description
books	A list of alternate rate books. Pass <code>null</code> to not override rate books.

This method has no return value.

Rate query lookup plugin

PolicyCenter provides the `IRateQueryLookupPlugin` plugin interface so that you can customize the rate query fail-over logic in PolicyCenter. The default configuration of PolicyCenter uses a built-in implementation of the plugin, `RateQueryWithFailoverLookupPlugin.gs`. You can implement your own instance of this plugin by copying and customizing this plugin implementation.

To edit the plugin registry for `IRateQueryLookupPlugin` in PolicyCenter Studio, open `IRateQueryLookupPlugin.gwp` in **configuration > config > Plugins > registry**.

You can use the rate query lookup plugin to implement rate table lookup fail-over. For most implementations, fail-over is not necessary. When a rate table lookup fails, it usually indicates that the table has a missing value that needs to be added to the table. With fail-over when the lookup fails, the plugin provides an alternate return value.

You can use this plugin to lookup values in a fail-over rate book. Extend the rate book entity to point to the fail-over rate book. For example, if you change rate table codes between the old and new editions of the rate book, you can extend the rate book to include a mapping table. Use the mapping table to fail-over to alternate rate tables.

You can also use this plugin to implementing capping. Capping uses the current renewal policy data and uses the prior policy period's rate book to determine the rate factor, or capping basis. Tables in the old edition of the rate book may be missing values for the renewal policy. For example, the old rate table does not contain values for new or changed coverages. With the plugin, lookup can fail-over to the table in the new edition of the rate book.

Rate query lookup plugin implementation

In Studio, the built-in implementation of this plugin is the `RateQueryWithFailoverLookupPlugin.gs` class in the `gw.plugin.rateflow` package.

Rate query lookup plugin methods

The rate query lookup plugin must implement these methods.

Preempt Query with Fail-over

The `shouldPreemptQueryWithFailover` method returns `true` to signal using fail-over if lookup fails.

Process Single Factor Query Fail-over

The `processSingleFactorQueryFailover` method handles fail-over for a single factor rate table.

If you are not using cascaded lookup, pass the new book in a `ThreadLocal` variable. If you are using cascaded lookup, pass the `PolicyPeriod` in the `ThreadLocal` variable to lookup the rate book hierarchy using `RateBook.selectRateBook`.

Process Multiple Factor Query Fail-over

The `processMultiFactorQueryFailover` method handles fail-over for a rate table with multiple factors.

If you are not using cascaded lookup, pass the new book in a `ThreadLocal` variable. If you are using cascaded lookup, pass the `PolicyPeriod` in the `ThreadLocal` variable to lookup the rate book hierarchy using `RateBook.selectRateBook`.

Adding line-specific cost methods using cost adapters

Each line of business defines its own root cost entity. Every root Cost entity must implement the `CostAdapter` interface and delegate it to a separate class that you create. For example, the root cost entity for the personal auto line is `PACost`, and the cost adapter is `PACostAdapter.gs` in the `gw.lob.ba.financials` package.

The cost adapter interface contains methods related to creating transactions and reinsurance. In addition, the interface contains methods related to rating worksheets and impact testing in Guidewire Rating Management.

See also

- *Application Guide*

Cost adapter methods

The classes that implement the cost adapter interface must implement these methods.

Create Transactions

The `createTransaction` method creates a `Transaction` object in the given policy period branch that is appropriate for the `Cost` type. The method signature is:

```
Transaction createTransaction( PolicyPeriod branch )
```

Get Reinsurable

The `getReinsurable` method gets the single reinsurable that is associated with this cost. The method signature is:

```
Reinsurable getReinsurable()
```

This method returns `null` if the cost does not represent a premium. For example, a cost for taxes or fees does not represent a premium.

Get Coverable

Implement a getter for the `Coverable` property. The getter returns a coverable if there is a coverable associated with this cost. Impact testing uses this getter. The signature is:

```
override property get Coverable()
```

This getter returns the coverable associated with this cost or `null` if this cost is not associated with a coverable. This method can return `null` if impact testing is not enabled for this line of business.

Get Name of Coverable

Implement a getter for the `NameOfCoverable` property.

The signature is:

```
override property get NameOfCoverable()
```

The getter returns the name of the coverable if there is a coverable associated with this cost. The name is usually, but not always, equivalent to the display name of the coverable. An exception is in the Personal Auto line. All premium costs are computed with respect to a vehicle. The `getNameOfCoverable` returns the `DisplayName` of the vehicle even where `getCoverable` returns the Personal Auto Line instance.

Is Matching Bean

For reporting differences, returns `true` if the two beans are a match. The signature is:

```
override function isMatchingBean(bean : KeyableBean)
```

The `bean` parameter is the other bean to compare with this one.

Get Policy Line

Implement a getter that returns the `PolicyLine` associated with this cost. The signature is:

```
override property get PolicyLine()
```


Configuring Reinsurance Management

This topic describes how to configure Reinsurance Management in PolicyCenter.

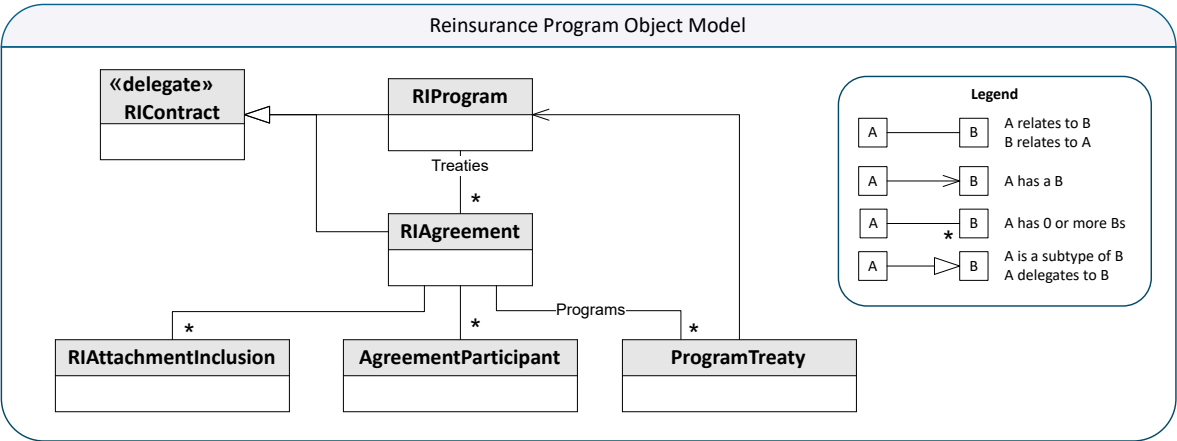
See also

- *Plugins, Prebuilt Integrations, and SOAP APIs*

Reinsurance Management object model

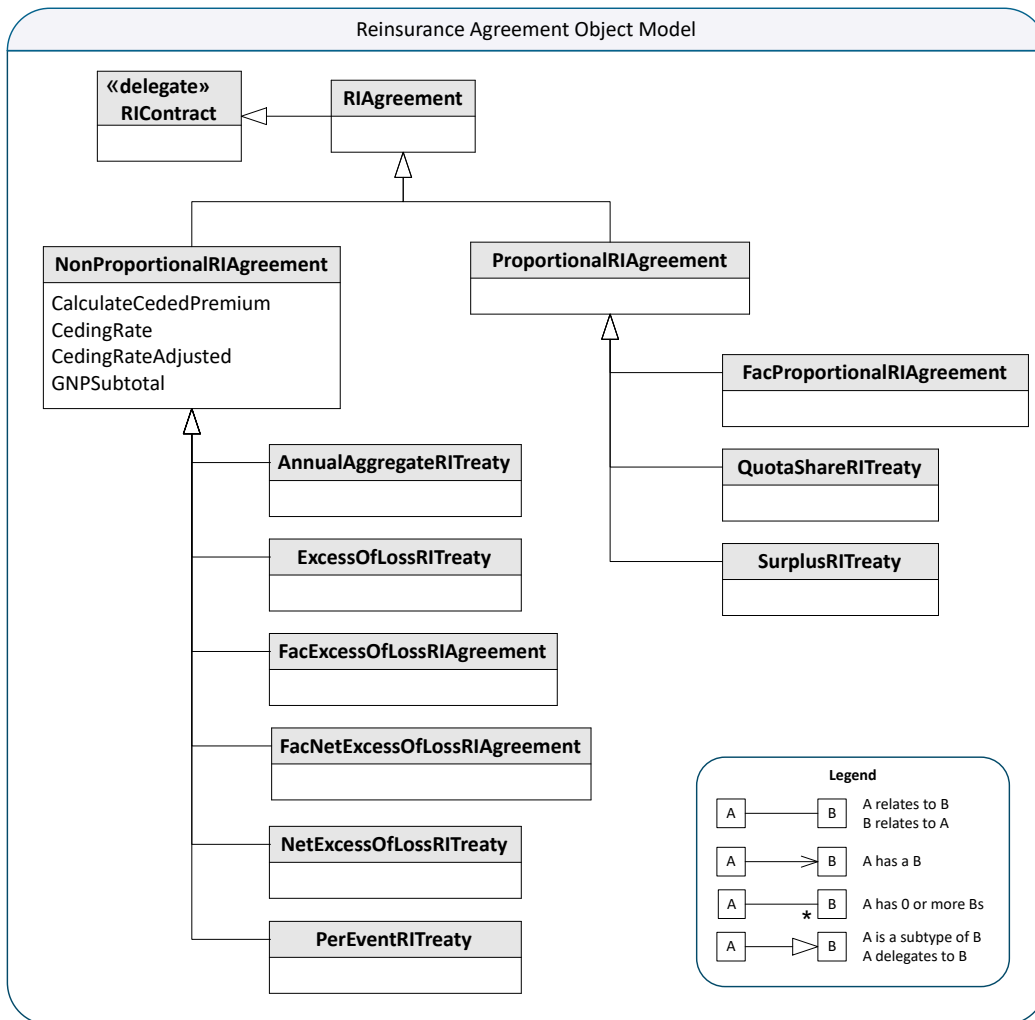
Reinsurance program object model

Reinsurance programs are represented by the RIProgram entity. Reinsurance programs contain one or more treaties. As shown in the following object model diagram, each reinsurance agreement is represented by the RIAgreement entity. A reinsurance agreement can be associated with one or more programs. You can access the programs that an RIAgreement is associated with through the Programs array. Each ProgramTreaty entity has a foreign key to an RIProgram.

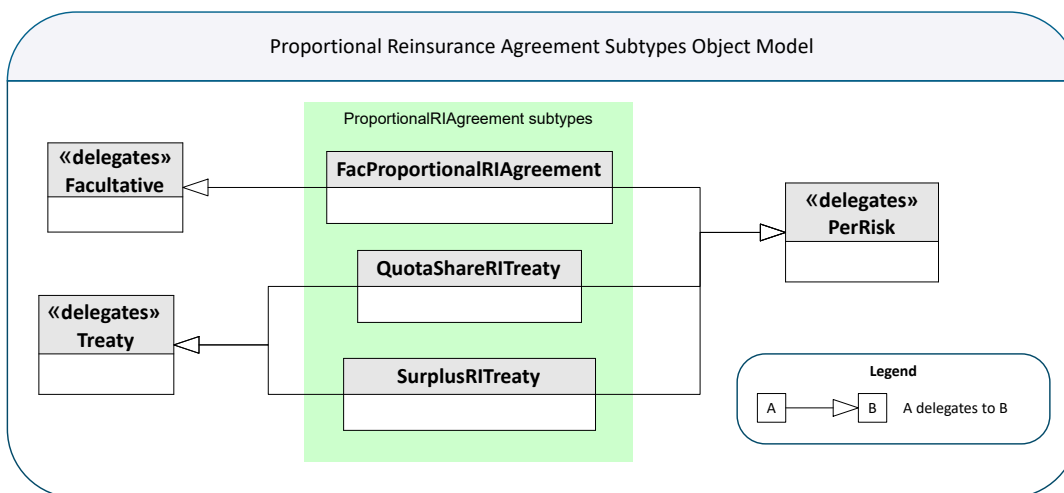


Reinsurance agreement object model

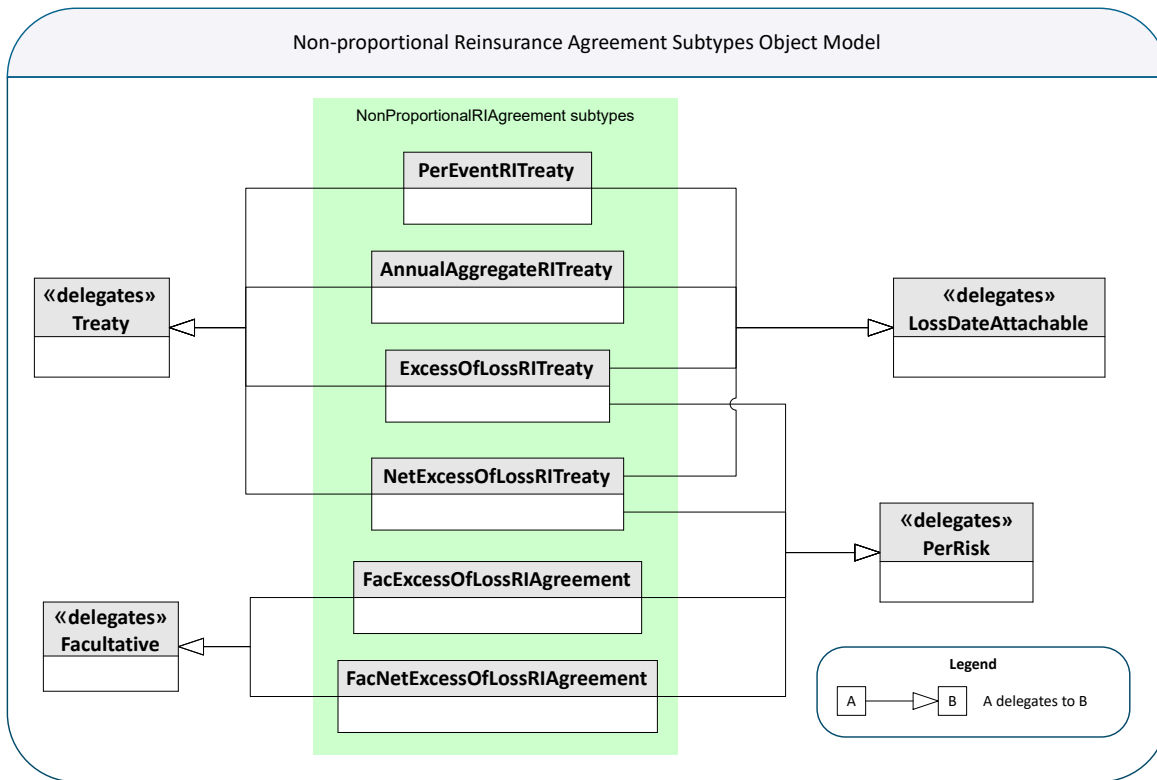
Reinsurance agreements are represented by the RIAgreement entity. The following illustration shows the subtypes of RIAgreement. Reinsurance agreements subtypes are proportional or non-proportional.



The following illustration shows the proportional reinsurance entities and their delegates.



The following illustration shows the non-proportional reinsurance entities and their delegates.



Treaty entities

All entities that represent treaties delegate to the Treaty entity. In the default configuration, the treaty entities are:

Proportional	Non-proportional
QuotaShareRITreaty	PerEventRITreaty
SurplusRITreaty	AnnualAggregateRITreaty
	ExcessOfLossRITreaty
	NetExcessOfLossRITreaty

Facultative agreement entities

All entities that represent facultative agreements delegate to the Facultative entity. In the default configuration, the facultative agreement entities are:

Proportional	Non-proportional
FacProportionalRIAgreement	FacExcessOfLossRIAgreement
	FacNetExcessOfLossRIAgreement

Loss date attachment agreement entities

All entities that represent loss date attachment agreements delegate to the LossDateAttachable entity. In the default configuration, the loss date attachment agreement entities are:

Non-proportional
PerEventRITreaty

Non-proportional

AnnualAggregateRITreaty

ExcessOfLossRITreaty

NetExcessOfLossRITreaty

Per risk entities

All entities that represent per risk agreements delegate to the PerRisk entity. In the default configuration, the per risk agreement entities are:

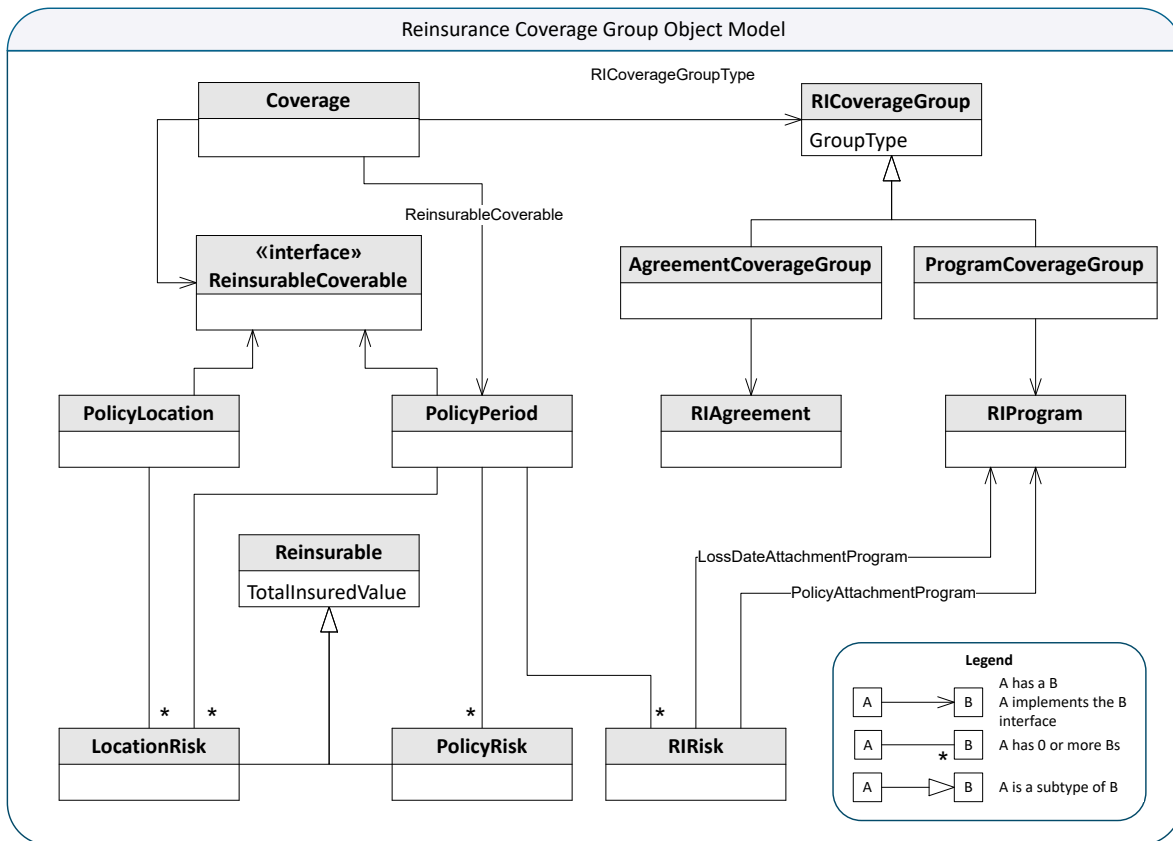
Proportional	Non-proportional
FacProportionalRIAgreement	ExcessOfLossRITreaty
QuotaShareRITreaty	NetExcessOfLossRITreaty
SurplusRITreaty	FacExcessOfLossRIAgreement
	FacNetExcessOfLossRIAgreement

Reinsurance coverage group object model

The Coverage entity has two fields related to reinsurance. The RICoverageGroup field returns the RICoverageGroupType of the reinsurable. In the default configuration, RICoverageGroupType is a typelist with the following values:

- Property
- Liability
- AutoPD
- AutoLiability
- WorkersComp

The RICoverageGroupType field groups coverages of the same reinsurance coverage group into reinsurable risks. Two coverages of different reinsurance coverage groups cannot be grouped into the same reinsurable risk. This field is also used in reinsurance programs and reinsurance agreements to specify which reinsurance coverage groups these programs and agreements apply to. The following object model diagram shows some of these relationships.

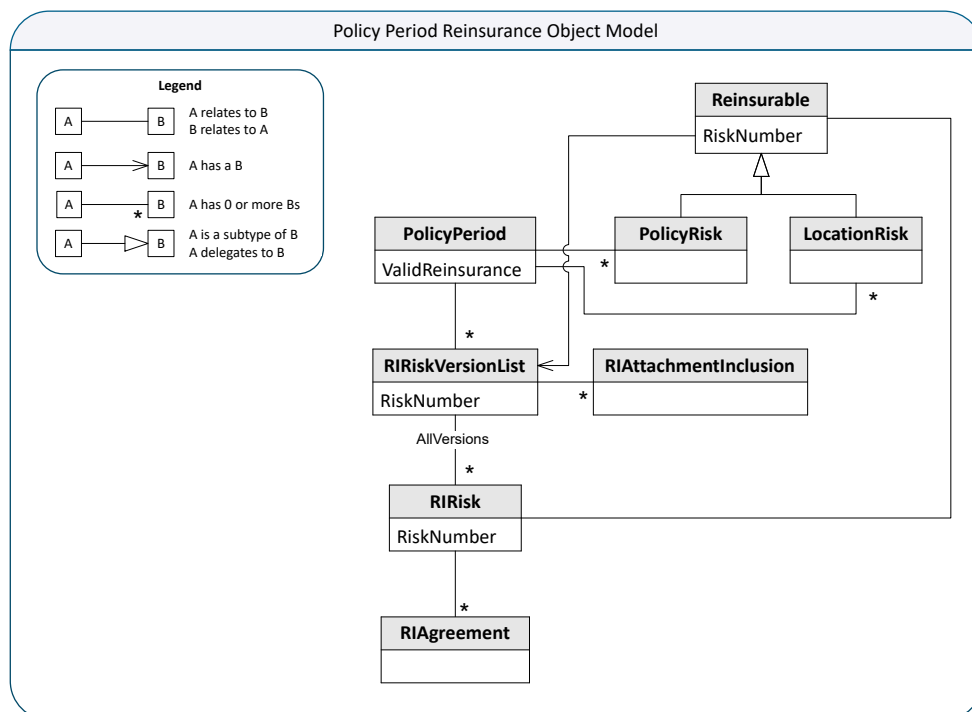


Coverages of same `RICoverageGroup` are not all grouped into a same reinsurable risk, but are divided further by what reinsurable coverable they apply to. In the default configuration, the reinsurable coverable entities are `PolicyLocation` and `PolicyPeriod`. The reinsurable risks associated with these coverables are `LocationRisk` and `PolicyRisk`, respectively. For example, if all property coverages for all buildings at the same policy location are grouped into a single reinsurable risk, the reinsurable coverable is the policy location.

Reinsurable coverables implement the `ReinsurableCoverable` interface. One `ReinsurableCoverable` can create multiple reinsurable risks depending upon how many `RICoverageGroup` entities the coverages under that `ReinsurableCoverable` have.

Policy period reinsurance object model

The following object model diagram shows some of the reinsurance entities associated with jobs and policy periods.



The RiskNumber field is used to uniquely identify a risk in various entities in the object model. For example, the RIRisk and PolicyRisk entities have the same risk if they have the same RiskNumber.

The RIRisk entity delegates to the SimpleEffDated delegate which has EffectiveDate and ExpirationDate fields. Each instance of this entity represent a reinsurance risk for a period of time when it unchanged. The RIRisk represents a version of the risk.

The RIRiskVersionList entity is the container for all versions of a RIRisk created by a policy period. The version list has a link back to the policy period which created it.

Reinsurance Management permissions

The following table lists the permissions related to Reinsurance Management:

Permission	Code	Description
Edit active RI programs	editreinsuranceactiveprogram	Permission to edit active reinsurance programs
Edit fac agreements	editreinsurancefacagreement	Permission to edit facultative agreements
Edit RI policies advanced	editreinsuranceforpolicyadvanced	Permission to edit advanced reinsurance fields for policies
Edit RI policies basic	editreinsuranceforpolicybasic	Permission to edit basic reinsurance fields for policies
Edit RI programs	editreinsuranceprogram	Permission to edit reinsurance programs
View RI policies	viewreinsuranceforpolicy	Permission to view reinsurance for policies
View RI programs	viewreinsuranceprogram	Permission to view reinsurance programs

Configuring reinsurance coverage groups

Configure reinsurance coverage group on individual coverages

About this task

For each coverage, you can configure whether the coverage is part of a reinsurance coverage group.

Procedure

1. In Product Designer, open a policy line and go to the **Coverages** page.
2. Select a coverage.
3. Go to the **Reinsurance** page for that coverage.
4. Select a **Reinsurance Coverage Group** from the drop-down list. You can also add a **Reinsurance Coverage Group Script**.
Selecting a **Reinsurance Coverage Group** sets the `RICoverageGroup` on the Coverage entity.

Disable Reinsurance Management for a line of business

About this task

You can disable Reinsurance Management for a line of business.

Procedure

In Product Designer, open the policy line and make the following changes to each coverage on the **Coverages > Reinsurance** page:

- Set **Reinsurance Coverage Group** to **<none selected>**.
- Remove any **Reinsurance Coverage Group Script**.

If no coverages are linked to a coverage group, then no reinsurable risk will be created. Therefore, there will be no checking for reinsurance or linking to reinsurance agreements.

Reinsurance net retention underwriting issue type

Reinsurance Management has one underwriting issue type.

Reinsurance Management raises an `RINetRetention` underwriting issue if a policy period has reinsurable risks with insufficient reinsurance. PolicyCenter checks for this issue at quote release, bind, and issuance.

PolicyCenter raises an underwriting issue if the net retention is greater than the **Target Max Retention** in the **Reinsurance** screen of a policy file. In the base configuration, the `DefaultUnderwriterEvaluator.gs` class contains code to raise this underwriting issue.

If you integrate with an external reinsurance management system and have defined your own reinsurance data model, you will need to rewrite the Gosu code for raising this underwriting issue.

See also

- The *Application Guide* for more information about the Reinsurance checking set.

Gosu methods for calculating the total insured value

Each line of business calculates the total insured value.

In the default configuration, the `calculateTotalInsuredValue` method in `gw.api.policy.AbstractPolicyLineMethodsImpl` calculates the total insured value. For each coverage, this method

calls the `getTIVForCoverage` method. Each line of business overrides the `getTIVForCoverage` method. For an example, see the `getTIVForCoverage` method in `gw.lob.ba.BAPolicyLineMethods.gs`.

If you implement a new policy line and do not use `AbstractPolicyLineMethodsImpl`, you must implement a `calculateTotalInsuredValue` method.

If the new policy line inherits from `AbstractPolicyLineMethodsImpl`, the policy line needs to implement the `getTIVForCoverage` method.

Configuring monetary data

This topic identifies the features used to configure monetary data.

Monetary data in PolicyCenter

Monetary data refers to any database column, object field, or user interface control that contains a financial value.

Unlike other types of numeric data, monetary data has special requirements. Insurers need to control how monetary data is stored in the database, how it is shown in the user interface, and how values are rounded.

Some insurers may also need to work with monetary data in different currencies. These insurers can enable the multicurrency features in PolicyCenter. They may also need to specify currency-specific storage and display settings.

Configuring PolicyCenter for a single currency

The following table identifies the issues to consider when you are configuring an implementation of PolicyCenter that will use a single currency.

Issue	See also
Determine the number of digits that will be stored in database columns that record monetary amounts.	"Configuring monetary data storage in the database" on page 219
Identify the currency that the implementation supports.	"Specifying the supported currencies" on page 220 "The default currency" on page 221
Optionally override the display properties specific to monetary data.	"Display properties for monetary data" on page 222

If you integrate two or more core applications in Guidewire InsuranceSuite, set the values of the following parameters in the following files to be the same in every application:

- datatypes.xml
 - precision
 - scale
- config.xml
 - DefaultApplicationCurrency

Configuring PolicyCenter for multicurrency

The following table identifies the issues to consider when you are configuring an implementation of PolicyCenter that will use multicurrency.

Issue	See also
Determine the number of digits that will be stored in database columns that record monetary amounts.	"Configuring monetary data storage in the database" on page 219
Identify the currencies that the implementation supports.	"Specifying the supported currencies" on page 220 "The default currency" on page 221
For each currency, optionally override the display properties and scale specific to that currency.	"Display properties for monetary data" on page 222
Enable multicurrency.	• <i>BillingCenter Configuration</i> (BillingCenter) • "Enabling multicurrency" (ClaimCenter) • <i>PolicyCenter Configuration</i> (PolicyCenter)
Associate currencies with territories as needed.	"Configuring currencies by territorial data" on page 225

If you are integrate two or more core applications in Guidewire InsuranceSuite, set the values of of the following parameters in the following files to be the same in every application:

- `datatypes.xml`
 - `precision`
 - `scale`
- `config.xml`
 - `DefaultApplicationCurrency`
 - `MulticurrencyDisplayMode`

Monetary data resources

The following table summarizes all resources related to monetary data configuration.

Guidewire Studio™ resource	Usage	Location
The <code>Currency.example.ttx</code> file	Identifies the currency or currencies supported in the implementation.	configuration > config > Extensions > Typelist
<code>config.xml</code>	Includes application parameters that affect how monetary data is handled: • <code>DefaultApplicationCurrency</code> • <code>DefaultRoundingMode</code> • <code>MulticurrencyDisplayMode</code>	configuration > config
<code>datatypes.xml</code>	Identifies how monetary data is stored in the database, specifically regarding: • <code>precision</code> • <code>scale</code>	configuration > config > fieldvalidators
The <code>localization_LocaleCode.xml</code> files	Used in single currency implementations to: • Override ICU library settings of the display values for monetary data	configuration > config > Localizations

Guidewire Studio™ resource	Usage	Location
The currency.xml files	Used in multicurrency implementations to: • Override ICU library settings of the display values for monetary data • Specify currency-specific storage scales	configuration > config > currencies > (locale) > currency.xml

Configuring monetary data storage in the database

BillingCenter has different money data types based on whether the money stored is negative, positive or zero. You can configure the precision and scale of the monetary data types.

Money data types

Monetary amounts, such as \$12 US Dollars, have two components: a currency (such as US Dollars) and a value (such as 12). Every monetary amount must have both components in order to be valid.

To simplify working with monetary amounts, the application uses the data types list below. Each data type has two components: a currency component and an amount component. Whenever an entity field uses one of these data types, the field is mapped to two columns in the corresponding database table: a currency column and a value column.

Money data type	Description
monetaryamount	Permits positive, negative, and zero values.
nonnegativemonetaryamount	Permits positive and zero values only.
positivemonetaryamount	Permits positive values only.

IMPORTANT: Do not use the currencyamount data type in your data model. Do not use the corresponding CurrencyAmount Gosu or Java class. Use the monetaryamount data type instead.

Precision and scale

The number of digits stored in database columns with a monetaryamount data type is controlled by two settings: precision and scale. *Precision* is the setting that controls the total number of digits of the value stored. *Scale* is the setting that controls the number of digits to the right of the decimal symbol. For example, if precision is set to 5 and scale is set to 2, then the maximum value that can be stored for a monetaryamount is 999.99 and the minimum value is -999.99.

Guidewire recommends that you set the scale to the maximum number of digits that you would possibly need in the future. Setting scale to an initially large value makes it easier to add currencies that require more digits than the currencies supported by your initial configuration.

The scale attribute also specifies the default number of digits that are shown in the user interface after the decimal symbol. When multicurrency is enabled, you can provide currency-specific values that override this default.

See also

- *Configuration Guide*

Set the precision and scale

About this task

IMPORTANT: When you integrate the core applications in Guidewire InsuranceSuite, Guidewire recommends setting the precision and scale to be the same in each application.

Procedure

1. In Guidewire Studio™, open the `datatypes.xml` file either by:
 - Searching for it with `Ctrl+Shift+N`.
 - Navigating in the Project window to **configuration** > **config** > **fieldvalidators** and double-clicking `datatypes.xml`.
2. Search for the `<MoneyDataType>` element.
3. Set the precision and scale attributes as needed.
 - You do not need to modify the `validator` attribute.

Specifying the supported currencies

The Currency typelist identifies the currency or currencies that are supported in a given instance of the application. The typelist must define at least one currency.

The base configuration defines the following currencies. All of these currencies are defined in the `Currency.example.ttx` typelist extension file. They are defined in an extension file so that an insurer can remove any currencies that will not be used in a new implementation.

- AUD – Australian dollar
- CAD – Canadian dollar
- EUR – European Union euro
- GBP – British pound
- JPY – Japanese yen
- RUB – Russian ruble
- USD – U.S. dollar

Note: Most typelist extension files are named *TypelistToExtend.ttx*. However, the file that extends the Currency typelist is named `Currency.example.ttx` (and not `Currency.ttx`). This has been done to simplify upgrades from previous versions of the application where the base configuration declared currencies in example files.

During the implementation, you add, remove, or retire currencies to match the currencies the instance will support.

Installation type	Action
New single currency installation	If necessary, add the currency to be supported to <code>Currency.example.ttx</code> . Remove all other currencies from <code>Currency.example.ttx</code> .
New multiple currency installation	If necessary, add the currencies to be supported to <code>Currency.example.ttx</code> . Remove all other currencies from <code>Currency.example.ttx</code> .
Upgrading previous installation	If necessary, add the currency to be supported to <code>Currency.example.ttx</code> . Retire all other currencies from <code>Currency.example.ttx</code> .

Add a currency

About this task

Procedure

1. In Guidewire Studio™, open the `Currency.example.ttx` file, either by:

- Searching for it with **Ctrl+Shift+N**.
 - Navigating in the Project window to **configuration > config > Extensions > Typelist** and double-clicking `Currency.example.ttx`.
2. In the toolbar, click the Add icon . Studio adds a row for the typecode and selects it automatically.
 3. Enter the following properties for the new currency.
 - **code** - The lowercase version of the three-letter ISO 4217 currency code. For example, `nzd`.
 - **name** - The uppercase version of the three-letter ISO 4217 currency code. For example, `NZD`.
 - **desc** - The country or jurisdiction that issues the currency, followed by the name of the currency. For example, `New Zealand Dollar`.

What to do next

In single currency mode, currency symbols are specified in the `localization_LocaleCode.xml` files. If you add a new currency, add a new `localization_LocaleCode.xml` file to correctly display the currency symbol. For more information, see “Display properties for monetary data” on page 222.

In multicurrency mode, currency symbols come from the ICU Library. There is typically no additional work needed to configure the currency symbol for a new currency.

Remove a currency

About this task

Procedure

1. In Guidewire Studio™, open the `Currency.example.ttx` file, either by:
 - Searching for it with **Ctrl+Shift+N**.
 - Navigating in the Project window to **configuration > config > Extensions > Typelist** and double-clicking `Currency.example.ttx`.
2. Click the currency to be removed, and then click **Remove** .

Retire a currency

About this task

Procedure

1. In Guidewire Studio™, open the `Currency.example.ttx` file, either by:
 - Searching for it with **Ctrl+Shift+N**.
 - Navigating in the Project window to **configuration > config > Extensions > Typelist** and double-clicking `Currency.example.ttx`.
2. Select the **Retired** cell of the currency to be retired.
3. Set the cell value to **true**.

The default currency

Every instance of the application must specify a default currency in `config.xml`. This is true for both single currency instances and multicurrency instances.

- In ClaimCenter, columns of type `currencyamount` without a `currencyProperty <columnParam>` are in the default application currency.

In the base configuration, the default currency is set to U.S. dollars.

Specify the default currency

About this task

In the base configuration, the default currency is set to `usd`.

IMPORTANT: The `DefaultApplicationCurrency` configuration parameter setting is permanent. Once you have started the application, you must not change it. If you do change the value, subsequent attempts to start the server will fail.

IMPORTANT: When you integrate the core applications in Guidewire InsuranceSuite (such as ClaimCenter and PolicyCenter), you must set the default application currency to be the same in each application.

Procedure

1. In Guidewire Studio™, open the `config.xml` file either by:
 - Searching for it with `Ctrl+Shift+N`.
 - Navigating in the Project window to **configuration** and double-clicking `config.xml`.
2. Search for this parameter: `<param name="DefaultApplicationCurrency" value="usd"/>`.
3. Set the `value` attribute to the typecode of the desired currency.

Display properties for monetary data

The ICU Library

The *International Components for Unicode (ICU)* is an open source set of libraries that provide support for software localization. One of the ICU libraries provides industry standards for the display of monetary amounts in different locales and currencies.

In general, the application shows currency data using the settings specified by the ICU Library. However, if you want use a different set of currency formats, you can override the ICU Library settings.

The settings that you can override include the following:

- **negativePattern** - How the application displays negative monetary amounts, such as `-$32.00` or `($32.00)`.
- **positivePattern** - How the application displays positive monetary amounts, such as `$32.00`.
- **zeroValue** - How the application displays zero amounts, such as `0 (zero)` or `- (dash)`.

The application has two sets of files where currency format overrides can be declared.

- In single currency mode, currency format overrides are declared in the set of `localization_LocaleCode.xml` files. There is one file for each locale.
- In multicurrency mode, currency format overrides are declared in the set of `currency.xml` files. There is one file for each currency.

Currency formats in a single currency implementation

When the application is in single currency mode, all monetary amounts are in the same currency. The application uses the `localization_LocaleCode.xml` file corresponding to the region selected by the user to format monetary amounts. If there is no `localization_LocaleCode.xml` file corresponding to the chosen region, the application uses the default locale's `localization_LocaleCode.xml` file. The localization files can be accessed in Guidewire Studio from **Project** window to **configuration** > **config** > **Localizations**.

In the base configuration, the `localization_en_US.xml`, `localization_fr_FR.xml`, and `localization_ja_JP.xml` files define regional formats customary to each region. However, the only file with currency format overrides is the `localization_en_US.xml` file. This is because the base configuration is set to single currency mode and uses US dollars as the default currency.

In general, the application shows currency data using the settings specified by the ICU Library. However, if you want use a different set of currency formats, you can override the ICU Library settings.

The `<CurrencyFormat>` attributes that you can override include:

- **negativePattern** - How the application displays negative monetary amounts. The application displays and formats the numeric value in place of the pound sign (#) in the pattern. The application displays all other characters in the pattern as listed without modification.
 - For example, the negative pattern `($#)` displays the numeric value -32 as `($32.00)`.
- **positivePattern** - How the application displays positive monetary amounts. The application displays and formats the numeric value in place of the pound sign (#) in the pattern. The application displays all other characters in the pattern as listed without modification.
 - For example, the positive pattern `##` displays the numeric value 32 as `$32.00`.
- **zeroValue** - How the application displays zero amounts. For example, the zero value pattern can be 0 (zero) or - (dash). If the numeric value of a monetary amount is null, the application displays the amount as empty or blank. The monetary amount must be a non-null 0.00 amount for the zero value pattern to be used.
 - For example, the zeroValue - displays the numeric value 0 as -.

The `localization_en_US.xml` file contains these overrides:

```
<Localization xmlns="http://guidewire.com/localization">
  <GWLocale code="en_US" name="English (US)" typecode="en_US">
    ...
    <CurrencyFormat negativePattern="($#)" positivePattern="##" zeroValue="-"/>
  </GWLocale>
</Localization>
```

Create currency format overrides in a single currency implementation

About this task

The base configuration contains one currency format override for single currency instances, which is for the US English locale. If you want to modify currency display for this locale, you must edit the existing file. For all other locales, you must create a new file as described in this task.

Procedure

1. In Guidewire Studio™, navigate to **config > Localizations**.
2. Right-click **Localizations** and select **New > File**.
3. Name the file `localization_xx_XX.xml`, where:
 - `xx` is the lowercase version of the language code. For example, `en`.
 - `XX` is the uppercase version of region code. For example, `US`.
4. Open the existing `localization_en_US.xml` file. Copy the contents of this file into the new file.
5. Modify the information in the `<GWLocale>` element to match the new locale.
6. Modify the `<CurrencyFormat>` element to specify the desired currency format overrides.
7. Remove the remaining elements from the file as needed.

What to do next

If your instance supports multiple locales, Guidewire recommends adding the same currency format overrides to all locale files. This ensures that currency values are formatted the same way, regardless of the locale chosen by the user.

Currency formats in a multicurrency implementation

When the application is in multicurrency mode, monetary amounts can be in different currencies. The application uses the `currency.xml` file corresponding to the relevant currency to format monetary amounts. If there is no `currency.xml` file corresponding to the chosen region, the application uses the settings specified in the ICU Library. The `currency.xml` files can be accessed in Guidewire Studio from **Project** window to **configuration > config > currencies > (locale)**.

In the base configuration, there is one `currency.xml` file for each of the defined currencies. Each file is stored in a folder whose name is the currency code:

- AUD – Australian dollar
- CAD – Canadian dollar
- EUR – European Union euro
- GBP – British pound
- JPY – Japanese yen
- RUB – Russian ruble
- USD – U.S. dollar

In general, the application shows currency data using the settings specified by the ICU Library. However, if you want to use a different set of currency formats, you can override the ICU Library settings.

Currency scale overrides

Your instance may need a different number of digits stored in the database and shown in the user interface depending on the currency. To set currency-specific storage values, use the `storageScale` attribute on the `<CurrencyType>` element in file `currency.xml`.

The `<CurrencyType>` attributes that you can override include:

- **storageScale** – The number of decimal digits to store in the database for this currency. For example, United States dollars have two digits after the decimal point indicating the number of cents. Therefore the storage scale for `usd` is "2".

For example, in the base configuration:

- File `currency.xml` for the Australian dollar defines the following `storageScale`:

```
<CurrencyType code="aud" desc="Australian Dollar" storageScale="2">
```

- File `currency.xml` for the Japanese yen defines the following `storageScale`:

```
<CurrencyType code="jpy" desc="Japanese Yen" storageScale="0">
```

Currency format overrides

The `<CurrencyFormat>` attributes that you can override include:

- **negativePattern** - How the application displays negative monetary amounts. The application displays and formats the numeric value in place of the pound sign (#) in the pattern. The application displays all other characters in the pattern as listed without modification.
 - For example, the negative pattern `($#)` displays the numeric value -32 as `($32.00)`.
- **positivePattern** - How the application displays positive monetary amounts. The application displays and formats the numeric value in place of the pound sign (#) in the pattern. The application displays all other characters in the pattern as listed without modification.
 - For example, the positive pattern `$#` displays the numeric value 32 as `$32.00`.
- **zeroValue** - How the application displays zero amounts. For example, the zero value pattern can be 0 (zero) or - (dash). If the numeric value of a monetary amount is null, the application displays the amount as empty or blank. The monetary amount must be a non-null 0.00 amount for the zero value pattern to be used.

- For example, the zeroValue - displays the numeric value 0 as -.

For example, the currency.xml file for US Dollars defines the following:

```
<?xml version="1.0" encoding="UTF-8"?>
<Currency xmlns="http://guidewire.com/currency">
  <CurrencyType code="usd" desc="US Dollar" storageScale="2">
    <CurrencyFormat zeroValue="-"/>
  </CurrencyType>
</Currency>
```

There is a currency format override for the zero value (so that zero values are shown as "-"). There are no overrides for the negative pattern and positive pattern.

Create currency format overrides in a multicurrency implementation

About this task

The base configuration contains one currency format override for every defined currency. If you want to modify any of these existing overrides, you must edit the existing files. To override formats for a newly added currency, you must create a new file as described in this task.

Procedure

1. In Guidewire Studio™, navigate to **config > currencies**.
2. Right-click **currencies** and select **New > Package**. This creates a new directory in the currencies folder. Name the directory after the lowercase version of the three-letter ISO 4217 currency code. For example, **nzd**.
3. Right-click the package created in the previous step and select **New > File**. Name the file **currency.xml**.
4. Open one of the existing currency.xml files for one of the other currencies. Copy the contents of this file into the new file.
5. Modify the XML to match the new currency and to specify the desired currency format overrides.

Configuring currencies by territorial data

PolicyCenter only. The RegionCurrencyMappingUtil class associates countries, jurisdictions, and addresses with currencies. The class has the following methods:

- getCurrencyMappingForCountry – Takes as a parameter a typecode from the Country typelist.
- getCurrencyMappingForJurisdiction – Takes as a parameter a typecode from the Jurisdiction typelist.
- getCurrencyMappingForAddress – Takes as a parameter an instance of an Address entity.

All methods return a typecode from the Currency typelist.

Categorizing territorial typelists by currency

For the RegionCurrencyMappingUtil class to function properly, you must:

- Categorize all Country and Jurisdiction typecodes by Currency typecodes.
- Categorize all State typecodes by Jurisdiction typecodes.

Categorize a Country or Jurisdiction typecode with a single Currency typecode only. Conversely, you can use a single Currency typecode to categorize many Country and Jurisdiction typecodes. In a similar manner, categorize a State typecode with a single Jurisdiction typecode only, but use a single Jurisdiction typecode to categorize many State typecodes.

Mapping currencies to jurisdictions

Given a Jurisdiction typecode, the getCurrencyMappingForJurisdiction method first looks up the Country typecode that categorizes the jurisdiction and then returns the Currency typecode that categorizes the country. If this

indirect mapping fails to produce a valid currency, the `getCurrencyMappingForJurisdiction` method looks up the Currency typecode that categorizes the jurisdiction directly.

Mapping currencies to jurisdictions indirectly through countries handles situations where certain jurisdictions, which are not countries in their own rights, use the currencies of other countries. For example, Guam and the US Virgin Islands use the US Dollar. They are jurisdictions that do not claim the US as their country. As jurisdictions however, they can be categorized directly by the US Dollar (usd).

Mapping currencies to addresses

Given an `Address` instance, the `getCurrencyMappingForAddress` method first looks up the Country typecode to find the Currency that categorizes it. Sometimes an `Address` instance has no Country typecode, or the typecode is not categorized by a currency. In that case, `getCurrencyMappingForAddress` uses the State typecode to find the Jurisdiction typecode that categorizes the State. Then the method looks up the Currency that categorizes the Jurisdiction.

Monetary amounts in Gosu

In Gosu, monetary amounts are managed by the `MonetaryAmount` data type. Like other numeric values, `MonetaryAmount` objects can be used to perform standard mathematical calculations.

A `MonetaryAmount` value is initialized when it is constructed. There are two constructors:

```
MonetaryAmount( BigDecimal amount, Currency currency )
MonetaryAmount( String value )
```

You can retrieve the default currency by calling the `gw.api.util.CurrencyUtil.getDefaultCurrency` method. The method returns a `Currency` object that matches the setting of the `config.xml` parameter `DefaultApplicationCurrency`. For other useful `CurrencyUtil` methods, refer to the [Gosu API Reference](#).

The format of the value string for the second constructor is "*Amount CurrencyCode*". A space character separates the two parameters. For example:

```
MonetaryAmount myBalance( "123.45 USD" )
```

A `MonetaryAmount` object can also be created from a `BigDecimal` object by calling the `BigDecimal` extension method `ofCurrency` or `ofDefaultCurrency`. The `ofCurrency` method accepts a `Currency` type argument. The `ofDefaultCurrency` method uses the default currency specified in `config.xml`. For example:

```
var baseBigD      = new java.math.BigDecimal( "123.45" )
var monetaryAmount01 = baseBigD.ofCurrency( Currency.TC_USD )
var monetaryAmount02 = baseBigD.ofDefaultCurrency()
```

For additional information about the `MonetaryAmount` data type, refer to the *Gosu API Reference*.

See also

- For information on accessing the *Gosu API Reference*, see the *Configuration Guide*.

Configuring multicurrency

Through multicurrency, PolicyCenter provides the ability to write policies that provide insurance for assets in different currencies. For example, an insurer offers a commercial property policy that can include properties in more than one country. With multicurrency, the policy values the properties in the currency of their locations.

See also

- *Application Guide*

Configuring PolicyCenter for a single currency

When you run PolicyCenter with `MultiCurrencyDisplayMode` disabled, currency fields are not visible in the user interface. Examples of currency fields in the user interface are currency selectors on screens and currency columns in tables.

In the base configuration, PolicyCenter runs with `MultiCurrencyDisplayMode` disabled.

To run PolicyCenter with `MultiCurrencyDisplayMode` disabled, set the following parameters in `config.xml`:

Parameter	Value	Description
<code>MultiCurrencyDisplayMode</code>	<code>SINGLE</code>	The multicurrency display mode. In the base configuration, this parameter is set to <code>SINGLE</code> .
<code>DefaultApplicationCurrency</code>	A typecode from the Currency typelist.	The default currency for the application. Set to the currency of your choice.

See also

- "`MultiCurrencyDisplayMode`" in *Configuration Parameters*
- "`DefaultApplicationCurrency`" in *Configuration Parameters*

Enabling multicurrency

The multicurrency feature is enabled using the `config.xml` parameter `MultiCurrencyDisplayMode`. The parameter's name references only multicurrency display, but the parameter enables all multicurrency behaviors.

The `MultiCurrencyDisplayMode` parameter can be set to one of the following values:

- `SINGLE` (the base configuration default)

- MULTIPLE

Note: This parameter is also included in ContactManager's `config.xml` file in the base application. If set to MULTIPLE, a person contact's preferred currency can be set in ContactManager.

In PolicyCenter, there is a second application configuration parameter related to multicurrency: `MultiSettlementCurrencyHandlingEnabled`. By default, this is set to false. When it is set to true, Advanced Product Design App generates additional codegen artifacts to ensure that the multiple settlement currencies are calculated correctly. This includes additional constructors and Gosu code.

- If your instance of PolicyCenter supports multicurrency, including multicurrency policies with multiple settlement currencies, you must set this to true.
- If your instance of PolicyCenter supports multicurrency, but it does not support multicurrency policies with multiple settlement currencies, Guidewire recommends you set this to false. There is technically no harm in setting this to true, but it will result in APD App codegen generating additional artifacts that your instance of PolicyCenter will never need.

Enable multicurrency

About this task

In the base ClaimCenter configuration, multicurrency reserving is disabled and ClaimCenter tracks all financial transactions in US Dollars (the default application currency).

In the base configuration for PolicyCenter and BillingCenter, multicurrency is disabled.

IMPORTANT: The `MulticurrencyDisplayMode` and `EnableMultiCurrencyReserving` (ClaimCenter only) configuration parameter settings are semi-permanent. You can change the values of these parameters only once, `MulticurrencyDisplayMode` from SINGLE to MULTIPLE and `EnableMultiCurrencyReserving` from false to true. If you have changed the value and started the application, you must not later change them back. If you do change the value of `MulticurrencyDisplayMode` back to SINGLE or `EnableMultiCurrencyReserving` back to false, subsequent attempts to start the server will fail.

IMPORTANT: When you integrate the core applications in Guidewire InsuranceSuite, you must set the value of `MulticurrencyDisplayMode` to be the same in each application.

Procedure

1. Open `config.xml`.
2. Search for this parameter: `<param name="MulticurrencyDisplayMode" value="SINGLE"/>`. Enable multicurrency by setting the value attribute to MULTIPLE.
3. If you plan to enable multicurrency reserving in ClaimCenter, search for this parameter: `<param name="EnableMulticurrencyReserving" value="false"/>`. Enable multicurrency reserving by setting the value attribute to true.
4. If you plan to support multicurrency policies with multiple settlement currencies in PolicyCenter, search for this parameter: `<param name="MultiSettlementCurrencyHandlingEnabled" value="false"/>`. Enable multiple settlement currency handling by setting the value attribute to true.
5. Set the currency attributes for multicurrency mode as needed. For more information, see “Currency formats in a multicurrency implementation” on page 224 and the *Setting the default currency* topic in the *Globalization Guide*.
6. Restart the application.

Multicurrency object model

This topic describes multicurrency object models.

Multicurrency properties

The following table shows some multicurrency properties on objects.

Property	Objects	Property description
PreferredCoverageCurrency	Account PolicyPeriod Coverable Coverable delegates	The preferred currency for coverages. This is a type key to Currency. Some policy lines are coverables, therefore they also have this property. The Commercial Property line, CommercialPropertyLine entity provides an example.
PreferredSettlementCurrency	Account Contact PolicyPeriod	The preferred currency for settlement or billing.
CoverageCurrency	Cost Reinsurable Transaction	The coverage currency.
SettlementCurrency	Cost Transaction WorkersCompLine	The settlement currency.

Multicurrency in a policy line

This topic describes objects in a multicurrency policy line using Commercial Property as an example. PolicyCenter displays the user interface for changing the various currencies when the MultiCurrencyDisplayMode parameter is enabled.

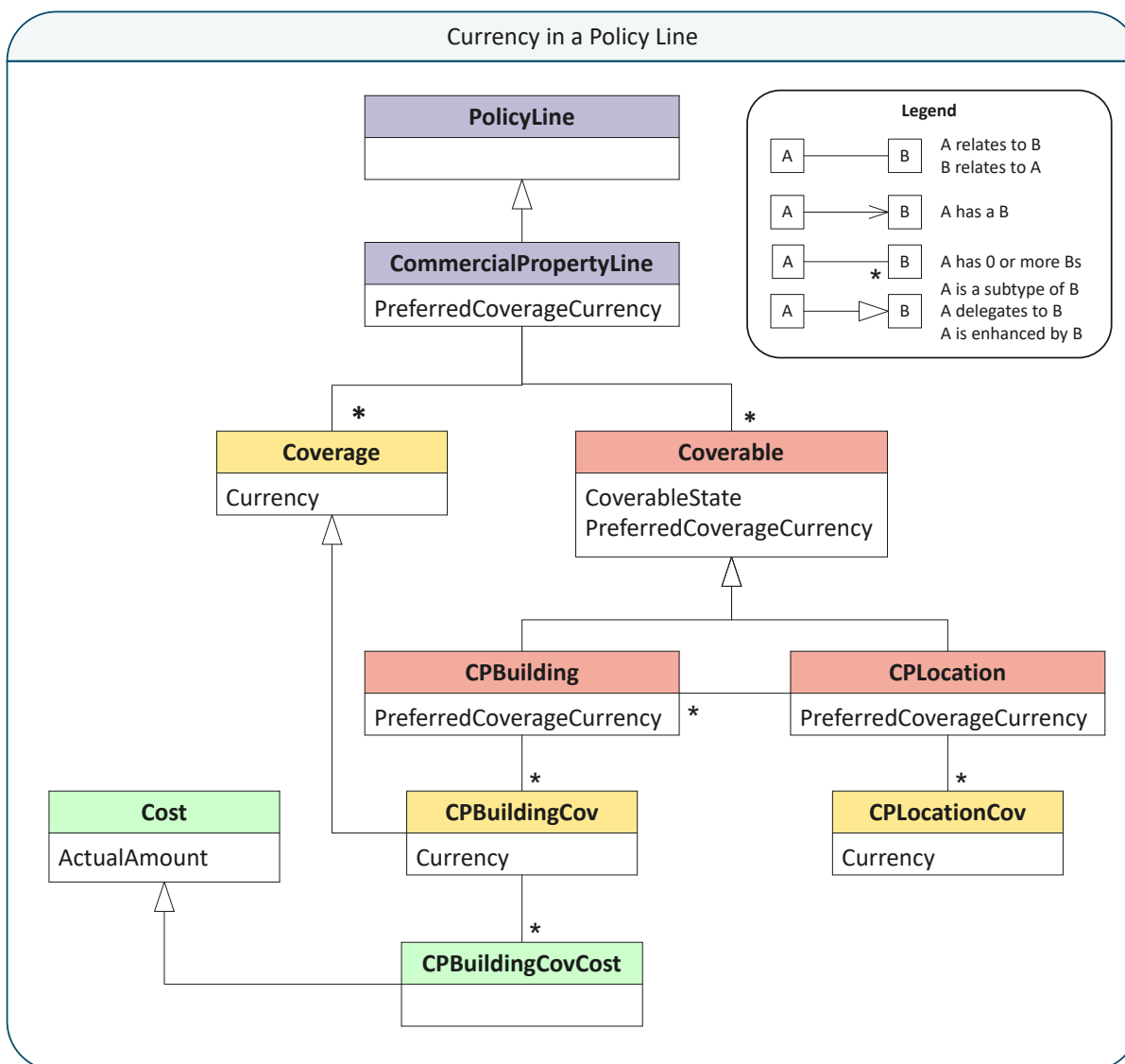
IMPORTANT: The MultiCurrencyDisplayMode configuration parameter setting is semi-permanent. You can change the value of this parameter only once, from SINGLE to MULTIPLE. If you have changed the value and started the application, you must not later change it back. If you do change the value of MultiCurrencyDisplayMode back to SINGLE, subsequent attempts to start the server fail.

In the base configuration, all coverages on the same coverable must have the same currency. Because of this, changing the currency on a specific coverage or clause is not exposed in the user interface. You change the currency on the coverable, and PolicyCenter propagate the currency to the coverages and clauses.

In a line of business, the coverable objects have a preferred coverage currency. You can set this property in the user interface. In the base configuration, PolicyCenter propagates the currency down to the individual clauses (coverages, exclusions, and conditions) for that coverage. In the Commercial Property line of business, the CommercialPropertyLine, CPLocation, and CPBuilding coverable objects have a PreferredCoverageCurrency property. The CommercialPropertyLine is a coverable object, although this object has no line level coverage in the base configuration.

The coverage, exclusion, and policy condition entities (Coverage, Exclusion, and PolicyCondition) include a currency property (Currency) which stores the currency for the clause. The currency property stores the legally binding currency associated with the coverage, exclusion, or policy condition. In the base configuration, PolicyCenter propagates the currency from the coverable; you can modify this behavior through configuration.

In the base configuration, the CPLocationCov and CPBuildingCov coverages have a Currency property. PolicyCenter propagates the currency from the coverable. The following diagram shows the Currency property on various entities in the commercial property line.



Both the PreferredCoverageCurrency and Currency properties have a type key to the Currency typelist. You can specify typecodes for additional currencies in this typelist.

The cost object (Cost) has a property for the actual amount (ActualAmount). This property is a MonetaryAmount type.

Creating multicurrency policy lines

This topic describes how to create policy lines that support more than one coverage currency. The tasks include:

- Identifying the coverage currencies that the line supports.
- Ensuring that the required foreign exchange conversions are available (not described in this topic)
- Adding appropriate coverage options for each of the currencies
- Managing rating in each of the currencies (not described in this topic)
- Enabling producers to write business in various settlement currencies

The examples use the Wright Construction account in the small sample dataset. The examples assume that Wright Construction is based in the United States. The insurer provides a commercial property policy that insures buildings and locations the United States and France. The following table contains currency information for these countries.

Country	Currency	Currency symbol	ISO 4217 Code
United States	U.S. dollar	\$	USD
France	Euro	€	EUR

IMPORTANT: The MulticurrencyDisplayMode configuration parameter setting is semi-permanent. You can change the value of this parameter only once, from SINGLE to MULTIPLE. If you have changed the value and started the application, you must not later change it back. If you do change the value of MulticurrencyDisplayMode back to SINGLE, subsequent attempts to start the server fail.

Adding coverage currencies to a policy line

This topic provides example of how to manage multiple coverage currencies for a given coverage. In these step-by-step instructions, you add the euro as a coverage currency for the commercial property line and add euro values to a money coverage term.

Create workspace and change list

Procedure

1. In Product Designer, create a workspace named “Multicurrency”.
2. In Product Designer, create a change list named “Multicurrency Coverage Currency” that uses the workspace.

What to do next

Now you can move to the next step, “Add a currency to a coverage” on page 231.

Add a currency to a coverage

Before you begin

Before you begin, you must complete “Create workspace and change list” on page 231.

Procedure

1. In Product Designer, navigate to **Product Model > Policy Lines > Commercial Property Line**.
2. Under **Available Coverage Currencies**, click **Add** and select **EUR** from the drop-down list. Product Designer displays both **EUR** and the existing coverage currency as choices for coverages.
3. Go to **Coverages** and select **Building Coverage**.
4. Go to **Terms** and select the **Limit** coverage term. The **Value Type** of this coverage term is **Money**.
5. Click to **Add** a currency.
6. For example, select **EUR** and enter **75,000** as a default value. Optionally enter minimum and maximum values such as **25,000** and **100,000**.
7. Go to **Terms > Deductible**. The **Value Type** of this coverage term is **Money**. This is a option coverage term which presents a list of values.
8. Go to **Options**.
9. Add options in euros, such as **250**, **500**, and **750**. Prefix the **Code** with **EUR** to distinguish the euro options, for example **EUR250**.
10. Commit the change list and synchronize the product model.

What to do next

Now you can go to the next step, “Check your work” on page 232.

Check your work

Before you begin

Before you begin, you must complete “Add a currency to a coverage” on page 231.

Procedure

1. Log in to PolicyCenter as a producer such as aapplegate.
2. Go to the Wright Construction account.
3. Create a Commercial Property submission.
4. On the **Policy Info** screen, verify that **USD** and **EUR** appear in the **Preferred Currency > Coverage** drop-down list and select **EUR**.
The preferred coverage currency will be the default currency for coverables on the policy.
5. Advance to the **Buildings and Locations** screen.
6. Add a building to the primary location and fill in any required fields on the **Building Information** screen. Verify that the **Coverages in** drop-down list includes **USD** and **EUR**, and the preferred coverage currency, **EUR**, is selected.
7. Fill in required fields on the **Details** tab.
8. Go to the **Coverages** tab.
Notice that the **Limit** is set to the default value for **EUR**. The **Deductible** drop-down list contains the **EUR** options you defined in Product Designer.
9. Deselect all coverages except for **Building Coverage**.
10. Advance to the **Policy Review** screen.
Notice that for the building, the **Coverages in** are in euros.
11. Quote the policy.
The quote is in U.S. dollars because that is the settlement currency.

Changing the settlement currency

The preferred settlement currency, PreferredSettlementCurrency, is a field on the contact, account, and policy period entities.

In a job, the default settlement currency is the settlement currency on the account. In a job, you can pick a settlement currency from the **Preferred Currency > Settlement** drop-down list on the **Policy Info** screen. The list of **Settlement** currencies is populated from the Currency typelist.

A producer can quote a policy in any configured currency. However, when you bind or issue a policy, the producer of record must have the authority to bind the policy in the settlement currency. In the base configuration, the user interface enables you to select only one currency per producer code. You can configure PolicyCenter to allow more than one currency per producer code. You can view producer codes by navigating in PolicyCenter to the **Administration > Producer Codes** screen.

PolicyCenter and product model configured for multicurrency

PolicyCenter does not start in single currency display mode if the product model is configured to use multiple currencies.

Product Designer allows users to configure multicurrency features without regard for the setting of the MultiCurrencyDisplayMode in the PolicyCenter config.xml file. If you configure multiple currencies in a product model object, and the value of the MultiCurrencyDisplayMode property is SINGLE, PolicyCenter does not start.

For example, if you add clauses that specify multiple coverage currencies and the PolicyCenter `MultiCurrencyDisplayMode` is set to `SINGLE`, the server fails to start and displays messages similar to the following:

```
ERROR ... PolicyLinePattern : ... [field : AvailableCoverageCurrencies] : - Multiple Coverage  
Currencies defined for product not allowed in single Currency mode  
ERROR ... [ExclusionPattern] : ... [DirectCovTermPattern] : ... [LimitsSet] : - The pattern specifies  
limits for different currencies in SingleCurrencyMode
```

If you intend to deploy a product model on PolicyCenter servers that are not configured for multiple currency display mode, do not configure multiple currencies in any part of the product model. However, you can test multicurrency products on a staging server configured for multicurrency while the production system is configured for a single currency.

Configuring the coverage currency

In the base configuration, the coverage currency appears in various places in accounts and policies. In the base configuration, accounts, policy periods, and coverables have coverage currency properties. Coverages and clauses have a currency property.

In the base configuration, you set the coverage currency on a coverable. PolicyCenter sets the currency for each coverage on the coverable to the same currency as the coverable. You can modify this behavior through configuration.

Coverage currency in accounts

Accounts have a preferred coverage currency. You can view or set this value on the **Account Summary** screen. The preferred coverage currency on account is in the `Account.PreferredCoverageCurrency` property. In the base configuration, PolicyCenter initially sets the account's preferred coverage currency to the contact's preferred settlement currency, `Contact.PreferredSettlementCurrency`. You can change this behavior through configuration.

In the base configuration, all contact objects have a preferred coverage currency even if PolicyCenter is configured for a single currency. (When running PolicyCenter with `MultiCurrencyDisplayMode` disabled, the currency does not appear in the user interface.) However, if a contact does not have a preferred coverage currency, then PolicyCenter sets the account's preferred coverage currency to the default system currency. A properly configured system always specifies the currency and does not rely on this fail-safe behavior.

Coverage currency in policy periods

Policy periods have a preferred coverage currency which PolicyCenter initializes when initializing the policy term. The preferred coverage currency on policy periods is in the `PolicyPeriod.PreferredCoverageCurrency` property. In the base configuration, this property is set to the account's preferred coverage currency property, `Account.PreferredCoverageCurrency`, if it exists. This is implemented in the `initializeCurrencies` method in the `PolicyPeriodPlugin` class in the `gw.plugin.policyperiod.impl` package.

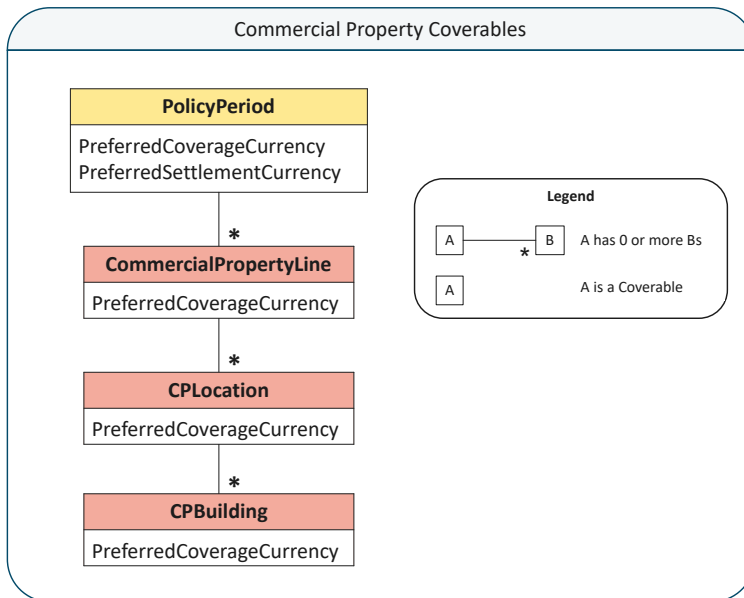
In the base configuration, all accounts have a preferred coverage currency even if PolicyCenter is configured for a single currency. However, if an account does not have this value, PolicyCenter sets the policy period's preferred coverage currency to the default system currency. A properly configured system always specifies the currency and does not rely on this fail-safe behavior.

To get the default preferred coverage currency for the specified policy period, you can use the `lookupDefaultPreferredCoverageCurrency` method in the `gw.job.PCPolicyCurrencyUtil` Gosu class.

Coverage currency in coverages and clauses

Coverages, exclusions, and policy conditions include a currency which is the currency for the clause. The `Currency` property on the `Coverage`, `Exclusion`, and `PolicyCondition` entities stores the currency. In the base configuration, all coverages on the same coverable have the same currency. Therefore, the user interface does not provide a way to change the currency on a specific clause. The user interface lets you change the currency on the coverable, and that currency is propagated to all coverages on the coverable.

A coverable inherits its default currency from the owning coverable or policy period. In Commercial Property, as shown in the following object model diagram, a new building (CPBuilding) automatically gets the preferred coverage currency, PreferredCoverageCurrency, of its parent coverable, CPLocation.



Coverage currency in coverables

Coverables have a preferred coverage currency which PolicyCenter uses to synchronize all coverages associated with a coverable. The preferred coverage currency on coverables is in the `Coverable.PreferredCoverageCurrency` property. After updating the preferred coverage currency on a coverable, PolicyCenter synchronizes the product model. The `synchronizeCurrencies` method in the `gw.web.policy.CoverableCoverageCurrencySynchronizer` Gosu class synchronizes the product model.

Configuring the settlement currency

This topic describes Gosu classes and object related to the settlement currency. The settlement currency is used in billing. In the base configuration, the contact, account, policy period objects have a settlement currency property.

Settlement currency in contacts

Contacts have a preferred settlement currency. The preferred settlement currency on contact is in the `Contact.PreferredSettlementCurrency` property. PolicyCenter initializes the preferred settlement currency in the `gw.account.AccountBaseEnhancement` class.

In the base configuration, the preferred settlement currency is based on the primary address of the contract. For example, if the primary address is in California, which is in the United States, then PolicyCenter sets the currency to U.S. dollars. The `createAccountForContact` method sets the preferred settlement currency.

In the base configuration, the jurisdiction on the address maps to a country, and each country has a currency. However, if for some reason an address does not map to a currency, the `createAccountForContact` method initializes this property to the system default currency. A properly configured system always specifies the currency and does not rely on this fail-safe behavior.

The `ContactCurrencyInitializer` class contains methods for editing the currency for a contact. PCF files that enable a user to edit the currency for a contact use these methods. The `ContactCurrencyInputSet` PCF file uses this class to set the **Preferred Currency** for a contact. You can see an example of this **Preferred Currency** field when you view a contact on the **Contact File Details** screen.

The **Account Summary** screen uses the preferred settlement currency on the contact to aggregate policy, claim, and billing metrics and display them in the settlement currency. The contact's preferred settlement currency becomes the default settlement currency for an account created from that contact.

Settlement currency in accounts

Accounts have a preferred settlement currency. The preferred settlement currency on account is in the `Account.PreferredSettlementCurrency` property. To create a new account with the given contact as the account holder, you can use the `createAccountForContact` method in `gw.account.AccountBaseEnhancement`. In the base configuration, this method initializes the account's preferred settlement currency based on the contact's primary address. The code assumes that if the coverage currency is set, the settlement currency is also set.

In the base configuration, the jurisdiction on the address maps to a country, and each country has a currency. For the rare jurisdiction that does not have a country, you can specify the currency directly for that jurisdiction. Map the jurisdiction to currency in the definition of the jurisdiction and country typekeys. For an example, see the typecode for Guam in the `Jurisdiction.ttx` typelist extension. However, if for some reason an address does not map to a currency, the `createAccountForContact` method initializes this property to the system default currency. A properly configured system always specifies the currency and does not rely on this fail-safe behavior.

When the account is initially created, the country of the account's address determines the preferred settlement currency. Methods in the `gw.web.financials.AccountCurrencyInitializer` class determine the currency.

The `AccountCurrencyInputSet` PCF file displays the account's settlement currency.

The `AccountFile_Claims` and `SubmissionManager` PCF files use the settlement currency to determine the currency on the **Account File Claims** and **Submission Manager** screens. These files use the settlement currency for computing and displaying the **Total Incurred** and **Total Cost** fields, respectively.

Settlement currency in policy periods

Policy periods have a preferred settlement currency. The preferred settlement currency is in the `PolicyPeriod.PreferredSettlementCurrency` property. PolicyCenter initializes this field when initializing the policy term. You can modify the initialization by overriding the `initializeCurrencies` method in the `IPolicyPeriodPlugin` interface.

When you create a policy in the user interface, you set the policy period's settlement currency (**Preferred Currency > Settlement**) on the **Policy Info** screen.

Configuring multicurrency and reinsurance

PolicyCenter attaches a reinsurance program to a risk in a policy if the *total insured value/sum insured* (TIV/SI) for that risk has the same currency as the risk. In the base configuration, the TIV/SI currency is the currency associated with the jurisdiction of the risk. Through configuration, you can modify how PolicyCenter chooses the TIV/SI currency.

The reinsurance currency is independent of the coverage and settlement currencies. In the base configuration, PolicyCenter sets the reinsurance currency to the currency specified for the coverable's jurisdiction in the `ReinsuranceConfigPlugin` plugin. This plugin implements the `IReinsuranceConfigPlugin` plugin interface. The plugin calculates the TIV/SI based on the coverage currency. If the coverage currency differs from the reinsurance currency, the plugin converts the TIV/SI to the reinsurance currency. The `Reinsurable` object stores the TIV/SI (`TotalInsuredValue`) and reinsurance currency (`ReinsuranceCurrency`). The `Reinsurable` is then used to create the reinsurable risk (`RIRisk`) objects. If an appropriate reinsurance program is available, PolicyCenter attaches the program to the reinsurable risk. The program has the same currency as the reinsurable risk.

In the `ReinsuranceConfigPlugin` plugin, the `getReinsuranceCurrency` method returns the currency that the `RIRisk` uses for TIV/SI. To change how PolicyCenter selects the reinsurance currency, you can override this method.

The input parameter to the `getReinsuranceCurrency` method is a list of all the coverages associated with a single `ReinsurableCoverable`. The `PolicyPeriodBaseEnhancement` Gosu class calls this method when creating `Reinsurables`. In the base configuration, the method gets the jurisdiction specified in the

Coverable.CoverableState property and looks up the currency associated with that jurisdiction. That mapping is returned by the RegionCurrencyMappingUtil class and configured through the Country, Jurisdiction, State, and Currency typelists.

The reinsurance program and reinsurance contract entities, RIProgram and RIAGreement respectively, delegate to the reinsurance contract entity, RIContract. The RIContract entity has a non-nullable currency property, Currency, that the RIProgram and RIAGreement entities implement. The program and agreements associated with the program must have the same currency.

When PolicyCenter attaches an RIProgram or an RIAGreement to an RIRisk, the Currency properties must match.

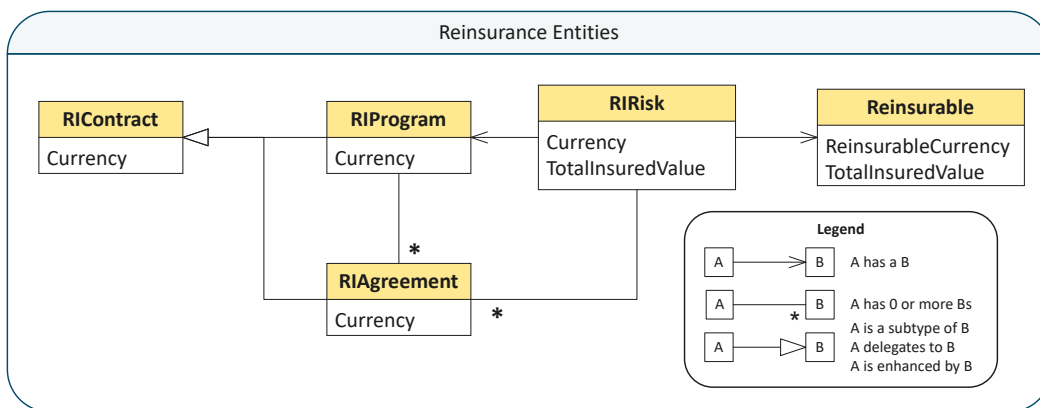
The Premium Ceding batch process runs and calls the PCCedingCalculator class to process premium transactions. This class does the conversion from the reinsurance currency to the settlement currency.

See also

- *Plugins, Prebuilt Integrations, and SOAP APIs*

Reinsurance objects and multicurrency

The following illustration shows some of the multicurrency properties on reinsurance objects.



Configuring multicurrency and rating

In the base configuration, the rating engine creates cost data in the currency of the associated coverage.

For costs not associated with a coverage, the rating engine uses the PreferredCoverageCurrency on the policy period associated with the line being rated.

The gw.rating.AbstractRatingEngineBase class defines a TaxRatingCurrency property and sets it to the PreferredSettlementCurrency on the policy period associated with the line being rated. The AbstractRatingEngineBase class is a read-only file.

See also

- *Application Guide*

Costs and multicurrency

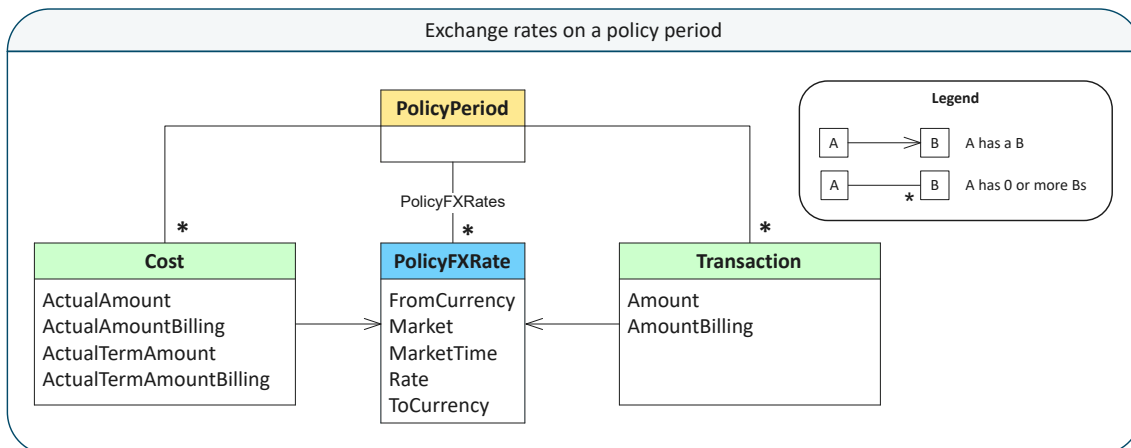
The rating engine rates the coverables in the coverage currency and returns cost data to PolicyCenter. PolicyCenter converts the cost data into Cost objects. PolicyCenter stores the currency and the amount on Amount properties on the Cost, for example Cost.ActualAmount. The Amount properties are of MonetaryAmount type which has properties for an amount and a currency. If necessary, PolicyCenter converts the amounts to the settlement currency. The rating engine then stores the amounts in duplicate billing amount properties (AmountBilling) on the Cost entity, for example Cost.ActualAmountBilling. If the coverage currency differs from the settlement currency, the rating engine also stores a pointer to the exchange rate. The Cost.CurrencyChanged Boolean property is set to true if the PolicyCenter converted the currency on the cost.

The `convertCostsToSettlementCurrency` method in the `AbstractRatingEngineBase` class performs the currency conversion and stores the converted amounts in duplicate `AmountBilling` properties.

The duplicate amount properties on the `Cost` entity are:

Property	Description
<code>ActualAmountBilling</code>	The <code>ActualAmount</code> in the settlement currency for accounting and reporting.
<code>StandardAmountBilling</code>	The <code>StandardAmount</code> in the settlement currency for accounting and reporting.
<code>OverrideAmountBilling</code>	The <code>OverrideAmount</code> in the settlement currency for accounting and reporting.
<code>ActualTermAmountBilling</code>	The <code>ActualTermAmount</code> in the settlement currency for accounting and reporting.
<code>StandardTermAmountBilling</code>	The <code>StandardTermAmount</code> in the settlement currency for accounting and reporting.
<code>OverrideTermAmountBilling</code>	The <code>OverrideTermAmount</code> in the settlement currency for accounting and reporting.
<code>PolicyFXRate</code>	If the coverage currency is not the same as the settlement currency, this property contains a foreign key to a copy of the exchange rate from the exchange rate service. PolicyCenter uses this <code>PolicyFXRate</code> to convert the as-rated costs from the coverage currency to billing costs in the settlement currency. When you configure PolicyCenter, do not keep more than one copy of the same <code>PolicyFXRate</code> on a single policy period.

As shown in the following object model diagram, the policy period has an array of exchange rates used for monetary amount conversion. This array is `PolicyPeriod.PolicyFXRates`.



Each `FXRate` object records an exchange rate and the time that it was obtained from the market. Exchange rates can vary over the life of the policy, come from different sources, or cover different currency pairs, therefore the policy period maintains an array of exchange rates.

The following table describes some of the properties on the `PolicyFXRate` object:

Property	Description
<code>Rate</code>	The exchange rate at which one currency can be converted to another.
<code>MarketTime</code>	The time at which the market indicated the rate went into effect.
<code>RetrievedAt</code>	The time at which the quotation was obtained from the market.
<code>FromCurrency</code>	The currency to convert from.
<code>ToCurrency</code>	The currency to convert to.
<code>Market</code>	The exchange rate market, <code>FXRateMarket</code> , which set the rate.

Property	Description
PolicyPeriod	A foreign key to the policy period to which this object belongs.

Transactions and multicurrency

In the base configuration, transaction amounts are computed using the as-rated currency amounts on the costs. PolicyCenter then converts the transaction amounts to the settlement currency using the same exchange rate, PolicyFXRate, as the cost associated with the transaction. This is one way that an insurer might handle exchange rate changes during a policy term. You can configure PolicyCenter to handle the exchange rate in other ways during transactions.

The createEntityTransactions method in the PolicyPeriodTransactionCalculator class converts the transaction amount to the settlement currency using the exchange rate. The PolicyPeriodTransactionCalculator class is a read-only file.

The properties on the Transaction entity are:

Property	Description
Amount	The transaction amount.
AmountBilling	The Amount in the settlement currency for billing.
PolicyFXRate	A foreign key to a copy of the exchange rate from the exchange rate service. This points to the same exchange rate used to calculate the cost associated with this transaction.

Configuring underwriting authority and multicurrency

PolicyCenter raises underwriting issues based on characteristics of the policy, including ones that may be related to monetary amounts. You can create and manage monetary underwriting issues with values, approvals, and authority limits that include both an amount and a currency. The base configuration and sample data contain underwriting issues that provide an example of how to compare values, approvals, and authority limits in different currencies.

In the base configuration examples, PolicyCenter creates an issue with a value in the currency on the policy. That currency may differ from the currency of the user's authority grant. In the base configuration, the monetary-issue comparator automatically converts the value being tested to the currency of the reference value. The value being tested might be the value of the user's approval. The reference value currency might be the currency of the user's authority grant. In the sample data, the authority grants are in U.S. dollars, but this is not a requirement. Through configuration, you can take a different approach.

See also

- “Configuring underwriting rules” on page 69
- *Application Guide*

Configuring underwriting issues and multicurrency

In the base configuration, certain underwriting rules handle multicurrency. An underwriting rule that handles multicurrency has the following features:

Field	Description
Comparator	Value Comparator set to At least or At most. These values correspond to the codes Monetary_GE or Monetary_LE, respectively.
DefaultValueAssignmentType	Set to OffsetAmount.
DefaultValueOffsetAmount	Not equal to 0.

Field	Description
ValueFormatterType	Optional. Set to MonetaryAmount to format the value as an amount and currency in the user interface.

The PATotalPremium underwriting rule is an example that shows multicurrency. In PolicyCenter, you can view this underwriting rule in **Administration > Business Rules > Underwriting Rules**. In the Personal Auto line of business, the evaluator Gosu class (PA_UnderwriterEvaluator.gs) determines whether to raise PATotalPremium underwriting issues.

As with other underwriting issues, the code that creates the issue determines the value for the issue. PolicyCenter displays this value in the underwriting screens, therefore consider keeping the value (and approval) in the original currency. In the base configuration, the value comparator for monetary issues does a conversion between currencies, if needed. This conversion enables you to specify authority grants in a single currency per issue, yet have that grant to apply to all currencies for the issue. Then PolicyCenter determines whether or not to raise an underwriting issue. The UWIssueValueComparatorWrapper class in the gw.job.uw package uses the MonetaryGEValueComparatorWrapper and MonetaryLEValueComparatorWrapper classes to convert the currency and compare monetary values.

[See also](#)

- “Configuring underwriting authority” on page 67
- *Application Guide*

Configuring authority profiles and multicurrency

For PolicyCenter users, you set the approval value and currency for these underwriting issue types in an authority grant within an authority profile. Access authority profiles by navigating to **Administration > Users & Security > Authority Profiles** in PolicyCenter.

[See also](#)

- “Configuring authority grants” on page 67

Implementing an exchange rate service

Each insurer has their own unique exchange rate requirements such as how often to refresh the exchange rate or which market provides the rates. In some cases, the exchange rate market is not a market in the conventional sense, but a set of rates specified by the company or one of its business associates. An insurer’s exchange rates may vary by line of business. Certain policyholders may have custom rates which are negotiated with the insurer. Therefore, the base implementation of PolicyCenter provides a simple exchange rate service which rates multicurrency policies. In this simple implementation, PolicyCenter obtains the exchange rates from a static table containing rates for conversion from one currency to another. Guidewire expects each insurer to implement their own exchange rate service.

[See also](#)

- *Application Guide*

The exchange rate service plugin interface

The Exchange Rate Service plugin interface (IFXRatePlugin) is the interface for obtaining the exchange rate. In the base configuration, the FXRateServicePlugin implements this interface. You can use the FXRateServicePlugin implementation as a starting point for your own implementation of the IFXRatePlugin interface.

The IFXRatePlugin interface has the following methods:

Method	Return value	Description
canConvert	Boolean	For two currencies, this method returns true if you can convert from the first currency to the second currency.

Method	Return value	Description
getFXRate	FXRate	This method returns the rate for converting from one currency to another. You can optionally specify: • A date on which the rate was in effect • The rate market from which to obtain the rate

The Built-in Implementation of the Exchange Rate Service Plugin Interface

The built-in implementation of the Exchange Rate Service Plugin (IFXRatePlugin) interface is the FXRateServicePlugin plugin. This simple implementation gets exchange rates from a static table defined in the plugin. This class is provided only as an example; it is not for production use. Although you can use this plugin as a starting point, you must create your own implementation of the IFXRatePlugin interface.

Configuring the BillingCenter multicurrency integration

The integration between BillingCenter and PolicyCenter requires that you perform certain tasks to enable multicurrency.

Plans and authority limits for multicurrency in BillingCenter

In the integration between BillingCenter and PolicyCenter, you define plans and authority limits in BillingCenter.

You can specify multiple currencies for each plan type. Plans have monetary values that reflect various applicable fees and financial thresholds. For example, billing plans contain an invoice fee. A Euro account must define the fee in euros. As another example, delinquency plans have monetary thresholds that control when BillingCenter makes a policy delinquent. Such thresholds must be defined for each currency.

The plans and authority limits that are created in BillingCenter are:

- Billing plans
- Payment plans
- Commission plans
- Delinquency plans
- Agency bill plans
- Authority limits

With the exception of agency bill plans, BillingCenter requires all of these be defined to issue a policy in a given currency.

For agency bill plans, you must define agency bill plans that include each currency in which you plan on issuing policies with agency billing. If the agency bill plans do not include a given currency, **Agency Bill** will not appear in PolicyCenter as a choice for billing method.

Billing plans in BillingCenter for multicurrency accounts

In the integration between BillingCenter and PolicyCenter, billing plans are defined in BillingCenter.

In BillingCenter, each account must have an associated billing plan. Billing plans determine how BillingCenter handles automatic distributions, special circumstances such as low balance, and invoicing at the account level. For each billing plan, you can specify one or more currencies.

[See also](#)

- “Billing plans” in the *BillingCenter Application Guide*

Payment plans in BillingCenter for multicurrency accounts

In the integration between BillingCenter and PolicyCenter, payment plans are defined in BillingCenter.

In BillingCenter, each policy must have an associated payment plan. A payment plan is a collection of settings declared in BillingCenter that defines how invoices are generated for a group of policies. It specifies the `_payment interval_`, the down payment amount, the maximum number of installments, and when to invoice one-time charges. In BillingCenter, you can specify one or more currencies for each payment plan. The payment plans that match the settlement currency of a policy appear in PolicyCenter as a list of choices for how to bill a policy.

See also

- “Payment plans” in the *BillingCenter Application Guide*

Agency bill plans in BillingCenter for multicurrency organizations

In the integration between BillingCenter and PolicyCenter, agency bill plans are defined in BillingCenter. In PolicyCenter, producer organizations are associated with agency bill plans. When integrated with BillingCenter, PolicyCenter retrieves the agency bill plans from BillingCenter. In a multicurrency system, you can select one or more agency bill plans on the **Agency Bill** tab. Each agency bill plan can provide one or more currencies. For the current producer organization, select currencies from the agency bill plans. For each producer organization, you can associate only one plan per currency. Therefore, if you select USD on Plan A, you cannot select USD on Plan B.

When entering the **Organizations** or the **New Organizations** screens, PolicyCenter synchronizes with BillingCenter to refresh the agency bill plans.

See also

- *Application Guide*

Delinquency plans in BillingCenter for multicurrency accounts

In the integration between BillingCenter and PolicyCenter, you define delinquency plans in BillingCenter.

BillingCenter delinquency plans define sequences of events to execute when a policy becomes past due. Delinquency plans can be applied to the past-due policy only or to all policies in the account.

A special case exists when all policies in the account are considered delinquent and the account supports multiple currencies. In this situation, the delinquency plan is applied only to the policies that use the same currency as the policy that initiated the delinquency. In other words, only policies contained in the initiating policy’s currency silo are considered to be delinquent. Other policies in the account that use a different currency (and are therefore in a different currency silo) are not considered delinquent. This behavior describes the operation of the BillingCenter base configuration. If you wish all policies in the multicurrency account to be considered delinquent, regardless of their currency, BillingCenter can be configured to support that behavior.

Similarly, when a delinquent account is reinstated, the base configuration reinstates only the policies that reside in the relevant currency silo. To reinstate all policies in the account, regardless of their currency, configuration can be written to support that behavior.

See also

- “Delinquency processing” in the *BillingCenter Application Guide*

Authority limits in BillingCenter for multicurrency integration

In the integration between BillingCenter and PolicyCenter, authority limits are defined in BillingCenter.

Each user in BillingCenter can be assigned an authority limit profile. An authority limit profile specifies the types of financial transactions the user is authorized to create or approve. The profile also specifies the maximum monetary amount allowed for each financial transaction type.

BillingCenter supports multicurrency authority limits. If an integration includes Japanese yen, European Union euros, and U.S. dollars, a separate authority limit must be created for each of the currencies. All three of the limits can then be included in the same authority limit profile.

See also

- “Authority limit profiles” in the *BillingCenter Application Guide*

Configuring personal data destruction

Note: The data destruction features described in these topics provide a set of features that help enable insurers to comply with some of their data destruction requirements. These requirements may be driven by insurers' policies and practices, as well as by their interpretation of various regulatory requirements. For example, such regulatory requirements may come from the European Union General Data Protection Regulation (GDPR) or the New York State Cybersecurity Requirements for Financial Services Companies law.

Data destruction is the process of requesting that data be destroyed, making the data impossible to retrieve. Data destruction is typically initiated with a request that specifies a contact or user whose data is to be destroyed. In the base configuration, the application provides a web service that is intended to be called by an external application. You use the external application to manage the destruction of the data across Guidewire applications.

Data destruction can be implemented in PolicyCenter and ClaimCenter as either *purging* or *obfuscation* of data, depending on the data to be destroyed.

Purging is a form of data destruction that completely removes claim data (from ClaimCenter), policy or account data (from PolicyCenter), or contact data. There can be multiple objects associated with the purged objects that are also removed as they are detected by traversing the entity domain graph.

Obfuscation is a form of data destruction that permanently overwrites fields, such as user contact fields, with data that replaces the original data. Some actual removal of data can also be involved, such as deletion of an address referenced only by one user.

In the base configuration, BillingCenter supports obfuscation of `UserContact` objects and objects associated with `UserContact`, such as `User` and `Credential`. You can extend BillingCenter to obfuscate additional objects associated with a contact, such as accounts and policy terms, and objects detected by traversing the entity domain graph.

Obfuscation might be required if destroying the data affects objects that cannot be destroyed. For example, purging user data for a former employee could affect hundreds or even thousands of claims. Therefore it makes more sense to obfuscate the data for the user and leave the other data alone.

Data destruction configuration parameters

The application does not enable data destruction by default in the base configuration of the application. Use the following configuration parameters in file `config.xml` to configure personal data destruction.

PersonalDataDestructionEnabled

Set this parameter to `true` to enable destruction of personal data. In the base configuration, this parameter is `false`.

Note: PolicyCenter won't start if both `PersonalDataDestructionEnabled` and `DisableDomainGraphSupport` are true.

DataDestructionMaxWaitForBlockingMessagesToFinish

ClaimCenter only. Used by the `DestroyContactForPersonalData` work queue to determine the maximum number of seconds to wait for Solr messages in the queue to clear before retrying personal data destruction purging requests. Personal data destruction requests will retry every second until the queue is clear or the parameter threshold is reached. In the base configuration, this parameter is 60 seconds.

ContactDestructionRequestAgeForPurgingResults

Used by the `RemoveOldContactDestructionRequest` work queue to determine the age of `PersonalDataDestructionRequest` objects that have a status of `Finished`. In the base configuration, this parameter is set to 10 days.

Data model configuration for data destruction

There are data model configurations that apply to the objects being destroyed. Some are general configurations, and some are specific to purging or obfuscation.

DestructionRootPinnable delegate

An entity that implements this delegate gets a `Do Not Destroy` flag that can be checked as part of the destruction process. An entity that is intended to be the root of an entity graph must implement the `DestructionRootPinnable` delegate if it is to be used in personal data destruction.

Root of entity graph in ClaimCenter and PolicyCenter

In ClaimCenter and PolicyCenter, the destruction process uses entity domain graphs to determine what to destroy. An object that implements the `DestructionRootPinnable` delegate and the `RootInfo` delegate is the root of an entity domain graph.

Do Not Destroy flag

A `Do Not Destroy` flag is provided in the `DestructionRootPinnable` delegate. If an entity implements this delegate, instances of the entity have a `DoNotDestroy` Boolean field. The default value of this field is `false`.

Note: If this field is on a `Contact` or a subtype of `Contact`, it is maintained only in the application. Even if the contact is linked, the field is not sent to `ContactManager`, nor is it updated from `ContactManager`.

ClaimCenter

In the base configuration of ClaimCenter, the entities `Contact`, `ContactInfo`, `Claim`, and `ClaimInfo` implement the `DestructionRootPinnable` delegate and therefore have `DoNotDestroy` Boolean fields. The default value of the field is `false`, which permits destruction of the entity. If this field is set to `true`, the entity cannot be purged.

You can set this field only for `Contact`, `ContactInfo`, and `ClaimInfo` objects. Use the `markDoNotDestroy` method to set the field. For example:

```
ClaimInfo.markDoNotDestroy(true)
```

You cannot set this field on a `Claim` object. In the base configuration, the ClaimCenter personal data destruction code does not check this field on a claim.

PolicyCenter

In the base configuration of PolicyCenter, the entities `Contact`, `Account`, `Policy`, `PolicyTerm`, and `PolicyPeriod` all implement the `DestructionRootPinnable` delegate and therefore have `DoNotDestroy` Boolean fields.

You can set this field in an instance of any of these entities, such as a specific contact or policy, by using the method `setDoNotDestroy`. The method takes the following parameters:

Boolean value

Set the flag to `false` to allow destruction of the entity, or to `true` to prevent destruction of the entity.

Callable<String> getDescription

A block that returns either a `String` to be used as a history event comment or `null`.

PolicyCenter checks this flag as part of the destruction process. For example:

- If `DoNotDestroy` is set to `true` for the contact, the associated accounts, policies, and policy periods are not destroyed.
- The contact specified in a personal data destruction request has two accounts, and only one of the accounts has `DoNotDestroy` set to `true`. The other account has `DoNotDestroy` set to `false`. PolicyCenter does a partial purge and deletes the account that has `DoNotDestroy` set to `false`.

BillingCenter

In the base configuration of BillingCenter, the `DoNotDestroy` Boolean field is set to `false` on `Contact`, which is the only entity that implements the `DestructionRootPinnable` delegate. The setter of this property is not available in Gosu.

Do Not Process flag

A Do Not Process flag is provided by the `Operable` delegate. If an entity implements this delegate, the entity then has a `DoNotProcess` boolean field that you can use as needed in your configuration. This flag is not used in the base configuration.

Display keys for data destruction messages

There are a number of display keys defined for use by the personal data destruction code. For example, error messages use display keys. You can see the error messages for U.S. English in Guidewire Studio. Navigate in the **Project** window to **configuration > config > Localizations > Resource Bundle 'display'** and double-click `display.properties` to open this file in the editor. In this file, press `Ctrl+F`, enter the search string `PersonalData`, and then click the down or up arrow button to go to each search result in the file. For example:

```
PersonalData.Error.ObfuscateNotImplemented = Obfuscation for '{0}' is not supported
```

Obfuscatable delegate

If you intend to use obfuscation with an entity, it must implement the `Obfuscatable` delegate. This delegate is necessary to support marking fields as personally identifiable information with the `PersonalData` tag.

Note: A Delegate is a reusable type that defines database columns, an interface, and a default implementation of that interface. A delegate permits an entity to implement an interface while delegating the implementation of that interface to another class, that of the delegate. Each delegate type provides additional columns on the affected tables.

The `Obfuscatable` delegate has one column, `ObfuscatedInternal`, which cannot be set in Gosu code. This limitation exists because after the `ObfuscatedInternal` column has been set to `true`, it must not be set to `false`.

Note: If the parent of an entity implements the `Obfuscatable` delegate, all child entities inherit that implementation.

The `Obfuscatable` delegate extends the interfaces `Obfuscator` and `ObfuscatablePublicMethods`. These interfaces provide the following methods:

markAsObfuscated

Marks the entity instance as having been obfuscated by setting `ObfuscateInternal` to `true` on the delegate. In `BillingCenter`, you cannot call this method in `Gosu`.

isObfuscated

Returns `true` if `obfuscate` has been called and completed successfully. Otherwise returns `false`.

obfuscate

Obfuscates the columns that are marked for obfuscation on this object.

obfuscateSimple

Works the same as `obfuscate`, but in `PolicyCenter` only for objects that are not `EffDated`.

Obfuscator interface

If you intend to use obfuscation with an entity, the entity must implement the `Obfuscator` interface. Each entity that implements the `Obfuscator` interface must also designate an implementation class, such as `UserContactObfuscator`.

Note: If the parent of an entity implements the `Obfuscator` interface, all child entities inherit that implementation, including the implementation class. You can override the parent implementation by declaring `implementsInterface` in the child entity.

For example, in the base configuration, `UserContact.etx` has the following definition:

```
<extension xmlns="http://guidewire.com/datamodel"
  entityName="UserContact">
  <implementsInterface
    iface="gw.api.obfuscation.Obfuscator"
    impl="gw.personaldata.obfuscation.UserContactObfuscator"/>
  <column-override
    name="EmployeeNumber">
    <tag
      name="PersonalData"
      value="ObfuscateDefault"/>
    </column-override>
  </extension>
```

In the base configuration, an entity can implement the `Obfuscatable` delegate but have an `Obfuscator` interface implementation of `UnsupportedObfuscator`. In this case, even if the entity is marked as `obfuscatable`, any attempt to obfuscate it results in an `UnsupportedOperationException`.

To be able to obfuscate an entity, you must use or write a suitable `Obfuscator`, such as one that extends `PersonalDataObfuscator` (for `ClaimCenter` and `BillingCenter`) or `PCPersonalDataObfuscator` (for `PolicyCenter`).

Note

If the configuration of `implementsInterface` is coded incorrectly, then the server startup verification check will not be able to detect the error. The error will be caught when the application is starting up instead.

For example, suppose you coded the `impl` part of `implementsInterface` with a misspelled class name, `OffuscatorImpl2`:

```
<implementsEntity
  name="Obfuscatable"/>
<implementsInterface
  iface="gw.api.obfuscation.Obfuscator"
  impl="gw.api.personaldata.obfuscation.OffuscatorImpl2"/>
```

The server startup verification check cannot detect this error. When the application starts up, it generates a `Class Not Found` exception. If you see this exception on server startup, correct the code in the configuration.

Marking entity fields for obfuscation

If an entity implements the `Obfuscatable` delegate, it is likely that you will want to tag some of its fields for obfuscation. For each field that contains personal data, add the tag `PersonalData`. The value of the tag determines the kind of obfuscation that will be applied to the field's contents. In the base configuration, two values are defined in the `typelist PersonalDataTagValue.tti`:

ObfuscateDefault

The field's contents are replaced with the default value or null if no default value is defined.

ObfuscateUnique

You specify how the field's contents are replaced.

Note: If a field is not nullable, it must be given a default value if you mark it for obfuscation. For one of these fields, obfuscation will fail if you tag the field `obfuscate_default` and provide no default value.

In the base configuration, all the entities that implement the `Obfuscator` interface have fields tagged `PersonalData`. Additionally, many of the fields tagged as `PersonalData` are in `.etx` files to enable you to modify them. For example, the `Name` field of `Contact` is defined in `Contact.etx` as follows:

```
<column-override
  name="Name">
  <tag
    name="PersonalData"
    value="ObfuscateUnique"/>
</column-override>
```

In addition, the following delegates have fields tagged `PersonalData`. An entity that implements one or more of these delegates does not have to implement `Obfuscator` and `Obfuscatable` unless you want the entity to support obfuscation.

- `AddressBookConvertible.etx`
- `GlobalAddress.etx`
- `GlobalAddress.Global.etx`
- `GlobalContactName.etx`
- `GlobalContactName.Global.etx`
- `GlobalPersonName.etx`
- `GlobalPersonName.Global.etx`

BillingCenter entities and fields that support obfuscation

Guidewire recommends that you obfuscate only certain fields in the base configuration entities. In general, do not obfuscate financial related entities and amounts, as well as producer related entities.

IMPORTANT: When you start `BillingCenter`, a routine runs to ensure that only entity fields approved by Guidewire can be tagged as personal data for destruction. The list of fields that you are permitted to obfuscate cannot be configured or extended. Tagging entity fields that are not on this list as personal data for destruction will prevent the server from starting. Custom entities and entity extensions are not included in this list, and you can tag them as needed for destruction.

You can use the **Data Destruction Config** screen on the **Internal Tools** tab (press `Alt+Shift+T` to see which entity fields are on the approved list for obfuscation).

In your own entity extensions, you can obfuscate any of the fields.

IMPORTANT: You must run database consistency checks periodically during development to ensure that your data obfuscation implementation does not introduce database inconsistencies.

See also

- “`BillingCenter Data Destruction Config` screen” on page 247

BillingCenter Data Destruction Config screen

The **Data Destruction Config** screen in `BillingCenter` enables you to:

- Identify the entity fields in the base configuration that you can mark for obfuscation.
- Keep track of the BillingCenter fields that are already marked for obfuscation.

Note: The screen is visible only if the configuration parameter `PersonalDataDestructionEnabled` is set to `true`.

To be able to display the **Data Destruction Config** screen, the server must be in development mode, and you must log in as a user with the `internaltools` permission.

After you log in, press **Alt+Shift+T**, and then click the Internal Tools tab. Then click **Data Destruction Config** in the Sidebar on the left.

- To see the entity fields already marked for obfuscation, click **Personal data tagged fields**.
- To see the base configuration entity fields that you are permitted to mark for obfuscation, click **Personal data tagging whitelist**.

Note: You can mark for personal data obfuscation any entities that you have created or extended.

See also

- “Marking entity fields for obfuscation” on page 246
- “BillingCenter entities and fields that support obfuscation” on page 247
- “Data destruction configuration parameters” on page 243

ContactInfo support for personal data destruction in ClaimCenter

IMPORTANT: Guidewire must enable archiving before the archiving functionality is available.

The `ContactInfo` entity can retain data about contacts that have been archived. This entity supports locating contacts that need to be purged that are live and those that are archived, identified by either `PublicID` or `AddressBookUID`.

Fields of `ContactInfo` that provide this support are:

ArchivedPublicID

Records the `PublicID` of an archived contact.

ArchivedAddressBookUID

Records `AddressBookUID` of the archived contact. This value is `null` if the contact is not linked.

When personal data destruction is enabled, as a claim is being archived:

- The `PublicID` and `AddressBookUID` values are stored in `ContactInfo`.
- All contacts on a claim have a corresponding `ContactInfo` entity created.

`ContactInfo` also remains backwards compatible with existing user interface pages for archiving. As a result, a contact can have more than one `ContactInfo` object generated for it if the contact is both an insured party and a claimant.

If personal data destruction is not enabled, `ContactInfo` is a denormalized entity that stores simple fields, such as a contact's first and last names, role, and some other simple data. Additionally, only contacts that have the role of Insured or Claimant have `ContactInfo` data created. This data can be used for simple searches and other user interface purposes.

Entity domain graphs

PolicyCenter and ClaimCenter

PolicyCenter and ClaimCenter use entity domain graphs to define the aggregate clusters of associated objects that are treated as a single unit for purposes of data destruction. Each aggregate cluster has a root and a boundary.

Note: There are multiple entity domain graphs in PolicyCenter. Therefore, PolicyCenter uses a pinnable hierarchy to determine the order in which a personal data destruction request uses the domain graphs.

- The *root* is a single specific entity that the aggregate cluster contains. The root entity is the main entity in the graph. A root entity is application-specific and must implement `DestructionRootPinnable`. *Pinnable root* is another term for one of these root entities.

In ClaimCenter, the root entity is `ClaimInfo`.

In PolicyCenter, the root entities are `Policy`, `Account`, `PolicyTerm`, `PolicyPeriod`, `PolicyTerm`, and `Contact`.

- The *boundary* defines what is inside the aggregate cluster of objects. It identifies all the entities that are part of the graph.

In ClaimCenter, the boundary defines the entities that relate to a `ClaimInfo` object, such as `Claim` and its related `Exposure`, `Coverage`, `Matter`, and other similar objects.

In PolicyCenter, the boundary defines the entities that relate to an `Account` or `Policy`, such as jobs and effective dated fields, or to a `Contact`, such as an address.

The unit of work for personal data destruction is a single instance of the domain graph, such as a single `ClaimInfo` object (ClaimCenter) `Account` (PolicyCenter), and all their associated entities.

To enforce the boundaries of the domain graph, all objects participating in the destruction process must implement the following delegate:

- **Extractable** – All entities in the domain graph must implement the `Extractable` delegate.

You cannot define a new entity domain graph for use in personal data destruction. However, you can configure an existing entity domain graph in the same way that you configure the archiving domain graph. For example, use the `archivingOwner` data model attribute and the `CrossDomainPublicID` data model tag. If personal data destruction is enabled, then at server startup, if there are validation issues with any entity domain graph, those issues are logged and the server fails to start. This process is similar to how archiving domain graph validation works.

BillingCenter

BillingCenter does not require the definition of an entity domain graph on the data model to allow for data destruction. To cascade a data destruction request through a set of dependent entities, you can make subsequent Gosu obfuscation calls on the child entities directly.

For example, you can define that an account and its dependencies be considered as a single unit of work. They could either all be obfuscated together or none be obfuscated at all. To implement this example:

1. Change `Account.etx` to implement `DestructionRootPinnable`.
2. Add a `shouldDestroyAccount` method to `PersonalDataDestructionForBCRootsPlugin`.
3. Override `shouldDestroyRoot` on the `BCPersonalDataDestructionPlugin`, making it call out to `shouldDestroyAccount` if the `DestructionRootPinnable` is an `Account`.
4. After a contact destruction request is passed to the destroyer, find all accounts the `Contact` is associated with. For each account:
 - a. Call `shouldDestroyAccount` to determine if the given account can be destroyed (only if the account is closed, for example).
 - b. Call `account.obfuscate` for each of the identified accounts.

In this scenario, you would create a Gosu implementation of the `Obfuscator` interface for the `Account` entity, extending the `DefaultPersonalDataObfuscator` class. Your implementation would cascade the obfuscation down to all the owned policy periods, for example.

Note: This example abstracts a lot of implementation details, such as error handling and avoiding cyclical obfuscation cascades, among others.

Identifying archived objects that affect destruction of data

IMPORTANT: Guidewire must enable archiving before the archiving functionality is available.

BillingCenter

In BillingCenter, if you implement obfuscation and you use archiving, you need to identify archived objects.

Note: In the base configuration, no entities that are part of the BillingCenter archiving graph are obfuscated, and so archive references are not used. If you fully implement personal data destruction, you must account for archived data, which can be identified through the use of archive references. For example, if a given invoice has invoice items pointing back to it. Calling `ArchiveDocumentReferencesUtil.isReferencedFromArchiveDocument(invoice)`, passing an account invoice as a parameter, will return true if the given invoice has archived invoice items. If you find archived references, you must restore the archived data, obfuscate it, and then archive it again.

If there are references from any archived documents to any object instances that are being obfuscated, the objects cannot be destroyed until the archived objects are restored from the archive. Therefore BillingCenter requires that a table of archived objects be built and maintained to use in determining if references from archived objects exist.

If you implement obfuscation that involves objects that are already archived in an existing installation of BillingCenter, you must run the `ArchiveReferenceTrackingSync` batch process. Run this process once to build the initial table of archived objects.

ClaimCenter

Destroying personal data can require navigating the entity domain graphs and possibly purging objects other than the root objects. Therefore, it is possible that a purge can be affected by archived objects. For example, purging a contact's data affects several claims, one of which is archived.

If there are references from any archived documents to any object instances in the entity graph, the objects cannot be destroyed until the archived objects are restored from the archive. Therefore ClaimCenter requires that a table of archived objects be built and maintained to use in determining if references from archived objects exist.

If you already have archived claims in an existing installation of ClaimCenter, you must run the `ArchiveReferenceTrackingSync` batch process once to build the initial table of archived objects.

Additionally, previously existing archived claims do not possess sufficient data for personal data removal and need to be updated. These existing archived claims have `ContactInfo` data that lacks `ArchivedPublicID` and `ArchivedAddressBookUID` information, and that data exists only for contacts in a claimant or insured role. Running `ArchiveReferenceTrackingSync` updates the `ContactInfo` data for contacts of all archived claims.

PolicyCenter

Destroying personal data can require navigating the entity domain graphs and possibly purging objects other than the root objects. Therefore, it is possible that a purge can be affected by archived objects. For example, purging a contact's data affects several policy periods, one of which is archived.

If there are references from any archived documents to any object instances in the entity graph, the objects cannot be destroyed until the archived objects are restored from the archive. Therefore PolicyCenter requires that a table of archived objects be built and maintained to use in determining if references from archived objects exist.

If you already have archived policy periods in an existing installation of PolicyCenter, you must run the `ArchiveReferenceTrackingSync` batch process once to build the initial table of archived objects.

Running the ArchiveReferenceTrackingSync process

IMPORTANT: Guidewire requires that you run the `ArchiveReferenceTrackingSync` process to use the personal data destruction feature. Running this process can slow your system noticeably. Guidewire recommends that you run the process before deploying the upgraded Guidewire application.

Batch process `ArchiveReferenceTrackingSync` adds entries to a reference table for every archived object that does not yet have an entry.

In the base configuration, the application calls the following method to determine if the `ArchiveReferenceTrackingSync` batch process has run:

```
DataDestructionParameterCheck.verifyPersonalDataDestructionEnabled
```

The application calls this method from the following work queues:

PersonalDataContactDestructionWorkQueue

The class that implements the `DestroyContactForPersonalData` work queue.

PersonalDataDestructionNotifyExternalSystemsWorkQueue

The class that implements the `NotifyExternalSystemForPersonalData` work queue.

After you run this process:

BillingCenter

As you archive policy periods, `BillingCenter` adds the archive objects to the reference table of archived objects. You can configure this table to be checked whenever a personal data destruction request is initiated.

ClaimCenter

As you archive claims, `ClaimCenter` adds the archived objects to the reference table of archived objects. `ClaimCenter` checks this table whenever a personal data destruction request is initiated.

PolicyCenter

As you archive policy periods, `PolicyCenter` adds the archived objects to the reference table of archived objects. `PolicyCenter` checks this table whenever a personal data destruction request is initiated.

Utility class `gw.api.archiving.ArchiveDocumentReferencesUtil` provides methods for determining the following:

- If an object or set of objects is referred to by an archived document.
- Which archived documents refer to a given object or set of objects.

Data Protection Officer

A Data Protection Officer is expected to be available to handle problems with data destruction. Therefore, in the base configuration, there is a Data Protection Officer role to which users can be assigned.

Data Protection Officer permissions

In the base configuration of the application, the Data Protection Officer role has the following permissions relating to personal data destruction:

- `requestcontactdestruction`
- `editobfuscatedusercontact`

In `PolicyCenter` and `BillingCenter`, a user named DataProtection Officer with login `dpofficer`, which has the Data Protection Officer role, is available in the sample data.

In `ClaimCenter`, this role also has additional permissions to enable the user to work with users and groups. These permissions include `groupcreate`, `groupdelete`, `groupedit`, `usereditattrs`, `usereditlang`, `useredit`, `usergrantroles`, `userviewall`, `grouptreereview`, `groupreview`, and `userview`.

In the base configuration, the application screens prevent a user from editing obfuscated user contacts if the user does not have permissions to do so. In addition, the application prevents a user without the correct permissions from adding obfuscated user contacts to or removing them from groups and roles. You can create additional permissions and configure the application to further limit editing of obfuscated objects.

Notifying the data protection officer

The PersonalDataDestruction plugin interface provides a method that enables notification of the Data Protection Officer, `notifyDataProtectionOfficer`.

ClaimCenter

In the base configuration, the class that implements the PersonalDataDestruction plugin interface, `CCPersonalDataDestructionSafePlugin`, overrides the `notifyDataProtectionOfficer` method. In this class, the `notifyDataProtectionOfficer` method logs messages to the system console if a destruction request fails. You can configure where these messages are logged as needed.

PolicyCenter

In the base configuration, the class that implements the PersonalDataDestruction plugin interface, `PCPersonalDataDestructionPlugin`, overrides the `notifyDataProtectionOfficer` method and notifies data protection officers when a destruction request has failed.

The action performed by the `notifyDataProtectionOfficer` method is defined in the class `NotifyDataProtectionOfficerVisitor`. This class creates an activity that uses the activity pattern `personal_data_destruction_error` and assigns the activity to each user present in the system that has the Data Protection Officer role. If no users are found with the Data Protection Officer role, in the base configuration, an exception is thrown.

Note: PolicyCenter can purge retired pinnable roots from the system. However, if there are any issues that occur while purging this data, an activity will be created against the retired root. This activity will be listed in the user interface for users with the Data Protection Officer role. However, the user will not be able to view the details of the activity because the root is retired.

BillingCenter

In the base configuration, the class that implements the PersonalDataDestruction plugin interface, `BCPersonalDataDestructionPlugin`, overrides the `notifyDataProtectionOfficer` method. This method notifies data protection officers when an asynchronous obfuscation request has failed to execute. It is called in the base configuration when `shouldDestroyUser` returns `MUST_NOT_DESTROY` for a user destruction request. You must call it when implementing your system-wide personal data destruction strategy. You can use the method in both synchronous and asynchronous personal data destruction.

The action performed by the `notifyDataProtectionOfficer` method is defined in the class `DataProtectionOfficerNotifier`. In the base configuration, if personal data destruction is implemented, this class verifies that it is a `UserContact` that is being destroyed. If so, the class uses the activity pattern `personal_data_destruction_error` to create an activity and assign it to each user present in the system that has the Data Protection Officer role. In the base configuration, if the activity is created and no users are found with the Data Protection Officer role, an exception is thrown.

Data destruction purge configuration

IMPORTANT: Data destruction proceeds regardless of the work items that might be pointing to an object to be destroyed. If you have a work item in progress, ensure that it is complete or removed before any object used by the work item is destroyed.

ClaimCenter

Purging is the process of completely removing data from the ClaimCenter database.

- A purge can start with a claim and proceed through the claim graph to include associated objects. That type of purge is generally associated with claims that are a certain age and no longer need to be preserved or archived. That type of purge is described in the *InsuranceSuite Database Administration*.

- A purge can start with a contact and proceed to one or more claims. That type of purge is called *personal data destruction*. There can be multiple objects associated with the claim or contact that are also removed as they are detected by traversing the entity domain graph. Configuring that type of purge is described in the topics that follow.

PolicyCenter

Purging is the process of completely removing contact, account, policy, policy term, and policy period data from the PolicyCenter database. There can be multiple objects associated with a contact, account, policy, policy term, and policy period that are also removed as they are detected by traversing the entity domain graph.

BillingCenter

To destroy data, BillingCenter requires that you obfuscate data and not *purge* data—remove data from the database. In the base configuration, obfuscation is provided only for `UserContact` and associated objects, such as `User` and `Credential`. You can implement obfuscation in BillingCenter that uses the Destroyer API, the personal data destruction plugin, and code that cascades a data destruction request through a set of dependent entities.

Purging and the ClaimCenter entity domain graph

In the base configuration of ClaimCenter, methods of the class `PersonalDataPurger` call ClaimCenter methods that traverse the `ClaimInfo` graph. These methods can purge objects that are on the graph and objects that are related to the claim but not on the graph, such as `MessageHistory` objects.

Note: Claim purging is a feature of ClaimCenter used by personal data destruction, but exists separately for destruction of claims that have aged out or are incomplete.

The following `PersonalDataPurger` methods are called from `CCPersonalDataDestroyer` and can also be called from Gosu classes:

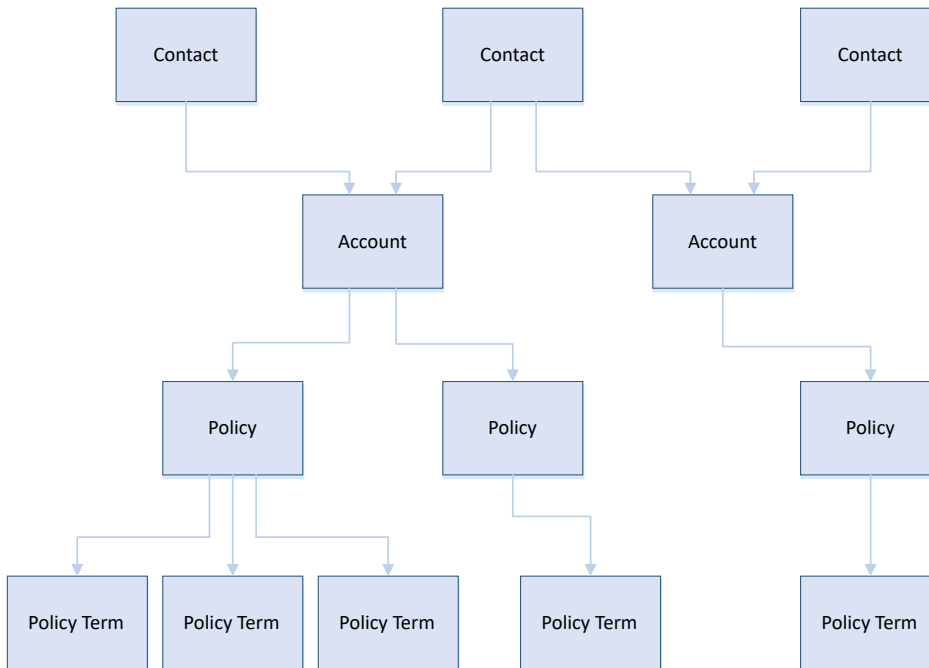
```
public static void purge(entity.ClaimInfo claimInfo)
public static void purge(entity.Claim claim)
```

Pinnable hierarchy in PolicyCenter

In PolicyCenter, because there are multiple entity domain graphs, it is necessary to use a hierarchy of domain graphs in PolicyCenter. The PolicyCenter pinnable hierarchy defines how entity domain graphs interact.

Each pinnable root defines its parents and children in an adapter. For PolicyCenter, all the pinnable roots can have multiple children. For parents, all the pinnable roots have only one parent except for account, which can be associated with multiple contacts.

The following figure shows an example of a pinnable hierarchy of domain graphs. Each box in the diagram is the root of a domain graph. Depending on how many contacts are on the account or how many accounts a contact is on, or both, the objects in the diagram could change. However, the order of the hierarchy would be the same.



PersonalDataPurgeTree class in PolicyCenter

In PolicyCenter, entity domain graphs enable purging of Contact, Account, Policy, PolicyTerm, and PolicyPeriod objects. The class `PersonalDataPurgeTree` creates a tree that stores the relationship of the relevant nodes to determine if the specified pinnable root can or cannot be purged.

The `PinnableDomainMethods` class defines methods that can be used to find parents, children, and descendants of the current object in the tree, like `getPinnableChildren`, `getPinnableDescendants`, and `getPinnableParents`.

The purge API can then be called, and best effort purging is performed. What gets purged is determined by the results of multiple passes through the domain graphs configured for each pinnable root. Purging is determined by the domain graphs configured for each pinnable root. The Account domain graph is a super-set of the Policy domain graph, and the Policy domain graph is a super-set of the PolicyTerm domain graph. However, the Contact domain graph intersects with the other domain graphs, but is not a super-set of the Account domain graph.

Note: Even though policy period and policy term are pinnable roots, in the base configuration, PolicyCenter does not provide the ability to purge them directly. Personal data purge must get to them through a higher level graph in the pinnable hierarchy.

The tree is constructed by using the pinnable hierarchy to determine the parents and children of each pinnable node. Starting from the specified start node, all its ancestors are included. For each of the descendants, if there are multiple parents, such as an account with multiple contacts, those parents will be included in the graph. Multiple parents are included because the ability of the parents to be destroyed can affect the start node and the purgeability of the children.

Purgeability for each node in the tree is then computed by performing a multi-step process for evaluating the nodes in the tree. Values are propagated to the appropriate ancestors, descendants, and siblings.

In the base configuration, `PolicyPeriod` and `PolicyTerm` are not destroyable if they have a sibling that is not destroyable.

Domain graphs are nested for Account, Policy, PolicyTerm, and PolicyPeriod. Consequently, purging at the account level removes roots below it and does not require an explicit call to each of the pinnable roots below the account. Based on the tree and ability of each node to be destroyed, the minimum set of roots necessary to purge all destroyable roots is calculated and stored in the tree.

The main place that you control the objects that can be purged is in your implementation of `PersonalDataDestructionPlugin`. In the class, you define `MUST_NOT_DESTROY`, `MAY_DESTROY`, and `MUST_DESTROY` return values for `shouldDestroyObject` methods.

The result of this process is one of the following values defined in `PersonalDataPurgeStatus`:

- `CompletePurgeExecuted`
- `PartialPurgeExecuted`
- `NothingPurged`
- `ExceptionThrownOnPurge`
- `PurgeHasNotBeenAttempted`

Traversing the pinnable hierarchy in PolicyCenter for personal data purge

In PolicyCenter, to identify entities that can be purged, PolicyCenter makes multiple passes through the pinnable hierarchy. PolicyCenter establishes the initial purge graph and then handles both the disposition of each node, like `MUST_NOT_DESTROY`, and any Do Not Destroy flags. Based on these passes through the pinnable hierarchy, PolicyCenter can perform a partial purge if necessary and maintain data integrity.

In general, the steps involve:

1. Starting with the root of the purge request, form a graph by traversing up and down the pinnable hierarchy.
2. Compute disposition of the root node and all descendants of the root. Exclude entities marked `MUST_NOT_DESTROY` and their ancestors.
3. Propagate the Do Not Destroy flag of any entity discovered while building the initial graph.
4. Purge entities that remain purgeable.
5. Report conflicts to the Data Protection Officer. For example, an entity that is marked `MUST_DESTROY` is not purgeable because it has a descendant with Do Not Destroy flag that is set.

Discovering the entity purge graph

In this initial step of the purge process, PolicyCenter makes an initial traversal through the pinnable hierarchy to discover the graph on the basis of which purge logic will execute. PolicyCenter traverses the relationships between contacts, accounts, policies, policy terms, and policy periods.

This traversal starts at the initial root of the personal destruction request and marks each entity as purgeable. It uses the following logic:

- Contact traverses to all related Account entities.
- Account traverses to all related Policy entities and all related Contact entities (primary and non-primary).
- Policy traverses to all related PolicyTerm entities.
- PolicyTerm traverses to all related PolicyPeriod entities.

Traversing the purge tree to determine disposition

This second step of purge graph traversal identifies entities that are eligible for purging. PolicyCenter excludes from purging entities that have a `MUST_NOT_DESTROY` disposition and entities whose presence is required to preserve data consistency. For an example of the latter, an entity in the graph that has a descendant that is not purgeable is excluded in this pass.

The traversal starts from the initial root of the personal destruction request and visits all descendant nodes.

For each visited node, PolicyCenter invokes disposition logic and marks the node not purgeable if any of the following are true:

1. The node has a disposition of `MUST_NOT_DESTROY`.

2. The node is an ancestor of a node that has a disposition of `MUST_NOT_DESTROY`.
3. The node is a `PolicyPeriod` or `PolicyTerm` sharing a `Policy` with another `PolicyPeriod` or `PolicyTerm` that has a disposition of `MUST_NOT_DESTROY`. For example, two pairs of `PolicyPeriod` and `PolicyTerm` objects are children of a `Policy` and are adjacent nodes to one another in the graph.
Sets of related `Policy`, `PolicyTerm`, and `PolicyPeriod` objects are kept or purged together. If a single object in the set has a disposition of `MUST_NOT_DESTROY`, the entire set is kept. For example, a single `PolicyTerm` that has a disposition of `MUST_NOT_DESTROY` prevents the purge of any `Policy`, `PolicyTerm`, and `PolicyPeriod` objects in the set.

Propagating the Do Not Destroy flag

In this step of the purge graph traversal, PolicyCenter processes any Do Not Destroy flags on entities in the hierarchy. A Do Not Destroy flag on `Contact`, `Account`, and `Policy` propagates to all ancestors and descendants, but not to siblings. A Do Not Destroy flag on `PolicyTerm` and `PolicyPeriod` does propagate to sibling `PolicyTerm` and `PolicyPeriod` objects that share a `Policy`.

The detailed traversal logic follows.

1. Mark nodes with a Do Not Destroy flag as not purgeable.
2. Mark all descendant nodes of nodes with a Do Not Destroy flag as not purgeable.
3. Mark all ancestor nodes of nodes with a Do Not Destroy flag as not purgeable.
4. Mark as not purgeable all sibling `PolicyPeriod` and `PolicyTerm` objects sharing a `Policy` with a `PolicyPeriod` or `PolicyTerm` object that is marked Do Not Destroy.

Sets of related `Policy`, `PolicyTerm`, and `PolicyPeriod` objects are kept or purged together. If a single object in the set has a Do Not Destroy flag, the entire set is kept. For an example, a single `PolicyTerm` with a Do Not Destroy flag prevents the purge of any `Policy`, `PolicyTerm`, and `PolicyPeriod` in the set.

Purging eligible entities

After the purge graph has been built and all nodes have been marked as needed, PolicyCenter purges eligible entities as follows:

1. Purge all purgeable nodes and associated non-root entities.
2. When purging a policy, if the `AccountContactRole` entity and the `AccountContact` entity on the account associated with the policy have been orphaned, they are purged also.
Contacts whose only link to the graph is the policy being purged are unlinked from the graph in this step. If the contact was marked not purgeable as a result of propagation of the Do Not Destroy flag, the contact becomes purgeable again. If the contact had a Do Not Destroy flag that was directly set, that flag is left alone and the contact remains not purgeable.
3. Purge any contacts that no longer have links with the graph and have a disposition other than `MUST_NOT_DESTROY`.

Reporting purge tree conflicts

Entities with a disposition of `MUST_DESTROY` might remain after the final step of the purge process. Typically these entities are not purged for data integrity reasons. For example, a descendant of an entity has a `MUST_NOT_DESTROY` disposition.

PolicyCenter reports these entities to the Data Protection Officer by using an activity.

Defining the destroyer

ClaimCenter

The `CCPersonalDataDestroyer` class implements the `PersonalDataDestroyer` interface and overrides the `PersonalDataDestroyer` interface methods that determine how a destruction request is carried out. The class provides methods that are called by the `PersonalDataDestructionAPI` web service and its work queues.

A personal data destruction request to delete a contact by AddressBookUID can correspond in ClaimCenter to many contacts with different PublicID values on many claims. The method `translateABUIDtoPublicIDs` finds all these PublicID values and returns them to the web service, and then the web service makes a series of `destroyContact` method calls to delete each PublicID.

If a personal data destruction request specifies deleting a contact by PublicID, there can be at most one corresponding claim, so the processing by the web service is simpler.

The following two methods are the primary Destroyer methods used by the web service.

`translateABUIDtoPublicIDs`

This public method overrides the `PersonalDataDestroyer` interface method `translateABUIDtoPublicIDs`. It finds all the contacts related to the AddressBookUID, and returns a list the PublicID values for those contact.

The method is called by the web service `PersonalDataDestructionAPI`. The web service uses the method to determine if an AddressBookUID exists and to get the PublicID if the original destruction request was specified by AddressBookUID.

`destroyContact by destruction request`

This public method overrides the `PersonalDataDestroyer` interface method `destroyContact`. It is the main entry point for destroying contacts.

This method takes a `PersonalDataDestructionRequest` and finds the corresponding `Contact` and its `Claim` from the registered PublicID. For an archived contact, the method uses the registered PublicID to find the `ContactInfo` and its associated `ClaimInfo`.

Once the `Contact` and `Claim` are found, a call to the `PersonalDataDestruction` plugin is made to determine whether the `Contact` and `Claim` can be purged.

- If the result of the plugin call is `MUST_NOT_DESTROY`, the request is not processed for destruction. The reason is logged in the `DATA_DESTRUCTION_REQUEST` logger and the Destroyer returns the status `ContactDestructionStatus.TC_NOTDESTROYED`.
- If the result of the plugin call is `MAY_DESTROY` or `MUST_DESTROY`, the request is processed for destruction and `PersonalDataPurger.purge(claim)` is invoked.
 - If the purge is successful the Destroyer returns the status `ContactDestructionStatus.TC_COMPLETED`.
 - If there is a purge error, the Destroyer returns the status `ContactDestructionStatus.TC_NOTDESTROYED` and the error or exception is logged in the `DATA_DESTRUCTION_REQUEST` logger.

It is possible that the contact was previously purged and cannot be found. For example, a previous `PersonalDataDestructionRequest` resulted in the `Contact` being purged. In that case, the method returns `ContactDestructionStatus.TC_COMPLETED`.

PolicyCenter

The `PCPersonalDataDestroyer` class extends `PinnablePersonalDataDestroyer`, which implements the `PersonalDataDestroyer` interface. `PCPersonalDataDestroyer` overrides methods that determine how a destruction request is carried out.

In the base configuration, the `PCPersonalDataDestroyer` class destroys contacts, accounts and policies. If a contact is of type `UserContact`, the contact is obfuscated by calling `user.obfuscate`. Otherwise the class uses the internal implementation of `PersonalDataDestroyer` that purges the entity graph of the contact, account, or policy.

In the base configuration, `PCPersonalDataDestroyer` provides the destroyer that is obtained by the `PCPersonalDataDestruction` plugin implementation class call to `getDestroyer`. `PCPersonalDataDestruction` is registered by default in the `PersonalDataDestruction` and `PersonalDataDestructionForPCRootsPlugin` plugin registries.

BillingCenter

The `BCPersonalDataDestroyer` class implements the `PersonalDataDestroyer` interface and overrides the `PersonalDataDestroyer` interface methods that determine how a destruction request is carried out. The class provides methods that are called by the `PersonalDataDestructionAPI` web service and its work queues.

In the base configuration, the `destroyContact` method throws `NotImplementedException`, which prevents destruction of a contact by a personal data destruction request from the web service. If you want to implement obfuscation of contacts and associated objects, you must implement this method.

For example, for a very simple data destruction request, you might find policies and accounts related to a given contact. You might then obfuscate the associated account number and name, policy number, and registered payment instruments, in addition to the contact entities.

PersonalDataPurge event in ClaimCenter and PolicyCenter

When a personal data purge of certain objects is committed, the application generates a `PersonalDataPurge` event that you can respond to in an Event Fired rule to send a message. For example, you might want to send a message to a downstream system.

PolicyCenter generates a `PersonalDataPurge` event when a `Contact`, `Account`, `Policy`, `PolicyTerm`, or `PolicyPeriod` purge is committed as part of a personal data purge. The EventFired rule `PersonalDataPurge`, in the EventMessage rule set category, creates a message in response to the firing of the `PersonalDataPurge` event. This message actually has no transport or destination, but you can define them or create your own rule.

ClaimCenter generates a `PersonalDataPurge` event when a `ClaimInfo` purge is committed as part of a personal data purge. In the base configuration, there is no rule that responds to this event, but you can create an EventFired rule in the EventMessage rule set category that sends a message.

In ClaimCenter, your rule could take the following form in the Guidewire Studio Rules editor:

```
USES:
    uses gw.api.locale.DisplayKey

CONDITION (messageContext : entity.MessageContext):
    return messageContext.EventName == "PersonalDataPurge"

ACTION (messageContext : entity.MessageContext, actions : gw.Rules.Action):
    var claimInfo = messageContext.Root as ClaimInfo
    messageContext.createMessage(
        "Claim with number ${claimInfo.ClaimNumber} has been successfully purged")
```

Specifying which objects in the graph can be destroyed

The `PersonalDataDestruction` plugin interface provides basic methods that define which objects can be destroyed and how to contact the Data Protection Officer. You define a plugin implementation class to define this behavior and register it in the `PersonalDataDestruction.gwp` plugin registry.

PersonalDataDestruction plugin interface

This interface provides the following methods for implementation by the application plugin implementation class:

```
PersonalDataDisposition shouldDestroyRoot(
    Pinnable root, Collection<Pinnable> descendants, Pinnable origin);
PersonalDataDisposition shouldDestroyUser(UserContact userContact);
void notifyDataProtectionOfficer(Pinnable root, String title, String message, Date dateOfError);
void notifyExternalSystemsContactHasBeenPurged(
    String AddressBookUID, String requestor, String requestID);
PersonalDataPurgeContext createContext(PersonalDataPurgeContext context);
void prepareForPurge(PersonalDataPurgeContext context);
void postPurge(PersonalDataPurgeContext context);
PersonalDataDestroyer getDestroyer();
```

ClaimCenter

In the base configuration, ClaimCenter registers the `CCPersonalDataDestructionSafePlugin` class as the default class that implements `PersonalDataDestruction`, which prevents destruction of any of the pinnable root entities. A more realistic starting point for your purge definition is the `CCPersonalDataDestructionSamplePlugin` class.

PolicyCenter

In the base configuration, PolicyCenter registers the `PCPersonalDataDestructionPlugin` class as the default class that implements `PersonalDataDestruction`, which prevents destruction of any of the pinnable root entities. A more realistic starting point for your purge definition is the `SamplePersonalDataDestructionPlugin` class.

BillingCenter

In the base configuration, BillingCenter registers the `BCPersonalDataDestructionPlugin` class as the default class that implements `PersonalDataDestruction`, which prevents destruction of the User entity. This class extends `AbstractPersonalDataDestructionPlugin`.

In the class `AbstractPersonalDataDestructionPlugin`, BillingCenter overrides the following methods and throws `UnsupportedOperationException` for each of them because purging data is not supported.

```
PersonalDataPurgeContext createContext(PersonalDataPurgeContext context);
void prepareForPurge(PersonalDataPurgeContext context);
void postPurge(PersonalDataPurgeContext context);
```

Personal data destruction plugin implementation classes

Personal data destruction plugin implementation classes for ClaimCenter

In the base configuration of ClaimCenter, the `CCPersonalDataDestructionSafePlugin` class is registered as the class that implements the `PersonalDataDestruction` plugin interface. This class provides default handling for destruction of pinnable root entities in the base configuration. It prevents destruction of any personal data.

`CCPersonalDataDestructionSamplePlugin` is the class you can use as an example when you implement your own personal data destruction class to define both `getDestroyer` and how specific pinnable roots are handled. You must then register your implementation class with the plugin registry `PersonalDataDestruction.gwp`.

These two classes define methods that control destruction of pinnable root entities by returning one of the following values defined in the enum `PersonalDataDisposition`:

MUST_NOT_DESTROY

The object must not be destroyed. If this value is in conflict with a `MUST_DESTROY` value in the domain graph, the Data Protection Officer must get involved.

MUST_DESTROY

The object must be destroyed.

MAY_DESTROY

The object can be destroyed.

CCPersonalDataDestructionSafePlugin

In the base configuration, `CCPersonalDataDestructionSafePlugin` calls `getDestroyer` to obtain the destroyer defined in `CCPersonalDataDestroyer`. Additionally, this class prevents data destruction by returning `MUST_NOT_DESTROY` for all calls to destroy pinnable root entities. For example:

```
override function shouldDestroyRoot(
    root: DestructionRootPinnable,
    descendants: Collection<DestructionRootPinnable>,
    origin: DestructionRootPinnable): PersonalDataDisposition {
    CCPersonalDataLogUtil.logNotDestroyed(
        root, "Safe plugin implementation always returns MUST_NOT_DESTROY")
    return MUST_NOT_DESTROY
}
override function shouldDestroyUser(
```

```

    userContact: UserContact): PersonalDataDisposition {
        CCPersonalDataLogUtil.logNotDestroyed(
            userContact, "Safe plugin implementation always returns MUST_NOT_DESTROY")
        return MUST_NOT_DESTROY
    }

```

CCPersonalDataDestructionSamplePlugin

You can use the class `CCPersonalDataDestructionSamplePlugin` as a guide for writing your own personal data destruction code. This class has examples that can return values other than `MUST_NOT_DESTROY` for a pinnable root entity.

Some examples:

- The class respects the setting of the `DoNotDestroy` field. For example, it does not destroy a `Contact` if `DoNotDestroy` is set to `true`.
- If a claim has been closed for at least two years, it can be purged.
- If a claim is archived, it can be purged.
- If a claim cannot be found, `MustNotDestroy` is returned. For example:
 - The `Contact` is a `BulkInvoice` payee.
 - The `Contact` exists only as a related contact without its own role on a claim.
 - The `Contact` is an *orphan*—has no connection to a claim or any other object.

See also

- “Do Not Destroy flag” on page 244

Personal data destruction plugin implementation classes for PolicyCenter

In the base configuration of PolicyCenter, the `PCPersonalDataDestructionPlugin` class is registered as the class that implements both the `PersonalDataDestruction` plugin interface and the `PersonalDataDestructionforPCRoots` plugin interface. This class provides default handling for destruction of pinnable root entities in the base configuration.

`SamplePersonalDataDestructionPlugin` is the class you can use as an example when you implement your own personal data destruction class to define both `getDestroyer` and how specific pinnable roots are handled. You must then register your implementation class with both plugin registries, `PersonalDataDestruction.gwp` and `PersonalDataDestructionforPCRoots.gwp`.

These two classes define methods that control destruction of pinnable root entities by returning one of the following values defined in the enum `PersonalDataDisposition`:

MUST_NOT_DESTROY

The object must not be destroyed. If this value is in conflict with a `MUST_DESTROY` value in the domain graph, the Data Protection Officer must get involved.

MUST_DESTROY

The object must be destroyed.

MAY_DESTROY

The object can be destroyed.

PCPersonalDataDestructionPlugin

In the base configuration, `PCPersonalDataDestructionPlugin` calls `getDestroyer` to obtain the destroyer defined in `PCPersonalDataDestroyer`. Additionally, this class prevents data destruction by returning `MUST_NOT_DESTROY` for all calls to destroy pinnable root entities. For example:

```

override function shouldDestroyPolicyTerm(
    policyTerm: PolicyTerm, descendants: Collection<DestructionRootPinnable>,
    origin: DestructionRootPinnable): PersonalDataDisposition {
    return MUST_NOT_DESTROY
}
override function shouldDestroyPolicy(

```

```

policy: Policy, descendants: Collection<DestructionRootPinnable>,
origin: DestructionRootPinnable): PersonalDataDisposition {
    return MUST_NOT_DESTROY
}
}
override function shouldDestroyAccount(
    account: Account, descendants: Collection<DestructionRootPinnable>,
    origin: DestructionRootPinnable): PersonalDataDisposition {
    return MUST_NOT_DESTROY
}
}
override function shouldDestroyContact(
    contact: Contact, descendants: Collection<DestructionRootPinnable>,
    origin: DestructionRootPinnable): PersonalDataDisposition {
    return MUST_NOT_DESTROY
}
}

```

SamplePersonalDataDestructionPlugin

You can use the class `SamplePersonalDataDestructionPlugin` as a guide for writing your own personal data destruction code.

`SamplePersonalDataDestructionPlugin` has examples that use other return values than `MUST_NOT_DESTROY` for the pinnable root entities. For example:

The method `shouldDestroyUser` determines if there is a `User` object associated with the `UserContact` object. If not, it returns `MUST_DESTROY`. If the database query indicates that the user's credential is active, the method returns `MUST_NOT_DESTROY`. Otherwise, the credential is not active and destroying the `UserContact` is permitted, so the method returns `MAY_DESTROY`.

The method `shouldDestroyPolicy` checks:

- If the policy is retired. If so, the method returns `MUST_DESTROY`.
- A number of scenarios that would prevent the policy from being destroyed, and returns `MUST_NOT_DESTROY` if any of them are true, such as:
 - Any open activities
 - Any policy terms marked `MUST_NOT_DESTROY`
 - Any pinnable dependents marked `MUST_NOT_DESTROY`
- If any pinnable dependents are marked `MUST_DESTROY`. If so, the method returns `MUST_DESTROY`.
- Returns a default value of `MAY_DESTROY` if the previous checks show that nothing is marked `MUST_NOT_DESTROY` or `MUST_DESTROY`.

There are additional overridden methods for `shouldDestroyPolicyTerm`, `shouldDestroyAccount`, and `shouldDestroyContact` that you can review to see how these pinnable root entities might be handled.

Personal data destruction plugin implementation classes for BillingCenter

The `BCPersonalDataDestructionPlugin` class extends `AbstractPersonalDataDestructionPlugin`, which implements the interfaces `PersonalDataDestroyer` and `PersonalDataDestructionForBCRootsPlugin`.

In the base configuration, the `BCPersonalDataDestructionPlugin` class prevents destruction of a `User` by returning `MUST_NOT_DESTROY` in the `shouldDestroyUser` method. You can change this method to return a different value under circumstances that you define.

The `BCPersonalDataDestructionPlugin` class includes the enum `PersonalDataDisposition`, which defines the following possible return values for a method like `shouldDestroyUser`:

MUST_NOT_DESTROY

The object must not be destroyed. If this value is in conflict with a `MUST_DESTROY` value in the domain graph, the Data Protection Officer must get involved.

MUST_DESTROY

The object must be destroyed.

MAY_DESTROY

The object can be destroyed.

Implementations of `PersonalDataDestructionForBCRootsPlugin` can have `shouldDestroyObject` methods to allow or block a destruction request on specific objects. You can add a `shouldDestroyRoot` method to this class to handle destruction of such a pinnable root, and you can add specific entity methods like `shouldDestroyUser`.

A destruction root entity implements the `DestructionRootPinnable` delegate. You can define data disposition logic for a destruction root entity by writing a `shouldDestroyRoot` method. For example, if a given contact is associated with three different accounts, your configuration code would determine for each of these accounts if they are all settled. If they are, then the code would determine if the contact can also be destroyed.

Data obfuscation in InsuranceSuite

In general, personal data destruction in PolicyCenter and ClaimCenter is done through removal of database records. However, the entities associated with an employee who works in your installation of the application are not conducive to removal because of the way data creation and changes are recorded. In particular, objects in the database are connected to the user that created them, and, in many cases, the user that last modified them.

IMPORTANT: BillingCenter supports only obfuscation, and not purging, of personal data. In addition, BillingCenter supports obfuscating or purging frozen entities.

Because an employee is likely to create or modify hundreds of thousands of objects, it would be computationally expensive to locate all those objects in the database. It would also be expensive to change those references to something else. It is not necessary to destroy the relationship between all the work that the employee performed and the fact that it was performed by a specific employee. If the employee's personally identifiable data is destroyed, the set of objects associated with the employee can remain and not violate the need to destroy personal data.

Therefore, in the base configuration, the application obfuscates data related to `UserContact` objects.

In the base configuration, BillingCenter provides a framework that enables you to implement personal data destruction through configuration. A public API capable of accepting data destruction requests of `Contact` instances is provided, together with pre-defined work queues that call interfaces that you implement to enable personal data obfuscation. In the base configuration, BillingCenter is capable of fully destroying personal data related to system users. You can use this implementation as an example of how to enable full personal data destruction capabilities in the system.

WARNING: In BillingCenter, obfuscation of financially related entities and amounts, as well as producer related entities, is not supported, and doing so will result in data inconsistency as well as unpredictable system behavior.

Obfuscating frozen entities in BillingCenter

It is possible obfuscate any field on a frozen entity that is legally marked as personal data. However, for `PolicyPeriod` entities, Guidewire does not allow personal data obfuscation on an archived `PolicyPeriod` object.

Obfuscatable fields of a frozen entity are still considered frozen and, other than allowing obfuscation, cannot be changed. During the call to `Obfuscator.obfuscate`, frozen personal data fields are temporarily unfrozen to allow them to be obfuscated.

Obfuscation of a frozen personal data field is possible only when using an `Obfuscator` implementation that extends from `gw.api.personaldata.obfuscation.PersonalDataObfuscator` and uses the `obfuscate` method.

BillingCenter obfuscation and interaction with PolicyCenter

In the base configuration of BillingCenter, using the obfuscation framework does not cause any data changes to be propagated to PolicyCenter. Extending the default obfuscation capabilities can have an impact on billing transactions that PolicyCenter sends to BillingCenter. For example, policy and account numbers might be obfuscated in BillingCenter but still active in PolicyCenter. You must evaluate the impact while implementing your requirements for personal data removal.

BillingCenter obfuscation and interaction with ContactManager

ContactManager is expected to be the last Guidewire application web service called by the external system to process a personal data destruction request. When ContactManager receives a contact destruction request from the web service, it makes calls to the core application implementation of `ContactAPI.isContactDeletable` to determine if the contact can be destroyed. The contact will be destroyed in ContactManager only if all the applications return a positive response saying the contact can be destroyed.

If you implement Contact obfuscation in BillingCenter, in the base configuration, obfuscating a contact sets its `AddressBookUID` to null, severing the link between the BillingCenter contact and the ContactManager contact. For such a contact, BillingCenter returns a positive response when ContactManager calls the BillingCenter implementation of `ContactAPI.isContactDeletable`, allowing ContactManager to delete the contact.

It is possible that this recommended process might not be followed, and ContactManager will try to destroy a contact before BillingCenter does. In that case, BillingCenter allows the given contact to be destroyed in ContactManager if it is not related to any account, policy, or producer. After ContactManager destroys the contact, it continues to exist in BillingCenter with an `AddressBookUID`. Any further action with this contact in BillingCenter will cause unpredictable system behavior.

IMPORTANT: If you implement contact obfuscation in BillingCenter, you must set the contact's `AutoSync` field to `Disallow` when implementing your `PersonalDataDestroyer` class. Setting this field prevents data changes from being sent to ContactManager.

Obfuscated objects

Each object can indicate whether it has been obfuscated in its `Obfuscated` flag. The system has no special handling for objects that have gone through obfuscation. Obfuscated objects act like any other active object in the system regarding search results, batch processes, and so on. You can implement additional functionality to filter obfuscated beans, according to the application configuration capabilities. In your obfuscation implementation, you must take into account how your custom obfuscation might affect existing processes in the application.

Preupdate rules

Data obfuscation works the same as a normal entity editing, so changes made during obfuscation will trigger preupdate rules for entity types that have rules registered.

Implementing the Obfuscatable delegate in an entity

To be obfuscatable, an entity must implement the `Obfuscatable` delegate. If the entity implements `DestructionRootPinnable`, the `DoNotDestroy` field is set to `false` by default, which enables the entity to be obfuscated.

If an entity implements this delegate, the fields to be obfuscated must be tagged `PersonalData`.

Note: Tagging is not supported for array references, nor does obfuscation cascade automatically through foreign keys. Arrays and cascading through foreign keys must be handled in Gosu code in the `Obfuscator` implementation class.

In BillingCenter, the `DoNotDestroy` field has a default value of `false` and cannot be changed in Gosu code.

Implementing the Obfuscator interface in an entity

PolicyCenter

To be obfuscatable, an entity must implement the `Obfuscator` interface and specify an implementation class other than `UnsupportedObfuscator`. For example, use a class that extends `PCPersonalDataObfuscator`.

The entities that have `Obfuscator` implementations that support obfuscation are:

- `Credential` – `CredentialDefaultObfuscator`

- OfficialID – OfficialIDObfuscator
- PCOfficialID – PCOfficialIDObfuscator. This entity is a subtype of OfficialID and inherits that entity's implementation of the Obfuscation delegate.
- User – UserDefaultObfuscator
- UserContact – UserContactObfuscator. This entity is a subtype of Person and inherits that entity's implementation of the Obfuscation delegate.

The following entities implement the Obfuscatable delegate, and in the base configuration their Obfuscator interface implementation is UnsupportedObfuscator:

- Address
- AddressReplacement
- AffinityGroup
- Contact
- ContactAddress
- ContactCategoryScore
- ContactContact
- ContactTag
- OutboundLocationRiskAssessmentTempStore

ClaimCenter

To be obfuscatable, an entity must implement the Obfuscator interface and specify an implementation class other than UnsupportedObfuscator. For example, use a class that extends DefaultPersonalDataObfuscator.

The entities that have Obfuscator implementations that support obfuscation are:

- Credential – CredentialDefaultObfuscator
Has fields and typekeys marked PersonalData.
- OfficialID – DefaultPersonalDataObfuscator
Has fields and typekeys marked PersonalData.
- User – UserObfuscator
Has fields and typekeys marked PersonalData.
- UserContact – UserObfuscator.
This entity is a subtype of Person, which is a subtype of Contact. It inherits the Contact implementation of the Obfuscatable delegate and overrides the Contact implementation of the Obfuscator interface. In the base configuration, the EmployeeNumber field is marked PersonalData.

The following entities implement the Obfuscatable delegate, and in the base configuration their Obfuscator interface implementation is UnsupportedObfuscator:

- Address
- Contact – Has fields and typekeys that are marked PersonalData.
- ContactCategoryScore
- Person – Inherits obfuscation settings from Contact. Has fields and typekeys that are marked PersonalData.

BillingCenter

To be able to obfuscate an entity, you must use a suitable Obfuscator, such as one that derives from DefaultPersonalDataObfuscator.

The entities that have Obfuscator implementations that support obfuscation are:

- Credential – CredentialDefaultObfuscator
- OfficialID – DefaultPersonalDataObfuscator
- User – UserDefaultObfuscator
- UserContact – UserUserContactDefaultObfuscator.

This entity is a subtype of Person, which is a subtype of Contact. It inherits the Contact implementation of the Obfuscatable delegate and overrides the Contact implementation of the Obfuscator interface.

The following entities implement the Obfuscatable delegate, and in the base configuration their Obfuscator interface implementation is UnsupportedObfuscator:

- Address
- Contact
- ContactCategoryScore

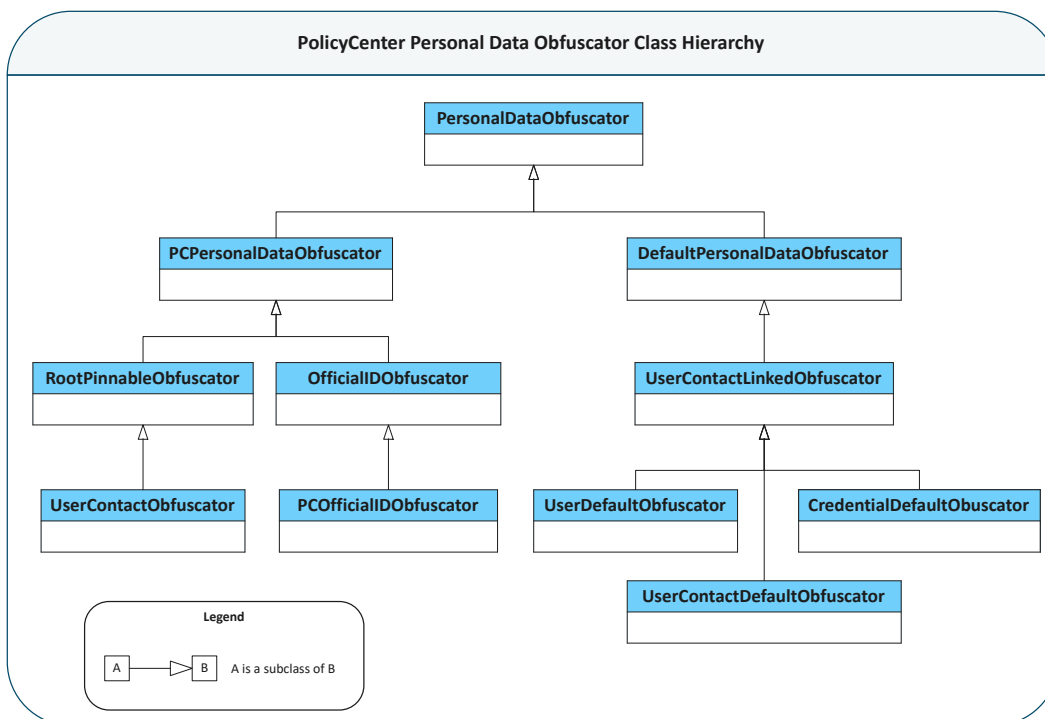
Personal data obfuscator classes

The personal data obfuscation classes, all of which ultimately inherit from PersonalDataObfuscator, define obfuscation for specific entities. The specific class that an entity uses is defined in its `implementsEntity` element for `gw.api.obfuscation.Obfuscator`.

Personal data obfuscation class hierarchy

PolicyCenter

As shown in the following object model diagram, PolicyCenter provides the following class hierarchy for personal data obfuscation:



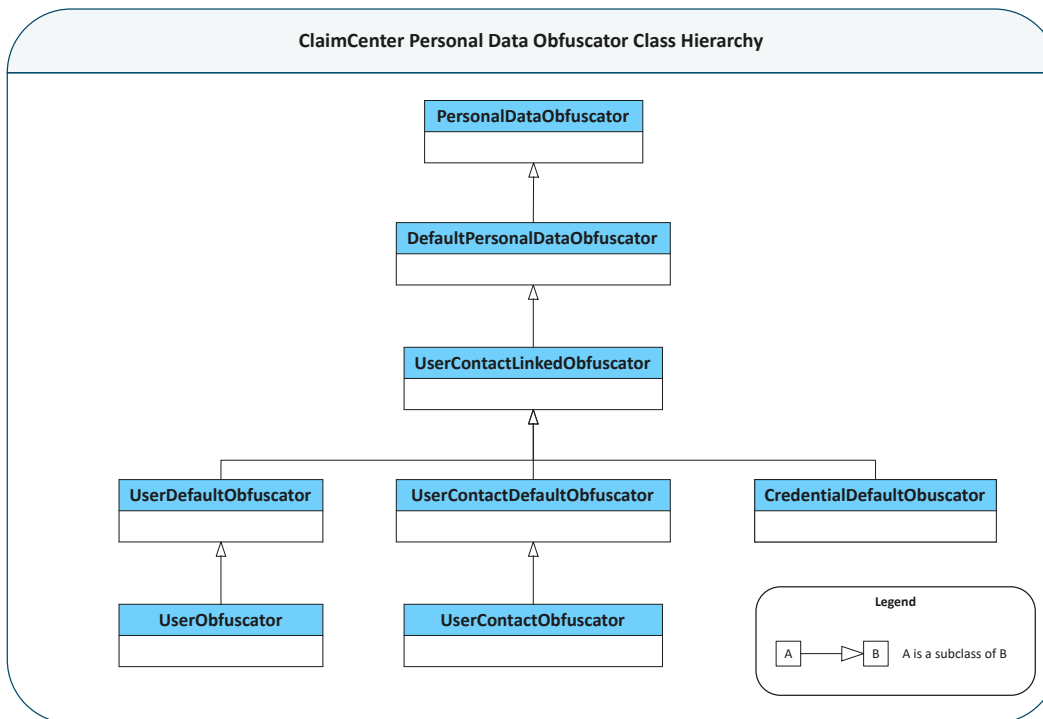
There are two main branches in this class hierarchy:

- DefaultPersonalDataObfuscator and its subclasses UserContactLinkedObfuscator, UserDefaultObfuscator, CredentialDefaultObfuscator, and UserContactDefaultObfuscator are available in all core applications. Its subclasses define base configuration behavior to enable obfuscating User, UserContact, and Credential and related objects. For example:

- `UserContactLinkedObfuscator` overrides the `shouldObfuscate` method. That method calls the `shouldDestroyUser` method defined in the `PersonalDataDestruction` plugin to get the setting for `UserContact`. For example, in the base configuration the setting is `MUST_NOT_DESTROY`.
- `UserDefaultObfuscator` overrides a `beforeObfuscate` method. That method throws an exception if the user's credential is active because that means the user is still active. If the user is not active, the method calls `user.Credential.obfuscate` and `user.Contact.obfuscate` and removes the arrays of join entities `AttributeUser` and `UserRegion`.
- `UserContactDefaultObfuscator` attempts to obfuscate or remove any contact addresses, official IDs, category scores, and tags associated with the `UserContact` that can be destroyed.
- `CredentialDefaultObfuscator` overrides the `beforeObfuscate` method, which stops the obfuscation if the credential is active.
- `PCPersonalDataObfuscator` is the base class of a set of classes used to define obfuscation of specific entities, such as `UserContact` and `PCOfficialID`. You can extend this class to define how obfuscation works on an entity. You specify the class in the entity's `implementsEntity` declaration for the `Obfuscation` class. For example:
 - `UserContactObfuscator` attempts to remove any associated primary address, contact addresses, and official IDs from the `UserContact` object. Additionally, the method `getObfuscatedValueForPersonalDataField` defines obfuscation behavior for a field that is tagged `ObfuscateUnique`.

ClaimCenter

As shown in the following object model diagram, ClaimCenter provides the following class hierarchy for personal data obfuscation:



`DefaultPersonalDataObfuscator` and its subclasses define base configuration behavior to enable obfuscating `User`, `UserContact`, and `Credential` and related objects. For example:

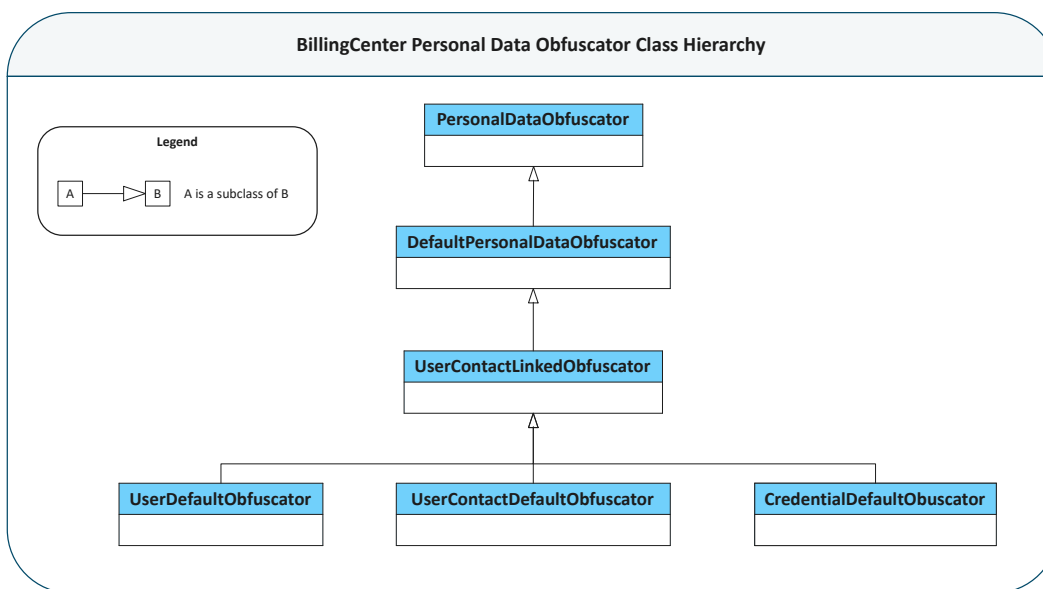
- `UserContactLinkedObfuscator` overrides the `shouldObfuscate` method. That method calls the `shouldDestroyUser` method defined in the `PersonalDataDestruction` plugin to get the setting for `UserContact`. For example, in the base configuration the setting is `MUST_NOT_DESTROY`.
- `UserDefaultObfuscator` overrides a `beforeObfuscate` method. That method throws an exception if the user's credential is active because that means the user is still active. If the user is not active, the method calls

`user.Credential.obfuscate` and `user.Contact.obfuscate` and removes the arrays of join entities `AttributeUser` and `UserRegion`.

- `UserContactDefaultObfuscator` attempts to obfuscate or remove any contact addresses, official IDs, category scores, and tags associated with the `UserContact` that can be destroyed.
- `CredentialDefaultObfuscator` overrides the `beforeObfuscate` method, which stops the obfuscation if the credential is active.
- `UserObfuscator` overrides the `beforeObfuscate` method to remove the user's authority profile, but only if there is only one owner.
- `UserContactObfuscator` overrides the `beforeObfuscate` method to remove unreferenced ETF records.

BillingCenter

As shown in the following object model diagram, `BillingCenter` provides the following class hierarchy for personal data obfuscation:



`DefaultPersonalDataObfuscator` and its subclasses define base configuration behavior to enable obfuscating `User`, `UserContact`, and `Credential` and related objects. For example:

- `UserContactLinkedObfuscator` overrides the `shouldObfuscate` method. That method calls the `shouldDestroyUser` method defined in the `PersonalDataDestruction` plugin to get the setting for `UserContact`. For example, in the base configuration the setting is `MUST_NOT_DESTROY`.
- `UserDefaultObfuscator` overrides a `beforeObfuscate` method. That method throws an exception if the user's credential is active because that means the user is still active. If the user is not active, the method calls `user.Credential.obfuscate` and `user.Contact.obfuscate` and removes the arrays of join entities `AttributeUser` and `UserRegion`.
- `UserContactDefaultObfuscator` attempts to obfuscate or remove any contact addresses, official IDs, category scores, and tags associated with the `UserContact` that can be destroyed.
- `CredentialDefaultObfuscator` overrides the `beforeObfuscate` method, which stops the obfuscation if the credential is active.

PersonalDataObfuscator

`PersonalDataObfuscator` is the parent class for the obfuscator classes. It is a general class that obfuscates the fields for any entity. You are required to extend this class or one of its subclasses when implementing obfuscation for entities that do not have obfuscator classes defined.

For PolicyCenter, `PersonalDataObfuscator` handles setting the obfuscation for effdated and non-effdated fields. If you have tagged the entity fields `PersonalData` with value `ObfuscateUnique`, and there is no need to do any special preprocessing of the fields, you can use this class.

For ClaimCenter and BillingCenter, `PersonalDataObfuscator` handles setting the obfuscation for marked fields. If you have tagged the entity fields `PersonalData` with value `ObfuscateUnique`, and there is no need to do any special preprocessing of the fields, you can use this class.

The following methods are provided:

isObfuscated

Checks the field `ObfuscatedInternal` and returns true or false depending on the value.

obfuscate

Finds all the columns that are marked for obfuscation on this object.

- PolicyCenter only. If the object is an instance of `EffDated` then the method obfuscates all versions of that column on the `EffDated` object.
- ◦ Otherwise, the method sets the field value with the `obfuscatedValue`.
- ClaimCenter and BillingCenter only. The method sets the field value with the `obfuscatedValue`.
- Before obfuscating the column, the method calls `preProcessPersonalDataFieldBeforeObfuscating` to do any preprocessing.
 - If the column is marked `PersonalData` with a value of `ObfuscateDefault`, the method sets the value to either null or the default value.
 - If the column is marked `PersonalData` with a value of something other than `ObfuscateDefault`, the method calls `getObfuscatedValueForPersonalDataField`. That method passes the tag value, and you must provide the object that the method uses to obfuscate the field.
- At the end, the method sets the `ObfuscateInternal` column to true by using `ObfuscatablePublicMethod.setObfuscated`.

obfuscateSimple

Works the same as `obfuscate`, but in PolicyCenter only for objects that are not `EffDated`.

If you want to do preprocessing before obfuscation, you can extend `PersonalDataObfuscator` or a subclass of this class and override the `beforeObfuscate` method. If you want to change the value of the personal data field before obfuscation, you can override `getObfuscatedValueForPersonalDataField`.

The method `getObfuscatedValueForPersonalDataField(Obfuscatable bean, IEntityPropertyInfo personalDataField, String tagValue)` defines default obfuscation behavior for fields that are tagged `ObfuscateUnique` and `ObfuscateDefault`.

UnsupportedObfuscator

This Java class provides a default implementation for any entity that implements the interface `Obfuscator` and declares `UnsupportedObfuscator` as the implementation. When `obfuscate` is called, this class throws `UnsupportedOperationException` for the field.

PersonalDataObfuscatorUtil

This Gosu class is in the same package as the personal data obfuscation classes, `gw.personaldata.obfuscation`. The class implements the method `computeMD5Padding`. If a personal data field has a `PersonalData` tag with value `ObfuscateUnique`, this method is called to obfuscate the field.

The method computes an MD5 String based on the type of entity and the `PublicID`, and then returns that string so the field can be obfuscated with that value.

For example, `DefaultPersonalDataObfuscator` calls this method in its `getObfuscatedValueForPersonalDataField` method when the field's `PersonalData` tag has the value `PersonalDataTagValue.TC_OBFUSCATEUNIQUE.Code`.

Testing data destruction in PolicyCenter during development

PolicyCenter includes a tool to help you test data destruction on a contact, policy, or account and test your implementation of data destruction. You can use this tool during development.

IMPORTANT: Do not include this tool in a production system.

To access this tool, log in to PolicyCenter, press **Alt+Shift+T**, and select **Internal Tools > Personal Data Destruction Test**.

The tool enables you to test purging or obfuscation.

Configuring automation

Autopilot Workflow Service and PolicyCenter

The Autopilot Workflow Service is an automation platform provided by Guidewire that allows you to externalize business logic and build automation for common InsuranceSuite tasks and processes.

The Autopilot Workflow Service is comprised of the following services:

- Workflow Designer: A tool for orchestrating and modeling workflows.
- Workflow Provisioner: A tool for onboarding the Autopilot Workflow Service for a planet.
- Autopilot Console: A tool for managing runtime workflow instances, organizing workflows into packages, and managing the package lifecycle.

When used with PolicyCenter, the Autopilot Workflow Service automates and streamlines policy workflows, which reduces manual intervention and improves PolicyCenter performance. The Autopilot Workflow Service enables you to model automated workflows for tasks and sequences that agents or underwriters typically perform in PolicyCenter. You can also model the workflow to support manual intervention from an underwriter or to continue with the automated workflow.

See the *Autopilot Workflow Service* documentation for more information.

A PolicyCenter workflow can automate any of the following policy sequences:

- Policy updates and changes.
- Policy renewals.
- Policy cancellations and delinquency management.
- Policy audits and responses.

Specific examples can include:

- (Policy submission) Follow up automatically with the insured on missing documentation.
- (Policy submission) Minimum dwelling coverage verification after a policy change.
- (Policy change) Automate the removal of the bank's information from a policy after a vehicle loan is paid off.
- (Policy change) Change the name of the mortgage servicing company.
- (Policy cancellation) Automate post cancellation activities such as sending updates to lienholder.
- (Policy cancellation) Automate non-pay cancellations or insured cancellations where defined thresholds or grace periods have been met.

PolicyCenter has been integrated with the Autopilot Workflow Service to support a smooth transition between an Autopilot underwriter and a human underwriter. Changes have been made to the default configurations for roles, authority profiles, and job assignments (see “Changes to PolicyCenter to enable automation” on page 272).

Note: Reference templates for PolicyCenter workflows are available on the [Autopilot Workflow Service Technical Solutions](#) page and on [Guidewire Marketplace](#).

Changes to PolicyCenter to enable automation

Autopilot underwriter user

You must enable support for the Autopilot Workflow Service in PolicyCenter by setting the `AutopilotUserRoleAssignment` flag to true in `config.xml`.

If support is enabled, PolicyCenter creates an autopilot underwriter user at server startup based on the settings contained in `autopilot_user.xml`:

```
<?xml version="1.0"?>
<import xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:noNamespaceSchemaLocation="http://www.guidewire.com/pc/import/pc_import.xsd">
  <User public-id="autopilot_uw:1">
    <Contact public-id="autopilot_uw:1"/>
    <Credential public-id="autopilot_uw:1"/>
    <roles>
      <UserRole>
        <role public-id="underwriter"/>
        <user public-id="autopilot_uw:1"/>
      </UserRole>
    </roles>
    <UseProducerCodeSecurity>false</UseProducerCodeSecurity>
    <UserSettings public-id="autopilot_uw:1"/>
    <UserType>underwriter</UserType>
    <!-- "systemusertype" is fixed internally. Do not try to set it here. -->
  </User>

  <UserSettings public-id="autopilot_uw:1">
    <startuppge>DesktopActivities</startuppge>
  </UserSettings>

  <UserContact public-id="autopilot_uw:1">
    <FirstName>Autopilot</FirstName>
    <LastName>Underwriter</LastName>
  </UserContact>

  <Credential public-id="autopilot_uw:1">
    <Password>PPNxQmp1UdwbZrn2G1Tj8+w01rI=</Password>
    <!-- "username" is fixed internally. Do not try to set it here. -->
  </Credential>
</import>
```

You can edit the autopilot underwriter user details before the file is loaded for the first time. You must add this user to a group in PolicyCenter before you execute any workflows.

New automation jobs are assigned to the autopilot underwriter user if it exists in the assigned group for the job. However, when an automated job is reassigned, it will not be reassigned to the autopilot underwriter user. It is instead assigned to the next available user in the same group.

The same assignment strategy applies to activities. If an activity is created on an automated job assigned to the autopilot underwriter user and the assignment is set to user automated assignment, then the activity is assigned to the next available user in the group.

Note: In addition, if an autopilot underwriter user attempts to bind a policy transaction, the bind is passed to the next available user in the group.

The assignment strategy can be demonstrated in Workflow Designer. When a submission is created that is eligible for automation, the workflow invokes the **Activate Autopilot** subflow (which assigns the job to the autopilot underwriter user) and the job status changes to **Currently Processing**. But if the submission is not eligible for automation, the workflow invokes the **Halt Autopilot** subflow. The **Halt Autopilot** subflow invokes the PolicyCenter API to reassign the job to someone else in the assigned group, and the job status changes to **Cannot Process**.

Changes to the PolicyCenter data model to support automation

The PolicyCenter data model supports automation by including an `Automation Status` field on the `Job` entity to indicate whether a job is automated. This field was also added to the Cloud API schema and the associated APP Event graph for use by the Workflow Service. The following events are defined to support the Workflow Service:

JobAutopilotEnabled

Generated when the `Automation Status` field is set to `Currently Processing` on the `Job` entity.

JobAutopilotDisabled

Generated when the `Automation Status` field is set to `Cannot Process` on the `Job` entity.

In addition, when the job `Automation Status` is set to disabled, the job is assigned to the next underwriter (and not the autopilot underwriter user).

Changes to PolicyCenter to support user intervention in the workflow

In the base configuration, PolicyCenter contains specialized Autopilot activity patterns (in `activity-patterns.csv`) to support user intervention and provide the workflow the ability to resume automation or refer the job for manual handling based on user review. These activity patterns are automatically loaded into new databases after server startup. For existing databases, the activity patterns are created automatically through a database upgrade trigger. The changes to PolicyCenter are summarized below:

- Two specialized activity patterns - `autopilot_action_required` and `autopilot_notification`.
- The `Activity` entity has been extended to add the `AutopilotHandlingDecision` field for capturing manual review decisions. This field has also been added to the Cloud API schema and associated App Event graph for use by the Autopilot Workflow service.
- User interface changes have been made to the **Activity Worksheet** to support **Approve** and **Deny** paths for automation after the manual review.

Configuring seasonal coverages in a line of business

The following sections explain how you can configure a line of business to support seasonal coverages on coverables.

Configure coverable(s) using seasonal coverage

Coverables that implement the Seasonal delegate will be able to store seasonal coverage data (seasonal date ranges that specify when the coverable period will be in-season). When the coverable period falls outside of the seasonal date range, those periods are off-season. If no seasonal data ranges are specified for the coverable, the coverable will be non-seasonal.

Add the Seasonal delegate to the coverable entity.

```
<implementsEntity  
  name="Seasonal"/>
```

Configure system tables with downgraded data to use when off-season

When a coverable is sliced based on the seasonal date ranges, it will have periods which are in-season and off-season. During the off-season periods, a coverable does not need the same amount of coverage. This means that the coverage can be lowered. The information that captures what the coverage minimum must be during off-season periods is currently stored in system tables and is looked up with the plugin method `ISeasonalCoverablePlugin#downgradeAllCoveragesToLegalMinimum()`.

Configure seasonal coverage system table with coverages that will be downgraded to legal minimum when off-season

Modify the existing `seasonalcoverage_configs.xml` or add a new system table file that is specific to the line of business (`seasonalcoverage_configs.<LOB>.xml`).

Add a `SeasonalCoverageConfig` entry for each coverage that will be downgraded when off-season.

```
<SeasonalCoverageConfig public-id="zj1jk03vfutm936mcieu0cnbm9">  
  <EffectiveDate>2000-01-01 00:00:00.000</EffectiveDate>  
  <Coverage>PALimitedMexicoCov</Coverage>  
  <LineOfBusiness>PersonalAutoLine</LineOfBusiness>  
  <Value>exclude</Value>  
</SeasonalCoverageConfig>
```

The `SeasonalCoverageConfig` entry can contain the following information:

Entry attribute	Condition	Description
EffectiveDate	required	The date when the entry becomes valid.
ExpirationDate	optional	The date when entry is no longer valid. If not specified, the entry is valid from effective date onwards.
Jurisdiction	optional	The jurisdiction that the entry applies to. If not specified, entry applies to all jurisdictions.
LineOfBusiness	required	The code identifier of the line of business that the entry applies to.
Coverage	required	The code identifier of the coverage that the entry applies to.
Value	required	Whether the coverage is included or excluded given the effective/expiration date, jurisdiction, line of business.

Configure seasonal coverage term system table with coverage terms that will be downgraded to legal minimum when off-season

Modify existing seasonalcovterm_configs.xml or add a new system table file for the line of business (seasonalcovterm_configs.<LOB>.xml).

Add SeasonalCovTermConfig entry for each coverage term that will be downgraded to the legal minimum when off-season.

```
<SeasonalCovTermConfig public-id="zqvj4tkij806130mh8fls5vd31b">
  <EffectiveDate>2000-01-01 00:00:00.000</EffectiveDate>
  <ExpirationDate>2021-01-01 00:00:00.000</ExpirationDate>
  <Coverage>PATowingLaborCov</Coverage>
  <CoverageTerm>TowingAndLaborLimit</CoverageTerm>
  <LineOfBusiness>PersonalAutoLine</LineOfBusiness>
  <Value>25</Value>
</SeasonalCovTermConfig>
```

The SeasonalCovTermConfig entry can contain the following information:

Entry attribute	Condition	Description
EffectiveDate	required	The date when the entry becomes valid.
ExpirationDate	optional	The date when entry is no longer valid. If not specified, the entry is valid from effective date onwards.
Jurisdiction	optional	The jurisdiction that the entry applies to. If not specified, entry applies to all jurisdictions.
LineOfBusiness	required	The code identifier of the line of business that the entry applies to.
Coverage	required	The code identifier of the coverage that the entry applies to.
CoverageTerm	required	The code identifier of the coverage term that the entry applies to.
Value	required	The value the coverage term must be downgraded to given the effective/expiration date, jurisdiction, line of business.

For coverage terms added to seasonalcovterm_configs, a corresponding coverage (its parent) must be added to seasonalcoverage_configs with a value of include.

```
<SeasonalCoverageConfig public-id="zhcjaaacmqff9acr149kr7255ub">
  <EffectiveDate>2000-01-01 00:00:00.000</EffectiveDate>
  <Coverage>PATowingLaborCov</Coverage>
  <LineOfBusiness>PersonalAutoLine</LineOfBusiness>
  <Value>include</Value>
```

Configure PCF files

Add seasonal ranges to seasonal coverable with SeasonalDatesListPanelSet

SeasonalDatesListPanelSet.pcf contains a ListView where you can add, edit, or view seasonal ranges for a seasonal coverable. This ListView is designed to be used with seasonal coverables in the job wizard.

Add PanelRef for SeasonalDatesListPanelSet to PCF with seasonal coverable.

```
<PanelRef
  def="SeasonalDatesListPanelSet(selectedVehicle, openForEdit, jobWizardHelper)"/>
```

Add slice selector for period quoted with seasonal coverable

When a policy transaction with seasonal coverables is quoted, the slices are generated based on the defined seasonal date ranges. However, the slice selector drop-down does not automatically show the coverages or coverage terms on different slices. To be able to use this drop-down, a modification needs to be made to `00SEPanelSet.pcf`.

To view the slices of the seasonal coverable, in the code section of the PCF, modify the return value for function `showSliceSelector()` so that the selector is shown for seasonal coverables.

Before:

```
return jobWizardHelper.PolicyPeriod.SliceDate > policyPeriod.EditEffectiveDate // always show if we're editing a
future slice
    or policyPeriod.Failed00SEValidation or policyPeriod.Failed00SEEvaluation // otherwise show if there are
errors in a future slice
```

After:

```
return jobWizardHelper.PolicyPeriod.SliceDate > policyPeriod.EditEffectiveDate // always show if we're editing a
future slice
    or policyPeriod.Failed00SEValidation or policyPeriod.Failed00SEEvaluation // otherwise show if there are
errors in a future slice
    or policyPeriod.Lines*.AllCoverables.whereTypeIs(Seasonal).HasElements // or if there are any seasonal
coverables
```

Optional configuration

The `ISeasonalCoverablePlugin` plugin manages seasonal coverables and allows you to customize the following information:

- How coverages are downgraded for a seasonal coverable for an off-season period (`downgradeAllCoveragesToLegalMinimum`).
- What gets validated for a seasonal coverable (`validateSeasonalCoverable`).
- The set of seasonal date ranges to use for a seasonal coverable (`getSeasonalPeriods`).
- The seasonal date ranges to use on a renewal (`updateSeasonalDateRangesOnRenewal`).

By default the plugin uses `SeasonalCoverablePlugin.gs`, which behaves as follows:

- `downgradeAllCoveragesToLegalMinimum` looks up off-season values for coverages and coverage term values in the `seasonalcoverage_configs` and the `seasonalcovterm_configs` system tables.
- `validateSeasonalCoverable` uses `SeasonalValidationBase`. Checks that the start and end months for a seasonal range are defined. Also validates that there are no overlapping months when there are multiple seasonal ranges defined for a seasonal coverable.
- `getSeasonalPeriods` returns sets of seasonal periods for the current year, prior year, and the next year based on the months defined in the seasonal ranges of the seasonal coverable.
- `updateSeasonalDateRangesOnRenewal` is not overridden.

Defining activity patterns

This topic discusses activity patterns, what they are, and how to configure them.

Activity patterns standardize the way that the application creates activities. Activity patterns describe the kinds of activities that people perform while handling policies within an organization. For example, reviewing and approving a policy renewal is a common activity. Thus, it has its own activity pattern that creates a reminder to perform this activity.

Patterns act as templates for creating activities. Activity patterns define the typical practices for each activity. For example, this is its name, its relative priority, and the standards for how quickly it is to complete (that is, its due dates). If a user (or a rule) adds an activity to the workplan for a policy, the application uses the activity pattern as a template to set default values for the activity. (For example, an activity pattern can set the subject, priority, or target date for the activity.)

You can set up and customize the **Activity Patterns** that make sense for your policies business processes from the **Administration** tab in the application. It is possible to create activities from activity patterns in different ways:

- You can manually create activities in the application.
- A business rule or some other Gosu code create activities as part of generating workplans or while responding to escalations, policy exceptions, or other events.
- The application automatically creates activities to handle manual assignment or approvals, for example.
- External applications create activities through API calls.

You can view the list of available **Activity Patterns** by selecting **New Activity** from the **Actions** menu on one of the **Summary** pages.

IMPORTANT: After an activity pattern is in production, do not delete it as there can be old activities tied to it. Instead, edit the activity pattern and change the **Automated only** field to **Yes**. This prevents anyone from creating new activities of that type.

An activity pattern does not control how the application assigns an activity. Instead, activity assignment methods in Gosu expressions control how the application assigns an activity. Using the pattern name, the assignment methods determine to whom to assign the activity.

Pattern types and categories

The application applies a type attribute to every activity pattern. You can also use a category attribute to classify patterns into related groups. This topic describes how the application makes use of these two attributes.

Activity pattern types

Each activity pattern has a set type, such as General or Approval. It is only possible to add a custom activity pattern of type General or Assignment Review through the application interface. It is not possible to create a custom Approval activity pattern through the application **Activity Patterns** screen.

Guidewire defines a number of internal activity pattern types in the base configuration. All pattern types other than General or Assignment Review are internal. Only internal application code can use an internal pattern type. Do not attempt to remove an internal activity pattern type as this can damage your installation. You can, however, customize attributes of the internal activity patterns, such as adjusting the due date.

The ActivityType typelist

Guidewire defines activity pattern types in the ActivityType typelist. Guidewire defines this typelist as final. Typelists marked as final are internal typelists and used by internal application code. You cannot add typecodes to—or delete typecodes from—a typelist marked as final. However, you can modify some of the fields on an existing typecode, if you wish. For more information on typelists marked as final, see *Data Model Configuration*.

Any pre-existing activity patterns of type General in the base configuration are examples that Guidewire provides. You can fully customize any of them. Activity patterns with other activity types are typically not available in the application interface. You use them only within Gosu and the application uses them internally.

Categorizing activity patterns

About this task

Guidewire recommends that you categorize your activity patterns so that it is possible to choose among the different activity categories during new activity creation. These categories serve as the first level of navigation in the application **New Activity** menu. The activity pattern categories appear only within the application interface.

The ActivityCategory Typelist

Guidewire defines activity categories in the ActivityCategory typelist. You are free to add or delete typecodes from this typelist. If you change a typelist, remember that you must restart the application server to view your changes in the application interface.

The application displays the activity categories in the **New Activity Pattern** editor screen.

Using activity patterns in Gosu

About this task

This topic applies only to PolicyCenter and BillingCenter

IMPORTANT: You must use the activity pattern code to refer to an activity pattern in Gosu code. Do not use a pattern ID or PublicID value.

There are two operations that you can perform in Gosu involving activity patterns:

- One is to test which activity pattern an existing activity uses.
- The other is to retrieve an activity pattern for use in creating a new activity.

To test for a specific activity pattern

Use the following Gosu code, which compares an activity pattern Code value with a string value that you supply.

```
Entity.ActivityPattern.Code == "activity_pattern_code"
```

To retrieve an activity pattern

To find (retrieve) a specific activity pattern, use one of the following Gosu find or get methods. The find method returns a query object and the get method returns an ActivityPattern object.

Calculating activity due dates

The activity made from a pattern always has a specific date as a deadline. Each activity pattern defines how to calculate the due date for a specific activity instance.

Target due dates (deadlines)

A *target date* (or *due date*) suggests the date to complete an activity. Settings in the **New Activity Pattern** editor determine how the application calculates the due date for an activity. The application can calculate a target due date in hours or days. The application calculates due dates using the following pieces of information:

- *How much time?* How much time to take or how many hours or days to allow to complete the activity. You specify this using the **Target days** or **Target hours** value.
- *What is the starting point?* What point in time does the application use as the start point in calculating the target date? You specify this using the **Target start point** field.
- *What days to count?* The application can count calendar days or only business days. You specify this with the **Target Include Days** field.

The application reports deadlines only at the level of days. For example, if something is due on June 1 of a particular year, it becomes overdue on June 2 of that year, not some time in the middle of the day on June 1. The application does track activity creation dates and marks completion at the level of seconds so that you can calculate average completion times at a more granular level.

If you do not specify **Target Days** or **Target Hours** as you define an **Activity Pattern Detail**, the application uses 0 for both. A target date is optional for activities.

The maximum number of hours that you can specify is 23, and the maximum number of days that you can specify is 1825. Any value greater than the maximum is automatically limited to the maximum.

Escalation dates

While the target date can indicate a service-level target (such as complete within five business days), there can possibly be some later deadline after which the work becomes dangerously late. (For example, this might be a 30 day state deadline.) The application calls this later deadline an escalation date.

The escalation date is the date at which activity requires urgent attention. While work is shown as overdue after the target date, the application does not actually escalate (take action on) an activity until the escalation date passes. Within Studio, you can define a set of rules that define what actions take place if an activity reaches its escalation date. For example, it could be company policy to inform a supervisor if an activity passes an escalation date. You might also want to reassign the activity.

The application calculates the escalation date using the methodology it uses for target dates. You can specify escalation timing in days and hours. If you do not specify **Escalation Days** or **Escalation Hours** as you define an activity pattern, the application uses 0 (zero) for both. An escalation date, like a target date, is optional for activities.

The maximum number of hours that you can specify is 23, and the maximum number of days that you can specify is 1825. Any value greater than the maximum is automatically limited to the maximum.

Defining the business calendar in ClaimCenter

Due dates within ClaimCenter depend on an understanding of the business calendar as defined by your company. For example, if something is due in five business days, exactly which days does this include? Does your company operate seven days a week or do you consider only Monday through Friday to be business days? Which days are company holidays? Another key concept in the business calendar is understanding how your company defines the point at which a week begins or ends. Is Friday the last day of the week or is Sunday?

You manage the business calendar through the **Administration > Holidays** screen within ClaimCenter.

Base configuration activity patterns

The application uses file `activity-patterns.csv` to load the base activity pattern definitions upon initial server startup after installation. You access file `activity-patterns.csv` in Guidewire Studio in the following location:

configuration > config > import > gen

IMPORTANT: Do not remove any internal (non-General type) activity patterns or change their type, category, or code values. Internal application code requires them. You can change other fields associated with these types, however.

See also

- *InsuranceSuite Database Administration*

Activity pattern objects

The following list describes the properties on the `ActivityPattern` object.

Property	User interface field	Description
ActivityClass	Activity Class	Indicates whether the activity is a task or an event. A task has a due date. An event does not.
AutomatedOnly	Automated only	A Boolean value that defines whether only automated additions (by business rules) to the workplan use the activity pattern. • If true, the activity pattern does not appear as a choice in the application interface. • If you do not specify this value, the default is false. Guidewire recommends that you set this flag set to true for all patterns with a non-general type. This ensures that they are not visible in the application interface.
Category	Category	The category for grouping <code>ActivityPatterns</code> in the application interface.
Code	Code	Any unique text with no spaces. Maximum length is 60 characters. This property is required. The Code property is used to identify the activity pattern when accessing the pattern in rules or Gosu code. You can see this value only through the Administration tab.
Command	Not Applicable	<i>Do not use.</i> For Guidewire use only.
Data-Set	Not Applicable	The value of the highest-numbered dataset of which the imported object is a part. The application typically orders a dataset by inclusion. Thus, dataset 0 is a subset of dataset 1, and dataset 1 is a subset of dataset 2, and so forth.
Description	Description	Describes the expected outcome at the completion of this activity. It is visible only if you view the details of the activity.
DocumentTemplate	Document Template	Document template to display if you choose this activity. Enter the document template ID.
EmailTemplate	Email Template	Email template to display if you choose this activity. Enter the email template file name.
EntityId	Not Applicable	<i>Required.</i> The unique public ID of the activity pattern.
EscalationDays	Escalation days	The number of days from the <code>escalationstartpt</code> to set the Escalation Date for an activity.

Property	User interface field	Description
EscalationHours	Escalation hours	The number of hours from the escalationstartpt to set the Escalation Date for an activity.
EscalationInclDays	Escalation Include Days	Specifies which days to include. You can set this businessdays or elapsed.
EscalationStartPt	Escalation start point	The initial date used to calculate the target date. If you specify escalationdays or escalationhours, you need to specify this parameter. Otherwise, this parameter is optional.
Mandatory	Mandatory	A Boolean value that defines whether you can skip an activity. Non-mandatory activities act as suggestions about what might be a useful task without forcing you into doing unnecessary work. This value is optional. If you do not specify a value, the application uses a default of true.
PatternLevel	Pattern Level	The level that this pattern is for. Valid choices are: - Account - All - Job
Priority	Priority	Used to sort more important activities to the top of a list of work. This property is required. You can set this property to the following values: - urgent - high - normal - low.
Recurring	Recurring	A Boolean value indicating that an activity is likely to recur on a regular schedule. If you do not specify a value, the application uses a default of true.
ShortSubject	Short Subject	A brief description of the activity used on small areas of the application interface such as a calendar event entry. Maximum length of 10 characters.
Subject	Subject	A short text description of the activity that the application shows in activity lists. This property is required.
TargetDays	Target days	The number of days from the targetstartpoint to set the activity's Target Date .
TargetHours	Target hours	The number of hours from the targetstartpoint to set the activity's Target Date .
TargetIncludeDays	Target Include Days	This field answers the “what days to count” part of calculating the target date. Your options are the following: - elapsed—the count all days - businessdays—as defined by the business calendar
TargetStartPoint	Target start point	The initial date used to calculate the target date. You need specify this value only if you specify targetdays or targethours. Otherwise, this value is optional.
Type	Type	This specifies what activity type to create. Note It is only possible to create a custom activity pattern of type General or Assignment Review. Guidewire reserves all other activity pattern types for internal use. See “Pattern types and categories” on page 279 for more information.

Use activity patterns with documents and emails

About this task

You can attach a document or email template to an activity pattern. Then, if the application displays an activity based on this activity pattern, it also displays a **Create Document** or **Create Email** button in the **Activity Detail** worksheet. This button indicates that this type of activity typically has a document or email associated with that activity.

To associate a document or email template with an activity pattern:

Procedure

1. Log into the application with an administrative account.
2. Click the **Administration** tab and navigate in the Sidebar to **Business Settings > Activity Patterns**.
3. Open the activity pattern edit screen either by creating a new activity pattern or by selecting an activity pattern to update.
 - To create a new activity pattern, click **New Activity Pattern**.
 - To edit an existing activity pattern, click the activity pattern name, and then click **Edit**.
4. Use the search icon next to the **Document Template** or **Email Template** field to open a search window.
5. Find the document or email template, and then add it to the activity pattern.

Results

If you associate a document or email template with an activity pattern, you can do the following in the application:

1. If you create a new activity from this activity pattern, the application automatically populates any template field for which you specified a template with the name of that template.
2. If you open this activity, the application displays a **Create Document** button or a **Create Email** button or both in the **Activity Detail** worksheet at the bottom of the screen.
3. If you then click the **Create Document** or the **Create Email** button, the application opens a pop-up window enabling you to create the document or email from the specified template.

Specifying document and email templates in CSV files

You can also specify the document or email template in file `activity-patterns.csv`. Add a column for that template and then enter either the document template ID or the email template file name as appropriate. See “Base configuration activity patterns” on page 282 for details of working with the `activity-patterns.csv` file.

Localizing activity patterns

The application stores activity pattern data directly in the database. Thus, it is not possible to localize fields such as the subject or description of an activity pattern by localizing a display string. In the base configuration, you can localize the following activity pattern properties through the application interface—if you configure the application for multiple locales:

- Description
- Subject

If you configure the application correctly to use multiple locales, then you see additional fields at the bottom of the **New Activity Pattern** screen. You use these fields to enter localized subject and description text for that activity pattern.

See also

- For information on how to make a database column localizable (and thus, an object property localizable), see the *Globalization Guide*.

Guidewire InsuranceSuite and email

Guidewire includes support for sending emails from the application. Sending an email is one of several possible actions to take, such as when a user escalates an activity.

The email includes the following items:

- To and From properties
- A subject
- The name of the template used to generate the body of the email
- The object that the message references

The application initially saves email messages and then sends them to an SMTP email server in a background process.

Emails and Gosu code

You can send an email from any Gosu code. Thus, you can create and send an email from Gosu rules or in Gosu embedded in the application PCF screens.

InsuranceSuite email support

The application provides support for the following email functionality:

- Support for various types of email recipients (To, CC, and BCC)
- Support for templates that can be used to populate the subject and body of the email
- Support for attaching documents stored in the configured DMS (Document Management System) to the email message
- Support for SMTP authentication using the `JavaxEmailMessageTransport` class, which supports encryption

Monitoring email messages

Because the application sends email messages using the same integration infrastructure as event-based messages, you can use the same administrative tools for monitoring the status of messages. The application **Message Queues** screen, available to administrators, provides information on the various message destinations, including the Email message destination. In addition, PolicyCenter provides a **Messages** screen in which you can search for, and view, information on various message types, including email messages.

Default email plugin implementation

Guidewire provides a default email message plugin named `emailMessageTransport`. To set the application for email notifications, configure the plugin parameters in Guidewire Studio.

Configure the email plugin class and parameters

Configure the class and parameters that affect the email plugin in the Studio Plugin Editor.

Procedure

1. In the Guidewire Studio **Project** window, expand **configuration > config > Plugins > registry**.
2. Double-click `emailMessageTransport` to open it.
3. If the **Java Class** is `com.guidewire.pl.plugin.email.internal.EmailMessageTransport`, change the class that implements the plugin:
 - a) Click **Remove Plugin**  to remove the class.
 - b) To use the supported Gosu plugin class in the base configuration, click **Add Plugin** , and then click **Add Gosu Plugin**.
 - c) In **Gosu Class**, type `gw.plugin.email.impl.JavaxEmailMessageTransport`, or click **Browse**  to navigate to the class.
If you have created a different Gosu or Java plugin class for the email plugin, use that class, specifying the appropriate plugin type, instead of the `JavaxEmailMessageTransport` class.
4. If the **Disabled** check box is checked, clear the check box to enable the plugin.
5. Enter appropriate values for the plugin parameters, including the following ones:
 - `CredentialPlugin.Key`
 - `defaultSenderAddress`
 - `defaultSenderName`
 - `smtpHost`
 - `smtpPort`

The `JavaxEmailMessageTransport` email plugin class also supports many other parameters for SMTP operation, and SSL and TLS security.
6. Save your changes.
7. Restart the application server.

What to do next

See also

- “Add an implementation to a Plugins Registry item”
- “Sending emails” in *Plugins, Prebuilt Integrations, and SOAP APIs*

The email object model

The application uses the following classes to define email messages:

- `gw.api.email.Email`
- `gw.api.email.EmailContact`

These classes are simple data containers with almost no logic. The classes also contain a number of utility methods that perform operations on the fields defined in the class.

`gw.api.email.Email`

The `Email` class contains the following fields, most of which are self-explanatory:

Field	Description
Subject	Subject of the email
Body	Content of the email
Sender	Sender of the email
ReplyTo	Contact object (It is possible for this to be different from the Sender.)
ToRecipients	List of Contact objects
CcRecipients	List of Contact objects
BccRecipients	List of Contact objects
Documents	List of DocumentBase entities for attachment to the email

[gw.api.email.EmailContact](#)

The EmailContact class contains the following fields:

Field	Description
Name	Name of contact
EmailAddress	Email address of contact
Contact	Contact object, which can be null. If this parameter exists, it sets the Name and EmailAddress fields to the appropriate values from the specific Contact entity.

Email utility methods

In addition to the Email and EmailContact classes, Guidewire also provides a set of static utility methods in the `gw.api.email.EmailUtil` class for generating and sending emails from Gosu, for example:

```
gw.api.email.EmailUtil.sendEmailWithBody( KeyableBean entity, Email email )

gw.api.email.EmailUtil.sendEmailWithBody( KeyableBean entity,
                                           Contact to,
                                           Contact from,
                                           String subject,
                                           String body )

gw.api.email.EmailUtil.sendEmailWithBody( KeyableBean entity,
                                           String toEmailAddress,
                                           String toName,
                                           String fromEmailAddress,
                                           String fromName,
                                           String subject,
                                           String body )
```

These methods all take an entity as the first parameter. This parameter can be null. However, if specified, use the application entity to which this email is related, such as a specific policy or activity. The application uses this parameter only while processing the email for transmission. See “Email transmission” on page 288.

Emails that use an Email object

One variation of the `sendEmailWithBody` method requires that you create a `gw.api.email.Email` entity, and then define its properties to build the Email entity. For example:

```
...
var testEmail : gw.api.email.Email
testEmail.Body = "This is a test."
testEmail.Subject = "Test"
...
gw.api.email.EmailUtil.sendEmailWithBody( thisClaim, testEmail )
```

You can also attach multiple To, CC, and BCC addresses to the email.

Emails that use Contact objects

Another variation of the `sendEmailWithBody` method uses `Contact` objects for the `to` and `from` parameters. In Gosu, you can obtain `Contact` objects from various places. For example, in a policy rule, to send an email from an insurance company employee to the insured, do the following:

- Set the `to` parameter to `Policy.insured`.
- Set `from` to `Policy.AssignedUser.Contact`.

The following Gosu example generates an email from the current assigned user to that user's supervisor:

```
gw.api.email.EmailUtil.sendEmailWithBody(thisPolicy, thisPolicy.AssignedGroup.Supervisor.Contact,
    thisPolicy.AssignedUser.Contact, "A policy got a PolicyValid event", "This is the text.")
```

Emails that use an email address

Use the following variation of the `sendMailWithBody` method if you do not have a full `Contact` object for a recipient or sender. It is possible that another application dynamically generated the `Contact` object, providing an incomplete version of the object. The `sendMailWithBody` method uses a name and email address instead of entire `Contact` records and does not require that you have access to a `Contact` record.

In the following example, all arguments are `String` objects:

```
gw.api.email.EmailUtil.sendEmailWithBody( Entity, toName, toEmail, fromName, fromEmail, subject, body)
```

Email transmission

The application, from the user's perspective, sends emails asynchronously by using the application Messaging subsystem. If there is a method call for one of the `EmailUtil.sendEmail` methods, the application creates a `Message` entity with the contents and other information from the `Email` object.

In the `sendEmail` method parameters:

- If the entity parameter is non-null, the application adds the `Message` entity to the entity transaction bundle. The application persists the `Message` entity any time that it creates the bundle.
- If the entity parameter is null, the application persists the `Message` entity immediately.

The application then processes the messages one at a time and sends out the emails associated with that message.

Note: You must configure a `MessageTransport` class to consume the email messages and perform the actual transmission of the email.

About email templates

You use email templates to create the body of an email message. Typically, email templates are generally suitable only for boilerplate text that does not require modification, or, for presenting in the application interface as the starting point for further modification. However, email templates do support Gosu expressions, both in the Subject string, and in the Body string. It is also possible for an email template to be in HTML-formatted text. However, you cannot use Email templates for mail-merge types of operations.

By default, email templates live in the following location in Studio:

configuration > config > resources > emailtemplates

Email template files

An email template consists of two separate files:

- A descriptor file that contains some metadata about the template. The file name must end in `.gosu.descriptor`,

- A template file that contains the desired contents of the email body. The file name must end in `.gosu`.

IMPORTANT: The file names of the descriptor and template files must match.

An email descriptor file contains the following fields:

Field	Description
body	The email body of the emails created using this template
keywords	A list of keywords for use in searching the templates
name	The name of the template
subject	The email subject of the emails created using this template
topic	The topic of the template

For example, `EmailReceived.gosu.descriptor` defines an *Email Received* descriptor file:

```
<?xml version="1.0" encoding="UTF-8"?>
<serialization>
  <emailtemplate-descriptor
    name="Email Received"
    keywords="email"
    topic="reply"
    subject="Email Received"
    body="EmailReceived.gosu"
  />
</serialization>
```

The `EmailReceived.gosu.descriptor` file pairs with the actual template file (`EmailReceived.gosu`):

Thank you for your correspondence. It has been received and someone will contact you shortly to follow up on your feedback.

Sincerely,

Create an email template file

Procedure

1. In the **Project** window in Guidewire Studio, expand **configuration > config > resources**.
2. Select **emailtemplates**, right-click, then select one of the following items from the context menu:

Template type	Menu path	Extension to add	Note
Text	New > File	.gosu	Select Text from the list of file types to associate with the <code>.gosu</code> extension, if necessary.
HTML	New > HTML File	.gosu.html	Set the HTML version to use in the HTML File dialog.

Studio opens an editable file with the given name in a new tab.

3. Enter the text of the email message to send in the template.
For example:

```
Greetings:

Please contact <%= activity.AssignedUser %> at <%= activity.AssignedUser.Contact.WorkPhone %>
in regards to <%= activity.Subject %>

Thank you for your patience while we resolve this issue.

Sincerely,
```

Create an email template descriptor file

Procedure

1. In the **Project** window in Guidewire Studio, expand **configuration > config > resources**.
2. Select **emailtemplates**, right-click, then select one of the following items from the context menu:

Descriptor type	Menu path	Extension to add	Note
Text	New > File	.gosu.descriptor	Select Text from the list of file types to associate with the .gosu.descriptor extension, if necessary.
HTML	New > HTML File	.gosu.html.descriptor	Set the HTML version to use in the HTML File dialog.

Studio opens an editable file with the given name in a new tab.

3. Enter text similar to the following, modifying the given text to suit your business needs.

```
<?xml version="1.0" encoding="UTF-8"?>
<serialization>
  <emailtemplate-descriptor name="Test Email" keywords="email" topic="reply" subject="Test Email"
    body="TestEmail.gosu" />
</serialization>
```

Localize an email template

Localizing an email template is generally a straightforward process of creating localized versions of the template and its descriptor file and placing the files in the correct location.

About this task

Note: To use a localized email template in the application, perform a search for the localized template as you create a new email.

Procedure

1. In Studio, create a locale folder for the template files in the **resources > emailtemplates** folder.
The following example illustrates how to create a locale folder for Japanese.
resources > emailtemplates > ja_JP
2. Within the locale folder, place localized versions of the email template file and its associated template descriptor file.

Sending emails from Gosu code

To send an email from Gosu, you need to do the following:

- First, create the email, typically through the use of email templates.
- Send the email using one of the `gw.api.email.EmailUtil.sendEmailWithBody` methods.

Transaction bundle commits

It is possible to send an email either from a Gosu rule or directly from Gosu code. Which method you use affects whether you need to commit the email transaction.

Rule execution	If you create and send an email as part of rule workflow, you do not need to commit the email transaction as it is already part of the rule transaction commit. Creating an email message and storing it in the Send Queue occurs as part of the same database transaction in which the rules run. This is the same as regular message creations triggered through rules.
Gosu execution	If you create and send an email directly from Gosu code, outside of a Gosu rule, it is more likely that you need to commit the email transaction, depending on the exact circumstances of your Gosu code.

Saving an email message as a document

The application does not store email objects created through the “gw.api.email.Email” class in the database. However, it is possible to save the contents, recipient information, and other information of an email as a document in the configured DMS application.

Because all variations of the `gw.api.email.EmailUtil.sendEmailWithBody` methods take all of the sending information explicitly, you can save the email information by using the following process in Gosu code:

1. Create the email subject and body and determine recipients.
2. Send the email by calling one of the `sendEmailWithBody` methods.
3. If the application does not encounter an exception, create a document recording the details of the sent email.

Example code

For a simple example of the code needed to save an email as a document, refer to either the `BillingCenter` or `ClaimCenter CreateEmailScreen.pcf` PCF file. To see the example, select the highest level element, then open the Code tab at the bottom of the screen.

If desired, it is possible to replicate this functionality in Gosu code using the sample code as the basis for your code. Possible improvements to the example code include the following:

- Add a header to the document with metadata to determine the `docContainer` object.
- Add a BCC (Blind Carbon Copy) to the email so as to create a way to archive the email addresses.
- Create a batch process that fetches the emails in a specific account and stores any outgoing email as a document.

It is important to understand that the sample code in the `CreateEmailScreen.pcf` PCF file performs a transaction bundle commit. In most cases, you do not need to perform the bundle commit in your code:

- If calling the code from a Gosu rule, the rule is already in a commit bundle.
- If calling the code from workflow, the workflow is already in a transaction bundle.
- If calling the code from a work queue, the workqueue is mostly likely already in a transaction bundle.

Creating an HTML email within code

The InsuranceSuite email subsystem sets the email content type to HTML only if the HTML property on the email is set to true. The following examples show how to work with HTML emails:

- The `EmailEnhancement` example is an email enhancement that sets the HTML property based on the template file extension. The enhancement also sets other useful header values that exist in an email object.
- The `CreateEmailScreen` example shows how to modify the base configuration `CreateEmailScreen` to add HTML template-related fields.

Note: If you are creating an email inline and you are sure that there is no possibility for an attack in the HTML, you can set the email HTML property directly. However, if you do so, ensure that you encode any user-supplied strings.

EmailEnhancement.gsx

The following code first populates a number of message headers, then provides a `useEmailTemplate` method that example PCF screen `CreateEmailTemplate` calls to set up the HTML email.

```
package gw.api.email

uses gw.api.util.DisplayableException
uses gw.document.TemplatePluginUtils
uses gw.plugin.email.IEmailTemplateDescriptor
uses java.io.StringReader
uses java.util.Map
uses java.util.HashSet
```

```

uses gw.api.util.LocaleUtil
uses gw.api.system.PLLLoggerCategory

enhancement EmailEnhancement : gw.api.email.Email {

    // list of headers, note the value should be ascii-7 or encoded
    property get AUTOMATION_HEADER() : String { return "x-gw-Automation" }

    property get EXPIRATION_HEADER() : String { return "Expiry-Date" }
    property get REPLY_TO_ID_HEADER() : String { return "In-Reply-To" }
    property get ID_HEADER() : String { return "Message-ID" }
    property get REF_IDS_HEADER() : String { return "References" } // space delimited

    property get RETURN_RECEIPT_ADDR_HEADER() : String { return "Return-Receipt-To" }
    property get RETURN_READ_ADDR_HEADER() : String { return "Disposition-Notification-To" }

    property get THREAD_HEADER() : String { return "Thread-Topic" }
    property get THREAD_INDEX_HEADER() : String { return "Thread-Index" }

    property get PRIORITY_HEADER() : String { return "Priority" }
    property get PRIORITY_LOW() : String { return "5" }
    property get PRIORITY_HIGH() : String { return "1" }

    property get IMPORTANCE_HEADER() : String { return "Importance" }
    property get IMPORTANCE_LOW() : String { return "low" }
    property get IMPORTANCE_HIGH() : String { return "high" }

    property get SENSITIVITY_HEADER() : String { return "Sensitivity" }
    property get SENSITIVITY_CONFIDENTIAL() : String { return "Company-Confidential" }
    property get SENSITIVITY_HIGH() : String { return "high" }

    function useEmailTemplate(template : IEmailTemplateDescriptor, beans : Map<String,Object>) {
        this.Html = template.Html
        try {
            var locale = template.Locale
            if (locale == null) {
                locale = LocaleUtil.getDefaultLocale()
            }
            TemplatePluginUtils.resolveTemplates( locale,
                {new StringReader(template.Subject), new StringReader(template.Body)},
                // setup the symbol table for the template processing, this is the same between email and note
                \ iScriptHost -> {
                    // load the symbols supplied by the caller
                    var seen = new HashSet<String>()
                    for (entry in beans.entrySet()) {
                        var bean = entry.getValue()
                        iScriptHost.putSymbol(entry.Key, typeof(bean) as String, bean)
                        seen.add(entry.Key.toLowerCase())
                    }
                    // now load (or copy from other possible symbol names) the symbols that could be expected
                    // cc: claim, ...
                    // pc: policy, account, policyperiod, job, ...
                    if (not seen.contains( "activity" )) {
                        iScriptHost.putSymbol( "activity", Activity as String, null )
                    }
                    if (not seen.contains( "user" )) {
                        iScriptHost.putSymbol( "user", User as String, User.util.CurrentUser )
                    }
                },
                // process the result of the template expansion
                \ results -> {
                    this.Subject = results[0]
                    this.Body = results[1]
                } )
        } catch (e : Throwable) {
            PLLLoggerCategory.MESSAGING_EMAIL.info("On ${template.getName()}", e)
            throw new DisplayableException("On ${template.getName()} caught ", e);
        }
    }
}

```

CreateEmailScreen.pcf

The following example code adds additional functionality to the base configuration application CreateEmailScreen PCF. This example version of the PCF screen adds HTML template-related fields to the screen, include a read-only view of the HTML template. This field (a `TextAreaInput` widget) contains a `PostOnChange` action that calls method `executeTemplate`, which, in turn, calls the `useEmailTemplate` method from `EmailEnhancement.gsx`.

Note: This example uses display keys that do not exist in the base configuration. You must add the missing display keys to `display.properties` to see the correct labels.

```

<?xml version="1.0"?>
<PCF
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

```

```

xsi:noNamespaceSchemaLocation="../../../pcf.xsd">
<Screen
  editable="true"
  id="CreateEmailScreen">
  <Require
    name="srcBean"
    type="Activity"/>
  <Require
    name="docContainer"
    type="Activity"/>
  <Require
    name="emailTemplateName"
    type="String"/>
  <Require
    name="documentsToSend"
    type="Document[]"/>
  <Variable
    initialValue="null"
    name="documentToSave"
    type="Document"/>
  <Variable
    initialValue="emailTemplateName == null"
    name="noDefaultTemplate"
    type="Boolean"/>
  <Variable
    initialValue="false"
    name="saveAsDocument"
    type="boolean"/>
  <Variable
    initialValue="false"
    name="showCC"
    type="boolean"/>
  <Variable
    initialValue="false"
    name="showBCC"
    type="boolean"/>
  <Variable
    initialValue="gw.api.util.LocaleUtil.getDefaultLanguageType()"
    name="language"
    type="LanguageType"/>
  <Variable
    initialValue="initializeSymbolTable()"
    name="symbolTable"
    type="java.util.Map<String, Object>"/>
  <Variable
    initialValue="new String()"
    name="content"
    type="String"/>
  <Variable
    initialValue="emailTemplateName == null ? null : fetchTemplate()"
    name="template"
    type="gw.plugin.email.IEmailTemplateDescriptor"/>
  <Variable
    initialValue="initNewEmail()"
    name="NewEmail"
    type="gw.api.email.Email"/>
  <ToolBar>
    <ToolBarButton
      action="sendEmailToRecipients(NewEmail)"
      available="true"
      id="ToolBarButton0"
      label="DisplayKey.get("Web.Activity.Email.SendEmail")"
      visible="true"/>
    <ToolBarButton
      action="CurrentLocation.cancel()"
      available="true"
      id="ToolBarButton1"
      label="DisplayKey.get("Web.Activity.Email.Cancel")"
      visible="true"/>
    <ToolBarDivider/>
    <PickerToolBarButton
      action="PickEmailTemplatePopup.push(createSearchCriteria())"
      id="EmailWorksheet_UseTemplateButton"
      label="DisplayKey.get("Web.Activity.Email.UseTemplate")"
      onPick="template = PickedValue;
        language = gw.api.util.LocaleUtil.toLanguageType(PickedValue.Locale);
        executeTemplate(NewEmail)"
      shortcut="P"
      visible="noDefaultTemplate"/>
  </ToolBar>
  <AlertBar
    id="NoDefaultTemplate"
    label="DisplayKey.get("Web.Email.Template.NotFound"; emailTemplateName)"
    showConfirmMessage="false"
    visible="emailTemplateName != null and noDefaultTemplate"/>
  <DetailViewPanel>
    <InputColumn>
      <TypeKeyInput
        editable="true"
        id="Language"

```

```

label="DisplayKey.get(&quot;Web.EmailTemplateSearch.Language&quot;);"
required="true"
value="language"
valueType="typekey.LanguageType"
visible="LanguageType.getTypeKeys( false ).Count > 1 and emailTemplateName != null">
<PostOnChange
    onChange="template = fetchTemplate(); executeTemplate(NewEmail)"/>
</TypeKeyInput>
<ListViewInput
    editable="true"
    id="ToRecipientLVInput"
    label="DisplayKey.get(&quot;Web.Activity.Email.ToRecipients&quot;);"
    labelAbove="true"
    visible="true">
<Toolbar
    visible="true">
<IteratorButtons
    addVisible="true"
    iterator="ToRecipientLV"
    removeVisible="true"/>
<ToolbarDivider/>
</Toolbar>
<ListViewPanel
    editable="true"
    id="ToRecipientLV"
    visible="true">
<RowIterator
    autoAdd="true"
    editable="true"
    elementName="ToRecipient"
    numEntriesRequired="1"
    numEntriesToAdd="1"
    toCreateAndAdd="var newEmailContact = new gw.api.email.EmailContact();
        NewEmail.addToRecipient(newEmailContact); return newEmailContact;"
    toRemove="NewEmail.removeToRecipient( ToRecipient )"
    validationLabel="DisplayKey.get(&quot;Web.Activity.Email.ToRecipients&quot;);"
    value="NewEmail.ToRecipients?.toArray()"
    valueType="gw.api.email.EmailContact[]">
<Row
    editable="true">
<TextCell
    editable="true"
    id="ToName"
    label="DisplayKey.get(&quot;Web.Activity.Email.Name&quot;);"
    maxChars="60"
    numCols="15"
    value="ToRecipient.Name"/>
<TextCell
    editable="true"
    id="ToEmail"
    label="DisplayKey.get(&quot;Web.Activity.Email.EmailAddress&quot;);"
    numCols="15"
    requestValidationExpression="VALUE == null ?
        DisplayKey.get(&quot;Web.Activity.Email
            .Error.AddressForToRecipientRequired&quot;): null"
    required="true"
    value="ToRecipient.EmailAddress"/>
</Row>
</RowIterator>
</ListViewPanel>
</ListViewInput>
<ButtonInput
    action="showCC = true"
    id="ShowCCRecipients"
    labelAbove="true"
    value="DisplayKey.get(&quot;Web.Activity.Email.AddCCRecipients&quot;);"
    visible="!showCC"/>
<ListViewInput
    editable="true"
    id="CcRecipientLVInput"
    label="DisplayKey.get(&quot;Web.Activity.Email.CCRecipients&quot;);"
    labelAbove="true"
    visible="showCC">
<Toolbar
    visible="true">
<IteratorButtons
    addVisible="true"
    iterator="CcRecipientLV"
    removeVisible="true"/>
</Toolbar>
<ListViewPanel
    editable="true"
    id="CcRecipientLV"
    visible="true">
<RowIterator
    editable="true"
    elementName="CcRecipient"
    toCreateAndAdd="var newEmailContact = new gw.api.email.EmailContact();
        NewEmail.addCcRecipient(newEmailContact); return newEmailContact;"
    toRemove="NewEmail.removeCcRecipient( CcRecipient )"

```

```

        value="NewEmail.CcRecipients?.toArray()"
        valueType="gw.api.email.EmailContact[]">
    <Row>
        editable="true">
            <TextCell>
                editable="true"
                id="CcName"
                label="DisplayKey.get(&quot;Web.Activity.Email.Name&quot;)"
                numCols="15"
                value="CcRecipient.Name"/>
            <TextCell>
                editable="true"
                id="CcEmail"
                label="DisplayKey.get(&quot;Web.Activity.Email.EmailAddress&quot;)"
                numCols="15"
                required="true"
                value="CcRecipient.EmailAddress"/>
            </Row>
        </RowIterator>
    </ListViewPanel>
</ListViewInput>
<ButtonInput>
    action="showBCC = true"
    id="ShowBCCRecipients"
    labelAbove="true"
    value="DisplayKey.get(&quot;Web.Activity.Email.AddBCCRecipients&quot;)"
    visible="!showBCC"/>
<ListViewInput>
    editable="true"
    id="BccRecipientLVInput"
    label="DisplayKey.get(&quot;Web.Activity.Email.BCCRecipients&quot;)"
    labelAbove="true"
    visible="showBCC">
    <Toolbar>
        visible="true">
            <IteratorButtons>
                addVisible="true"
                iterator="BccRecipientLV"
                removeVisible="true"/>
            </Toolbar>
    <ListViewPanel>
        editable="true"
        id="BccRecipientLV"
        visible="true">
        <RowIterator>
            editable="true"
            elementName="BccRecipient"
            toCreateAndAdd="var newEmailContact = new gw.api.email.EmailContact();
                NewEmail.addBccRecipient(newEmailContact); return newEmailContact;"
            toRemove="NewEmail.removeBccRecipient( BccRecipient )"
            value="NewEmail.BccRecipients?.toArray()"
            valueType="gw.api.email.EmailContact[]">
            <Row>
                editable="true">
                    <TextCell>
                        editable="true"
                        id="BccName"
                        label="DisplayKey.get(&quot;Web.Activity.Email.Name&quot;)"
                        numCols="15"
                        value="BccRecipient.Name"/>
                    <TextCell>
                        editable="true"
                        id="BccEmail"
                        label="DisplayKey.get(&quot;Web.Activity.Email.EmailAddress&quot;)"
                        numCols="15"
                        required="true"
                        value="BccRecipient.EmailAddress"/>
                    </Row>
                </RowIterator>
            </ListViewPanel>
        </ListViewInput>
    <InputDivider/>
    <CheckBoxInput>
        editable="true"
        id="SaveAsDocument"
        value="saveAsDocument"
        valueLabel="DisplayKey.get(&quot;Web.Activity.Email.SaveAsDocument&quot;)">
    </InputColumn>
<InputColumn>
    <TextInput>
        editable="true"
        id="SenderName"
        label="DisplayKey.get(&quot;Web.Activity.Email.SenderName&quot;)"
        value="NewEmail.Sender.Name"/>
    <TextInput>
        editable="true"
        id="SenderEmail"
        label="DisplayKey.get(&quot;Web.Activity.Email.SenderEmail&quot;)"
        value="NewEmail.Sender.EmailAddress"/>
    <TextInput>

```

```

        editable="true"
        id="Subject"
        label="DisplayKey.get(&quot;Web.Activity.Email.Subject&quot;)"
        requestValidationExpression="VALUE == null ?
            DisplayKey.get(&quot;Web.Activity.Email.Error.SubjectRequired&quot;) : null"
        required="true"
        value="NewEmail.Subject"/>
<TextAreaInput
    editable="true"
    id="Content"
    label="template.ContentPrompt"
    numCols="60"
    numRows="10"
    required="false"
    value="content"
    visible="template.Html">
    <PostOnChange
        onChange="executeTemplate(NewEmail)"/>
</TextAreaInput>
<TextAreaInput
    editable="true"
    id="Body"
    label="DisplayKey.get(&quot;Web.Activity.Email.Body&quot;)"
    numCols="60"
    numRows="10"
    requestValidationExpression="VALUE == null ?
        DisplayKey.get(&quot;Web.Activity.Email.Error.BodyRequired&quot;) : null"
    required="true"
    value="NewEmail.Body"
    visible="!template.Html"/>
<ListViewInput
    editable="true"
    id="AttachedDocuments"
    label="DisplayKey.get(&quot;Web.Activity.Email.AttachedDocuments&quot;)">
    <Toolbar>
        <PickerToolbarButton
            action="PickExistingDocumentPopup.push(docContainer)"
            id="AddDocumentButton"
            label="DisplayKey.get(&quot;Web.Activity.Email.AddDocument&quot;)"
            onPick="NewEmail.addDocument(PickedValue)"
            shortcut="A"
            visible="true"/>
        <IteratorButtons
            addVisible="false"
            iterator="AttachedDocumentsLV"/>
    </Toolbar>
    <ListViewPanel
        editable="true"
        id="AttachedDocumentsLV">
        <RowIterator
            editable="true"
            elementName="AttachedDocument"
            toRemove="NewEmail.removeDocument( AttachedDocument )"
            value="NewEmail.Documents?.toTypedArray()"
            valueType="entity.Document[]">
            <Row>
                <TextCell
                    editable="true"
                    id="Document"
                    label="DisplayKey.get(&quot;Web.Activity.Email.DocumentName&quot;)"
                    value="AttachedDocument.Name"/>
            </Row>
        </RowIterator>
    </ListViewPanel>
</ListViewInput>
</InputColumn>
</DetailViewPanel>
<TemplatePanel>
    <TemplatePanelContents><![CDATA[
        <% if (template != null && template.Html) printContent(NewEmail.Body, false) %>]]>
    </TemplatePanelContents>
</TemplatePanel>
<Code><![CDATA[function initializeSymbolTable() : java.util.Map<String,Object> {
return { "activity" -> srcBean }
}

function createSearchCriteria() : gw.api.email.EmailTemplateSearchCriteria {
var rtn = new gw.api.email.EmailTemplateSearchCriteria()
rtn.Language = language
rtn.AvailableSymbols = symbolTable.Keys?.toTypedArray()
return rtn
}

function initNewEmail() : gw.api.email.Email {
var rtn = new gw.api.email.Email()
if (documentsToSend != null) {
for (document in documentsToSend) {
rtn.addDocument( document )
}
}
}

```



```

    if (template != null) {
        executeTemplate(rtn)
    }
    return rtn
}

function fetchTemplate() : gw.plugin.email.IEmailTemplateDescriptor {
    template = gw.plugin.Plugins.get(gw.plugin.email.IEmailTemplateSource)
        .getEmailTemplate(gw.api.util.LocaleUtil.toLanguage(language), emailTemplateName);
    if (template == null) {
        noDefaultTemplate = true
        throw new gw.api.util.DisplayableException(DisplayKey
            .get("Web.Activity.EmailTemplate.Language", emailTemplateName, language))
    }
    return template
}

function executeTemplate( email : gw.api.email.Email) {
    email.useEmailTemplate(template, { "activity" -> srcBean, "content" -> content })
}

function sendEmailToRecipients(emailToSend : gw.api.email.Email) {
    var warnings = gw.api.email.EmailUtil.emailContentsValid(emailToSend)
    if (warnings.length > 0) {
        throw new gw.api.util.DisplayableException(warnings)
    }
    if (saveAsDocument) {
        var templatePlugin = gw.plugin.Plugins.get(gw.plugin.document.IDocumentTemplateSource)
        var emailSentDocTemplate = templatePlugin
            .getDocumentTemplate("EmailSent.gosu.htm", gw.api.util.LocaleUtil.getDefaultLocale())
        if (emailSentDocTemplate == null) {
            throw new gw.api.util.DisplayableException("Could not save email as a document
                because the \"EmailSent.gosu.htm\" does not exist!")
        } else {
            documentToSave = documentToSave != null ? documentToSave : new Document()
            documentToSave.Name = emailToSend.Subject
            documentToSave.MimeType = emailSentDocTemplate.MimeType
            documentToSave.Type = typekey.DocumentType.get(emailSentDocTemplate.TemplateType)
            documentToSave.Section = typekey.DocumentSection
                .get(emailSentDocTemplate.getMetadataPropertyValue( "section" ) as String)
            documentToSave.SecurityType = typekey.DocumentSecurityType
                .get(emailSentDocTemplate.DefaultSecurityType)
            documentToSave.Status = TC_FINAL
            var recpName = emailToSend.ToRecipients.first().Name
            documentToSave.Recipient = recpName == null ? emailToSend.ToRecipients.first().EmailAddress : recpName
            documentToSave.Activity = docContainer
            documentToSave.DateCreated = gw.api.util.DateUtil.currentDate()
            documentToSave.Author = User.util.CurrentUser.DisplayName
            documentToSave.Inbound = false

            var paramMap = new java.util.HashMap()
            paramMap.put("User", User.util.CurrentUser)
            paramMap.put("Email", emailToSend)
            paramMap.put("DateSent", gw.api.util.DateUtil.currentDate())
            gw.document.DocumentProduction
                .createAndStoreDocumentSynchronously(emailSentDocTemplate, paramMap, documentToSave);
        }
    } else if (documentToSave != null) {
        documentToSave.remove();
    }
    gw.api.email.EmailUtil.sendEmailWithBody(srcBean, emailToSend);
    // it didn't throw so reset email activity.EmailTemplate so that other templates can be used
    if (emailTemplateName != null and srcBean.typeis Activity) {
        if (srcBean.EmailTemplate == emailTemplateName) {
            srcBean.EmailTemplate = null
        }
    }
    CurrentLocation.commit();
}}]></Code>
</Screen>
</PCF>

```


Guidewire PolicyCenter job configuration

Jobs are the equivalent of policy transactions, such as submissions and policy changes, in the user interface. To configure policy transactions, you make changes to jobs.

Configuring jobs

Configuring PolicyCenter jobs involves making changes to the product model, Gosu classes and rules, page configuration files (PCF files), wizards, configuration parameters, plugins, and sometimes workflows. Each can be configured based on your business requirements. It is the combination of these elements that allows you to create or modify a policy.

Note: In PolicyCenter, the user interface uses the term policy transaction to refer to submissions, policy changes, and other policy transactions. Policy transactions are implemented as jobs in the data model, and referred to as jobs in PCF files, Gosu classes, and other configuration files. Therefore, the configuration documentation refers to policy transactions as jobs.

You can configure a job to use either the cloud job process (CJP) or the local job process (LJP). There are several differences between these two processes:

- You can configure the classes in the LJP, but you can only extend the classes in the CJP.
- With CJP, you can configure products to use Quoting Services. For more information, see the *Application Guide*.
- The CJP does not support high-volume quoting, side-by-side quoting, or quick quote.

In contrast to jobs, the product model defines the types of policies that you can offer to your customers. Each product can contain one or more lines of business offering various coverages. For more information, see the *Application Guide*.

Local job process

Local job processes (LJP) run quoting processes within PolicyCenter. You can configure the Gosu classes for LJP jobs.

Gosu classes for local job processes

The Gosu job code contains most of the programming logic for the job (policy transaction). In Studio, you can view the code for each job type by navigating to **Configuration > gsrc > gw > job > local**. The types of files that make up the job code are:

- **Job Class** – Each job type defines a subclass of the `JobProcess` class. For example, `SubmissionProcessImpl.gs` implements the `SubmissionProcess` interface that ultimately implements the `JobProcess` interface. This class defines the steps of a job. It also defines the core functionality and core properties of the job, such as request quote or bind. The job class is marked as `@Export`, so you can modify the class directly.

Be careful when you modify the job class because it is the primary controller for how jobs work. Modifications can easily cause the job process to break.

For information about classes, see the *Gosu Reference Guide*.

- **Job Subclass** – Rather than modifying the job class directly, you can indirectly change or add methods and properties to the job by defining a subclass. You may find it easier to maintain changes in a subclass rather than the job class because the subclass contains your code only.

For example, to create a subclass for the submission class, you create a file such as `YourCompanySubmissionProcessImpl.gs` that extends the `SubmissionProcessImpl` class:

```
class YourCompanySubmissionProcessImpl extends SubmissionProcessImpl {
    ...
}
```

Be careful when you modify a subclass because it modifies the job process and can easily cause the job process to break.

If you create a subclass, you must modify the job process customization plugin. For more information, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

- **Enhancement** – You can use a Gosu enhancement to add methods and properties to a class. These methods and properties are available to all objects of the enhanced type. The default application contains some enhancements to the job entities, and you may modify these or add your own enhancements. Enhancements often contain code that is not the core functionality of the job and that you can customize to meet your business needs. If you need to modify the enhancement for the submission class, edit `SubmissionEnhancement.gsx`.

IMPORTANT: The job class code may change in future releases. Therefore, be sure to use comments in the code to clearly delimit and document your changes. As part of moving to a new release, you may have to implement (not just upgrade) your changes again. Review your modifications to the job class and enhancement as well your modifications to job subclasses.

See also

- *Gosu Reference Guide*

Cloud job process

Jobs that use cloud job process can run multiple Quoting Services in parallel. While the underlying job process creation behavior is similar to the local job process, the way that you can customize the job processes differs. To customize cloud job process jobs, you extend the corresponding job process implementation class under `gw.cloud.jobprocess` and implement pre and post hooks.

Products managed by APD use cloud job process by default. However, configuration is required to enable cloud retrofit product with LOB-specific endpoints to use the cloud job process.

See also

- “Configure cloud job processes” on page 302
- For information on Quoting Services, see *Application Guide*
- For information on the `JobProcessCreationPlugin`, see *Plugins, Prebuilt Integrations, and SOAP APIs*.
- For information about how local job process products can adopt cloud job process, see [InsuranceSuite Product Guidance](#)

Configure cloud job processes

You can extend the cloud job process in Studio using a combination of the `CloudJobProcessCreationPlugin` and the cloud job process implementations available in the `gw.cloud.jobprocess.*` packages. In Studio, you can view `CloudJobProcessCreationPlugin` by navigating to **Configuration > gsrc > gw > cloud > plugin**.

To customize the cloud job process,

1. Extend the `CloudJobProcessCreationPlugin` class.

For example:

```
class CustomCloudJobProcessCreationPlugin extends CloudJobProcessCreationPlugin {}
```

2. Configure which products use the cloud job process.

By default, installed and visualized products that are managed by APD use the cloud job process. To use cloud job process for additional products, override the `useOrchestratedJobProcess` method in an extended `CloudJobProcessCreationPlugin` class.

For example, you can add SBO as a product that uses cloud job process:

```
protected function useOrchestratedJobProcess(period : PolicyPeriod) : boolean {
  return period.Policy.Product.ApdManaged || period.Policy.ProductCode == "Manual" ||
    period.Policy.ProductCode.equalsIgnoreCase("sbo")
}
```

3. Customize cloud job process implementations.

The cloud job process can be customized by extending the relevant classes. For example, you can override the `canStartCopyPolicyData` method in the `PolicyChangeProcessImpl` or `RenewalProcessImpl`.

4. Use pre and post hooks to implement customizations to methods you cannot override.

Methods such as `requestRate`, `requestQuote`, `bind`, and `issueJob` cannot be overridden. For backward compatibility, `runPreRate` invokes `runPreQuote` by default.

	Rate	Quote	Bind	Issue
Pre hook	<code>runPreRate</code>	<code>runPreQuote</code>	<code>runPreBindAndIssue</code>	<code>runPreBindAndIssue</code>
Post hook	<code>runPostRate</code>	<code>runPostQuote</code>	<code>runPostBindAndIssue</code>	<code>runPostBindAndIssue</code>

Example

You can customize the `SubmissionProcessImpl`. Create a custom class `CustomSubmissionProcessImpl` that extends `SubmissionProcessImpl`.

```
class CustomSubmissionProcessImpl extends SubmissionProcessImpl {
  construct(period : PolicyPeriod, jobTypePermissions : JobTypePermissions) {
    super(period, jobTypePermissions)
  }

  override function runPreBindAndIssue(bindAndIssue : boolean, warningsThrowException : boolean) {
    ...
  }
}
```

Then, in a `CustomCloudJobProcessCreationPlugin`, which extends the `CloudJobProcessCreationPlugin`, replace the call to `SubmissionProcessImpl` with `CustomSubmissionProcessImpl`.

```
class CustomCloudJobProcessCreationPlugin extends CloudJobProcessCreationPlugin {
  ...
  protected function createOrchestratedJobProcess(period : PolicyPeriod) : IJobProcess {
    switch (typeof period.Job) {
      ...
      case RewriteNewAccount:
        return new RewriteNewAccountProcessImpl(period, getJobTypePermissions(period, new
        RewriteNewAccountPermissions(period.Job)))
      case Submission:
        return new CustomSubmissionProcessImpl(period, getJobTypePermissions(period, new
        SubmissionPermissions(period.Job)))
      default:
        CloudJobProcessLogger.logError(PCLoggingMarker.CLOUD_JOB_PROCESS_CREATION, "Unhandled job subtype " +
        period.Job.DisplayType)
        throw new IllegalArgumentException("Unhandled job subtype " + period.Job.DisplayType)
    }
  }
}
```

Related Information

- To configure how the cloud job process reports validation warnings to the Observability plugin, see *InsuranceSuite Observability*.

Wizards for jobs

A wizard is a type of PCF file that contains a series of user interface screens. Wizards collect information and perform operations while guiding the user through a complex business transaction.

Each job (policy transaction) has a wizard associated with it. For example, the submission job has the `SubmissionWizard.pcf` file which contains the steps for collecting information about a submission. The wizard pre-qualifies the applicant, collects policy information, performs risk analysis, reviews the policy, views a quote, prints forms, and sets up billing.

Each wizard step has a screen such as the **New Submissions**, **Pre-Qualification**, **Policy Info**, **Quote**, and **Submission Bound** screens. Each screen is a PCF file that defines the screen and its component parts.

As the user moves through the wizard, the wizard calls functions in the job code for processing and may indirectly access rule sets and workflows. The job code contains most of the code for the job and also interacts with the rule sets and workflows. In some cases, the wizard accesses the workflow directly.

You can modify wizards to gather the data that you want users to collect while processing a job. In Studio, you can access the PCF and wizard files for jobs by navigating to **Page Configuration > pcf > job**.

Job workflows

PolicyCenter uses workflows for asynchronous steps and for automated processes. In the default installation, only renewal and cancellation jobs have workflows. Cancellation has a workflow that handles the potential waiting period between the time the cancellation is scheduled or issued and completed. Renewal has workflows to handle automated renewals.

The workflow calls and is called by the job code and by rules in the rule sets. In some cases, the wizard calls the workflow and sends control back to the wizard.

See also

- "Guidewire workflow" in *Workflow Configuration*

Job rule sets

Another way to add business logic for jobs and workflows is through rule sets. You can use rules to check for a simple set of conditions and then follow with a set of simple actions. Rule sets are grouped by function for the purpose of customizing a process, such as a submission, to follow established business procedures.

Note: Instead of using rules, you can also add your business logic in the Gosu job process code. Inserting logic in the job code allows for a more centralized job flow and is easier to debug. The default application puts most of the business logic in the Gosu job process code.

See also

- *Gosu Rules Guide*

System configuration parameters for jobs

There are configuration parameters that determine the behavior of the various jobs. You can find these parameters in `config.xml` which you can view in Studio by navigating to **configuration > config**. For more information about these parameters see:

- "Job-related parameters" in *Configuration Parameters*

- "Job expiration parameters" in *Configuration Parameters*

Changing jobs to expired status

A job in an Expired status indicates that it has been closed due to inactivity. An expired job is in a terminal state, and you cannot reopen it. However, you can clone an expired job into a new, open job.

The Job Expire batch process changes jobs from **New**, **Draft**, or **Quote** status to **Expired** if they have passed an expiration threshold. In the default configuration, the Job Expire batch process expires submission jobs with these statuses that are at least seven days past the effective date of the policy. In addition to submission jobs, you can enable expiration for other job types. You can configure the expiration threshold as the number of days past the effective date or the creation date.

Expiring jobs at an appropriate time can impact searching and reporting, and have a critical impact on business operations including purging. Select an expiration threshold that is consistent with the intended behavior of impacted business operations.

A job is expired after a sufficient and configurable interval of time has elapsed. A policy version can be expired when its status is either New, Draft, or Quoted. If the policy version is already closed by being withdrawn, issued, or not taken, then it will not be expired because it is already closed. When a policy version expires, its status changes to Expired. You can view the status in the **Submission Manager** screen or in the toolbar if you are in the submission wizard. A branch is not editable.

Configuring the job expire batch process

The Job Expire batch process triggers job expiration. The batch process, by default, runs every day at 6 A.M. as configured in the `scheduler-config.xml` file. The batch process expires all versions of a policy in a job that can be expired.

You can configure both the scheduling and the amount of time that must elapse before expiration. Configure the batch process by modifying the following parameters in the `config.xml` file:

- `JobExpirationEffDateThreshold` is the number of days past the effective date of the policy version before a job can be expired. PolicyCenter compares this value with the effective date of the policy version, the default value is 7 (a week).
- `JobExpirationCreateDateThreshold` is the number of days past the create date of the submission before a submission can be expired. The create date is the day the submission entered the system. PolicyCenter compares this value with `Job.CreateTime`.

Configuring job history events

History events record the progress of a job (policy transaction). PolicyCenter displays history events in the **History** menu link in the **Policy** tab. In the default configuration, the submission and renewal jobs log history events. In addition, creating a new account and changing an account creates history events. You can add history events to a job by using the `createCustomHistoryEvent` method of the `Job` class. This method has two signatures:

- The first `createCustomHistoryEvent` method creates a History with the given `CustomHistoryType` and description. The description argument is wrapped in a block so that it can be localized to the primary language of the policy or account.

```
function createCustomHistoryEvent(type : CustomHistoryType, description : block() : String) : History
```

- The second `createCustomHistoryEvent` method is similar, but allows you to provide the `originalValue` and `newValue` fields of the History.

```
function createCustomHistoryEvent(type : CustomHistoryType, description : block() : String,
    originalValue : String, newValue : String) : History
```

For examples, view the job classes, such as `RenewalProcess.gs`, in Studio.

The **History** screen displays a **Type** and **Description** for each event. In Studio, type names are defined in the `CustomHistoryType.ttx` typelist. Descriptions are defined in **Display Keys**. The submission job display keys are defined in `Submission.History` entries. The renewal job display keys are defined in `Job.Renewal.History` entries. You can localize both the type and description.

Example Code that Localizes the Event Message to the Primary Language

The description argument of `createCustomHistoryEvent` is wrapped in a block so that it can be localized to the primary language of the policy or account. This example describes the code that does this.

Code in `CancellationProcess.gs` calls the `Job.createCustomHistoryEvent` method:

```
private function startScheduledCancellation(processDate : Date) {
    ...
    if (InitialNotificationsHaveBeenSent) {
        Job.createCustomHistoryEvent(CustomHistoryType.TC_CANCEL_RESCHEDULE, \ ->
            displaykey.Job.Cancellation.History.Reschedule(processDate))
    }
    ...
}
```

The second parameter of `createCustomHistoryEvent` method is defined as:

```
description : block() : String
```

The description parameter is a method, `block()`, that returns a `String`. In code above, the parameter is the `displaykey.Job.Cancellation.History.Reschedule` method.

The `createCustomHistoryEvent` method contains code which temporarily switches the locale to the primary language of the policy and runs the description method. Therefore, the history event message is localized to the language of the policy.

See also

- *Gosu Reference Guide*
- *Globalization Guide*

History events in the default configuration

In the default configuration, history events are logged when changes are made in an account, job, and policy term.

Account

PolicyCenter logs the following history events in an account:

- An **Account created** event is created when new accounts are created through the user interface or the AccountAPI.
- An **Account changed** event is created:
 - When the account status changes from **Pending** to **Active** at the start of a submission.
 - When the account holder is changed to a different contact. The event contains the old and new values.

You can add additional history events by configuring PolicyCenter.

Job

PolicyCenter logs the following history events for jobs (policy transaction):

- Cancellations
 - A **Cancel reschedule** event is created when a cancellation is rescheduled.
- Policy change
 - A **PolicyChange created** event is created when the job is created.
 - A **PolicyChange Effective Date change** event is created when the edit effective date of the policy change changes.

- Questions
 - An **Answer changed** event is generated when you change an incorrect answer to a blocking question to a correct answer.
- Renewals
 - A **Renewal** event is created for numerous events in a renewal job. Since renewals are typically automated, the histories provide insight into the progress of the job. The **Renewal** event is created:
 - When an automated renewal is escalated or manually edited.
 - When a user presses the **Renew**, **NonRenew**, **Not taken**, or **Issue Now** buttons.
 - When the automated renewal enters the pending renewal, pending non-renewal or pending not taken states.
 - If the product is no longer available and the renewal will be non-renewed.
 - When the renewal is quoted.
 - When the renewal is withdrawn.
 - When the renewal completes issuance, non-renewal, or not taken.
- Rewrite
 - A **Rewrite created** event is created when the rewrite job starts.
- Submissions
 - A **Submission bound** event is generated when the job is bound.
 - A **Submission created** event is generated when the job is created.
 - A **Submission decline** event is generated when the job is declined.
 - A **Submission issued** event is generated when the job is issued.
 - A **Submission quoted** event is generated when the job is quoted.

Policy Term

PolicyCenter logs the following history event for policy terms:

- Pre-renewal
 - A **Pre-renewal** event is created when a user sets the pre-renewal direction of a policy.
 - A **Pre-renewal** event is created when a user non-renews a renewal. Although the non-renewal action occurs in the renewal job, the history event is added to the `PolicyTerm`. The history description in the user interface is made by concatenating certain pre-defined strings when the user adds or removes a non-renewal explanation pattern.

Configuring history search criteria

You can configure the search criteria for history events in the Gosu class `gw.history.HistorySearchCriteria`.

Multiple revision jobs and the job selected branch property

To support multi-revision jobs (policy transactions), there is a property `Job.SelectedVersion`, which indicates for multi-revision jobs which branch is the selected branch. Knowing which branch is the selected branch is important especially where there is more than one non-withdrawn branch in the job.

There is always a selected branch for any job. The selected branch is always one of the non-withdrawn branches, if one or more remain. This makes the selected branch a safe way to obtain a `PolicyPeriod` from a `Job`. The selected branch may or may not be the one that the user is actively modifying in the user interface. The selected branch depends upon the user's actions and any business logic that has been implemented. In some display and reporting logic, the selected branch is the one that is used when more than one branch exists.

The `SelectedVersion` property is an edge foreign key from Job to PolicyPeriod. In the database, this creates a join table called `pc_jobpolicyperiod` where the `OwnerID` column points to the job, and the `ForeignEntityID` property points to the PolicyPeriod.

If you withdraw a revision from a multi-revision branch and there is only one non-withdrawn branch, PolicyCenter automatically sets that branch to the active branch.

In the user interface for multi-revision jobs, you can select one revision by clicking **Selected**.

Selecting the underwriting company through segmentation

Guidewire PolicyCenter provides the ability to set the underwriting company for a policy on the **Policy Info** page for Issuance, Rewrite, Renewal, and Submission jobs.

- If the user has the requisite underwriting permission (Multiple company quote), then PolicyCenter displays a list of valid underwriting companies. The user can then select from the list and set the underwriting company.
- If the user does not have the necessary underwriting permission, then PolicyCenter displays an **Autoselect** button instead.

Segmentation Example

Segmentation classes determine the segmentation, and, therefore, the available underwriting companies for a policy. Large insurers typically license more than one subsidiary to underwrite policies on behalf of the insurer. The primary reason for having multiple subsidiaries is to accommodate state regulatory requirements. (Many states do not allow insurers to have more than one set of rates per underwriting company.) Therefore, if an insurer desires to offer multiple sets of rates in that state, the insurer must file each set of rates under a separate underwriting company for that state.

Underwriters typically profile an account to determine the segment (category) into which an account falls. The segment determines which underwriting company actually underwrites the policy. By extension, segmentation sets the rates for which an account is eligible. The underwriter then uses appropriate set of rates to quote the policy.

For example, a workers' compensation insurer can segment accounts into three different segments:

- High hazard
- Medium hazard
- Low hazard

The insurer has three underwriting companies corresponding to each of the segments. To generate a quote for the policy, an underwriter indicates the underwriting company with which to place the policy. The rating engine takes this information and uses it to calculate a quote for the policy with the appropriate set of rates.

In the following example from `WC_SegmentEvaluator`, segmentation sets the segmentation level for any policy written for the state of Hawaii to high risk.

```
override property get IsHighRisk() : boolean {
    return _policyPeriod.PrimaryLocation.State == TC_HI
}
```

Segmentation classes

The following Gosu classes implement segmentation:

- `SegmentEvaluator` – The interface
- `AbstractSegmentEvaluator` – Implements the `SegmentEvaluator` interface
- Subclasses – `DefaultSegmentEvaluator` and line specific ones like `WC_SegmentEvaluator` that is created from `PolicyLineMethods#createSegmentEvaluator`.

PolicyPeriodBaseEnhancement

The `PolicyPeriodBaseEnhancement.gsx` enhancement (`gw.policy.PolicyPeriodBaseEnhancement`) contains code for auto-selecting an underwriting company. In particular, it contains the following method:

```
autoSelectUWCompany
```

Guidewire intends that you customize this enhancement to meet your specific business needs.

This method contains simple logic for determining the best underwriting company for the `PolicyPeriod` based on segment and lowest price.

A user triggers the `autoSelectUWCompany` method within PolicyCenter if the user clicks **Autoselect** for underwriter. (As mentioned, this button is available only if the user does not have the `Multiple company quote underwriting` permission.)

You set this method on `PolicyPeriod` in Studio, in any of the following PCF files:

- `IssuanceWizard_PolicyInfoDV`
- `RenewalWizard_PolicyInfoDV`
- `RewriteWizard_PolicyInfoDV`
- `SubmissionWizard_PolicyInfoDV`

To see this in the base configuration, open one of these files (`IssuanceWizard_PolicyInfoDV`, for example). Then, find the **Autoselect** button on the PCF page and select it. View the action property for this widget (at the bottom of the page). You see the following:

```
policyPeriod.autoSelectUWCompany()
```

[getUWCompaniesForStates](#)

Use the following method to return a set of underwriting companies based on the effective dates and all the states needed for the `PolicyPeriod`.

```
PolicyPeriod.getUWCompaniesForStates(boolean allStates)
```

For the `allStates` parameter:

- If `true`, the method returns only the underwriting companies that are valid for all states for the `PolicyPeriod`. For example, suppose that the company requesting the policy does business in three different states (Colorado, New Mexico, and Arizona). You might, therefore, want to find only those underwriting companies that can do business in all three of those states.
- If `false`, the method returns underwriting companies that are valid for any state for the `PolicyPeriod`. For example, suppose that you have the same company that does business in Colorado, New Mexico, and Arizona. This time, you only want to find the set of underwriting companies that do business in at least one of those states. You might want this if you wanted to split the policy between multiple underwriting companies based on the state.

To verify that you have a valid underwriting company, use the following method:

```
PolicyPeriod.getUWCompaniesForStates(true).contains(myUWCompany)
```


Configuring submissions

This topic describes how to configure submission jobs (submission policy transactions).

Note: In PolicyCenter, the user interface uses the term policy transaction to refer to submissions, policy changes, and other policy transactions. Policy transactions are implemented as jobs in the data model, and referred to as jobs in PCF files, Gosu classes, and other configuration files. Therefore, the configuration documentation refers to policy transactions as jobs.

Use Studio to view and edit the Gosu classes and other files related to the renewal process.

Key files include:

- **Gosu classes** such as: `SubmissionProcessImpl.gs`. This class defines and begins the new submission process.
- **Gosu enhancement files** such as: `SubmissionEnhancement.gsx`. This enhancement allows you to augment classes and other types with additional methods and properties. An example is the `addToGroup` method which groups it into the appropriate Submission group or creates a new one if a valid group does not exist.
- **Page configuration files (PCF files)** such as `SubmissionWizard.pcf` or `SubmissionWizard_PaymentScreen.pcf`.
- **Typelists** such as `JobGroup`, or `QuoteType`.

Note: Since the submission process for any insurer can be configured based on business requirements, all discussions apply to the default application only.

See also

- “Configuring jobs” on page 301
- “Selecting the underwriting company through segmentation” on page 308
- *Application Guide*

Submission process Gosu class

Gosu classes contain the code which primarily controls the submission process. In Studio, open `SubmissionProcessImpl.gs` class in the `gw.job.selfmanaged` package. This is the primary class for submissions. The job classes are part of the default application.

This class contains the main actions involved in a submission. This class ultimately extends the `JobProcess` class. The `JobProcess` class contains methods that enable you to get the job, the policy period, and a quote if it exists. It cancels any open activities on the submission, auto assigns the user role, and checks to see if the job can be expired, withdrawn, or approved by an underwriter. It also checks to see if the job can be started as new.

The submission process starts a new submission and assigns a requestor and producer. The `beginEditing` method moves a new submission to **Draft** status. Segmentation classes determines the available underwriting companies for a policy.

Next, the submission process determines whether quoting can be initiated. The process determines if the branch can be quoted in its current state by calling the `canRequestQuote` method. It also determines whether the submission needs to be reviewed by an underwriter. If the submission began as a quick quote, it also converts it to a full application. That in turn allows a job to be bound `bindOnly` and `issue`.

After PolicyCenter starts the bind process for the submission (branch), the submission may be rejected, marked for review by an underwriter, or returned to **Draft** for additional editing. Otherwise, binding begins, which puts the submission in the **Binding** status, updates the policy number, and sends an event for additional processing.

To complete binding, PolicyCenter calls the `finishBinding` method. This method completes the binding of a policy period without issuing it. The policy period is marked as bound and the job is finished. However, if issuance is being held, then PolicyCenter promotes the branch to the main policy period. Branches with no hold status must go through the `finishIssuing` method. This method promotes the policy period by marking it **Bound** and the completes job.

After issuing a submission, the user can bill the submission. The billing system plugin interface `IBillingSystemPlugin` defines the contract between PolicyCenter and the billing system. For example, the billing plugin implementation sends messages to the billing system about new billing instructions associated with a new submission.

Note: The submission job no workflows because it has no background processes. PolicyCenter uses workflows only for background processes.

Modifying the Submission Process

If you need to change or add methods to the submission process, you can edit the job class directly or create a subclass (such as `YourCompanySubmissionProcessImpl.gs`).

For more information about job classes and job subclasses, see “Gosu classes for local job processes” on page 301.

If you create a subclass, you must modify the job process customization plugin. For more information, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

Submission enhancements

There are other submission enhancements that contain additional functionality. You can modify these files.

- `SubmissionEnhancement.gsx` – This enhancement creates a copy of a submission and commits it. However, PolicyCenter does not validate the copied submission during the commit.

In the `copyPolicyPeriod` method, you can insert code for any additional copying. For fields that you do not want to copy, you can erase or reset them to default values.

For entities that you do not want to copy, you can implement the `PolicyPeriod` plugin method `returnTypesToNotCopyOnNewPeriod`. In addition, you can implement the `postCopyBranchIntoNewPolicy` method of this plugin which is called just after the copy. See *Plugins, Prebuilt Integrations, and SOAP APIs*.

The `SubmissionEnhancement.addToGroup` method groups a submission job into the appropriate submission group or creates a new one if a valid group does not exist.

- `SubmissionGroupLettersEnhancement.gsx` – This enhancement returns an array of the producers of the submissions in the submission group. It also verifies, through the `canAnySubmissionSendLetter` method, if any submission in the batch can have a letter generated for it. If it does, then you see that option in the user interface. Lastly, there is a `sendConfirmationLetter` that is a placeholder for implementing through integration, a confirmation letter. Its purpose is to send a confirmation letter with every possible submission in the batch.

For more information about job enhancements, see “Gosu classes for local job processes” on page 301.

Submission integration

Submission web services

JobAPI primarily adds an activity to a job by using an activity pattern. It also has methods to find and withdraw jobs.

PolicyPeriodAPI is a web service for performing various operations on policy periods within PolicyCenter. There are methods to add the following:

- Add a note to the policy and policy period
- Add a document to the policy period
- Add a referral reason code to the policy period

WorkflowAPI is a web service for performing various operations on workflows within PolicyCenter. It can:

- Start an action (by invoking a trigger) to advance the workflow to the next step.
- Resume a workflow or all the workflows. The workflow engine can attempt to advance the workflow (or all workflows) to the next step. If there is an error, then it is logged.
- Suspend a workflow.

Submission plugins

Submissions use the following plugins.

Name	Description
IAccountPlugin	This account plugin interface contains methods for detecting existing accounts, performing name clearance, generating account numbers, and doing risk reservation.
IBillingSystemPlugin	This plugin notifies an external system about billing events.
IEffectiveTimePlugin	This plugin determines the initial time component of effective/expiration dates (the PeriodStart / PeriodEnd dates) for a policy period when starting a new job. Submission is one of the affected jobs. All of the methods of this interface have a PolicyPeriod parameter, which can be used to access additional information that might be used to determine the desired time, such as: • The line or lines of business • The base jurisdiction • Other revisions The return value from each method is a Date, with its time component, HH:mm:ss, set to the desired time. The plugin discards the day component, dd/MM/yyyy. As a convenience, each method returns null to leave the time component set to the default (midnight).
IJobNumberGenPlugin	This plugin generates a new unique job number.
AccountLocationPlugin	For account locations and policy locations, customize how PolicyCenter: • Determines that two locations are the same • Clones a location
IPolicyNumGenPlugin	This plugin returns an identifier for a new period of a policy. The policy number may vary from period to period. Call this plugin for the initial period of a policy as part of a Submission job. You can also call this plugin if you want to generate a new identifier when renewing or rewriting a policy.

IPolicyPeriodDiffPlugin This plugin does the following:

Name	Description
	- Compares two policy periods, returning a list of <code>DiffItems</code> representing the differences between them. The following call the <code>compareBranches</code> method: - Multi-version quote - Comparing <code>PolicyPeriods</code> of a Policy - Comparing Jobs of a Policy - Filters a list of <code>DiffItems</code> that originate from the database.
<code>IPolicyPeriodPlugin</code>	This plugin mainly calculates period end dates and creates the job process for that period.
<code>IVinPlugin</code>	For a VIN (Vehicle Identification Number), the <code>getVehicleInfo</code> method returns the model, make, year, and color of vehicle.

See also

- The *Plugins, Prebuilt Integrations, and SOAP APIs* for information about these plugins

Expiring submissions

The Job Expire batch process changes submission jobs from **New**, **Draft**, or **Quote** status to **Expired** if they have passed an expiration threshold. In the base configuration, the batch process expires submission jobs with these statuses that are at least seven days past the effective date of the policy. The batch process, by default, runs every day at 6 A.M. as configured in the `scheduler-config.xml` file. The batch process expires all versions of a policy in a job that can be expired. If the policy version is already closed by being withdrawn, issued, or not taken, then it will not be expired.

See also

- “Changing jobs to expired status” on page 305

Configuring the copy submission feature

You can copy any single `PolicyPeriod` on a submission. To modify the code, open the `SubmissionEnhancement.gsx` in the `gw.job` package in Studio. Find the method `copyPolicyPeriod`.

Configuring issuance

This topic describes how to configure issuance jobs (issuance policy transactions).

Note: In PolicyCenter, the user interface uses the term policy transaction to refer to submissions, policy changes, and other policy transactions. Policy transactions are implemented as jobs in the data model, and referred to as jobs in PCF files, Gosu classes, and other configuration files. Therefore, the configuration documentation refers to policy transactions as jobs.

The Gosu class, `gw.job.IssuanceProcessImpl.gs`, contains the main logic of an issuance job.

The `beginEditing` method starts the process of issuing by moving the policy to Draft status so it can be changed if necessary. Like the submission job, it also invokes segmentation.

The `issue` method starts issuance for the policy period and withdraws any other active periods. The policy has an Issuing status. The method also sends the policy to the `FormInferenceEngine.gs` class. The `FormInferenceEngine` serves as the main access point for the rest of the application logic into the form inference logic.

Note: The issuance job no workflows because it has no background processes. PolicyCenter uses workflows only for background processes.

See also

- “Selecting the underwriting company through segmentation” on page 308

Modifying the issuance process

If you need to change or create new methods in `IssuanceProcessImpl` class, you can edit the class directly or create a subclass (such as `YourCompanyIssuanceProcessImpl.gs`).

For more information about job classes and job subclasses, see “Gosu classes for local job processes” on page 301.

If you create a subclass, you must modify the job process customization plugin. For more information, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

Issuance integration

Issuance web services and plugins

The issuance job has the following web services and plugins.

Issuance web services

The issuance job has no web services. The JobAPI and PolicyPeriodAPI apply to jobs in general.

Issuance plugins

The issuance job uses the following plugin.

Billing System Plugin

When you complete an issuance policy transaction in the PolicyCenter user interface, PolicyCenter calls the `finishIssuing` method, and the method performs some final checks. The method changes the policy status to Bound and invokes the `IBillingSystemPlugin`.

The billing system plugin interface, `IBillingSystemPlugin`, defines the contract between PolicyCenter and the billing system. This plugin is the primary mechanism for PolicyCenter to get information from a billing system or to notify the billing system of changes to a policy.

Configuring renewals

This topic describes how to configure renewal jobs (renewal policy transactions).

Note: In PolicyCenter, the user interface uses the term policy transaction to refer to submissions, policy changes, and other policy transactions. Policy transactions are implemented as jobs in the data model, and referred to as jobs in PCF files, Gosu classes, and other configuration files. Therefore, the configuration documentation refers to policy transactions as jobs.

In PolicyCenter, you can configure the renewal job (policy transaction) through Studio. The job process is controlled by:

- **Gosu classes** such as: `RenewalProcessImpl.gs` in the `gw.job` package. This class contains the main functionality of the renewal process.
- **Gosu enhancement files** such as: `RenewalEnhancement.gsx`. This enhancement allows you to augment classes and other types with additional methods and properties. An example is the `addToGroup` method which groups the job into the appropriate renewal group or creates a new one if a valid group does not exist.
- **Workflows** such as `StartRenewalWF`.
- **Page configuration files (PCF files)** such as `PreRenewalDirectionPage.pcf`.
- **Typelists** such as `RenewalCode`.
- **Rule Sets** such as `Renewal AutoUpdate`.
- **System tables** such as the `NotificationConfig` system table.

Note: Since the renewal process for any insurer can be configured based on business requirements, all discussions apply to the default application only.

See also

- “Selecting the underwriting company through segmentation” on page 308
- *Application Guide*

Renewal process Gosu class

The `gw.job.RenewalProcessImpl.gs` class is the main class controlling the renewal process. Use Studio to view and edit the Gosu classes related to the renewal process.

This class contains the main logic of the renewal job, including referral direction handling. PolicyCenter checks multiple methods to see if the renewal can be automatically processed based on certain conditions. If PolicyCenter

cannot process the job automatically, it blocks the renewal from advancing, raises an issue, and refers the renewal to an underwriter so that it can be manually processed.

Note: The `JobProcess.gs` Gosu class contains methods that are common to all jobs, including renewal. You can view that class in Studio and see the complete list of methods.

The renewal process class includes methods that do the following:

- Starts the renewal
- Sends renewal documents
- Escalates the renewal
- Issues and binds the renewal
- Handles user actions such as:
 - **Issue Now** – Immediately binds the policy.
 - **NonRenew** – Puts the policy period in `NonRenewing` status.
 - **Not Taken** – Puts the policy period in `NotTaking` status.
 - **Renew** – Puts the policy period in `Renewing` status.
 - **Edit Policy Transaction** – Puts the period back in `Draft` mode for editing and checks the conditions for which a new version of the policy period can be created.
 - **Withdraw Transaction** – Withdraws the policy period. If the renewal is at a point where it cannot be withdrawn, then a message appears stating that the renewal cannot be withdrawn.

See also

- *Gosu Reference Guide*

Modifying the Renewal Process Class

If you need to change or create new the methods in that class, you can edit the class directly or create a subclass (such as `YourCompanyRenewalProcessImpl.gs`).

For more information about job classes and job subclasses, see “Gosu classes for local job processes” on page 301.

If you create a subclass, you must modify the job process customization plugin. For more information, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

Renewal enhancement

Enhancement files are editable, and you can add additional logic to the renewal process in `RenewalEnhancement.gs`.

Renewal workflows

Note: PolicyCenter uses workflows in certain job (policy transaction) situations. It is not necessary to use workflows to start a job. Only job steps that are asynchronous use workflows. For synchronous steps, Gosu classes and enhancements contain the logic.

In the default configuration of PolicyCenter, if the renewal process job is automated and not dependent on user actions, then a workflow controls the process. However, if manual intervention is necessary, then the renewal workflow is not used. The renewal process class handles manual renewal steps. See “Renewal process Gosu class” on page 317.

The renewal job uses the following workflows:

- “Start Renewal Workflow”
- “Renewal Timeout Workflow”

Start Renewal Workflow

In the `RenewalProcess.gs` class, the `start` method begins the `StartRenewalWF` workflow.

This simple workflow has `Begin` and `Done` steps. The `Begin` step gets the user which the automated renewal will use, and starts the automatic renewal by calling the `beginAutomaticRenewal` method defined in `RenewalProcess.gs`.

Renewal Timeout Workflow

The `scheduleTimeoutOperation` method calls the `Renewal Timeout` workflow. Various methods in the renewal process class call this method to:

- Wait until a timer expires
- Edit the policy and change the policy to draft mode.
- Withdraw the policy
- Issue the policy renewal

See also

- “Renewal process Gosu class” on page 317

Renewal rule sets

In PolicyCenter, you can also embed your business logic in rule sets. Use the rule sets if there is a need to check for a simple set of conditions. Follow with a set of simple actions if those conditions are met. Rule sets are grouped by function for the purpose of customizing a process, such as a renewal, to follow established business procedures. Edit rules in Studio in the **configuration > config > Rule Sets** directory.

Note: You can also add your business logic in Gosu, for example, putting additional renewal business logic in the `RenewalEnhancement.gsx` enhancement. Inserting logic in Gosu allows for a more centralized job flow and is easier to debug.

See the `Renewal` topic in the *Gosu Rules Guide* for additional information.

The following table lists the rule sets that PolicyCenter provides in the base configuration.

Rule set	Rule	Acts on	Description
Renewal	RenewalAutoUpdate	PolicyPeriod	Called when PolicyCenter creates a renewal job. This rule set can be used to perform automatic changes to policy information that need to occur during a renewal.

Renewal integration

Renewal web services

Renewal jobs have the following web service APIs.

Name	Description
PolicyRenewalAPI	The policy renewal web service provides methods for managing renewals in PolicyCenter. The policy renewal web service provides methods to: • Start renewals on policies that exist in PolicyCenter. There is no gap in coverage between the renewal and its based-on policy period. • Import a policy from a legacy system and renew the policy. There is no gap in coverage between the renewal and its based-on policy period.

Name	Description
	Process a renewal in a billing system For more information see <i>Plugins, Prebuilt Integrations, and SOAP APIs</i>.
JobAPI	The job web service primarily adds an activity to a job by using an activity pattern. This web service also has methods to find and withdraw jobs. The job ID of the activity is set to the given job ID. The previous UserID of the activity is set to the current user. The Assignment Engine assigns the newly created activity to the specified group and user. Finally, the activity is saved to the database, and the public ID of the newly created activity is returned. For more information, see <i>Plugins, Prebuilt Integrations, and SOAP APIs</i>.
PolicyPeriodAPI	The policy period web service provides methods for performing various operations on policy periods within PolicyCenter. It allows you to do the following: - Add a note to the policy and policy period - Add a document to the policy period - Add a referral reason code to the policy period For more information, see <i>Plugins, Prebuilt Integrations, and SOAP APIs</i>.
WorkflowAPI	The workflow web service provides methods for performing various operations on workflows within PolicyCenter. This web service can: - Invoke a trigger that starts an action to advance the workflow to the next step. - Resume a workflow or all workflows. The workflow engine can attempt to advance the workflow (or all workflows) to the next step. If there is an error, then the API logs it. - Suspend a workflow. For more information, see <i>Plugins, Prebuilt Integrations, and SOAP APIs</i>.

Renewal plugins

Renewal jobs use the following plugins. For the complete list, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

Name	Description
IPolicyRenewalPlugin	Determines whether to start a renewal job. For more information, see <i>Plugins, Prebuilt Integrations, and SOAP APIs</i> .
INotificationPlugin	Called by IPolicyRenewalPlugin. Returns the date a notification must be mailed. For more information, see <i>Plugins, Prebuilt Integrations, and SOAP APIs</i> .
IEffectiveTimePlugin	This plugin determines the initial time component of a policy period's effective/expiration dates (the PeriodStart / PeriodEnd dates) when starting a new job. Renewal is one of the affected jobs. All of this interface's methods have a PolicyPeriod parameter, which can be used to access additional information that might be used to determine the desired time, such as: - The line or lines of business - The base jurisdiction - Other revisions The return value from each method is a Date, with its time component, HH:mm:ss, set to the desired time. The plugin ignores the day component, dd/MM/yyyy. As a convenience, each method returns null to leave the time component set to the default (midnight)
IJobNumberGenPlugin	This plugin generates a new unique job number.
AccountLocationPlugin	For account locations and policy locations, customize how PolicyCenter: Determines that two locations are the same

Name	Description
	Clones a location
IPolicyNumGenPlugin	This plugin returns an identifier for a new period of a policy. The policy number may vary from period to period. Call this plugin for the initial period of a policy as part of a Submission job. You can also call this plugin if you want to generate a new identifier when renewing or rewriting a policy.
IPolicyPeriodDiffPlugin	This plugin does the following: - Compares two policy periods, returning a list of DiffItems representing the differences between them. The compareBranches method is called in the following: - Multi-version quote - Comparing PolicyPeriods of a Policy - Comparing Jobs of a Policy - Filters a list of Diff terms that originate from the database.
IPolicyPeriodPlugin	This plugin calculates period end dates and creates the job process for that period.
IPolicyPlugin	This plugin contains methods including canStartRenewal which checks to see whether the PolicyPeriod can be renewed.
IVinPlugin	For a VIN (Vehicle Identification Number), the getVehicleInfo method returns the model, make, year, and color of vehicle.

Renewal events

Events are changes in PolicyCenter that might be of interest to an external system. The renewal job uses the following events:

- SendRenewalDocuments
- SendNonRenewalDocuments
- SendNotTakenDocuments
- SendNotTaken
- SendNonRenewal
- IssueRenewal

See also

- Plugins, Prebuilt Integrations, and SOAP APIs*

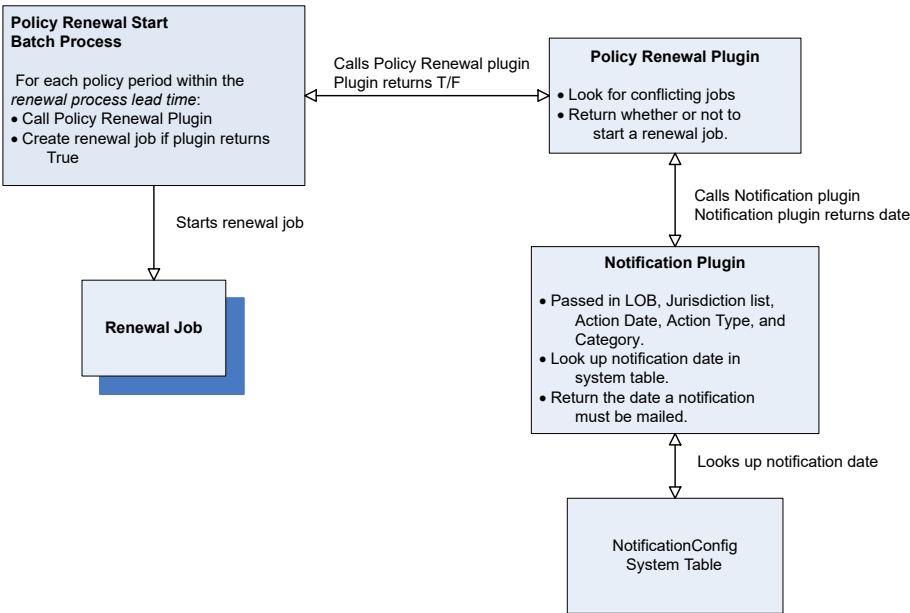
Configuring batch process renewals

PolicyCenter has a batch process that automatically finds policies that are ready for renewal. In the default configuration, PolicyCenter starts renewals based upon the expiration date, the *renewal process lead time*, line of business, jurisdiction, and time of year. If the expiration date of the policy period falls within the renewal process lead time, then the Renewal plugin calculates whether to start a renewal for the policy. Because the renewal process lead time is checked first, no policy will start automatic renewal sooner than this.

Renewal process lead time is specified in the RenewalProcessLeadTime parameter in the config.xml file.

Note: There are several factors to consider in scheduling the batch process. For more information, see the *Application Guide*.

The following illustration shows how the Policy Renewal batch process evaluates whether to create renewal jobs for policy periods.



PolicyCenter handles the renewal job process through a series of workflows that monitor its progress and move it to the next step. This series of workflows is the optimal way to begin renewals.

See also

- The *Application Guide*, which includes information on configuring the renewal process lead time.

Renewal plugin calculation

The Renewal plugin (IPolicyRenewalPlugin) determines whether or not to start a renewal on a policy period. You can modify this plugin to incorporate regulatory requirements and insurer practices for starting renewal processing.

In the default configuration, the Renewal plugin determines whether to start a renewal for the policy period in this way. It first determines what the regulations require. Then it adds additional time for company practices. Finally, it adds a delay for concurrent jobs, if any. These factors are described in:

- “Renewal lead time in notification config system table” on page 323
- “Example lead time by line of business” on page 323
- “Example lead time by time of year” on page 323
- “Delay for a conflicting job” on page 323

A renewal starts for a policy period when the current date is greater than or equal to the RenewalStartDate. The renewal start can be delayed for a conflicting job.

The RenewalStartDate for a policy period is the expiration date of the current period, moved earlier in time by the sum of the following:

Lead time	See
Renewal lead time in Notification Config system table	“Renewal lead time in notification config system table” on page 323
Example lead time by line of business	“Example lead time by line of business” on page 323
Example lead time by time of year	“Example lead time by time of year” on page 323

If the policy has multiple lines of business, then use the longest lead time determined across all lines of business on the policy. You can configure how to calculate the renewal lead time.

See also

- *Plugins, Prebuilt Integrations, and SOAP APIs*

Renewal lead time in notification config system table

The Notification Config system table is one of the factors in calculating the renewal process lead time.

Note: If the policy has multiple lines of business, then default configuration uses the longest lead time determined across all lines of business on the policy.

The Policy Renewal plugin calls the `getMaximumLeadTime` method in the Notification plugin. This method returns the maximum lead time based on the values in the Notification Config table. There are two signatures for this method. The Policy Renewal plugin calls the method with the signature that takes the `Category`, not the `ActionType`. You can find this table by navigating to **System Tables** in Product Designer and opening `notification_configs.xml`.

See also

- *Product Model Guide*

Example lead time by line of business

For renewals, the product lead time by line of business in the default configuration is listed in the following table.

Line of Business	Lead Time
Business Auto	30
Commercial Property	60
Commercial Package	90
General Liability	60
Inland Marine	60
Personal Auto	30
Workers Compensation	60

If the policy has multiple lines of business, then the Renewal plugin uses the longest lead time determined across all lines of business on the policy.

You can configure these values in the Renewal plugin.

Example lead time by time of year

The following table shows the default configuration of lead time by time of year (date ranges are inclusive):

Date Range	Lead Time
January 1 through January 15	15
January 16 through July 31	0
August 1 through September 15	-10
September 16 through end of year	0

You can configure this in the Renewal plugin. For more information, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

Delay for a conflicting job

The batch process does not start a renewal job if there is a conflicting job. A conflicting job is an open policy change or cancellation job. A policy that would normally renew on a given day will wait three additional days to start if there is a

conflicting job. After three days, the next run of the batch process starts the renewal, whether or not there is an open conflicting job.

The three day delay is set in the `CONCURRENT_JOB_DELAY_DAYS` variable which is defined in the Renewal plugin. You can modify how this plugin calculates and uses the delay for a conflicting job.

Note: The frequency of the batch process is a factor in when a delayed renewal starts. For example, you have a batch process that runs every Sunday. A delayed renewal that could start on Wednesday will not start until the batch process runs on Sunday, several days later.

Configuring explanations in pre-renewal directions

In a pre-renewal direction, the user can select a direction to not renew the policy. The user can add explanations to non-renewal directions. These explanations are configured as non-renewal explanation patterns. In Studio, **configuration > config > Import > gen** contains configuration files related to non-renewal explanation patterns:

File	Description
<code>nonRenewalExplanationPatterns.csv</code>	Definitions for non-renewal explanation patterns.
<code>importfiles.txt</code>	In the default configuration, this file specifies that <code>nonRenewalExplanationPatterns.csv</code> is imported when PolicyCenter is starts.

See also

- *Application Guide*

Configuring cancellations

This topic describes how to configure cancellation jobs (cancellation policy transactions).

Note: In PolicyCenter, the user interface uses the term policy transaction to refer to submissions, policy changes, and other policy transactions. Policy transactions are implemented as jobs in the data model, and referred to as jobs in PCF files, Gosu classes, and other configuration files. Therefore, the configuration documentation refers to policy transactions as jobs.

Use Studio to access the necessary files and code to configure the cancellation process to your business needs.

The key configuration files include:

- **Gosu job process class:** `gw.job.CancellationProcessImpl.gs`. This class contains the code behind the cancellation job (policy transaction). For more information, see “Cancellation Gosu classes”.
- **Gosu enhancement file:** `CancellationEnhancement.gsx`. This enhancement allows you to modify or add additional functions and properties to the cancellation job class. For more information, see “Cancellation Gosu classes”.
- **Workflow files:** `CompleteCancellationWF.1`. This file describes the workflow for completing the asynchronous parts of cancellation. `CancellationProcess.gs` starts the workflow. Clicking **Bind Options > Schedule Cancellation** or **Cancel Now** on the **Cancellation** screen calls the code to start the workflow. For more information, see “Complete cancellation workflow” on page 326.
- **Page configuration files (PCF files)** such as `CancellationWizard.pcf`. To view these PCF files, navigate in Studio to **configuration > config > Page Configuration > pcf > job > cancellation**.
- **Typelists** such as `CancellationSource` or `CalculationMethod`. You can view this typelist by navigating in Studio to **configuration > config > Metadata > Typelist**.
- **System tables** such as `notification_configs.xml`. You can view this table under **System Tables** in Product Designer.

Note: Since the cancellation process for any insurer can be configured based on business requirements, all discussions apply to the default application only.

[See also](#)

- *Application Guide*

Cancellation process class

Navigate in Studio to **configuration > gsrc** to view or edit the `gw.job.selfmanaged.CancellationProcessImpl.gs` Gosu class which contains the majority of cancellation logic. This class includes functions to start, schedule, quote, rescind, and issue a cancellation.

Start Function

Pressing **Start Cancellation** (in `StartCancellation.pcf`) indirectly invokes the `start` function. The **Start** button calls the `job.startJobAndCommit` method which indirectly calls `start`.

Modifying the Cancellation Process Class

If you need to change or create new methods in that class, you can edit the class directly or create a subclass (such as `YourCompanyCancellationProcessImpl.gs`).

For more information about job classes and job subclasses, see “Gosu classes for local job processes” on page 301.

If you create a subclass, you must modify the job process customization plugin. For more information, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

Cancellation enhancements

The cancellation enhancement, `CancellationEnhancement.gsx`, enhances the `Cancellation` entity with additional functionality. You can edit this file.

This enhancement contains functions that do the following:

- Determines which refund calculation functions displays based on the cancellation reason.
- Calculates the earliest and latest effective date of the cancellation.
- Returns the last date of the underwriting period.
- Withdraws later cancellations with the same based on period.

Complete cancellation workflow

Note: PolicyCenter uses workflows primarily for automation and coordination of activities between PolicyCenter and external systems and for observing time periods and other automation.

In a cancellation, the job process involves automation and waiting. You wait for a period of time before the cancellation job completes and the policy is canceled. During this time period, the insured may remit payment and the cancellation may be rescinded. The `CompleteCancellationWF.1` workflow manages the processes that are not user driven.

Complete Cancellation Workflow Steps

The workflow begins when you select **Bind Options > Schedule Cancellation** or **Cancel Now** through the PolicyCenter user interface.

In the default application, selecting **Schedule Cancellation** or **Cancel Now** in the user interface calls the `scheduleCancellation` method in `CancellationProcess.gs`. After notices are successfully sent through `sendNotices`, the `finishSendNotices` method starts the workflow.

Cancellation integration

PolicyCenter integrates with many types of external systems and custom code through a diverse toolbox of services. This topic describes web services and events that the cancellation job uses.

A common integration is with a billing system to start a cancellation for non-payment or to rescind a cancellation after receiving payment.

Another common integration is with a rating engine to calculate the premium amount.

Cancellation web services

The **Cancellation API** is a web service for use by external systems to begin, reschedule, rescind, or find a cancellation.

Invoke the `beginCancellation` method to begin a cancellation job. This method does not attempt to automatically push the cancellation through to completion. The method throws an exception if the cancellation cannot be started for any reason.

The `rescheduleCancellation` method allows you to reschedule a cancellation. You must specify the new cancellation effective date. Optionally, you can provide a new reason description. The cancellation can be in draft, quoted, or canceling status.

The `rescindCancellation` method rescinds a cancellation job. The method throws an exception if the rescind cannot be performed for any reason.

[See also](#)

- *Plugins, Prebuilt Integrations, and SOAP APIs*

Cancellation plugins

The following plugins might be useful for cancellation.

Name	Description
<code>IBillingPlugin</code>	If you need to integrate with an external billing system, use this plugin to notify an external billing system about billing events.
<code>MessageTransport</code>	Use this plugin to send a message to an external/remote system by using any transport protocol.
<code>IEffectiveTimePlugin</code>	Determines the time of day (since midnight) for a job. Use to calculate the effective time.
<code>IJobNumberGenPlugin</code>	Generates a new, unique job number. Use to generate cancellation job numbers.
<code>IPolicyPlugin</code>	Lets PolicyCenter know whether, based on dynamic calculations on the policy, it is OK to start various jobs. Use to determine if a cancellation job can be started.
<code>INotificationPlugin</code>	Returns cancellation lead time.

[See also](#)

Plugins, Prebuilt Integrations, and SOAP APIs

Cancellation events

PolicyCenter generates events for changes in PolicyCenter that might be of interest to an external system. You can also programmatically generate an event. For example, in the `CancellationProcess.gs` class, the `sendNotices` method generates a `SendCancellationNotices` event.

The Cancellation entity has the following events:

- `CancellationAdded`
- `CancellationChanged`
- `CancellationRemoved`

The PolicyPeriod entity has the following events pertaining to cancellation:

- `SendCancellationNotices` – Message to an external system to send cancellation notices.
- `IssueCancellation` – Message to an external System of Record (SOR) to send cancellation notices.
- `SendRescindNotices` – Message to an external system to send rescind cancellation notices.

[See also](#)

- *Plugins, Prebuilt Integrations, and SOAP APIs*

Calculating the cancellation effective date

PolicyCenter calculates the earliest cancellation effective date, based on cancellation reason, jurisdiction, line of business, and other factors. Methods in `CancellationEnhancement.gsx` calculate the date. In some cases, the calculation calls the Notification plugin which references the `notification_configs.xml` system table. The `notification_configs.xml` system table is where you specify the cancellation lead time by jurisdiction, line of business, and cancellation reason. You can view this table by navigating to **System Tables** in Product Designer.

Calculating the cancellation effective date if the source is insured

If the **Source** is **Insured**, return the current date. However, if the reason is that the policy is not taken, return the date that the policy went into effect.

Calculating the cancellation effective date if the source is insurer

This topic describes how PolicyCenter calculates the cancellation effective if the **Source** is **Insurer**.

Note: The cancellation lead time returned by the `notification_config.xml` system table is described in the *Product Model Guide*. This topic also describes how the application determines the lead time.

1. If the cancellation reason is equal to **Policy rewritten or replaced (flat cancel)**, then the cancellation effective date is equal to starting date of the policy period. You are done.
2. Else, determine if the policy is in the underwriting period.
 - a. Calculate the number of the days (`xDays`) between the Policy Effective Date and the current date.
 - b. The Notification plugin retrieves the underwriting period from the `notification_configs.xml` system table. The underwriting period is specified in rows with `ActionType` of `Underwriting Period`. The underwriting period is the value in the `LeadTime` column.
If `xDays` is less than or equal to the `LeadTime`, then the policy is in the underwriting period.
If the policy is in the underwriting period, the application will use underwriting period action types and categories in the `notification_configs.xml` system table to determine the cancellation lead time.
3. The Notification plugin retrieves the maximum lead time required for notification from the `notification_configs.xml` system table. The plugin retrieves the maximum lead time based on the cancellation reason, whether the policy is in the underwriting period, the policy jurisdiction, and the line of business.

Cancellation reason	Category	ActionType
Fraud	Cancellation	fraudcancel
	UW Period Cancellation	uwperiodfraudcancel
Non payment	Cancellation	nonpaycancel
	UW Period Cancellation	uwperiodnonpaycancel
Any other reason	Cancellation	othercancel
	UW Period Cancellation	uwothercancel

Note: If a policy includes multiple jurisdictions, there can be multiple matching rows. A policy that includes more than one line of business (such as a commercial package policy) can include both multiple jurisdictions and lines of business. In this case, the Notification plugin retrieves the maximum lead time. Therefore, the plugin returns the longest lead time specified for any of the jurisdiction and line of business combinations in the policy.

The cancellation effective date is equal to the current date plus the `LeadTime` for the matching row plus one day. Cancellation does not take place until the full notification period has passed. If ten days notification is required, cancellation does not take place earlier than 10:01 a.m. on day 11.

Configuring the premium calculation method

You cannot add or change the refund method (pro rata, short rate, and flat) except to change the calculation of penalty in short rate. In short rate, a rating engine calculates the amount of penalty. The default configuration contains rating system plugins. You can modify the plugin algorithms or integrate your own rating system to work with PolicyCenter.

Configuring policy change

This topic describes how to configure policy change jobs (policy change transactions).

Note: In PolicyCenter, the user interface uses the term policy transaction to refer to submissions, policy changes, and other policy transactions. Policy transactions are implemented as jobs in the data model, and referred to as jobs in PCF files, Gosu classes, and other configuration files. Therefore, the configuration documentation refers to policy transactions as jobs.

Gosu classes primarily control policy changes. In Studio, navigate to **configuration > gsrc** to view or edit the `gw.job.PolicyChangeProcessImpl.gsu` Gosu class that contains most of the policy change code.

In a policy change job, the `JobProcess.gsu` class, the `QuoteProcess.gsu` class, and the `PolicyChangeProcessImpl.gsu` class contain code for the main job actions. The default application spreads the policy change code among multiple classes for better organization and use of code. For example, changing the quote logic can be done in one location (`QuoteProcess.gsu`). You can modify the quote logic in one location and many types of jobs can access it.

Note: In the default configuration, the policy change job has no workflows because it has no background processes. PolicyCenter uses workflows only for background processes.

See also

- *Application Guide*

Policy change process class

The `PolicyChangeProcessImpl` class contains method for handling policy change jobs. The following table summarizes some of the methods contained in this class.

Method	Description
Starting a policy change	
<code>start</code>	This method starts the job and automatically assigns a requestor and producer.
<code>startAutomatic</code>	This method calls three methods which start the job, request a quote, and bind the policy. The <code>PolicyChangeAPI</code> calls this method.
Binding a policy change	
<code>canBind</code>	<code>canBind</code> – Checks that the policy can be bound. Some of the checks include: • Has the policy been quoted?

Method	Description
	Does the user have the correct permissions to bind the policy?
<code>bind</code>	Begins the binding process for a <code>PolicyPeriod</code>. This method verifies if the branch can be bound. For multi-version jobs, there is only one selected branch on the job which is specified by <code>Job.SelectedVersion</code>. That branch must have a valid quote. If the branch can be bound, then <code>bind</code> starts the binding process and sets the policy status to <code>Binding</code>. This method ensures that there is a producer of service on the policy. This method also calls out to the <code>FormInferenceEngine</code>, which is the main integration point for forms. Lastly, the <code>startBinding</code> method has commented out code to add an <code>IssuePolicyChange</code> event.
<code>finishBinding</code>	Completes the policy change job. This method also checks new conditions and sends billing information through the <code>IBillingSystemPlugin</code> .
<code>failBinding</code>	If binding fails, then the policy can be edited again by an underwriter.
Editing the effective date	
<code>canStartChangeEditEffectiveDate</code>	Check to see if Actions > Edit > Effective Date is available.
<code>canFinishChangeEditEffectiveDate</code>	Check to see if the current policy change can change its effective date to the input parameter value.
<code>changeEditEffectiveDate</code>	<code>PolicyCenter</code> invokes this method when you select Effective Date in the Actions menu. If <code>canStartChangeEditEffectiveDate</code> is false, then this method is not called. This method calls <code>canFinishChangeEditEffectiveDate</code> to verify that the change is valid before applying it.
Handling future bound jobs	
<code>applyChangesToFutureBoundRenewal</code>	Applies changes from this policy change to a renewal that is bound in the future.
<code>applyChangesToFutureUnboundRenewal</code>	Applies changes from this policy change to a renewal that is unbound in the future.
Flags	
<code>canRequestQuote</code>	Indicates whether quoting can be initiated on the <code>PolicyPeriod</code> .
<code>canWithdraw</code>	Checks to see if a branch can be withdrawn.

This class also identifies and raises underwriting issues with the policy, using underwriting rules to identify the types of issues to raise.

See also

- “Configuring underwriting authority” on page 67
- “Adding a new checking set” on page 71
- Application Guide*

Modifying the policy change process class

If you need to change or create new methods in that class, you can edit the class directly or create a subclass (such as `YourCompanyPolicyChangeProcess.gs`).

For more information about job classes and job subclasses, see “Gosu classes for local job processes” on page 301.

If you create a subclass, you must modify the job process customization plugin. For more information, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

Policy change integration

Policy change web services

For the policy change job, the PolicyChangeAPI includes two methods to start policy change jobs:

- The `startAutomaticPolicyChange` method starts a policy change job for a policy (specified by policy number) and an effective date. It returns the job number of the new job (its `JobNumber` property). In the default configuration, this method does not make any changes to the policy data. You can add your own methods to make changes to the data.
- The `startManualPolicyChange` method has the same arguments and return values, but does not attempt to bind and quote automatically. The policy change starts and waits in draft mode for the user to quote and finish it manually.

Policy change plugins

The policy change job has the following plugins.

IPolicyPlugin

The `canStartPolicyChange` method returns an error message if a change cannot be started for a policy and an effective date. Changes that are started internally or externally call this plugin.

For more information, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

IPolicyPeriodPlugin

The `postCreateNewBranchForChangeEditEffectiveDate` method is called to handle a change to the effective date of a job such as a policy change. In the default configuration, policy change is the only job that calls this method.

Policy change events

In PolicyCenter, events are changes (such as a new policy change) that might be of interest to an external system. In the `bind` method of the `PolicyChangeProcess.gs` class, you can generate the `IssuePolicyChange` event. In the default configuration, this code is commented out. The code notifies an external system to issue a policy change.

See also

- *Plugins, Prebuilt Integrations, and SOAP APIs*

Configuring reinstatement

This topic describes how to configure reinstatement jobs (reinstatement policy transactions).

Note: In PolicyCenter, the user interface uses the term policy transaction to refer to submissions, policy changes, and other policy transactions. Policy transactions are implemented as jobs in the data model, and referred to as jobs in PCF files, Gosu classes, and other configuration files. Therefore, the configuration documentation refers to policy transactions as jobs.

Use Guidewire Studio to access the files and code to configure the reinstatement process to your business needs.

Key files include:

- **Gosu job process class** – `ReinstatementProcessImpl.gs`. This class contains the code behind the reinstatement job. You can view or edit this class in Studio by navigating to **configuration > gsrc** and selecting `gw.job.ReinstatementProcess.gs`. For more information, see “Reinstatement process Gosu class” on page 335
- **Gosu enhancement file** – `ReinstatementEnhancement.gsx`. This enhancement allows you to modify or add additional functions and properties to the reinstatement job Gosu class. For example, this enhancement class has just one function which is called when the reinstatement job finishes. This function restarts any withdrawn renewals on the policy.
- **Page configuration files (PCF files)** such as `ReinstatementWizard.pcf`. To view these PCF files, navigate in Studio to **configuration > config > Page Configuration > pcf > job > reinstatement**.
- **Typelists** such as `ReinstateCode`. You can view this typelist by navigating in Studio to **configuration > config > Metadata > Typelist**.

Note: Since the reinstatement process for any insurer can be configured based on business requirements, all discussions apply only to the default application.

To learn more about how jobs work in Gosu and how to use workflows, see “Configuring jobs” on page 301.

[See also](#)

- *Application Guide*

Reinstatement process Gosu class

Gosu classes primarily control reinstatements. In Studio, navigate to **configuration > gsrc** to view the `gw.job.selfmanaged.ReinstatementProcessImpl.gs` Gosu class which contains most of the reinstatement code.

This class contains methods for reinstatement. The `JobProcess.gs` contains general functions used by different types of jobs.

This class ultimately extends the `JobProcess` Gosu class. The `JobProcess` class contains functions that allow you to get the job and the policy period. It cancels open activities on the reinstatement and auto assigns the user role. It also checks to see if the job can be expired or withdrawn, and checks if the job can be started as new.

This class identifies and raises underwriting issues with the policy, using underwriting rules to identify the types of issues to raise.

Methods in this class start the job as a draft, then automatically assign the producer and underwriter roles to the job.

Note: The reinstatement job has no workflows because it has no background processes. PolicyCenter uses workflows only for background processes.

See also

- “Configuring underwriting authority” on page 67
- *Application Guide*

Modifying the Reinstatement Process Class

If you need to change or create new methods in that class, you can edit the class directly or create a subclass (such as `YourCompanyReinstatementProcessImpl.gs`).

For more information about job classes and job subclasses, see “Gosu classes for local job processes” on page 301.

If you create a subclass, you must modify the job process customization plugin. For more information, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

Reinstatement integration

Reinstatement web services

The `ReinstatementAPI` is a web service to start a reinstatement from an external system.

The `ReinstatementAPI.beginReinstatement` function can be invoked to begin a reinstatement job. It does not attempt to automatically push the reinstatement job through to completion. The `policyNumber` and `reinstatementCode` arguments identify which policy is to be reinstated and the reason for the reinstatement. The function throws an exception if the reinstatement cannot be started for any reason.

See also

- *Plugins, Prebuilt Integrations, and SOAP APIs*

Reinstatement plugins

The following plugins may be useful to integrate with reinstatement.

Name	Description
<code>IBillingPlugin</code>	If you need to integrate with an external billing system, use this plugin to notify an external billing system about billing events.
<code>MessageTransport</code>	Use this plugin to send a message to an external/remote system by using any transport protocol.
<code>IEffectiveTimePlugin</code>	Determines the time of day (since midnight) for a job. Use to calculate the effective time.
<code>IJobNumberGenPlugin</code>	Generates a new, unique job number. Use to generate reinstatement job numbers.
<code>IPolicyPlugin</code>	Lets PolicyCenter know whether to start various jobs, based on dynamic calculations on the policy. Use to determine when a reinstatement job can be started.

See also

- *Plugins, Prebuilt Integrations, and SOAP APIs*

Reinstatement events

PolicyCenter generates events for changes that might be of interest to an external system. You can also programmatically generate an event. For example, in the `ReinstatementProcess.gs` class, the function `startReinstate` generates an `IssueReinstatement` event.

The `Reinstatement` entity has the following events:

- `ReinstatementAdded`
- `ReinstatementChanged`
- `ReinstatementRemoved`

The `PolicyPeriod` entity has the following events pertaining to reinstatement:

- `IssueReinstatement` – a message that notifies an external system to issue a reinstatement for a particular policy period.

See also

- *Plugins, Prebuilt Integrations, and SOAP APIs*

Configuring rewrite

This topic describes how to configure rewrite jobs (rewrite policy transactions).

Note: In PolicyCenter, the user interface uses the term policy transaction to refer to submissions, policy changes, and other policy transactions. Policy transactions are implemented as jobs in the data model, and referred to as jobs in PCF files, Gosu classes, and other configuration files. Therefore, the configuration documentation refers to policy transactions as jobs.

Use Studio to access the necessary files and code to configure the rewrite job (policy transaction) to your business needs.

Key files include:

- **Gosu job process class** – `RewriteProcessImpl.gs`. This class contains the code behind the rewrite job.
- **Page configuration files (PCF files)** such as `RewriteWizard.pcf`. To view these PCF files, navigate in Studio to **configuration > config > Page Configuration > pcf > job > rewrite**.
- **Typelists** such as `RewriteType`. You can view this typelist by navigating in Studio to **configuration > config > Metadata > Typelist**.

Note: Since the rewrite process for any insurer can be configured based on business requirements, all discussions apply only to the default application.

See also “Configuring jobs” on page 301 to learn more about how jobs work in Gosu and how to use workflows.

[See also](#)

- “Selecting the underwriting company through segmentation” on page 308
- *Application Guide*

Rewrite process Gosu class

Gosu classes primarily control the rewrite process. In Studio, navigate to **configuration > gsrc** and open the `gw.job.selfmanaged.RewriteProcessImpl.gs` Gosu class which contains most of the rewrite code.

This class ultimately extends the `JobProcess` Gosu class. The `JobProcess` class contains functions that allow you to get the job and the policy period. It cancels open activities on the rewrite and auto assigns the user role. It also checks to see if the job can be expired, or withdrawn, and checks if the job can be started as new.

Methods in this class start the rewrite job, request a quote, and rewrite the policy period.

Note: The rewrite job has no workflows because it has no background processes. PolicyCenter uses workflows only for background processes.

Modifying the Rewrite Process Gosu Class

If you need to change or create new methods in that class, you can edit the class directly or create a subclass (such as *YourCompanyRewriteProcessImpl.gs*).

For more information about job classes and job subclasses, see “Gosu classes for local job processes” on page 301.

If you create a subclass, you must modify the job process customization plugin. For more information, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

Rewrite events

PolicyCenter generates events for changes in PolicyCenter that might be of interest to an external system. You can also programmatically generate an event. For example, in the *RewriteProcess.gs* class, the *rewrite* function generates an *IssueRewrite* event (by calling the *startBinding* function).

The Rewrite entity has the following events:

- RewriteAdded
- RewriteChanged
- RewriteRemoved

The PolicyPeriod entity has the following events pertaining to rewrite:

- IssueRewrite – A message that notifies an external system to issue a rewrite for a particular policy period.

See also

- *Plugins, Prebuilt Integrations, and SOAP APIs*

Configuring rewrite new account

This topic describes how to configure rewrite new account jobs (rewrite new account policy transactions).

Note: In PolicyCenter, the user interface uses the term policy transaction to refer to submissions, policy changes, and other policy transactions. Policy transactions are implemented as jobs in the data model, and referred to as jobs in PCF files, Gosu classes, and other configuration files. Therefore, the configuration documentation refers to policy transactions as jobs.

See also

- *Application Guide*

Rewrite new account job

The main PCF file for the rewrite new account is the `RewriteNewAccountWizard`.

The primary Gosu class for this job is `gw.job.selfmanaged.RewriteNewAccountProcessImpl`. The main methods in this class are:

- `start`
- `rewriteNewAccount`
- `finishRewriteNewAccount`

If you need to change or add methods to the rewrite new account process, you can edit the job class directly or create a subclass (such as `YourCompanyRewriteNewAccountProcessImpl.gs`).

For more information about job classes and job subclasses, see “Gosu classes for local job processes” on page 301.

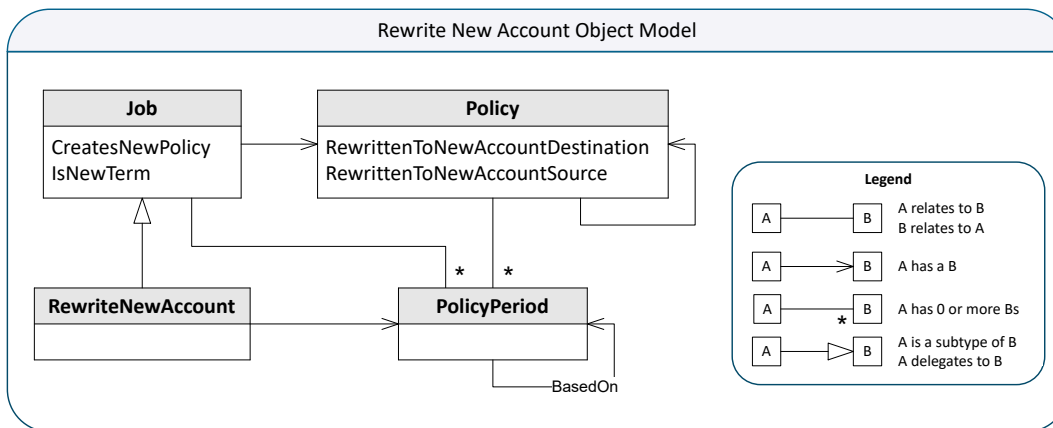
If you create a subclass, you must modify the job process customization plugin. For more information, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

Copying contacts and locations in rewrite new account job

When a rewrite new account job starts, it copies contacts and locations from the source policy account to the target account. For example, the source policy period references the driver, John Doe. The job checks for a corresponding `AccountContact` and `Driver` for John Doe on the target account and creates these entities if they do not exist. In the rewritten policy, the corresponding policy driver points to the copied account level driver.

Rewrite new account object model

The following object model diagram shows some of the entities and fields related to the rewrite new account job (policy transaction).



Rewrite New Account Entity

The RewriteNewAccount entity represents the rewrite new account job. This entity has a PolicyPeriod field which returns the policy period associated with the job.

The PolicyPeriod entity associated with the RewriteNewAccount entity has a BasedOn field that returns the policy period on which the source policy is based.

Policy Entity

The Policy entity has the following fields related to the rewrite new account job:

Field	Description
RewrittenToNewAccountDestination	If this policy was rewritten to a new account, returns the destination policy.
RewrittenToNewAccountSource	If this policy was rewritten to a new account, returns the source policy.

Job Entity

In a rewrite new account job, the Job entity has the following values:

Field	Description
CreatesNewPolicy	Value is True if this job creates a new policy with a new policy number. This value is True for rewrite new account jobs.
IsNewTerm	Value is True if this job creates a new policy term. This value is True for rewrite new account jobs.

Rewrite new account history events

The rewrite new account job (policy transaction) creates the following history events:

History event	Description
rewr_new_acct_created	This event appears on the source policy term when the rewrite new account job starts.
rewr_new_acct_bound	This event appears on the target policy term when the job is bound.

Configuring premium audit

In the base configuration, PolicyCenter provides two types of premium audit jobs: final audits and premium reports. Audits are included in the Workers' Compensation and General Liability lines of business. You can configure final audits and premium reports to meet your requirements. You can add and configure new audit types such as checking audits. You can add audit to additional lines of business.

This topic describes how to configure premium audit jobs (premium audit policy transactions).

Note: In PolicyCenter, the user interface uses the term policy transaction to refer to submissions, policy changes, and other policy transactions. Policy transactions are implemented as jobs in the data model, and referred to as jobs in PCF files, Gosu classes, and other configuration files. Therefore, the configuration documentation refers to policy transactions as jobs.

Use Guidewire Studio to access files and code to configure the audit job (policy transaction) to your business needs.

Key files include:

- **Gosu job process class:** `AuditProcessImpl.gs`. This class contains the code behind premium audit jobs. You can view or edit this class in Studio by navigating to **configuration > gsrc** and opening `gw.job.selfmanaged.AuditProcessImpl.gs`. For more information, see “Audit Gosu classes” on page 344.
- **Gosu enhancement files:**
 - `AuditInformationEnhancement.gsx` – This enhancement allows you to modify or add additional functions and properties to the premium audit job Gosu class.
 - `AuditAssignmentEnhancement.gsx` – This enhancement assigns the premium auditor to the job if the premium auditor was not already assigned. You can view or edit this class in Studio by navigating to **configuration > gsrc** and opening `gw.assignment.AuditAssignmentEnhancement.gsx`.
- **Rule Sets** – The **Audit > Reporting Trend Analysis > Ratio Out of Range** rule set checks to see if the reporting trend analysis ratio is within an acceptable range. The default configuration has rules for premium reports. The acceptable range is from 90% to 110%. If the ratio is outside the acceptable range and the number of reporting days is greater than 60, the code creates a `RatioOutOfRange` activity and assigns it to an underwriter. For information about reporting trend analysis, see the *Application Guide*.
- **Page configuration files (PCF files)** such as `AuditWizard.pcf`. To view these PCF files, navigate in Studio to **configuration > config > Page Configuration > pcf > job > audit**.
- **Typelists** such as `AuditMethod`. Typelists are in the **configuration > config > Metadata > Typelist** folder in Studio.

IMPORTANT: Since the audit process for any insurer can be configured based on business requirements, all discussions apply to the default application only.

See also

- *Application Guide*
- “Configuring jobs” on page 301 to learn more about how jobs work in Gosu and how to use workflows
- *Product Model Guide*

Audit Gosu classes

Gosu classes primarily control premium audits. The `gw.job.selfmanaged` package contains most of the audit code.

- `AuditProcessImpl.gs` is the main class and contains methods for audit. The `JobProcess.gs` contains general methods used by different types of jobs.

This class extends the `JobProcess` Gosu class. The `JobProcess` class contains methods that allow you to get the job and the policy period. It auto assigns the user role, and checks to see if the job can be expired, or withdrawn, and checks if the job can be started as new.

Methods in this class start the job as a draft, and verify the conditions for which the audit can be edited. For example, does the user have edit permissions? Is the job complete? A method checks to see if an audit package can be created. The `complete` method moves a policy period into the `AuditComplete` status, marking the audit as complete. From then on the policy period is read-only, and any new work on the audit must be done by revising the audit. Lastly, since an audit can be waived through the `waive` method, it locks the audit and it too cannot be edited.

- The `AuditInformationEnhancement.gsx` file contains methods that check to see if the audit is revisable. An audit is revisable if it is completed and was not waived, reversed, or already revised.
- The `PolicyPeriodAuditEnhancement.gsx` file contains methods for the policy period that are related to audit. This enhancement contains methods to schedule audits and remove a final audit from a policy period.

This enhancement also contains a `reverseFinalAudits` method which reverses audits. This method makes a call to the billing system to reverse charges related to the final audit. This method also sets the `AuditInformation.ReversalDate` to the current date. You can see which audits have been reversed by examining the `ReversalDate`.

PolicyCenter indicates that a policy has been billed by making an integration call to the billing system and calling the `PolicyPeriod.markBilled` function.

- The `AuditScheduler.gs` class contains methods for scheduling audits.

Note: The premium audit job has no workflows because it has no background processes. PolicyCenter uses workflows only for background processes.

Modifying the audit process Gosu class

If you need to change or create new methods in the `AuditProcessImpl` Gosu class, you can edit the class directly or create a subclass (such as `YourCompanyAuditProcessImpl.gs`).

For more information about job classes and job subclasses, see “Configuring jobs” on page 301.

If you create a subclass, you must modify the job process customization plugin. For more information, see *Plugins, Prebuilt Integrations, and SOAP APIs*.

Modifying the audit scheduler Gosu class

If you define a new audit type that requires schedules, you must modify the `AuditScheduler` Gosu class.

If you define a new audit type that requires schedules, add code that schedules the audits for a policy period. The `scheduleAllAudits` method creates schedules for the audit types in the base configuration. Define a method to schedule your new audit type. Call your method from the `scheduleAllAudits` method.

Methods in the `AuditScheduler` class call the `createAuditInformation` method on the `AuditSchedulePattern` interface. The `createAuditInformation` method creates an `AuditInformation` entity instance but does not return a

pointer to the instance. If you need to modify the `AuditInformation` entity instance, you can retrieve it by creating a query. For example, you added properties to the `AuditInformation` entity definition, and need to set these values in the entity instance. After you retrieve the instance, set values for the properties you added.

The `AuditInformation` entity has properties for information defined in the audit schedule. The properties include values for the audit method, audit schedule type, start date, and due date, among others.

Premium audit integration

Integration with BillingCenter

The base configuration of PolicyCenter contains an integration with Guidewire BillingCenter. For information on activating, using, and configuring this integration, see the *Application Guide*.

Audit web services

There are no premium audit web services.

Audit plugins

The following plugins may be useful to integrate with premium audit.

Name	Description
<code>IAuditSchedulePatternSelectorPlugin</code>	In the base configuration, this plugin sets the default audit schedule for final audits. Cancellation final audits are scheduled by phone and expiration final audits are scheduled physically. If the audit method is dependent on the premium value, you can configure it in this plugin.
<code>IBillingSystemPlugin</code>	If you need to integrate with an external billing system, use this plugin to notify an external billing system about billing events.

For further information, see *Plugins*, *Prebuilt Integrations*, and *SOAP APIs*.

Audit events

PolicyCenter generates events for changes that might be of interest to an external system. You can also programmatically generate an event.

There are no events for premium audit.

Defining audit schedules

PolicyCenter uses audit schedules to specify how to calculate start dates, due dates, and methods (physical, voluntary, or by phone) of audit transactions on a policy. PolicyCenter calculates the start date and the due date based on the period end date. The premium report schedule has other fields including frequency and default deposit.

In Product Designer, you can add audit schedules and modify properties of audit schedules. For example, in Product Designer you can change the audit schedule's method from voluntary to phone.

Final audit schedules

The default application comes with a variety of final audit schedules including schedules for cancellations audits and expirations audits. You can view and modify these schedules in Product Designer by navigating to **Audit Schedules**.

In Product Designer, you can view **Expiration Audit by Physical**, which provides an example of a final audit schedule. The audit is set to start 30 calendar days before the audit period end date. It is due 45 calendar days after audit period end date. If either the due or start date falls on a non-business day, it is delayed until the next business day.

The default final audit schedules are:

- **Cancellation Audit by Phone** for all cancellation final audits
- **Expiration Audit by Physical** for all non-cancellation final audits

The `IAuditSchedulePatternSelectorPlugin` determines the default final audit schedule. You can add additional audit schedules and modify the audit schedule pattern selector plugin code. For example, you can modify the plugin to auto-assign the schedule based on criteria such as policy premium.

Premium report schedules

Premium report scheduled items may be configured to utilize policy months or calendar months. In the default application, the premium report schedules are:

- **Monthly Reports by calendar months, exclude last month**
- **Monthly Reports by policy month, exclude last month**
- **Quarterly Reports by calendar quarter for full policy term**
- **Quarterly Reports by calendar quarter, exclude last quarter**
- **Quarterly Reports by policy quarter, exclude last quarter**

If you select policy months, the start and end day of the month is based on the policy's effective date. The following table shows reports for an audit schedule frequency of **Monthly reports by policy months**.

Policy effective date 3/28/09	Policy effective date 03/10/09
03/28/09 to 04/28/09	03/10/09 to 04/10/09
04/28/09 to 05/28/09	04/10/09 to 05/10/09
05/28/09 to 06/28/09	05/10/09 to 06/10/09
...	...

If you select calendar months, an additional audit schedule field comes into play. This audit schedule field is the *round to next month* field. This field allows you to control for a first report that is very short. If the policy effective date is after the *round to* date, then the first report covers the first short month and the second month. Subsequent periods correspond to calendar months. The following table shows reports for an audit schedule of **Monthly reports by calendar months with round to next month on day 15**.

Policy effective date 3/28/09	Policy effective date 03/10/09
03/28/09 to 05/01/09	03/10/09 to 04/01/09
05/01/09 to 06/01/09	04/01/09 to 05/01/09
06/01/09 to 07/01/09	05/01/09 to 06/01/09
...	...

In Product Designer, you can view and modify **Monthly Reports by calendar month, excluding last month**, which provides an example of a premium report schedule. The schedule specifies that the audit method is voluntary and has a 10% deposit. In the schedule details, the minimum audit period length is set to 15 days. The last audit period is excluded. In the period details, the audit is set to start five calendar days before the audit period end date. The audit is due 15 calendar days after the audit period end date.

Create audit schedule

About this task

You can create a new audit schedule in Product Designer.

Procedure

1. In Product Designer, navigate to **Product Model > Audit Schedules**.
2. Click **Add**.
3. Enter a **Code**, **Name**, select an **Audit Type**. The **Audit Type** drop-down list displays the audit types defined in PolicyCenter.
4. If the audit schedule type can be either single or series, such as checking audit, then select a **Schedule Type**.
5. Click **OK**.
Product Designer displays the new audit schedule with properties initially based on properties specified in the audit schedule properties files for this audit type. See “Audit schedule properties file” on page 349.
6. In the audit schedule, add and change properties such as a description, when to initiate the audit, and the type of days.

Configuring audit types

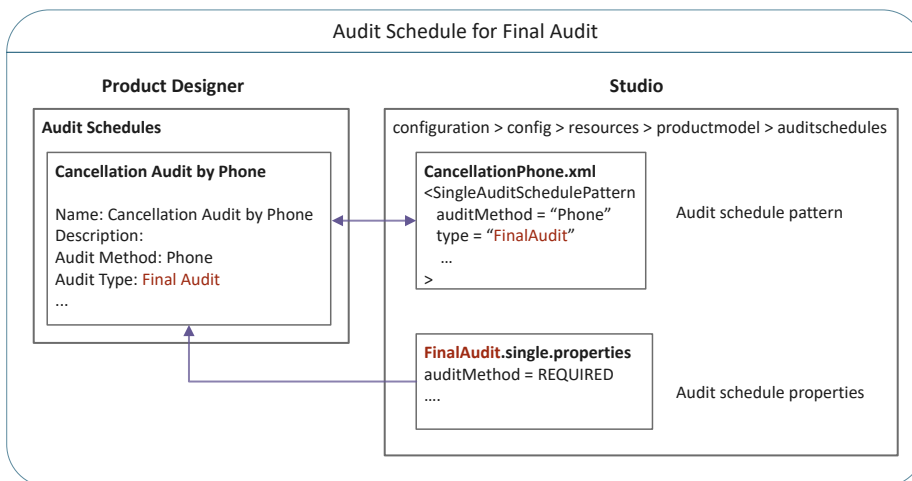
The base configuration contains the following audit types:

- Final Audit – Configured in the Workers’ Compensation and General Liability lines of business
- Premium Report – Configured in the Workers’ Compensation line of business
- Checking Audit – Not configured in any line of business

In addition, you can define and configure your own audit types.

Most audit types require that schedules of those audits be created for policy periods. Each audit schedule is for a particular audit type. The schedule properties are based on the audit schedule properties file for that audit type.

Using Final Audit as an example, the following illustration shows how audit schedules are linked to their properties files. The audit schedule is based on the audit schedule properties file for audit schedules of that audit type.



In an *audit schedule properties file*, you define properties of the audit schedule as well as basic properties of the audit type. Basic properties of the audit type include the audit method. This file also defines the *schedule type*, which specifies whether the audit has a single occurrence or occurs in a series. You can edit the properties file in Studio. However, only Product Designer uses this file. PolicyCenter does not.

In Product Designer, you can add new audit schedules based on the audit types. When you save an audit schedule and commit the changes, Product Designer writes the audit schedule to an *audit schedule pattern* XML file in PolicyCenter.

Add new audit type

About this task

To add a new audit type to a line of business, you must do the following:

Procedure

1. Extend the audit schedule typelist to include the new audit type. This enables you to configure the new audit type and define audit schedules based upon the new audit type in Product Designer. See “Extend audit schedule type typelist” on page 348.
2. Add code to handle the new audit type in all lines of business for which this audit type applies. For guidance, see:
 - *Product Model Guide*
 - “Configuring premium audit” on page 343

You may also need to add or modify PCF files and entities to configure the new audit type.

For audit types, such as Final Audit and Premium Reports, that require schedules, you must do the following steps. In the base configuration, Retrospective Rating is an audit type that does not require schedules.

3. Add audit schedule properties files for the new audit type. Product Designer loads your new audit schedule properties files when you log out, switch change lists, or restart Product Designer. See “Audit schedule properties file” on page 349.
4. Define audit schedules in Product Designer for the new audit type. When you commit changes, Product Designer updates the audit schedule pattern XML file. See “Audit schedule pattern” on page 351.

Extend audit schedule type typelist

About this task

If you add a new audit type, you must add the typecodes to the `AuditScheduleType` typelist. If the audit type requires schedules, then you can make the audit type available to Product Designer. In Product Designer, you can create audit schedules based on this audit type.

Procedure

1. In **configuration > config > Extensions > Typelist**, open `AuditScheduleType.ttx`. Or if the file does not exist, create a new **Typelist Extension** extending `AuditScheduleType`.
2. Add a typecode for new audit type.
3. For code, enter the `AuditType` as specified in the name of audit schedule properties file. For example, enter the code `FinalAudit` for `FinalAudit.series.properties`.
4. For name, enter the display name of the audit type.
5. If the audit type requires schedules, right-click the typefilter named `ConfigurablePatterns` and select **Add New > Include Into Filter**. Select the typecode for the new audit type. Add the typefilter if it does not exist. This is the same name as the typefilter on the base typelist.
6. Right-click the typefilter and select **Add New > Include Into Filter**. Select the typecode for the new audit type.
7. Click **Next**.
8. In the **Select filter** tab, select **ConfigurablePatterns**.
9. Click **Finish**.

Audit schedule properties file

The *audit schedule properties file* defines which properties are required or optional for audit schedules of a particular audit type. Each audit type that requires schedules has at least one audit schedule properties file.

The *schedule type* specifies whether the schedule type is *single* or *series*. A single audit occurs once. A series audit occurs one or more times. An audit type can have both single and series schedule types. In the base configuration, Checking Audit is defined for both schedule types.

In Studio, create a new audit schedule properties file in **configuration > config > resources > productmodel > auditschedules**. Only Product Designer uses this file. PolicyCenter does not.

You can base this file on existing properties files in this directory. For example, `FinalAudit.single.properties` is the audit schedule properties file for a Final Audit audit type. Name the audit schedule properties file:

```
AuditType.scheduleType.properties
```

Where:

- *AuditType* – The name of the audit type.
- *scheduleType* – The schedule type can be *single* or *series*. A single audit occurs once. A series audit occurs one or more times.

For example, Final Audits are always single audits because they only occur once per policy term. Therefore, there is only a `FinalAudit.single.properties` file.

Premium Reports are generally series audits, because there is a series of them during a policy term. Therefore, there is only a `PremiumReports.series.properties` file. Some types of audits such as Checking Audits, may happen both as single audits and as series audits. These audits require audit schedule properties files for single and series.

In the audit schedule properties file, specify one of the following values for each property. The value has the following effect when defining an audit schedule in Product Designer:

- **REQUIRED** – When defining an audit schedule of this type, the user must specify a value for this field.
- **EDITABLE** – The user can specify a value for this field. The field is not required.
- **HIDDEN** – This field is hidden.

In the properties file, specify values for the following properties for schedules for your new audit type. If you do not specify a value, the value defaults to **EDITABLE**.

Properties	Product Designer label	Description
<code>auditMethod</code>	Audit Method	A drop-down list to select how to conduct the audit. The drop-down lists values are defined in <code>AuditMethod</code> typelist.
Schedule Details fields that only apply to series audit schedules		
<code>frequency</code>	Scheduled	A drop-down list to select how often audits occur. Values are defined in the <code>AuditFrequency</code> typelist.
<code>intervalComputationType</code>	Schedule Basis	A drop-down list to select the schedule basis. Values are defined in the <code>AuditIntervalComputeType</code> typelist. In the base configuration, the values are: • Calendar Month • Policy Month
<code>calendarMonthRoundDate</code>	Round to Next Month on Day	A text box to enter which day rounds up to the next month.

Properties	Product Designer label	Description
minimumAuditPeriodLength	Minimum Audit Period (Days)	A text box to enter the minimum audit period.
excludeLastAuditPeriod	Exclude Last Audit Period	A check box to select whether to exclude the last audit period when creating a schedule of audits.
Schedule Details fields that only apply to single audit schedules		
endDateDelayAmount	Single Checking Ends After Policy Period Start (days)	A text box to enter the number of days after the policy period start the single audit ends.
endDateDelayUnit	Type of Days	A drop-down list to specify the type of days to use when calculating the end date for the audit. Values are defined in the DateCalcUnit typelist. In the base configuration, values are: • Calendar Days • Business Days
Audit Report Initiation fields that apply to both single and series audit schedules		
initDelayDirection	Audit Initiated	A drop-down list to specify whether the audit is initiated before or after the audit period end date. Values are defined in AuditReportDateDirection typelist. This typelist is final.
initDelayAmount	Days Before Days After	Depending upon the Audit Initiated selection, a text box to enter the number of days, before or after, to adjust the audit initiation date from the audit period end date.
initDelayUnit	Type of Days	A drop-down list to specify the type of days. Values are defined in the DateCalcUnit typelist.
initBusinessDayAdjustment	If Initiated on a Non-Business Day	A drop-down list to specify how to adjust the calculated audit initiation date if it occurs on a non-business day. Values are defined in the AuditBusinessDayAdjust typelist.
Audit Deadline fields that apply to both single and series audit schedules		
dueDelayDirection	Audit Due	A drop-down list to specify whether the audit is due before or after the audit period end date. Values are defined in AuditReportDateDirection typelist. This typelist is final.
dueDelayAmount	Days Before Days After	Depending upon the Audit Due selection, a text box to enter the number of days, before or after, to adjust the audit due date.
dueDelayUnit	Type of Days	A drop-down list to specify the type of days. Values are defined in the DateCalcUnit typelist.
dueBusinessDayAdjustment	If Due on a Non-Business Day	A drop-down list to specify how to adjust the audit due date if it occurs on a non-business day. Values are defined in the AuditBusinessDayAdjust typelist.
Escalation fields that apply to both single and series audit schedules		
numDaysAfterFirstEscalation	Escalate Number of Days after First Prompt	A text box to enter the number of days to wait before the first escalation.
firstEscalationPrompt	First Escalation Prompt	A drop-down list to specify what prompts the first escalation. Values are defined in the EscalationPromptType typelist.

Properties	Product Designer label	Description
numDaysAfterSecondEscalation	Escalate Number of Days after Second Prompt	A text box to enter the number of days to wait before the second escalation.
secondEscalationPrompt	Second Escalation Prompt	A drop-down list to specify what prompts the second escalation. Values are defined in the EscalationPromptType typelist.
Miscellaneous fields		
paymentPlanCode	Payment Plan Code	A text box to enter the payment plan code. For an example, see <code>PremiumReport.series.properties</code> and <code>ReportPolicyMonthExclLast.xml</code> .
reportingDefaultDepositPct	Reporting Default Deposit %	A text box to enter a default deposit percentage. For an example, see <code>PremiumReport.series.properties</code> and <code>ReportPolicyMonthExclLast.xml</code> .

In the base configuration, PolicyCenter does not use the escalation fields. You can write code to escalate audits when they are not completed.

Product Designer loads your new audit schedule properties files when you log out, switch change lists, or restart Product Designer.

Audit schedule pattern

After you add or modify the audit type, you can add audit schedules in Product Designer that use this audit type. When you add an audit schedule and commit the change in Product Designer, an XML file representing the new audit schedule appears in the PolicyCenter configuration resources. You can view the XML file in Studio by navigating to **configuration > config > resources > productmodel > auditschedules**. For example, the `CancellationVoluntary.xml` file is the audit schedule for the **Cancellation Audit by Voluntary** audit schedule.

Configuring side-by-side quoting

This topic describes how to configure side-by-side quoting for jobs (policy transactions).

Note: In PolicyCenter, the user interface uses the term policy transaction to refer to submissions, policy changes, and other policy transactions. Policy transactions are implemented as jobs in the data model, and referred to as jobs in PCF files, Gosu classes, and other configuration files. Therefore, the configuration documentation refers to policy transactions as jobs.

[See also](#)

- “Multiple revision jobs and the job selected branch property” on page 307
- *Application Guide*

Changing the maximum number of versions in side-by-side quoting

In the default configuration, the maximum number of side-by-side quotes is four. You can change the maximum number of side-by-side quotes for each job (policy transaction) type. In `config.xml`, the configuration parameters are:

- `RenewalMaxSideBySideQuotes`
- `SubmissionMaxSideBySideQuotes`
- `PolicyChangeMaxSideBySideQuotes`

On the Job entity, the derived property `MaxNumberOfSideBySideQuotes` returns the maximum number of side-by-side quotes specified in `config.xml` for that job.

[See also](#)

- “Side-by-side quoting parameters” in *Configuration Parameters*

Side-by-side quoting object model

This topic describes the object model for side-by-side quoting.

[Job Entity](#)

The Job entity has the following fields related to side-by-side quoting:

Field	Description
SideBySide	Specifies whether this job has side-by-side quoting.
SelectedVersion	This foreign key points to the PolicyPeriod of the selected version.
MaxNumberOfSideBySideQuotes	This derived property returns the maximum number of side-by-side quotes specified in config.xml for the job type.

Base Data Entities

The `gw.api.logicalmatch.LogicalMatcher` interface provides methods which determine if two things are logically equivalent. There are other interfaces which extend the `LogicalMatcher` interface, such as the `EffDatedLogicalMatcher` interface. Entities which implement this interface use the `isLogicalMatcherUntyped` method to determine whether the entities are equivalent. If this method returns `true` then the two entities are logically equivalent in side-by-side periods.

If you add a new entity that you would like to be base data, add a matcher that compares two entities and determines whether the entities are equivalent.

The `gw.lob.pa.PersonalVehicleMatcher` is an example matcher for the `PersonalVehicle` entity. The entity definition shows that the entity implements the `LogicalMatcher` interface. In Studio, press `CTRL+SHIFT+N` and enter `PersonalVehicle.eti`. Then double-click the search result to open the file in the editor. View the `implementsInterface` row with the **Primary Value** equal to `gw.api.logicalmatch.EffDatedLogicalMatcher`. This interface is implemented by the `gw.lob.pa.PersonalVehicleMatcher` class (**Secondary Value**). In general, the matcher is in the same package as the entity for which it provides matching.

Configuring the side-by-side quoting screen

The PCF file for the **Side-by-Side Quoting** screen is `SideBySideScreen.pcf`. In Studio, press `CTRL+SHIFT+N` and enter the file name. Then double-click the search result to open the file in the editor.

You can configure PolicyCenter to display base data fields on the **Side-by-Side Quoting** screen. For example, you can add widgets for base data fields in the columns for each side-by-side version.

IMPORTANT: Guidewire requires that base data entities or fields on the **Side-by-Side Quoting** screen not be editable in more than one place on a given screen.

Placing an editable widget for a base data field in the columns replicated for each version is a violation of this requirement. This requirement applies to fields that are implicitly base data, such as contact or location information that can be synchronized.

Posting changes to the server

For performance reasons, the **Side-by-Side Quoting** screen in the base configuration posts changes to the server only as needed.

The default configuration follows these rules for posting changes to the server on the **Side-by-Side Quoting** screen:

- **Quoted** – If the period is in the quoted state, PolicyCenter posts when the user edits any field on that period. In the **Side-by-Side Quoting** screen, PolicyCenter requires the post when you edit a quoted period because PolicyCenter must open that period for edit. (The user does not explicitly open the period for edit.) Opening the period for edit invalidates the premiums, and moves the policy period from quoted to draft status.
- **Not quoted** – If the edited period is not in the quoted state,
 - A change to a widget, such as a check box, that adds or removes a coverage triggers a post to the server. The post ensures that PolicyCenter reevaluates the availability logic in the product model.
 - An edit to a coverage term value does not trigger a post.

In personal auto, the one exception to this rule is the PALiability coverage and its coverage terms. The PALiability coverage has dependencies with other coverages in the policy period, therefore a post is necessary.

Regardless of the policy period status:

- Clicking **Quote All** or **New Version** triggers a post and a page refresh.
- Clicking **Use Defaults** for an offering triggers a post and a page refresh.

Disable post on enter

About this task

Most changes on the **Side-by-Side** screen post changes back to PolicyCenter. However, posting is not necessary for certain types of changes. By default, post on change is not selected for PCF elements.

To programmatically determine whether to post on change:

Procedure

1. In Studio, open a PCF file, such as `SideBySideCovTermInputSet.bit.pcf`.
2. Select a PCF element, such as the `BooleanRadioInput`.
3. In **Properties**, select the **PostOnChange** tab, then select **Enable targeted Post On Change**.
4. In the `disablePostOnEnter` property, enter code to set this property.

When enabled, PolicyCenter checks the value of `disablePostOnEnter` to determine whether to post changes.

Configuring initial behavior of the side-by-side quoting screen

In personal auto, when you view the **Side-by-Side Quoting** screen, PolicyCenter creates two additional periods for a total of three side-by-side periods. The new periods are copies of the initial period. The `configureInitialPolicies` method in `gw.job.sxs.PersonalAutoSideBySideInitialConfig` initializes the two new periods. You can modify the code in this class.

The `configureInitialPolicies` method checks to see if the initial period has an offering. If the initial period has an offering, the side-by-side periods are the same as the initial period. If there is no offering, the method applies the basic, standard, and premium offerings to each side-by-side version, respectively.

In policy periods that have the **Standard Program** offering, the `configureInitialPolicies` method also removes `PACollisionCov` and `PAComprehensiveCov` on vehicles over 10 years old.

Copying base data for side-by-side quoting

Base data is data which appears to be common to all versions in a side-by-side job (policy transaction). When you make a change to base data, PolicyCenter copies that change to the other side-by-side versions. When PolicyCenter copies the change to the other versions, it creates, removes, and updates entities across all versions as necessary.

When PolicyCenter copies base data to the other side-by-side periods, PolicyCenter opens the target periods for edit if the target periods are quoted. PolicyCenter moves the target periods from quoted to draft status and invalidates the quote.

In the personal auto line of business, some examples of base data are:

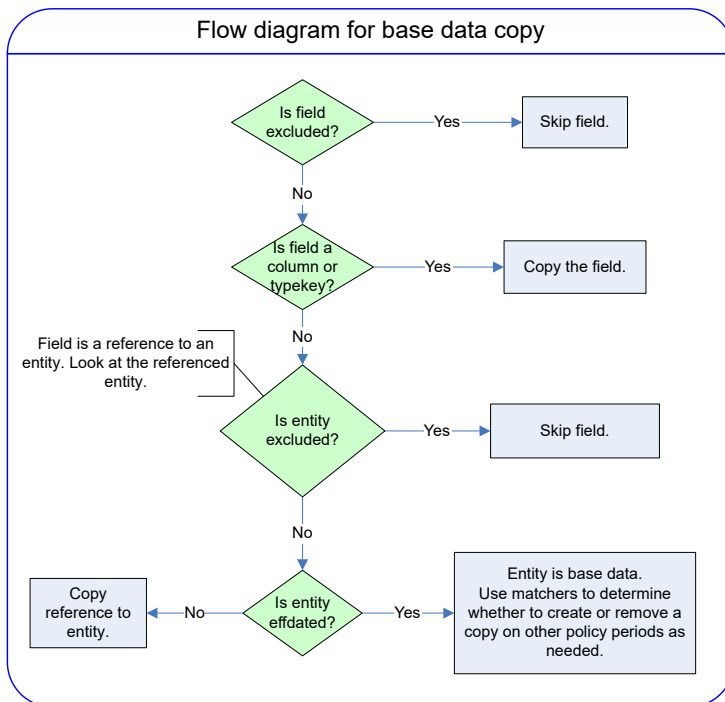
- Policy location
- Driver information such as motor vehicle records
- Line modifiers such as a multi-policy discount

High-level view of base data copy

Base data copy uses the following logic to traverse the policy period graph to:

- Exclude side-by-side entities and fields from base data copy
- Copy base data entities and fields

The following illustration is a flow diagram for base data copy. Base data copy uses this logic while traversing each node of the policy graph.



If the field is an array, one-to-one, foreign key, or edge foreign key, then base data copy checks to see if the entity pointed to is effective dated. Revisioned entities implement the `EffDated` delegate. Base data copy skips arrays of entities that are not effective dated because they are implicitly side-by-side data. Base data copy stops recursion on an entity that is side-by-side.

For a complete list of excluded entities and fields, see “Excluding side-by-side data from base data” on page 358.

Gosu methods for base data copy

Base data copy is implemented by the `SideBySideBaseDataCopy` method in `gw.job.sxs`. When base data is modified on an underlying `PolicyPeriod`, this method copies the change to the other side-by-side policy periods. As necessary, this method creates and removes entities so that the base data appears to be shared among the versions in the side-by-side job.

Note: In the policy graph, data that is not base data is referred to as side-by-side data. Unlike base data, side-by-side data is not synchronized between the policy graphs.

The `SideBySideBaseDataCopy` class uses the `MatchableTreeTraverser` class to traverse the policy graph of the source and destination periods, collecting information for use during base data copy. Base data copy starts at `PolicyPeriod` then recursively traverses each link until reaching data excluded from base data.

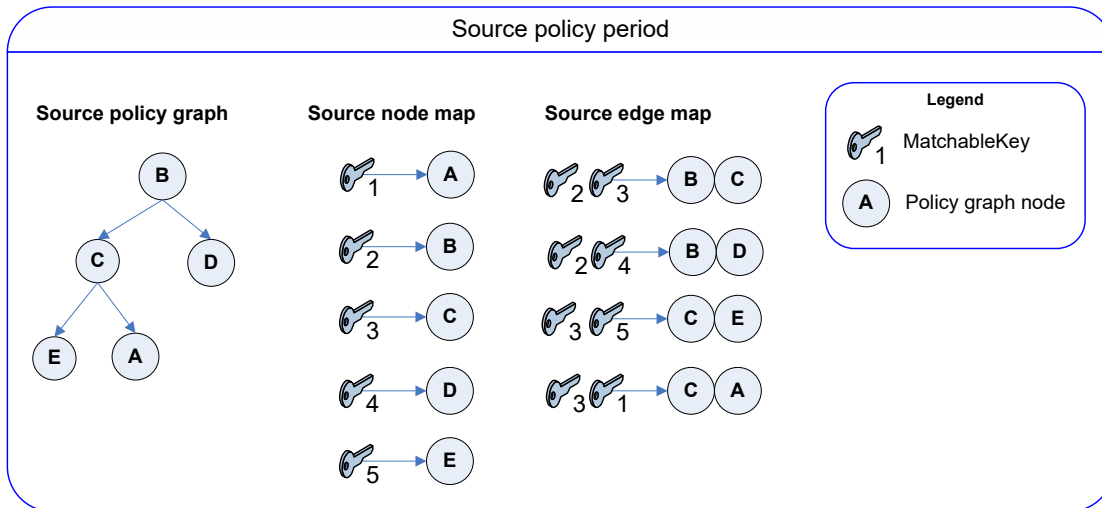
In `SideBySideBaseDataCopy`, the main method for copying base data is the `copySlice` method. This method has three phases.

Phase 1: Build map of matchable keys and edge map

The `copySlice` method calls the `MatchableTreeTraverser` class to build a map of `MatchableKey` objects for each node in the policy graph. The `MatchableKey` objects are used to determine if nodes from two different policy periods are logically equivalent.

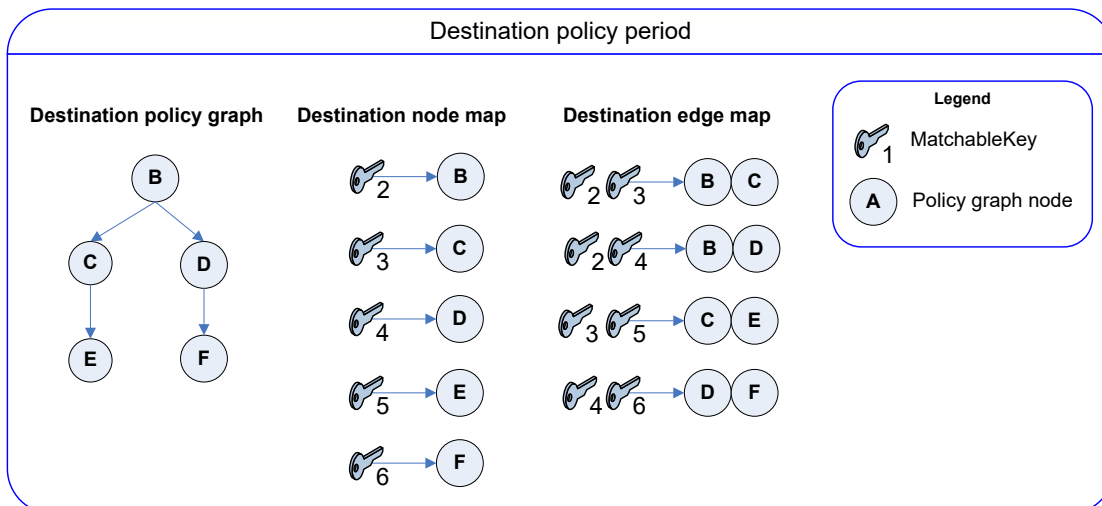
The `copySlice` method also calls the `MatchableTreeTraverser` class to build an edge map. The edge map contains a set of node to node edges for each relationship between nodes in the policy graph.

The following illustration shows the node and edge maps for the policy graph of a source policy period.



The `copySlice` method calls `MatchableTreeTraverser` to build node and edge maps for each destination policy period.

The following illustration shows the node and edge maps for the policy graph of a destination policy period.

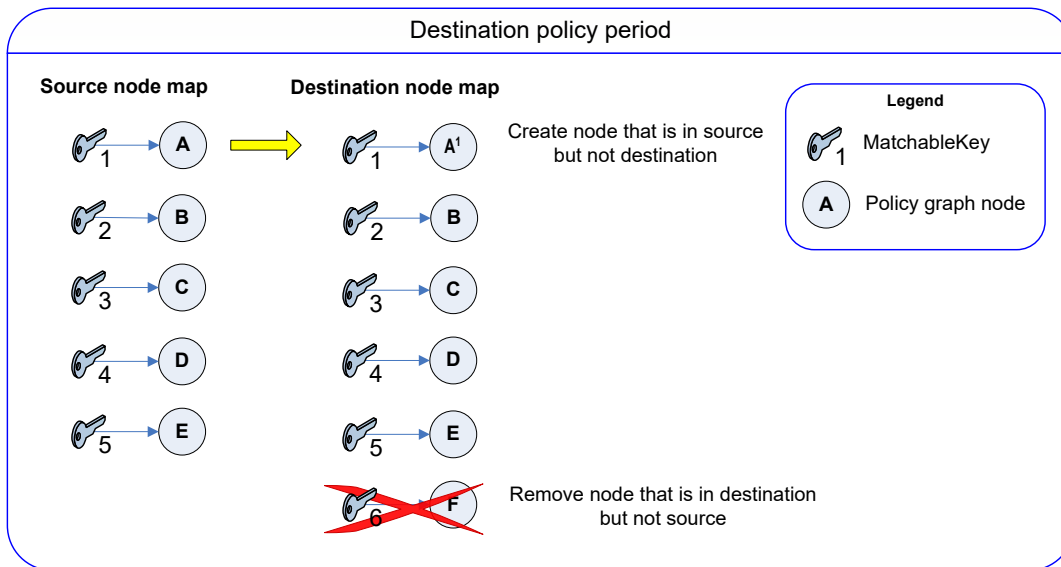


Phase 2: Compare source and destination node maps

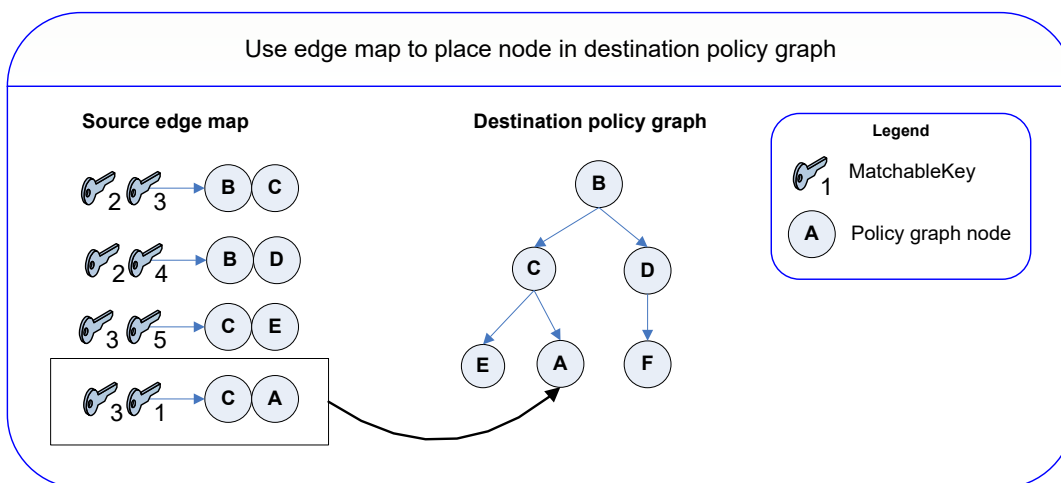
The `copySlice` method compares the source and destination node maps and creates or removes nodes from the destination policy graph.

In the following illustration, `MatchableKey 1` exists in the source but not the destination. Therefore, `copySlice` creates a corresponding `MatchableKey` and a copy of the A node in the destination.

The `MatchableKey 6` exists only in the destination node map. Therefore, `copySlice` removes the `MatchableKey` and the node.



Using the edge map of the source, the new node is inserted into the policy graph of the destination, as shown in the following illustration.



Phase 3: Update field values in destination

The copySlice method updates field values on the destination policy graph to match values on the source.

Excluding side-by-side data from base data

Side-by-side data are types and fields that PolicyCenter excludes from base data. Types include entities and delegates. Excluding these entities and fields prevents PolicyCenter from copying base data in a way that would cause inconsistencies in the policy data.

In PolicyCenter, the levels of excluded entities and fields are:

- “Side-by-side data that base data copy must exclude” on page 359 – These are not configurable.
- “Side-by-side data that base data copy can exclude” on page 359 – These are configurable.
- “Side-by-side data in personal auto excluded from base data copy” on page 361 – These are configurable.

Note: PolicyCenter also excludes data which is only accessible through excluded entities.

Side-by-side data that base data copy must exclude

Note: You cannot configure the side-by-side data that Guidewire excludes.

Guidewire excludes the following entities and fields from base data copy:

Exclude from base data	Details
Non-effective dated entities	Exclude all entities that are not EffDated.
Autonumber fields	Exclude all fields that point to AutoNumberSequence.
Costs and transactions	Exclude entities and their subtypes that implement the following delegates because the rating engine maintains them: • Cost • Transaction
Policy period	Exclude the following fields on the PolicyPeriod entity and its subtypes: • ActiveWorkflow • BranchName • BranchNumber • Failed00SEEvaluation • Failed00SEValidation • Job • Policy • PolicyTerm • PrimaryInsuredName (denorm) • Status • TermNumber • ValidQuote • Workflows
Policy location	Exclude the following field on PolicyLocation: • LocationNum
Building	Exclude the following field on Building: • BuildingNum
Policy contact role	Exclude the following fields on PolicyContactRole: • ContactDenorm • SeqNumber

Side-by-side data that base data copy can exclude

The data excluded from base data copy on all lines of business are defined in Gosu and can be configured.

The following table lists data on all lines of business that base data copy excludes in the default configuration. Guidewire strongly suggests that your implementation also exclude these.

Exclude from base data	Details
Coverages Exclusions Conditions	Exclude entities and their subtypes that implement the following delegates because the Product Model and other availability logic determine whether to add or remove them: • Coverage • Exclusion • PolicyCondition Exclude the following entities and their subtypes: • CoverageSymbol • CoverageSymbolGroup Exclude the following fields on entities that implement the Coverable delegate: • InitialCoveragesCreated • InitialExclusionsCreated • InitialConditionsCreated
Forms	Forms inference determines whether to add or remove forms based on the contents of the policy. Therefore, exclude the following entities and their subtypes: • Form • FormAssociation • FormEdgeTable
Reinsurance	Reinsurance Management determines whether to add or remove reinsurance-related entities and fields based on the contents of the policy. Therefore, exclude the following entity and its subtypes: • Reinsurable Exclude the following field on PolicyPeriod: • RIRiskVersionList
Underwriting Issues	Underwriting issues have complex logic that determines whether to add or remove issues. Underwriting issues also have an interaction with Policy. Therefore, exclude the following entity and its subtypes: • UWIssue The following fields are excluded on PolicyPeriod: • QuoteHidden • EditLocked
Grandfathering	Exclude fields related to grandfathering on an entity that implements the following delegates: • Coverage • Exclusion • Modifier In the default configuration, the following field is related to grandfathering, and therefore excluded: • ReferenceDateInternal Some entities, such as PersonalAutoLine, implement the ReferenceDateInternal field directly rather than through a delegate. Exclude these fields.
Billing	Exclude billing and payment-related fields because they are managed indirectly. Therefore, exclude the following fields on PolicyPeriod: • DepositCollected • InvoiceStreamCode

Exclude from base data	Details
	• NewInvoiceStream • ReportingPatternCode • SingleCheckingPatternCode • SeriesCheckingPatternCode • OverrideBillingAllocation • BillImmediatelyPercentage • DepositOverridePct • DepositAmount • WaiveDepositChange • PaymentPlanID • PaymentPlanName • AltBillingAccountNumber • AllocationOfRemainder • RefuncCalcMethod • BillingMethod • MinimumPremium • PaymentPlans Exclude the following field on PolicyLine and its subtypes: • MinumumPremium
Reporting fields	Exclude the following reporting-related denormalization fields on PolicyPeriod: • TotalCostRPT • TotalPremiumRPT • TransactionCostRPT • TransactionPremiumRPT
Additional policy-related fields	Exclude the following fields on PolicyPeriod: • BaseState • WrittenDate Exclude the following field on PolicyLine: • NumAddInsured

Side-by-side data in personal auto excluded from base data copy

The following are strongly-suggested excluded data in the personal auto line of business. In the default configuration, PolicyCenter excludes these entities and fields.

Exclude from base data	Details
Grandfathering	Exclude the following field on PersonalAutoLine and PersonalVehicle: • ReferenceDateInternal
Quick quote	Exclude the following field on PersonalVehicle and PolicyDriver: • QuickQuoteNumber

Exclude from base data Details

Personal auto	Exclude the following entities: • PersonalVehicle • PersonalAutoCov Exclude the following field: • EffectiveDatedFields.OfferingCode
Two paths to entity	Exclude the following entities because they have pointers to side-by-side entities, and there are potentially two paths to reach them: • VehicleDriver • PAVhcleAddlInterest See “Exclude entities reachable through two paths” on page 362.

The side-by-side data are configured in the `gw.job.sxs.PersonalAutoSideBySideBaseDataConfig` class.

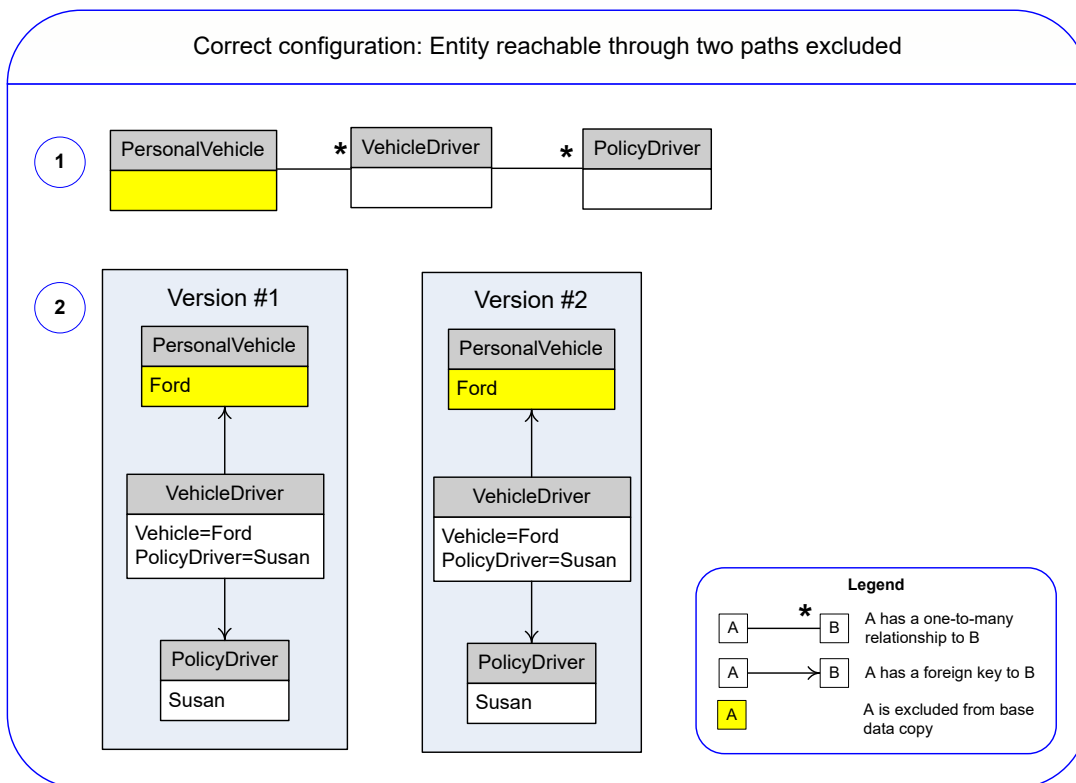
Exclude entities reachable through two paths

Base data copy can produce incorrect results in the following situation:

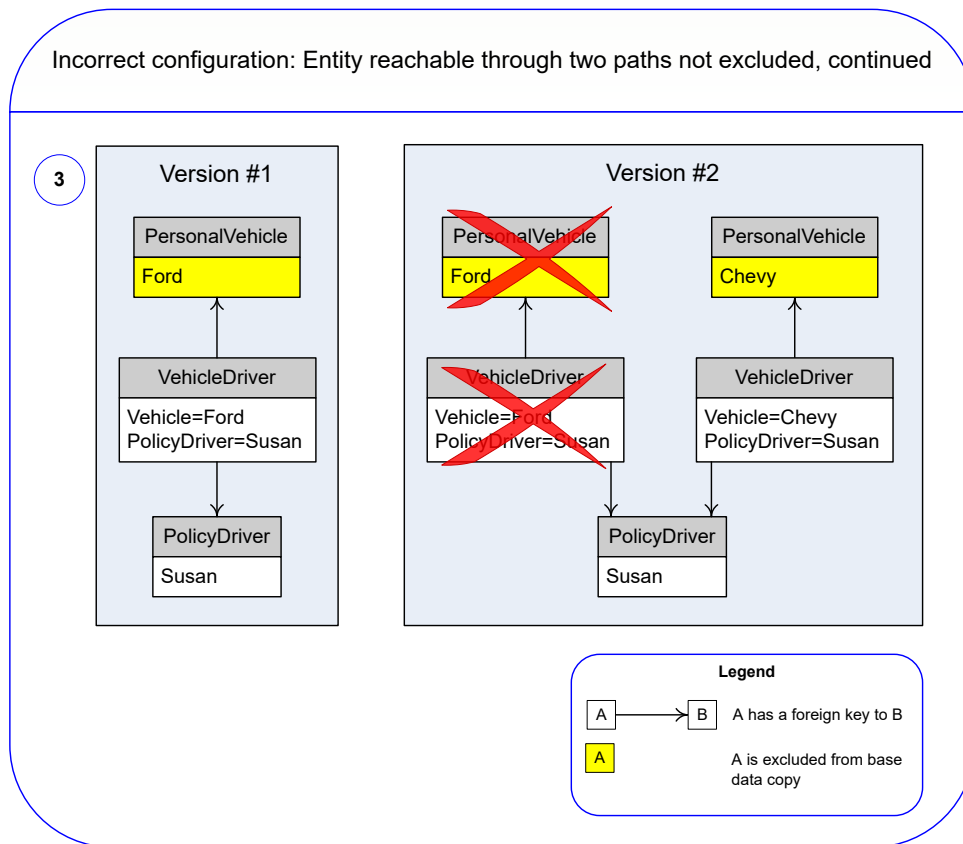
- If an entity is not excluded, and
- That entity is reachable by foreign key or array through two paths, one of which is through an excluded entity and the other is not.

Note: Guidewire recommends that you exclude an entity reachable through two paths if one of the paths is through an excluded entity. Guidewire especially recommends that an entity reachable through two paths be excluded if the foreign key is non-nullable.

In personal auto, `VehicleDriver` entity provides an example as shown in the following illustration of an incorrect configuration.



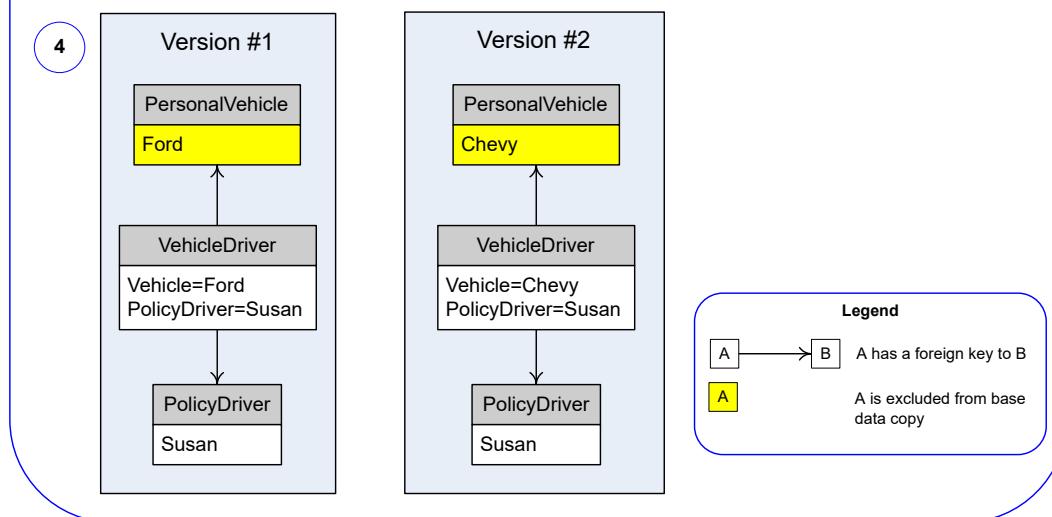
1. The VehicleDriver entity can be reached through PolicyDriver and PersonalVehicle. However, only PersonalVehicle is excluded from base data copy.
2. Initially, PersonalVehicle and VehicleDriver in Version #1 and Version #2 have the same values. The following illustration shows changes that happen when the agent edits policy Version #2.



3. The agent edits Version #2 and makes the following changes:
 - Removes the Ford and the VehicleDriver connecting Susan to the Ford.
 - Adds a Chevy and a VehicleDriver connecting Susan to the Chevy.

The following illustration shows the problems that base data copy encounters.

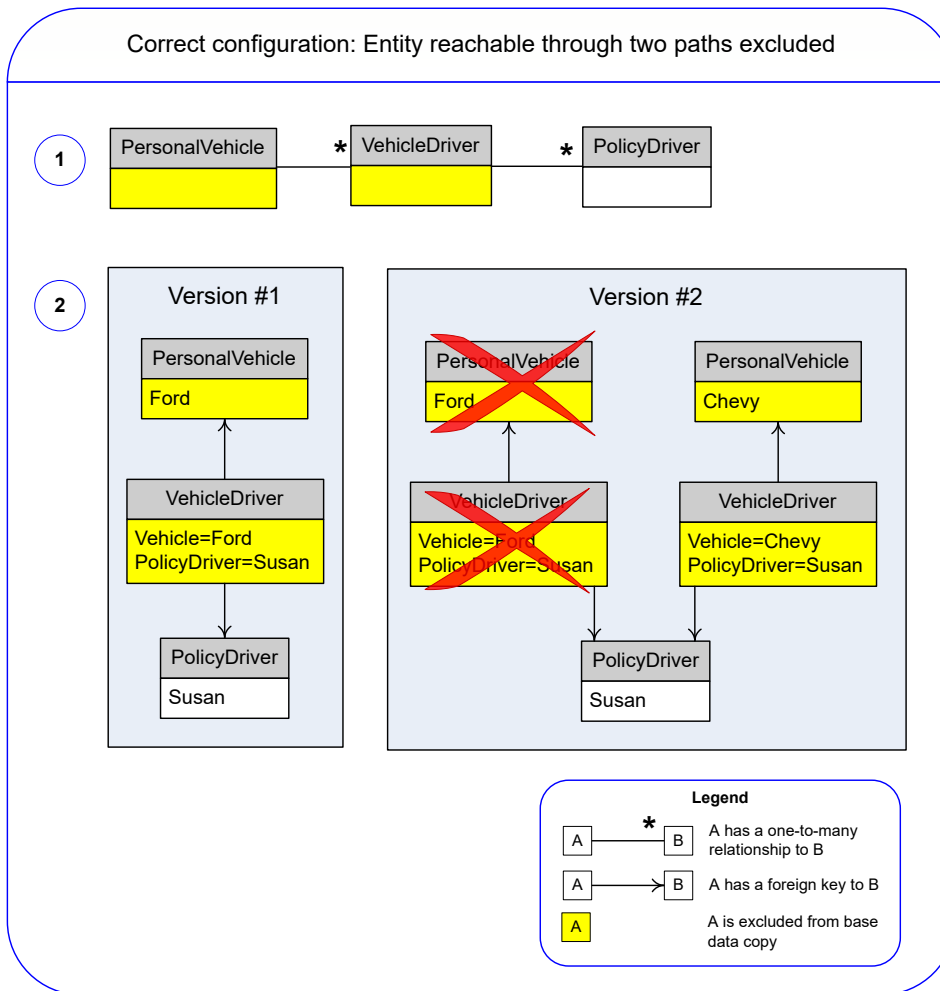
Incorrect configuration: Entity reachable through two paths not excluded, continued



4. When the agent saves the edits, PolicyCenter does the base data copy.

Removing the Ford and adding the Chevy is not a problem because `PersonalVehicle` is side-by-side data and does not affect other versions. However, `VehicleDriver` is a base data entity and must be the same in all versions. In Version #1, the `Vehicle` field points to the Ford. In Version #2, it points to the Chevy. Version #1 would contain bad data if base data copies `Vehicle=Chevy` to Version #1.

The `VehicleDriver` entity is reachable through two paths. Therefore, Guidewire recommends making the `VehicleDriver` entity side-by-side data by excluding it from base data copy, as shown in the following illustration.



1. Base data copy excludes both the PersonalVehicle and VehicleDriver entities.
2. It does not matter that the PersonalVehicle and VehicleDriver differ between the versions because base data copy excludes them.

Configuring data excluded from base data

To make data side-by-side data, you must exclude the field or type from base data. Fields include foreign keys, edge foreign keys, arrays, and one-to-one relationships. Fields also include scalar values such as bit, integer, and text. Types include entities and delegates. To exclude a field or type, add it to the `gw.job.sxs.PersonalAutoSideBySideBaseDataConfig` class:

- Add your new field to `sideBySideProperties`
- Add your new side-by-side type to `sideBySideTypes`

Configuring Quote All to ignore validation warnings

In the default configuration, **Quote All** in the **Side by Side Quoting** screen produces quotes only if there are no validation warnings. This behavior prevents PolicyCenter from generating invalid quotes. Guidewire expects that the default implementation is suitable for most implementations.

However, in some limited cases, your implementation may need to **Quote All** even though there are validation warnings. To quote when there are validation warnings, set the `SideBySideQuoteBlockedByWarnings` parameter in `config.xml` to `false`. The default value is `true`. If this parameter is not in `config.xml`, you need to add it.

Changing this parameter can affect product configuration files and Gosu code. Some code may assume that side-by-side quoting only occurs if there are no validation warnings. Test thoroughly to avoid problems.

In `gw.job.sxs.SideBySideProcess`, in the call to the `requestQuote` method has the `SideBySide` parameter as an argument.